HashiCorp Certified: Terraform Associate 004

Exam Prep Course

Bryan Krausen

Why Learn Terraform?

66% of Organizations are Increasing Cloud Spend YoY

Organizations are continuing to migrate workloads to one or more public cloud providers.

79% of Organizations have or are Planning Multi-Cloud Deployments

Management of multi-cloud environments requires cloud-agnostic solutions.

64% of Organizations are Experiencing a Shortage of Skilled Staff

Don't have the staff expertise they need to support their cloud infrastructure strategy.

Source: HashiCorp State of Cloud Strategy Survey

How This Course Will Transform Your Skills

Gain Real Skills, Build Confidence, and Open New Doors in Cloud Infrastructure

Hands-On Learning, Fast Results

Get practical experience with real-world Terraform scenarios from the start. You'll build and manage infrastructure right away, gaining skills you can apply immediately.

Confidence in Infrastructure Automation

By the end, you'll feel confident using Terraform to automate and manage cloud infrastructure, laying a solid foundation for advanced cloud projects.

Unlock Opportunities in Your Cloud Career

Terraform is a critical skill in the cloud ecosystem, and this course will give you the edge to stand out, whether you're looking to advance or break into cloud roles.

☝ HASHICONF 2024

INSTRUCTOR

Bryan Krausen

Terraform AUTHORIZED INSTRUCTOR

Welcome! I'm a Premier Instructor, a HashiCorp Ambassador, and a seasoned technologist who is passionate about cloud infrastructure and security. I help organizations realize the potential of HashiCorp's tools.

btk.me/btk

Why Trust Me?

I Consulted with VERY Large Organizations

During my (many) years of consulting on HashiCorp, I've designed, implemented, fixed, and managed it all ☺

I'm VERY Involved with the Community

I've spoken at 10+ HashiCorp User Groups (HUGs), 5 x HashiConf conferences, and I am one of three people who has been a HashiCorp Ambassador since the start

I Help Write HashiCorp Certification Exams

I've contributed to EVERY single certification exam that HashiCorp offers and continue to do so...

How Do We Get There?

A Variety of Content to Teach, Show, and Get you Hands-On Experience

- **Lectures to Build Understanding**
- **Demos for Real-World Examples**
- **Hands-On Labs to Apply Skills**

Access All Course Code on GitHub

All code used in this course is published in a GitHub repository, so you can follow along, review, and experiment with every example provided.

github.com/btkrausen/hashicorp

Udemy In-Course Hands-On Labs

✅ Preferred Method

Integrated Directly Into The Course in Your Browser

Isolated Environment for Hands-On Lab or Experimenting

All Prerequisites and Code are Preloaded for You

HashiCorp Terraform
Hands-On Labs

Get access to FREE hands-on labs to practice with Terraform configurations, settings, and components...all right in your browser using GitHub Codespaces.

github.com/btkrausen/terraform-associate-labs

Bryan Krausen

Learning Terraform: Tips for Your Journey

Start Small and Build Gradually

Learning Terraform can feel overwhelming but remember that every small step adds up.

Mistakes Are Part of the Process

Errors are expected and even helpful. Don’t be discouraged if things don’t work right away. Debugging is a great way to deepen your understanding.

Ask Questions and Seek Community

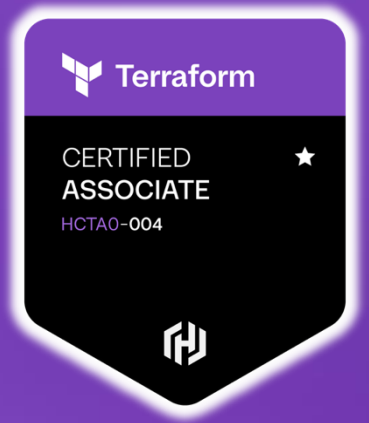
You’re not alone on this journey. Reach out for help when you need it, whether in the Q&A, forums, or with peers.

LET'S GET! STARTED!

Bryan Krausen

Terraform Associate 004 Certification

Everything You Need to Know



The Value of Terraform Associate Certification

Industry Recognized Credential

Often preferred or required for many modern DevOps-related or infrastructure roles

Opens Doors to Cloud Engineering Roles

Demonstrates commitment to professional development and your skillsets.

Validates Your Foundational Terraform and IaC Skills

Potential (and current) employers want to know that you can develop and manage Terraform and IaC workflows.

Is This Exam Right for You?

Ideal Candidates:

- Cloud Engineers & DevOps Professionals
- Recommended to have 6+ Months of Terraform Experience (not a hard requirement)
- Solid practice with all objectives/features in a lab environment
- Basic terminal skills
- Understanding of cloud architecture fundamentals

Exam Essentials

Duration: **60 Minutes***

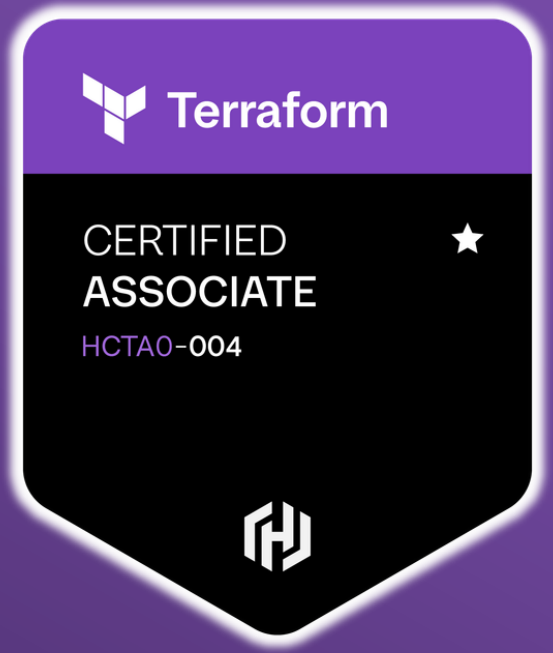
Delivery: **Online Proctored** (Certiverse)

Language: **English Only**

Cost: **$70.50 USD** (No free retake)

Validity: **Two-years** (retake the exam to extend**)

Results: **Immediate Pass/Fail Result**



* Non-Native English Speakers can request additional time during registration

** Extend by passing the Terraform Professional

Exam Format

What Type of Questions Should You Expect on the Exam?

Multiple Choice (one answer)

Multi-Select (multiple answers)
Questions will indicate how many to choose

True/False Questions

- Cloud-Agnostic but Mentions Different Platforms
- Scenario and Code-Based Questions
- Not Intentionally Tricky

Exam Logistics

Scheduling and Delivery

Book through HashiCorp Certification Portal

developer.hashicorp.com/certifications/signin

There is a check-in Process Before You Start

Government ID Required (must match registration)

Quiet, Private Room with Stable Internet

Seriously, make sure people aren't walking in as you're taking your exam

Webcam and Microphone Needed

Proctor will be watching you. Software will alert if your eyes continuously move off screen, etc.

EXAM OBJECTIVES

Objective 1

Infrastructure as Code (IaC) with Terraform

Objective 2

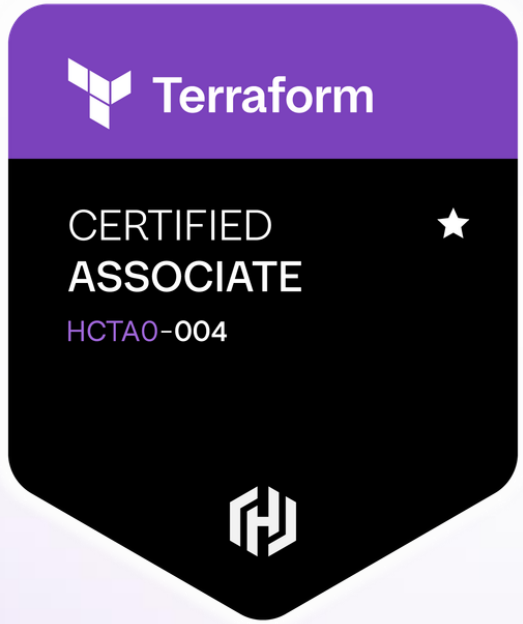
Terraform Fundamentals

Objective 3

Core Terraform Workflow

Objective 4

Terraform Configuration



Objective 5

Terraform Modules

Objective 6

Terraform State Management

Objective 7

Maintain Infrastructure with Terraform

Objective 8

HCP Terraform

What's Changed?

Differences Between Terraform Associate 003 and 004

Bryan Krausen

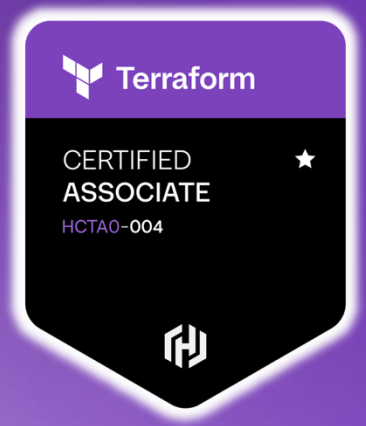
Terraform Associate 004

- The Terraform Associate 003 was retired in January 2026 in favor of the newer Terraform Associate 004.
- The last day to take 003 was January 8, 2026.
- Moving forward, all Terraform Associate will be on the 004 exam version.

What Stays the Same?

The core exam structure remains unchanged. Same format, same delivery, same question types.

Approximately 70% of content overlaps between versions.



Similar but Reorganized Objectives
Lots carried over, but there are additional sub-objectives to cover more modern features

60-Minute Testing Format
Same number of questions, same timeframe to take your exam

Same Type of Questions
Multiple choice, multi-select, and true/false
I do believe they removed any fill-in-the-blank

Online Proctored Delivery
In 2025, HashiCorp did drop PSI in favor of using Certiverse for delivery (better experience)

Same Two-year Validity
Either the 003 or the 004 gets you a Terraform Associate certification valid for 2 years

Four Major Additions to the Terraform Associate 004

Explicit Dependencies & Lifecycle Rules

Updates include the `depends_on` and `create_before_destroy` lifecycle rules.

Custom Validation Conditions

Know how to validate configurations using conditions, like the validate block for input variables.

Ephemeral Values & Write-Only Arguments

Know how to protect sensitive data and keep it out of state using ephemeral values and write-only arguments.

Expanded HCP Terraform Coverage

Now includes more HCP Terraform features, such as workspaces, projects, governance, collaboration tools, and more.

What This Means for You!

If you Studied and Passed TA 003:

- **Most of Your Knowledge Transfers**
- **Focus on the Four New Areas**
- **Review and Use the HCP Terraform Features**

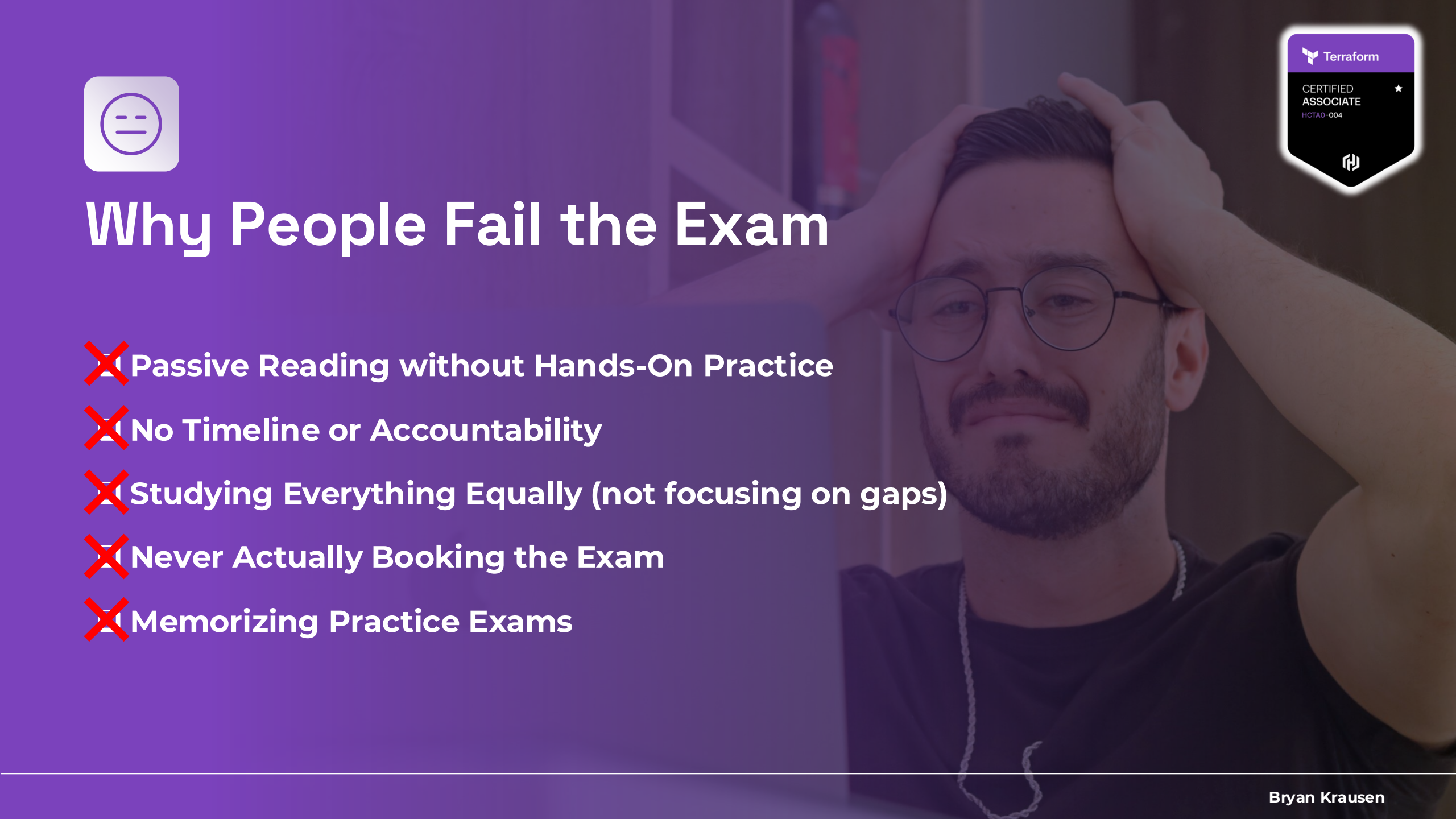
Your Path to Certification

Study Plans that Work

Bryan Krausen

Why People Fail the Exam

- ❌ Passive Reading without Hands-On Practice
- ❌ No Timeline or Accountability
- ❌ Studying Everything Equally (not focusing on gaps)
- ❌ Never Actually Booking the Exam
- ❌ Memorizing Practice Exams



There's a Better Way!

Using a strategic, timeline-driven approach can help make you more successful.

Choose a Pathway Based on Experience
You likely have some knowledge that applies to the exam

Book Your Exam Date NOW
Make the Ultimate Commitment and set a Target Date

Focus on Weak Areas
Don't Study Everything Equally, Focus on Things You Don't Know

70% Hands-On Practice, 30% Theory
Hands-On Practice is Proven to Help You Learn and Remember

Structure Weekly Schedule
Based on the Exam Date, Plan out Your Weeks to Ensure You Stay on Track

Pick Your Path to Success

🎓 Beginner

- ✓ Little to No Terraform Experience
- ✓ 8-12 Weeks
- ✓ Full Coverage of All Objectives

⚙️ Practitioner

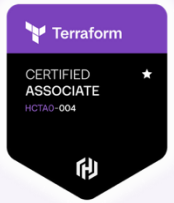
- ✓ Some Terraform Experience
- ✓ 4-6 Weeks
- ✓ Focus on Gaps + HCP Terraform

🚀 Experienced

- ✓ Regular Terraform User
- ✓ 2-3 Weeks
- ✓ Focus on 004 Updates and Exam Prep

Beginner Pathway

Building Skills from Zero to Terraform Certified



Weeks 1-3

Foundations

- **Objectives 1-2:** IaC Concepts and Fundamentals
- **Hands-On:** First Resources and Using Basic Workflow
- **Milestone:** Using Basic Configurations

Weeks 4-7

Core Skills

- **Objectives 3-4:** Workflow and Configuration
- **Hands-On:** Multi-Resource Deployments
- **Milestone:** Write Configurations From Scratch

Weeks 8-10

Advanced Skills

- **Objectives 5-8:** Modules, State, HCP Terraform
- **Hands-On:** Build Modules, Use Remote State
- **Milestone:** Understanding State and Modules

Weeks 11-12

Exam Preparation

- **Practice Exams:** Use Exams to Identify Gaps
- **Review Weak Areas:** Read Docs, Practice
- **Take Your Exam!**

Bryan Krausen

Practitioner Pathway

Fill in the Gaps and Formalize Knowledge

Start with a Practice Exam

If you are already using Terraform, use practice exams to identify weak objectives/areas and ensure you've got the fundamentals down.

- Deep Dive of 2-3 Weakest Subjects and 004-specific content (lifecycle, validation, ephemeral)
- Build Complete Demo Environments and Practice All Workflows. Create Modules from Scratch
- Integrate Workflows with HCP Terraform
- Take Multiple Full Practice Exams for Final Gap Identification

Experienced Pathway

Experienced Users Have 6+ Months of Experience Using Terraform in a Production Environment.

Comfortable Developing Multi-Resource and/or Multi-Cloud Terraform Configurations and Using Modules.

Focus on 004 Additions

`depends_on`, `lifecycle`, `validation`, `ephemeral` and write-only, HCP Terraform

Multiple Practice Exams

Identify any Specific Gaps with Videos and Documentation, Deep Dive into HCP Terraform

Timed Practice Exams

Final Confidence Building, Practice taking the test under real conditions

The Most Important Rule

The 70/30 Rule

Do NOT underestimate hands-on practice. Deploy free resources, break things, fix it, practice different configurations.

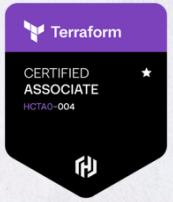
You CANNOT learn Terraform by reading alone!

70% - Hands-On Practice

Write actual configurations
Deploy real resources
Break things and fix them
Practice CLI commands
Work through error messages

30% - Theory and Reading

Documentation Review
Video content
Concept understanding



Book Your Exam!

Don't wait until you 'feel ready.' Pick your pathway, add the study time, and **book your exam TODAY.**

- ✅ Creates a hard deadline (no more "I'll do it someday")
- ✅ Gives you skin in the game ($70.50 investment)
- ✅ Motivates consistent study (exam is coming!)
- ✅ Prevents endless preparation without action

LECTURE

Introduction to HashiCorp Terraform

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

What is Terraform?

Created by HashiCorp

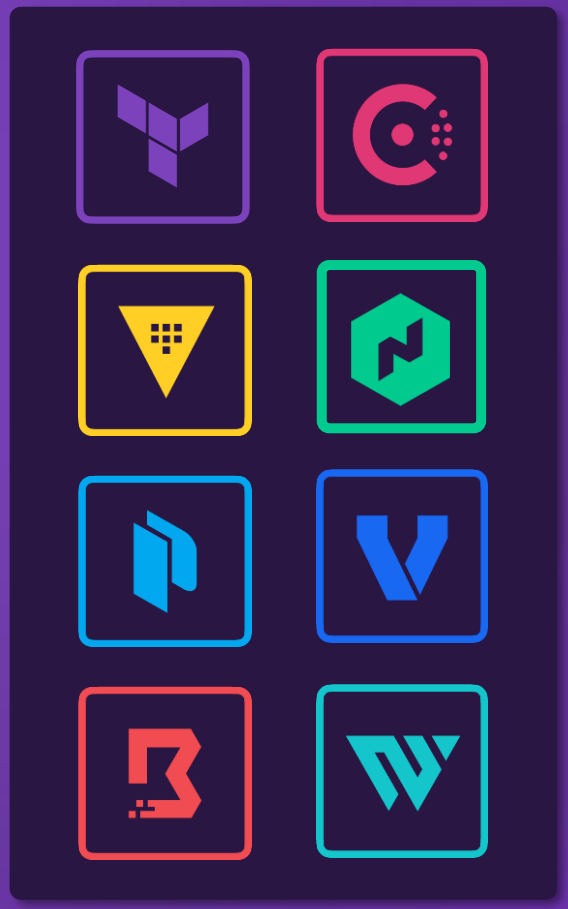
Often more famous by its tools rather than its name. It also offers many industry-standard solutions

Declarative Approach

Focuses on describing "what" needs to be done rather than explicitly stating "how" to do it, allowing the system to determine the best execution steps

Based on HashiCorp Configuration Language (HCL)

A human-readable language used in Terraform to define infrastructure resources in a clear, structured, and reusable way



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

What is Terraform?

The industry standard Infrastructure as Code (IaC) tool that automates the provisioning, management, and versioning of infrastructure.

Codify Your Infrastructure

Rather than clicking around in a console, create a repeatable, versioned workflow to provision and manage resources

Standardize Workflows

Use a single tool to deploy and manage resources required for application workloads. Quickly scale up using modules and other features.

Platform Agnostic

Deploy infrastructure as code using HCL to provision resources from any cloud or datacenter.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

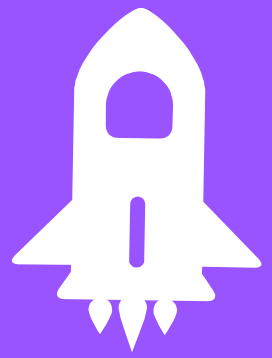
Benefits of Terraform

VERSION CONTROL AND AUDITIBILITY

Infrastructure changes are tracked in version control systems, providing a clear history and ability to revert

AUTOMATION AND EFFICIENCY

Automate the provisioning and management of infrastructure, reducing manual tasks and accelerating deployment times



SCALABILITY & FLEXIBILITY

Quickly scale infrastructure up or down to adapt to changing requirements by modifying code instead of manually changing resources.

COLLABORATION

Teams can collaborate on infrastructure changes just like application code, improving transparency and reducing silos in operations

CONSISTENCY & REPEATABILITY

Ensures that infrastructure is consistently deployed across environments, reducing configuration drift and human errors

Terraform is Platform Agnostic

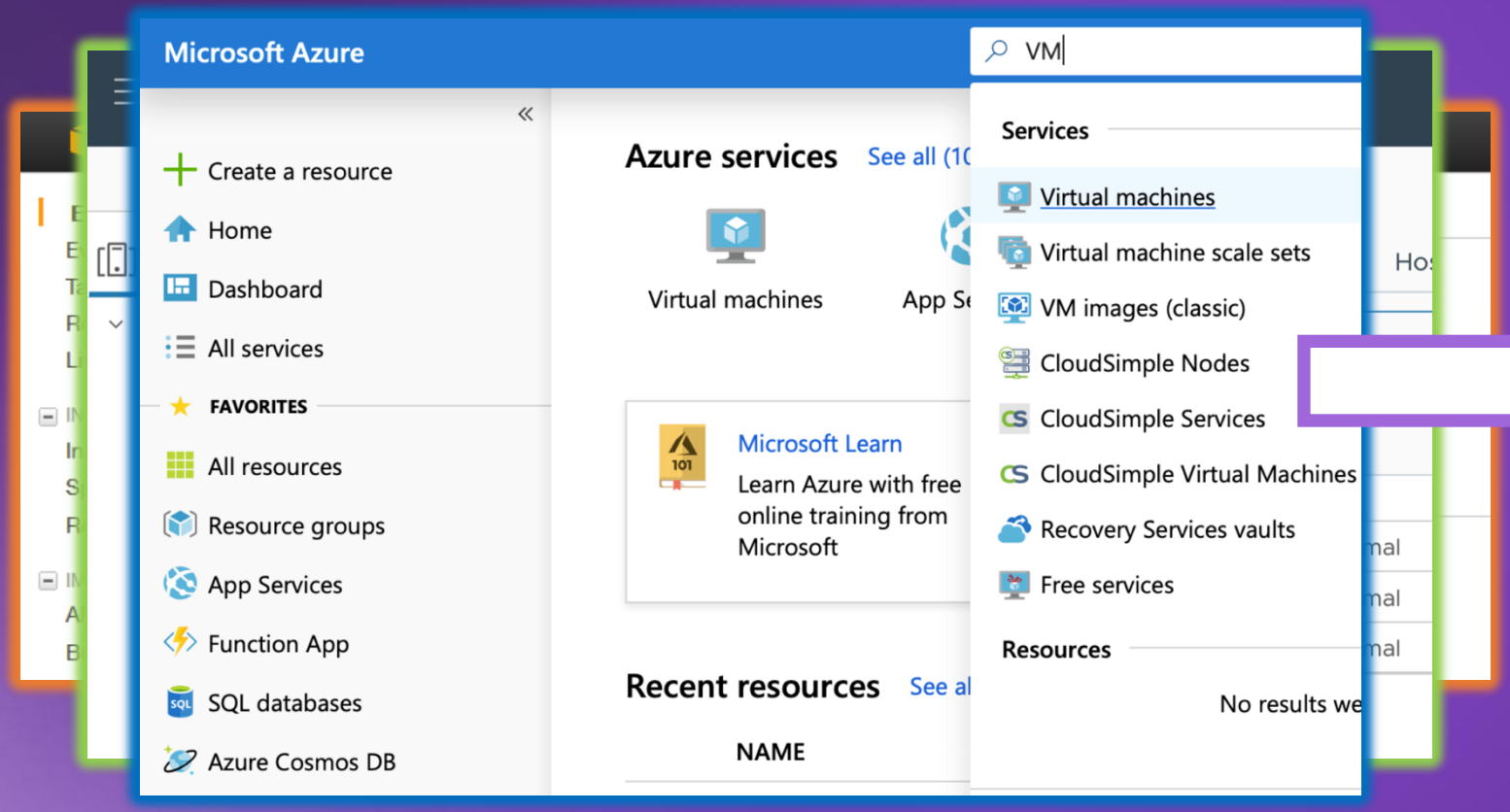
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

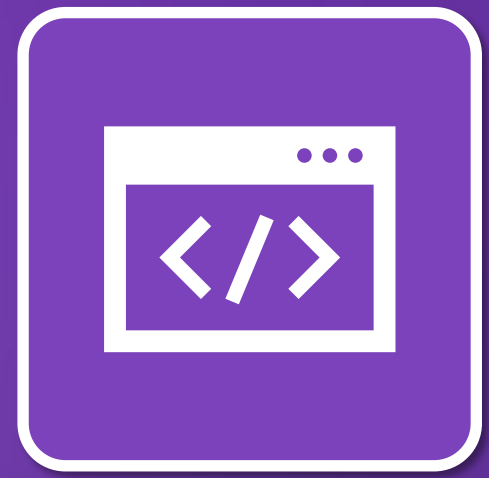
Before Using Terraform, You Need to First Be Proficient on the Target Platform

While it might feel like it, Terraform isn't magic. You need to understand the target platform before using Terraform.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Traditional Process of Provisioning Resources

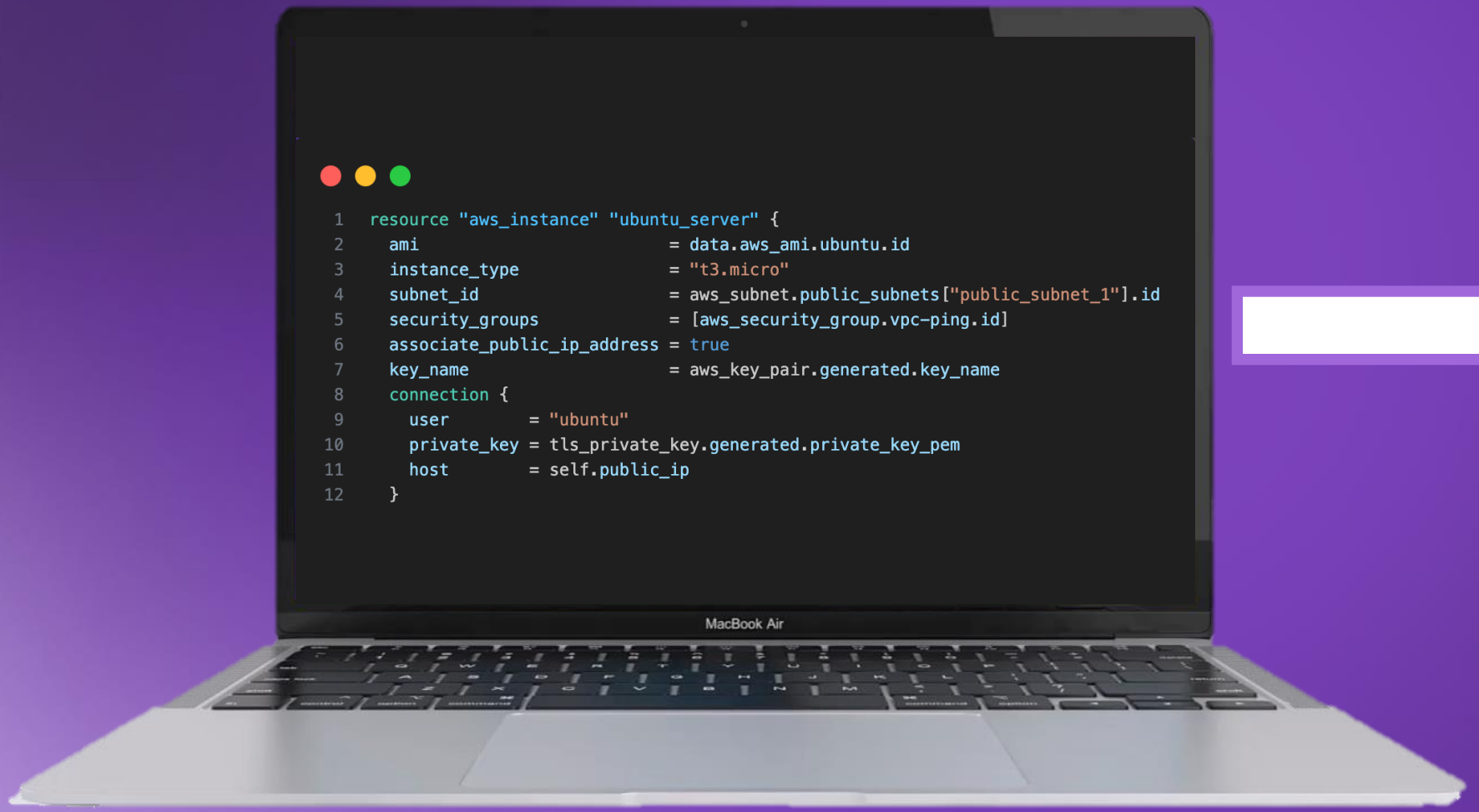




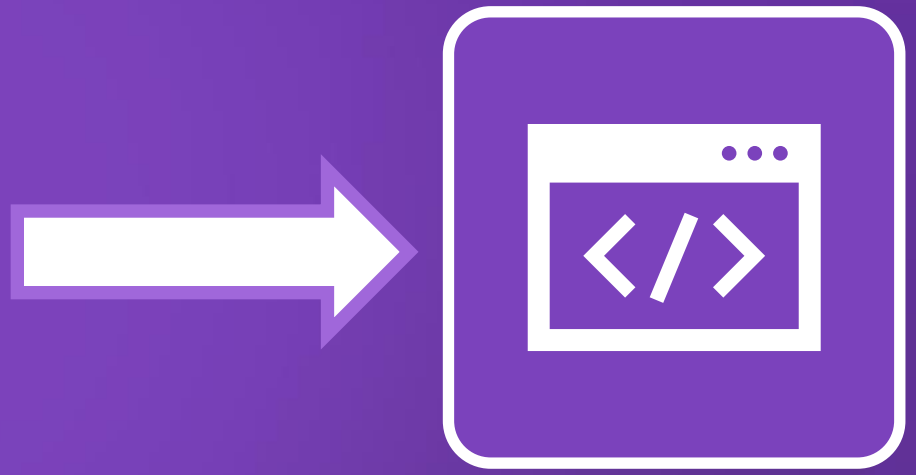
- ✓ Virtual Machine
- ✓ Container
- ✓ Network

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Provisioning with Terraform




```
resource "aws_instance" "ubuntu_server" {
  ami                         = data.aws_ami.ubuntu.id
  instance_type               = "t3.micro"
  subnet_id                   = aws_subnet.public_subnets["public_subnet_1"].id
  security_groups             = [aws_security_group.vpc-ping.id]
  associate_public_ip_address = true
  key_name                    = aws_key_pair.generated.key_name
  connection {
    user        = "ubuntu"
    private_key = tls_private_key.generated.private_key_pem
    host        = self.public_ip
  }
```



Application Workload

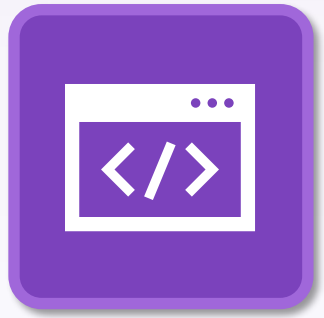
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Reusing Code with Terraform

Input Variables

AMI = Ubuntu
Subnet = 10.16.7.0
App = Web Server

```
resource "aws_instance" "basic_server" {
  ami                         = data.aws_ami.image.id
  instance_type               = var.instance_size
  subnet_id                   = aws_subnet.public_subnets.id
  security_groups             = [aws_security_group.ingress-ssh.id]
  associate_public_ip_address = true
  key_name                    = aws_key_pair.generated.key_name
  }
```

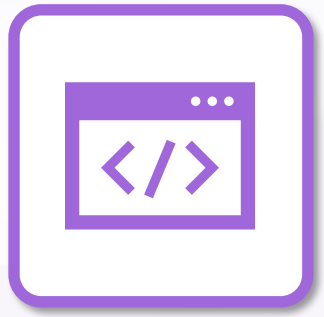


Web Application

Input Variables

AMI = Windows
Subnet = 10.0.3.0
App = Data Processing

```
resource "aws_instance" "basic_server" {
  ami                         = data.aws_ami.image.id
  instance_type               = var.instance_size
  subnet_id                   = aws_subnet.public_subnets.id
  security_groups             = [aws_security_group.ingress-ssh.id]
  associate_public_ip_address = true
  key_name                    = aws_key_pair.generated.key_name
  }
```

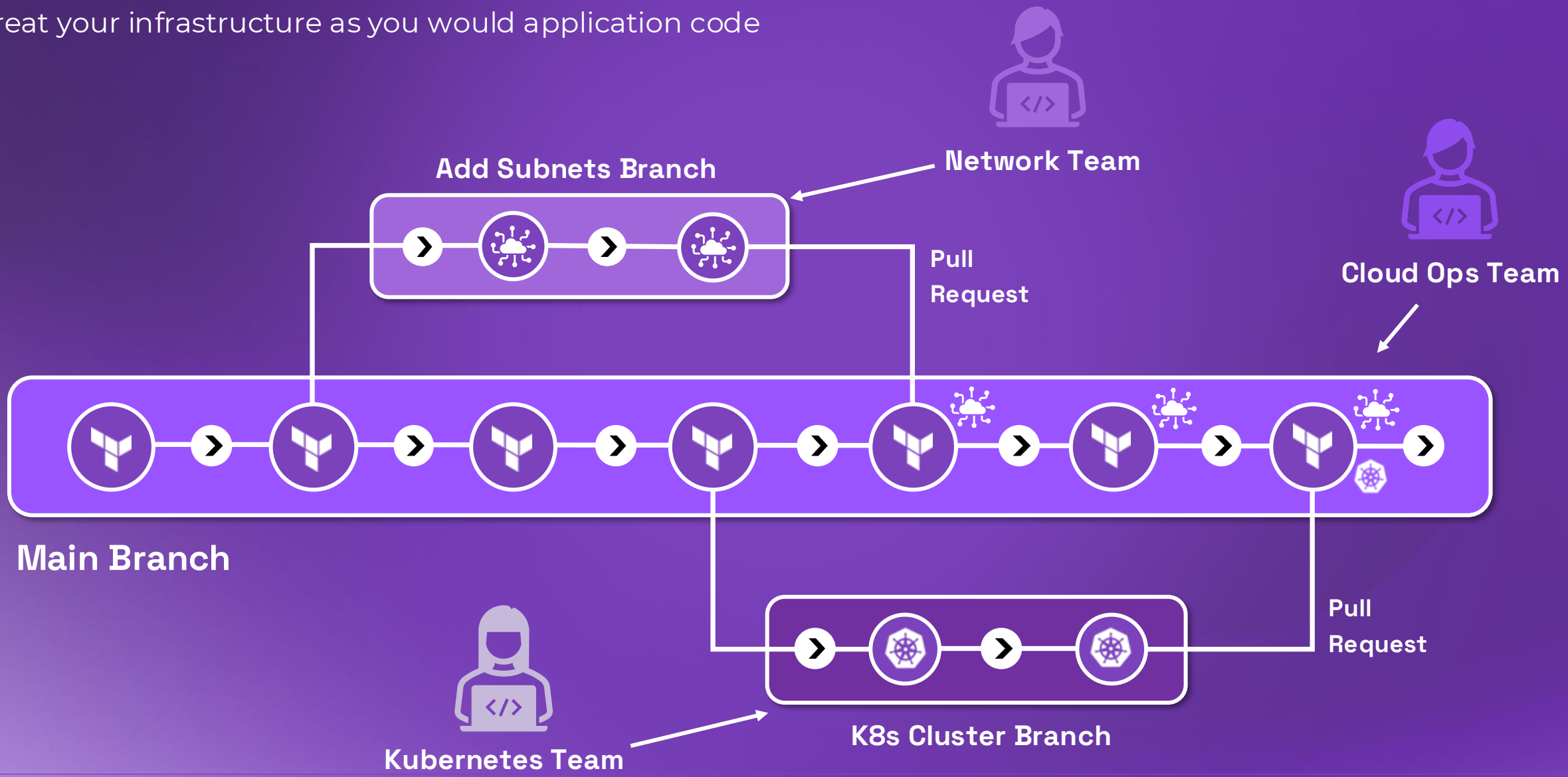


Data Application

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Infrastructure as Code

Treat your infrastructure as you would application code



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Speed & Consistency

Terraform can help you quickly spin up and decommission infrastructure for development, testing, QA, and production.

By defining infrastructure as code, you can reduce the risk of mistakes. You can create standardized components that can be integrated across projects consistently.

You can save infrastructure resources and their dependencies as code in a specified environment.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Core Components

Terraform Core
CLI tool that provisions and manages infrastructure resources as defined in Terraform configuration files

Providers
Extends the functionality of Terraform for specific platforms, such as public cloud providers, SaaS offerings, etc.

Resources
Infrastructure components or services that are managed by Terraform (virtual machines, networks, DNS records, etc.)

State
How Terraform maps the desired configuration with real-world resources on the target platform

Modules
Reusable and sharable blocks of code that can be called over and over again

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Editions

Terraform Community — Command-Line Based

HCP Terraform — SaaS Platform

Terraform Enterprise — Self-Hosted & Managed

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Compare Terraform with Other Tools

Terraform is often confused with other popular tools commonly found in many organizations.

Let's set the record straight...

Similar to Terraform

Infrastructure as Code Tools

- Infrastructure Provisioning and Management
- Declarative Language
- Immutable Infrastructure
- Modular Design

Different but Complementary

Configuration Management Tools

- Installing Packages
- Managing Files
- Manage Configuration Systems
- Automating Routine Tasks
- Orchestrating Multi-Step Tasks

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

What Does It Mean to Be Declarative?

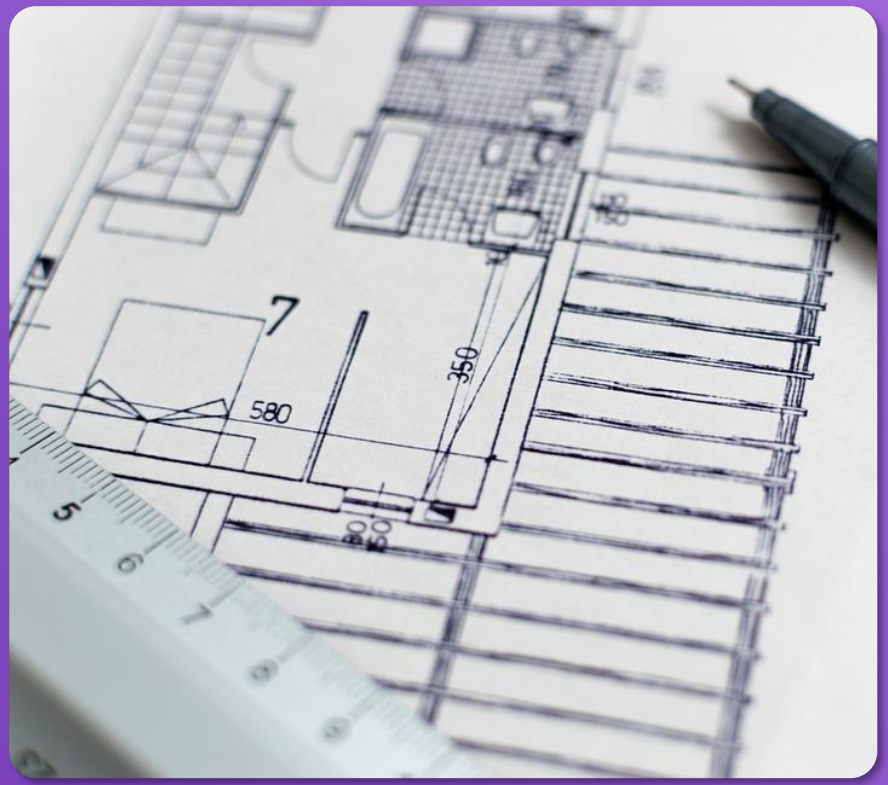
A declarative approach focuses on defining *what* you want the outcome to be, not *how* to achieve it.

DECLARATIVE vs IMPERATIVE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

How Terraform Defines Desired State

The desired state is the blueprint of your infrastructure: resources, properties, and relationships.



What you write in your Terraform configuration is known as the desired state. You are telling Terraform "Create resources exactly how I've defined it within my Terraform configuration."

Terraform compares the current state of 'real-world' resources with the desired state and makes the appropriate changes.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

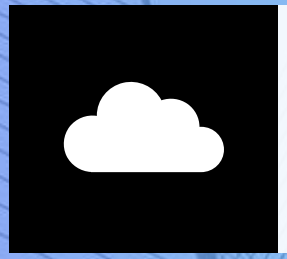
How Do I Get Started with Terraform?

1. **Download and Install Terraform**
   Get the proper binary for your machine.
2. **Prepare Your Workspace**
   Install your favorite IDE, create directories, etc.
3. **Obtain Credentials for Target Platform**
   Terraform needs a way to communicate with your cloud
4. **Write Your First Terraform Config**
   Start with basic resources and iterate from there.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

SUMMARY

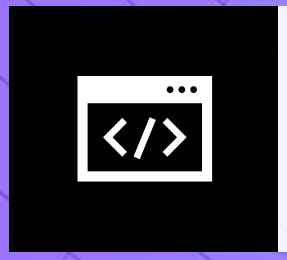
Terraform is a tool created by HashiCorp that is used to manage infrastructure as code.



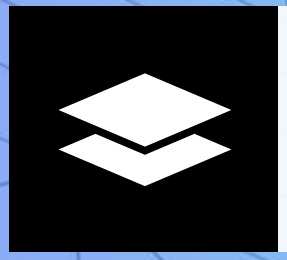
Terraform is platform-agnostic. It can be used to manage infrastructure on any cloud platform or data center.

Terraform takes a declarative approach, which means that users describe what they want and Terraform will figure out

Benefits of Terraform include version control, scalability, increased collaboration, and repeatability



Terraform uses the HashiCorp Configuration Language (HCL) to define infrastructure



Terraform's core components include Terraform Core, Providers, Resources, State, and Modules.

Bryan Krausen

LECTURE

Learn the Basics of HCL

Bryan Krausen

HashiCorp Configuration Language (HCL)

Terraform configuration language to describe the infrastructure that Terraform manages.

Declarative Language

Used in Terraform (and other HashiCorp products) to define infrastructure configurations

Easy to Read and Write

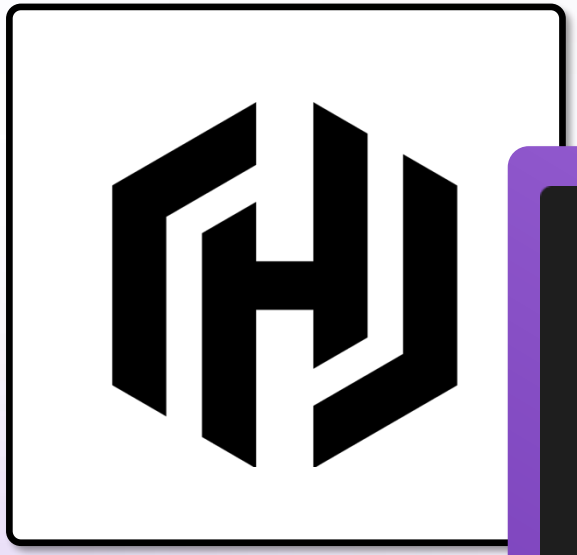
HCL provides a clean and human-readable syntax for expressing resources and dependencies

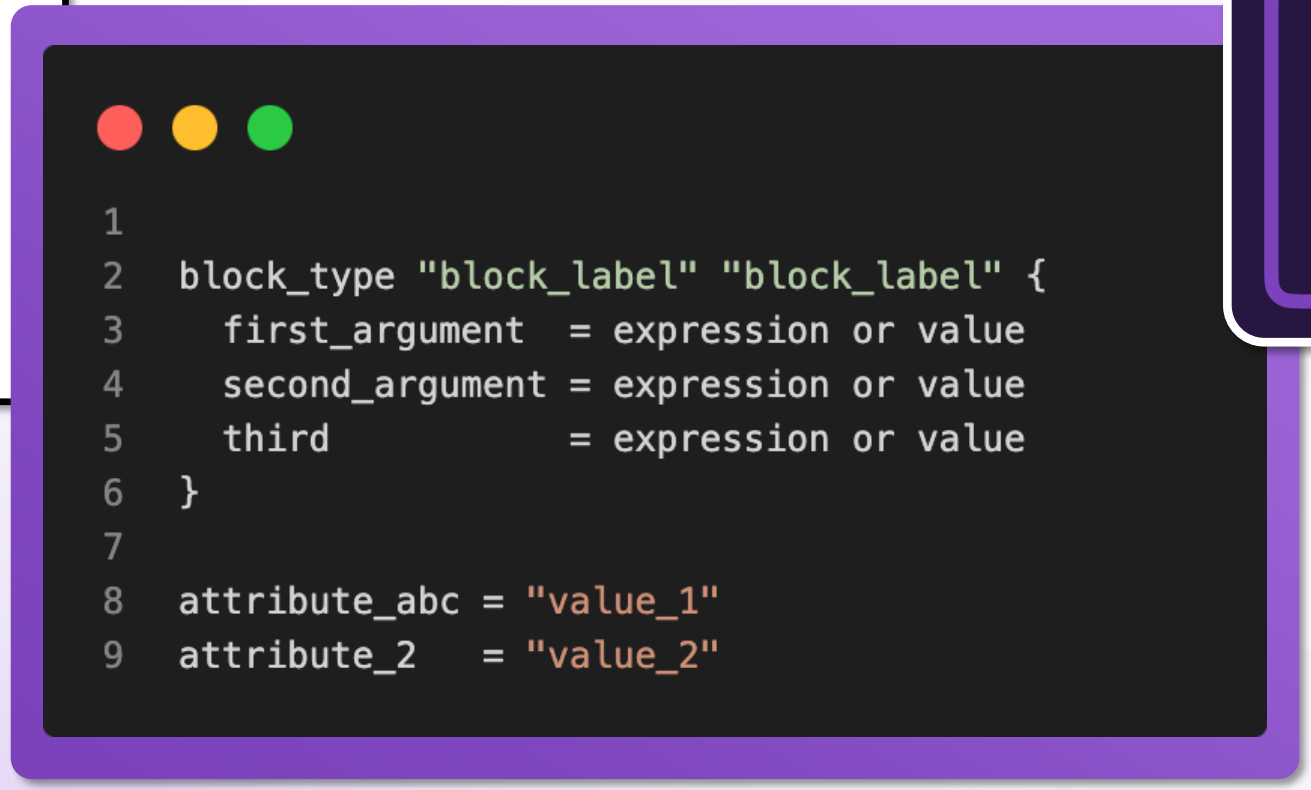
Terraform's Primary Interface

Configuration files that you write tell Terraform what infrastructure you want to create or what data to retrieve

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

HashiCorp Configuration Language (HCL)

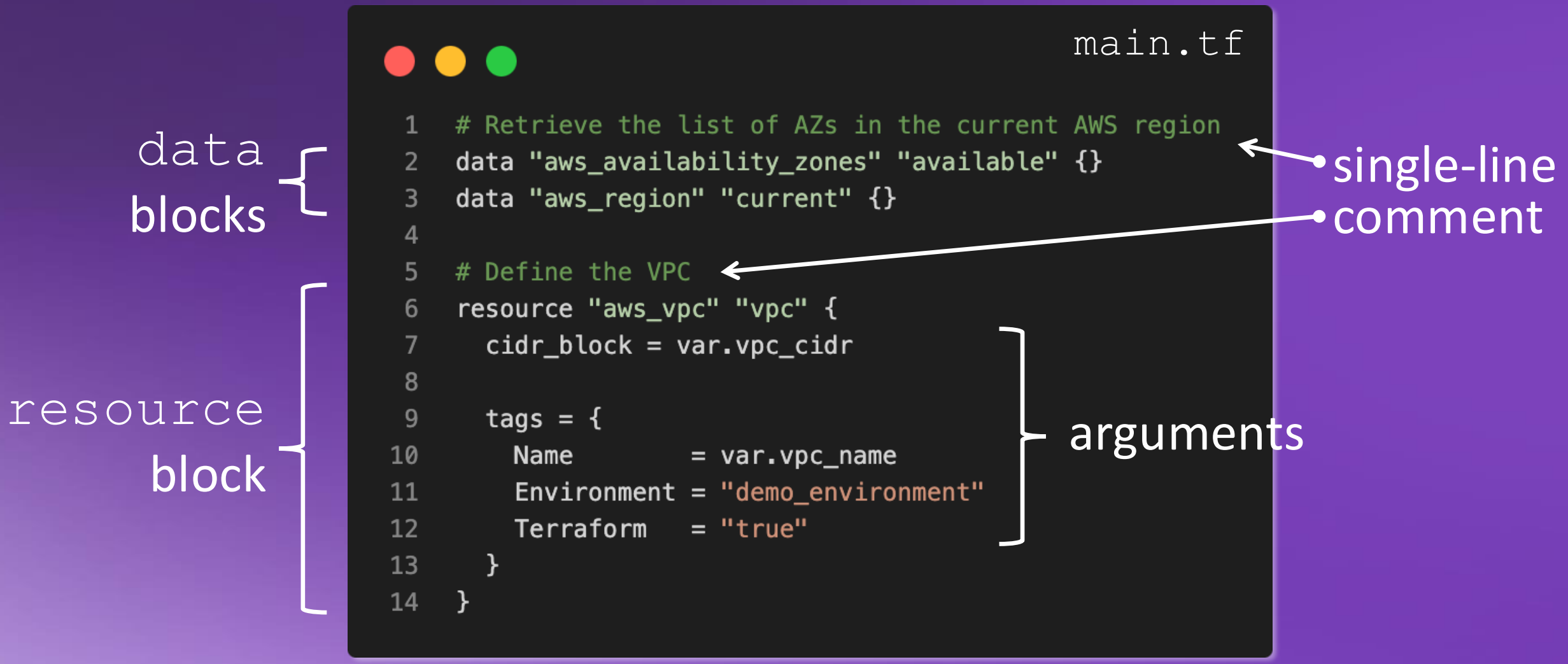




```
1
2  block_type "block_label" "block_label" {
3    first_argument  = expression or value
4    second_argument = expression or value
5    third           = expression or value
6  }
7
8  attribute_abc = "value_1"
9  attribute_2   = "value_2"
```

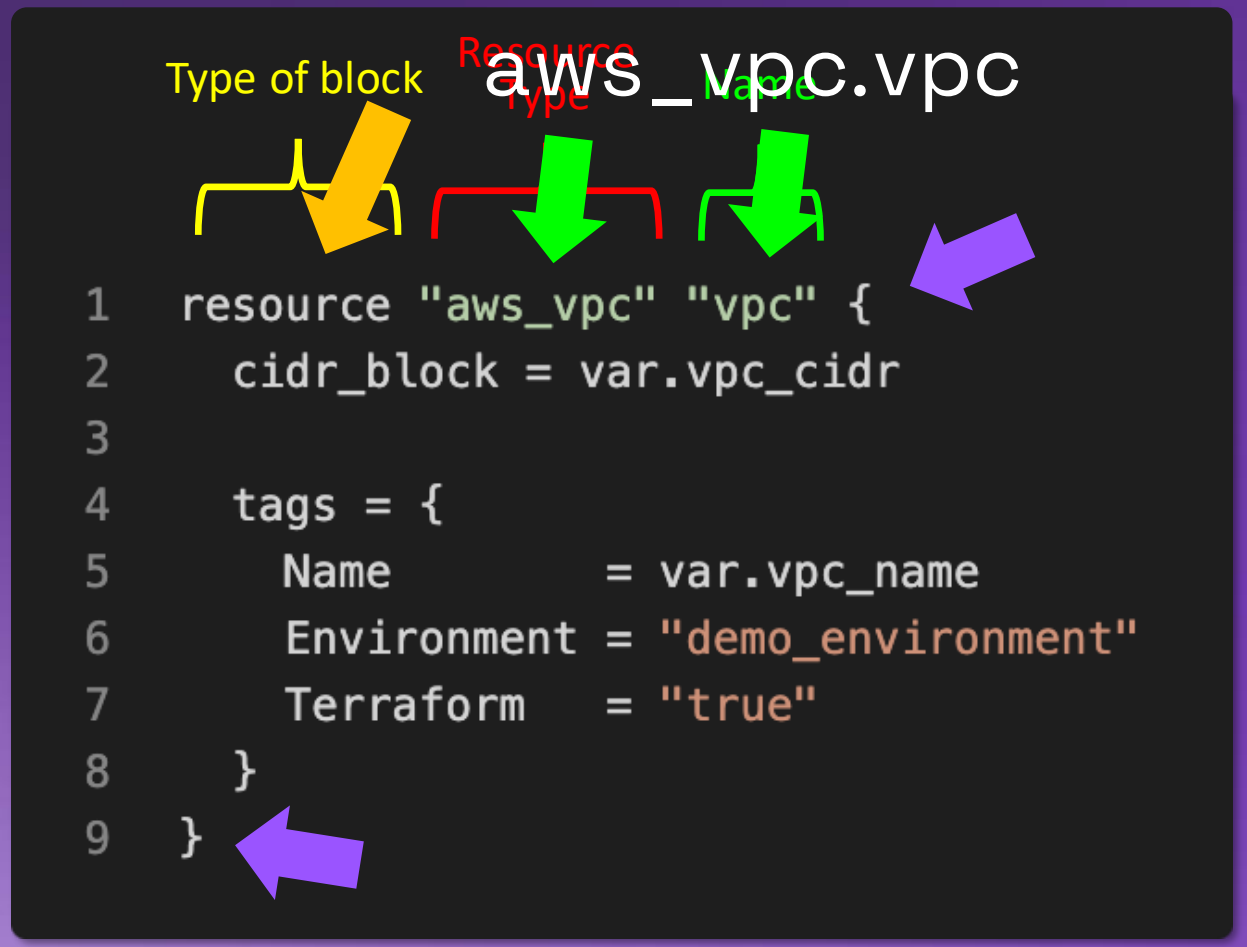
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

HCL Example



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Block Example



Blocks are delineated by { }
All contents within the block are arguments some type of value

Blocks start with a keyword
Defines what type of block it is, such as resource, data, variable, data, etc.

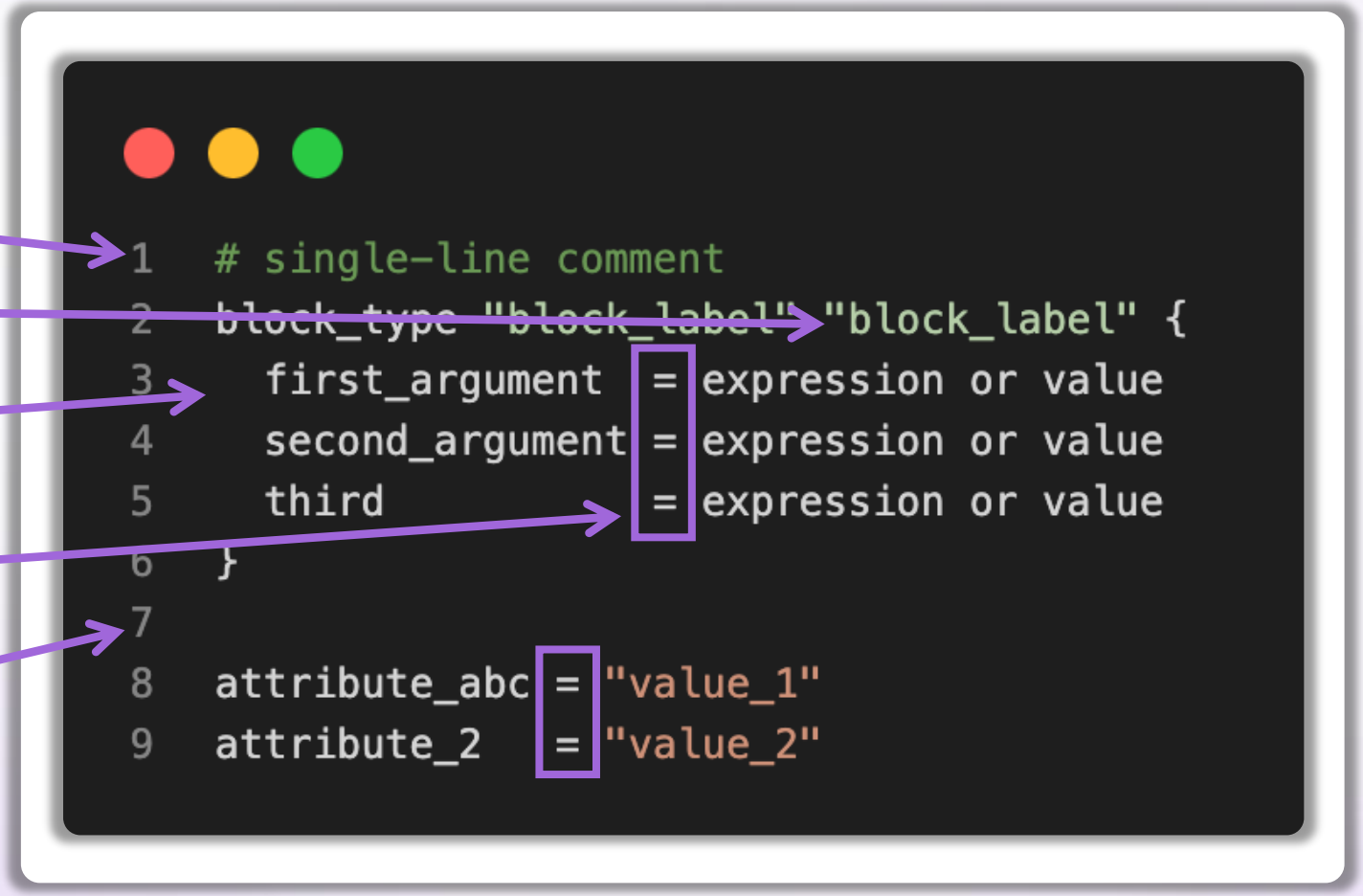
Blocks have named values
Terraform can access information created by or obtained by a block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

HCL Style Guide

Writing Terraform code in a consistent style makes it easier to read and maintain.

- Use # for single and multi-line comments
- Use underscores to separate multiple words in names
- Indent two spaces for each nesting level
- Align equal signs for values on consecutive lines
- Use empty lines to separate groups of arguments in a block



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Referencing

Resource referencing allows you to connect resources within your configuration, making it easy to share data between resources and build dependencies.

Create More Dynamic Configurations

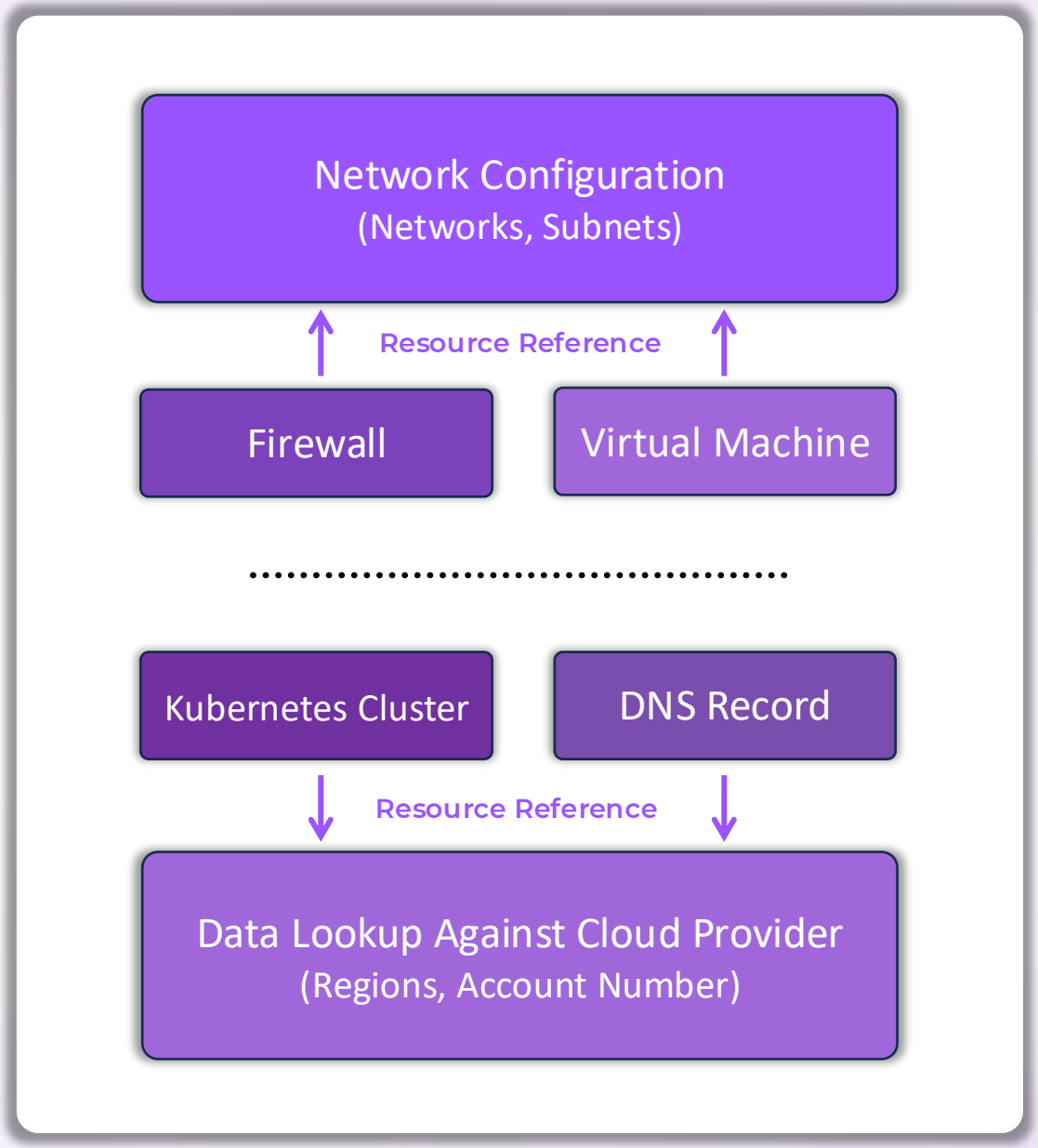
Terraform can use properties from one resource in another and avoid hardcoding values, creating a dynamic configuration

Automatic Dependency Mapping

Terraform automatically determines the order of resource creation based on references.

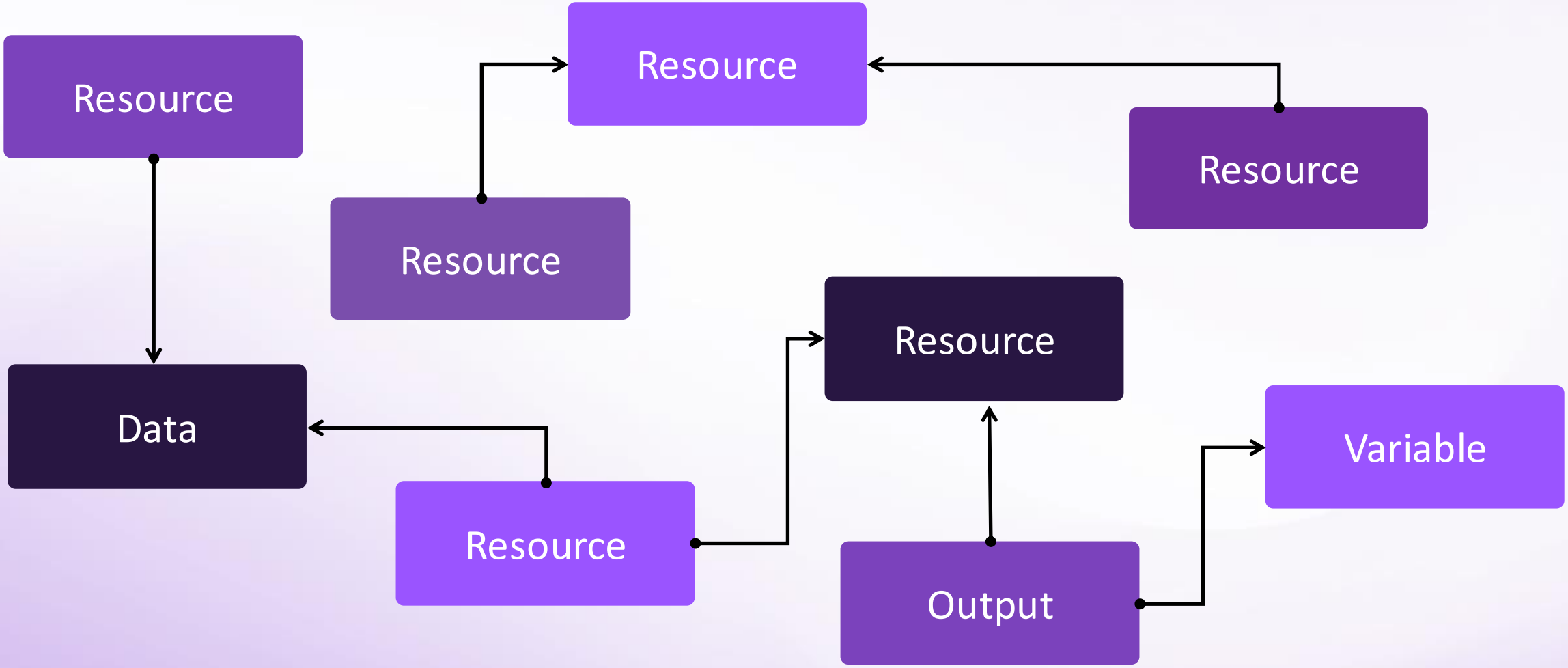
Resource Identifiers

Each Terraform block is identified by a **type** and a **name**, which makes it easy to reference in other parts of your configuration



Resource Referencing

The Reality of Terraform



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

DEMO

Learn the Basics of HCL

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Best Practices for HCL

File Extensions

Terraform HCL files use the .tf .tfvars file extension. This allows Terraform and related tools to recognize and apply configurations properly

Formatting

Correctly formatting files improves readability and maintainability. Follow the style guide or use built-in tools to format your files

Organization

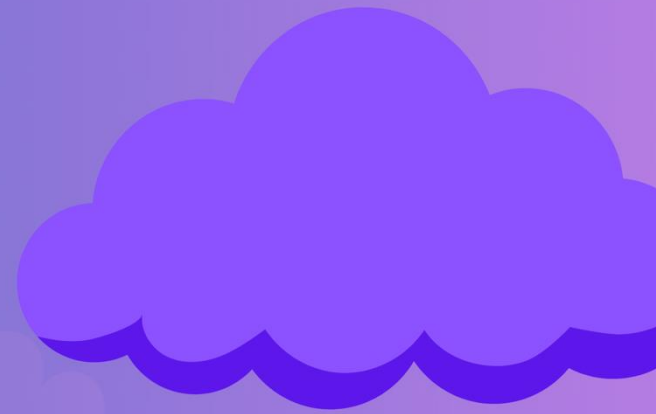
Organize functionality within files (variables in a variables file, outputs in an outputs file, etc.

Commenting and Documentation

Include comments and README.md files to provide context and instructors, making it easier for others to understand and use the code

Avoid Hardcoding Values

Use variables for values that may change which makes configurations for flexible and reuable.



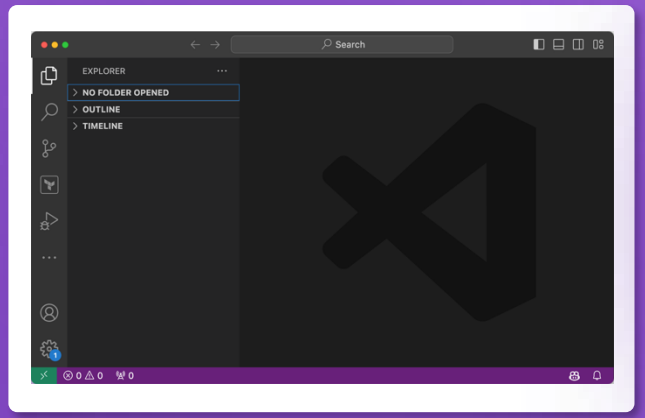
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Enhancing Code Quality

Using Linters, Extensions, and Built-In Tools for Code

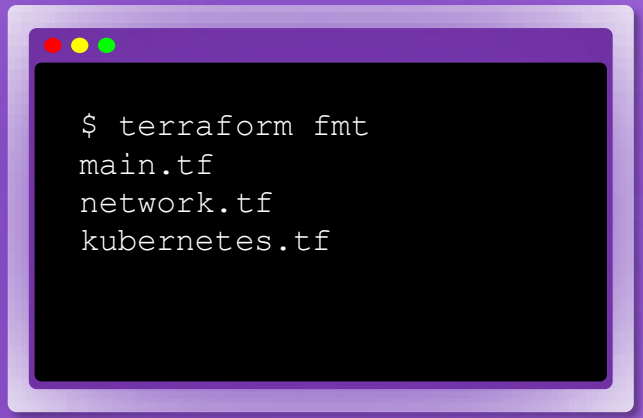
Properly formatted code improves readability and reduces errors. Use a VSCode extension or linter for real-time syntax checks and best-practice suggestions.

Or run `terraform fmt` to automatically format HCL files with a Terraform-related extension (`.tf`)



Linters and Extensions

Linters provide immediate feedback on code issues, while extensions offer additional features like autocompletion and integrated commands



Built-In Tools

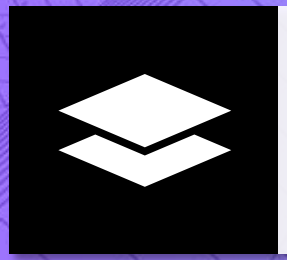
`terraform fmt` automatically formats Terraform code to ensure consistency and readability, making collaboration easier across teams.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

SUMMARY

HCL is a declarative language that is easy to read and write, and is Terraform's primary interface.

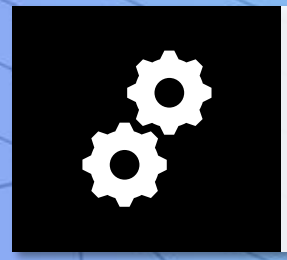
A single Terraform file can have one or many blocks to define your infrastructure



HCL uses blocks for data, resources, etc., and these blocks can be nested.

Use the '.tf' file extension so Terraform and related tools can recognize and apply configurations properly.

Single-line and block comments are used in HCL.



Use linters, extensions, and built-in tools like 'terraform fmt' to enhance code quality.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Introduction to the Terraform Workflow

The Terraform Workflow (Official)

1

Write
Create or Modify your Terraform Configuration Files

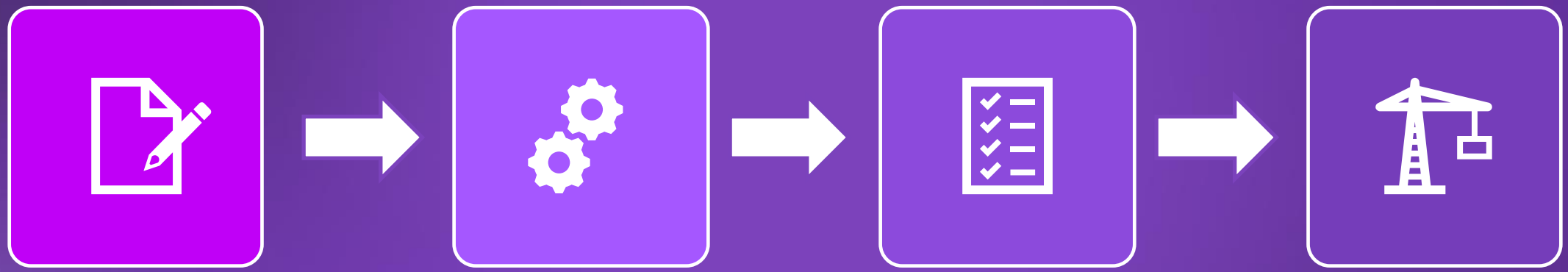
2

Plan
Preview your changes before applying them (aka dry run)

3

Apply
Implement the Changes According to your Configurations

Terraform Workflow



Write Configuration (HCL)

`Initialize`

`Plan`

`Apply`

Developing Terraform Configuration

What Does ”Develop Configuration” Mean?

The Write stage involves creating one or more configuration files that define your desired state – the end result you want for your infrastructure.

Terraform configurations are iterative, meaning you can build and refine your configuration incrementally.

Defining Desired State

You describe the infrastructure components (e.g., VMs, Networks) and the properties for each. Terraform ensures that the "real-world" infrastructure matches this state

`terraform init`

`terraform init` is the first command you run in a new Terraform project to set up your working directory

What Does It Do?

It prepares the environment to run other Terraform commands. Downloads the required providers (e.g., AWS, Azure, GCP) and modules used in the Terraform configuration.

Backend Management

`terraform init` also configures the backend for storing Terraform state and creates the lock file

When Do You Use It?

At the start of a new project or any time you add, remove, or upgrade providers/modules or change the backend settings in your configuration.

The Terraform Lock File

Every time you run `terraform init`, Terraform creates or updates a file called `.terraform.lock.hcl` in your working directory

Purpose

Locks dependency versions for consistency across your environments

File Name and Location

The name is `.terraform.lock.hcl` and it's stored in your working directory (same as .tf files)

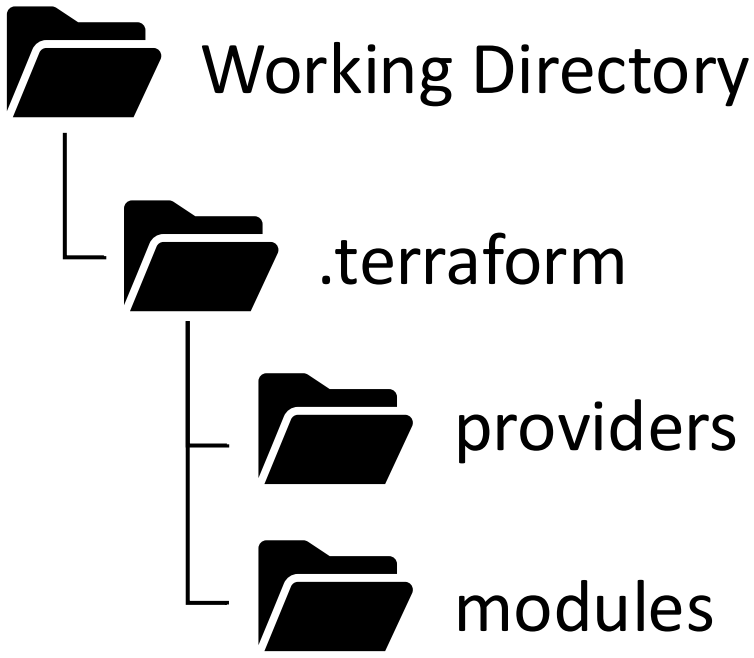
Best Practices

Commit to Version Control and Managed Automatically by Terraform

terraform init

TERMINAL

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/vsphere...
- Finding latest version of hashicorp/random...
- Finding latest version of hashicorp/vault...
- Installing hashicorp/vault v4.5.0...
- Installed hashicorp/vault v4.5.0 (signed by HashiCorp)
- Installing hashicorp/vsphere v2.10.0...
- Installed hashicorp/vsphere v2.10.0 (signed by HashiCorp)
- Installing hashicorp/random v3.6.3...
- Installed hashicorp/random v3.6.3 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```



The `.terraform` Directory

Terraform uses the `.terraform` directory to store the project's providers and modules

- The `.terraform/providers` directory stores cached versions of the configuration's providers.
- The `.terraform/modules` directory contains downloaded versions of any referenced modules.

Working with the .terraform Directory

01

Should the `.terraform` directory be in version control?

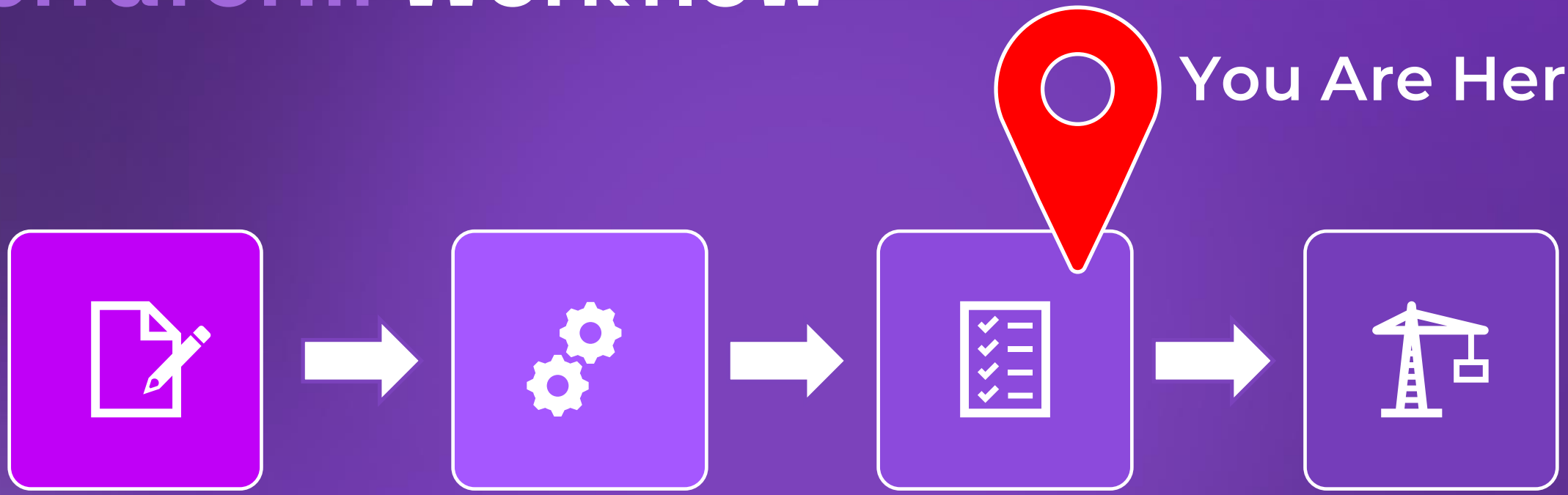
- No, add to your .gitignore file
- Contains easily recreatable files
- Can be large in size and contain platform-specific files

02

Can you delete the `.terraform` directory?

- Technically, yes
- Run terraform init to create it
- Useful for troubleshooting or cleaning up old dependencies

Terraform Workflow



terraform plan

`terraform plan` generates an execution plan showing the changes that Terraform will make to the 'real-world' resources in order to match the desired state configuration

What Does It Do?

Provides you with an output of the changes that will happen if you apply the changes It shows you the resources to be created (+), updated (~), or destroyed (-).

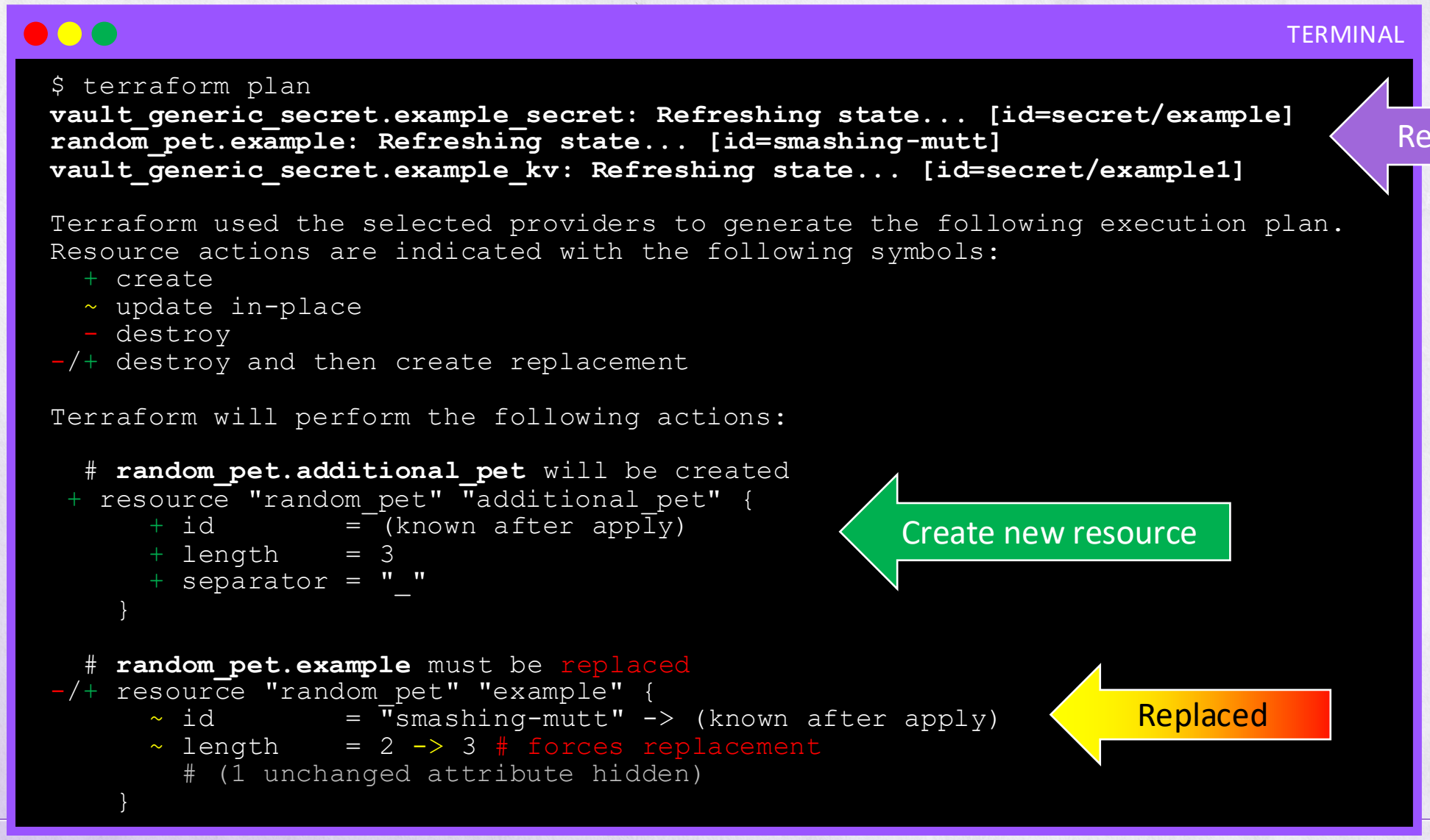
Perform a Dry-Run of Your Changes

`terraform plan` gives you the ability to perform a 'dry-run' without impacting 'real-world' resources. It helps avoid unintended changes by showing exactly what Terraform will do

Why a Plan is Important

It is crucial for reviewing changes in production environments to prevent downtime out errors.changes,uld be run before applying any changes but it is **NOT** required for a `terraform apply`

terraform plan

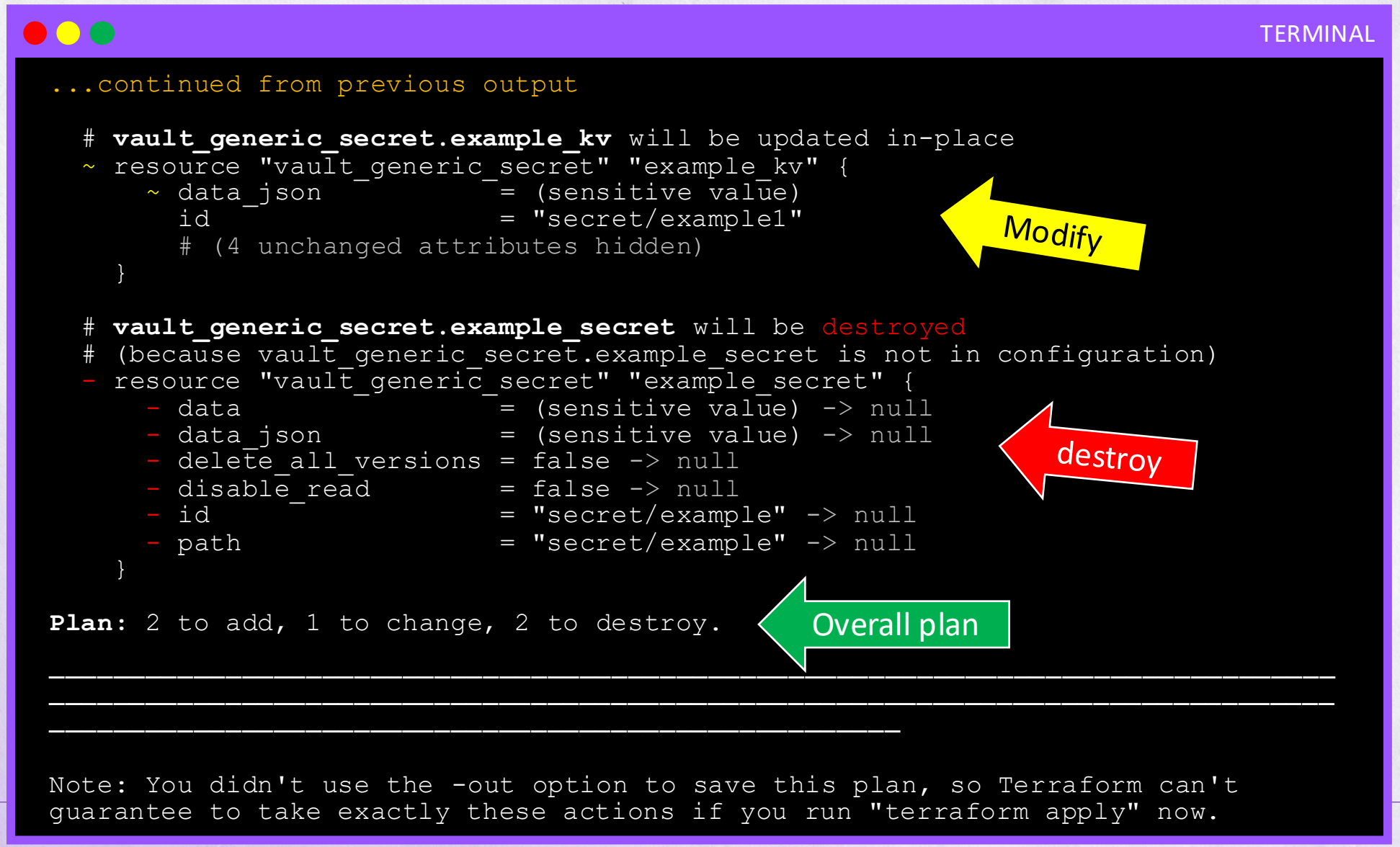


Refreshes State

Create new resource

Replaced

terraform plan



Saving a Plan

Guaranteeing Execution for Review and Approval Workflows

Benefits of Saving Plans

The exact actions in the saved plan will be executed, no surprises

Review & Approval Workflows

Save plan, have it reviewed, then apply later with confidence

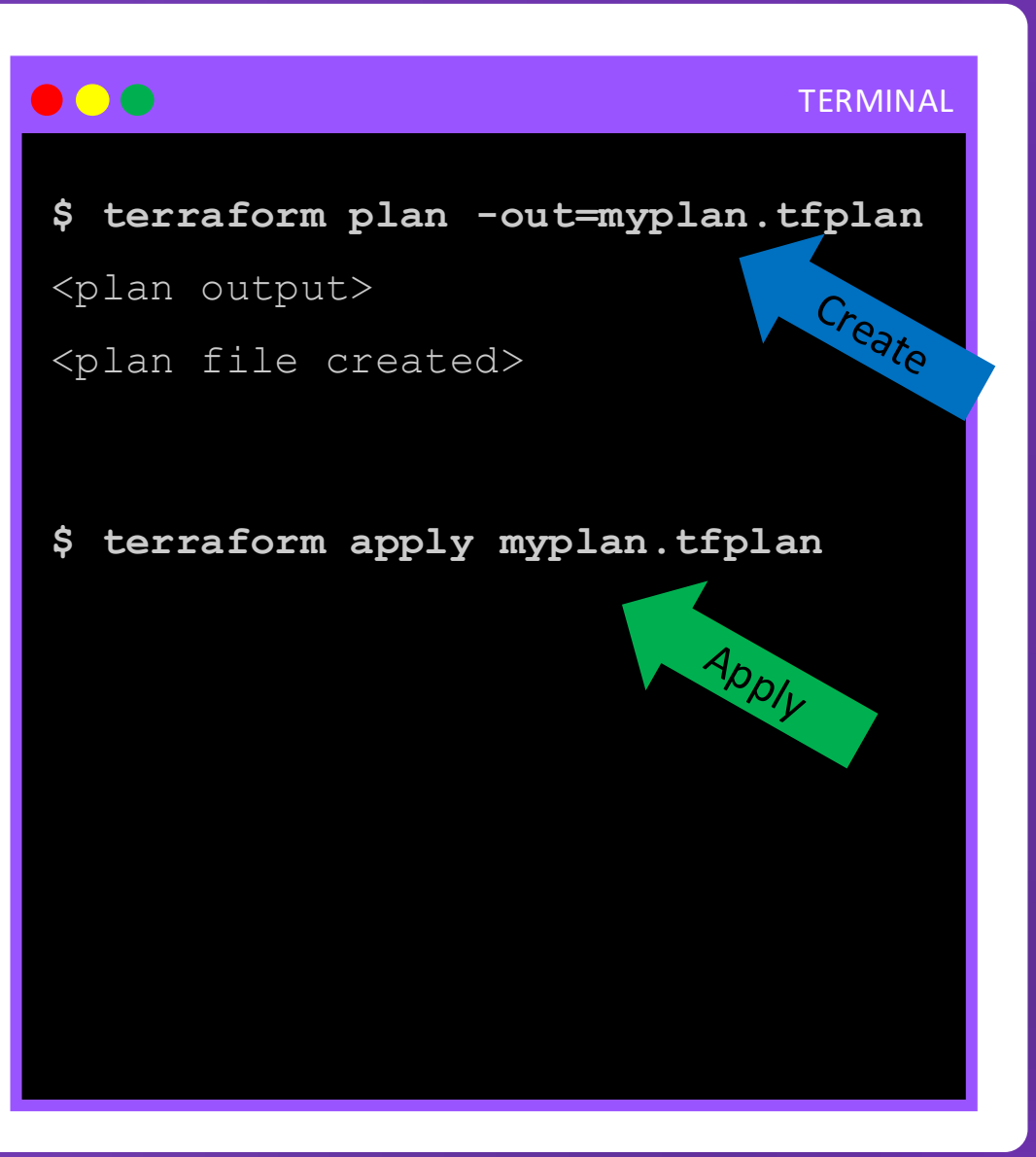
How to Save a Plan

```
$ terraform plan -out=myplan.tfplan
```

(file name can be anything you choose)

How to Apply a Saved Plan

```
$ terraform apply myplan.tfplan
```

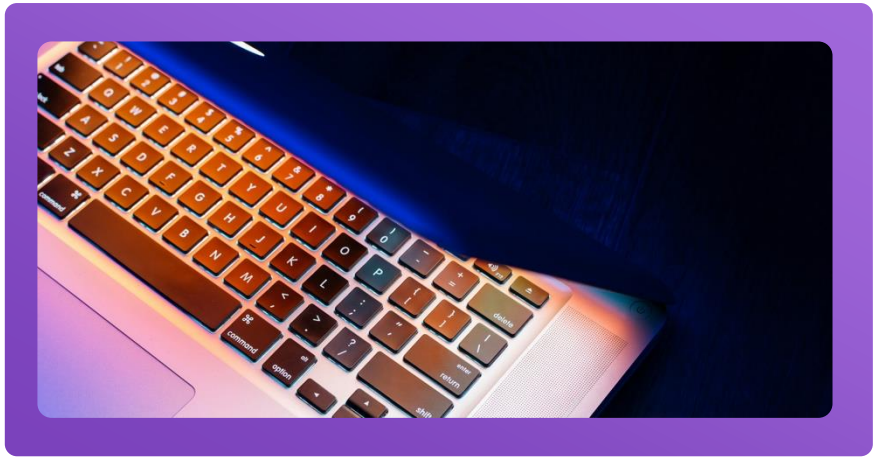


Resource Graph

So what actually happens when you run a Terraform plan?

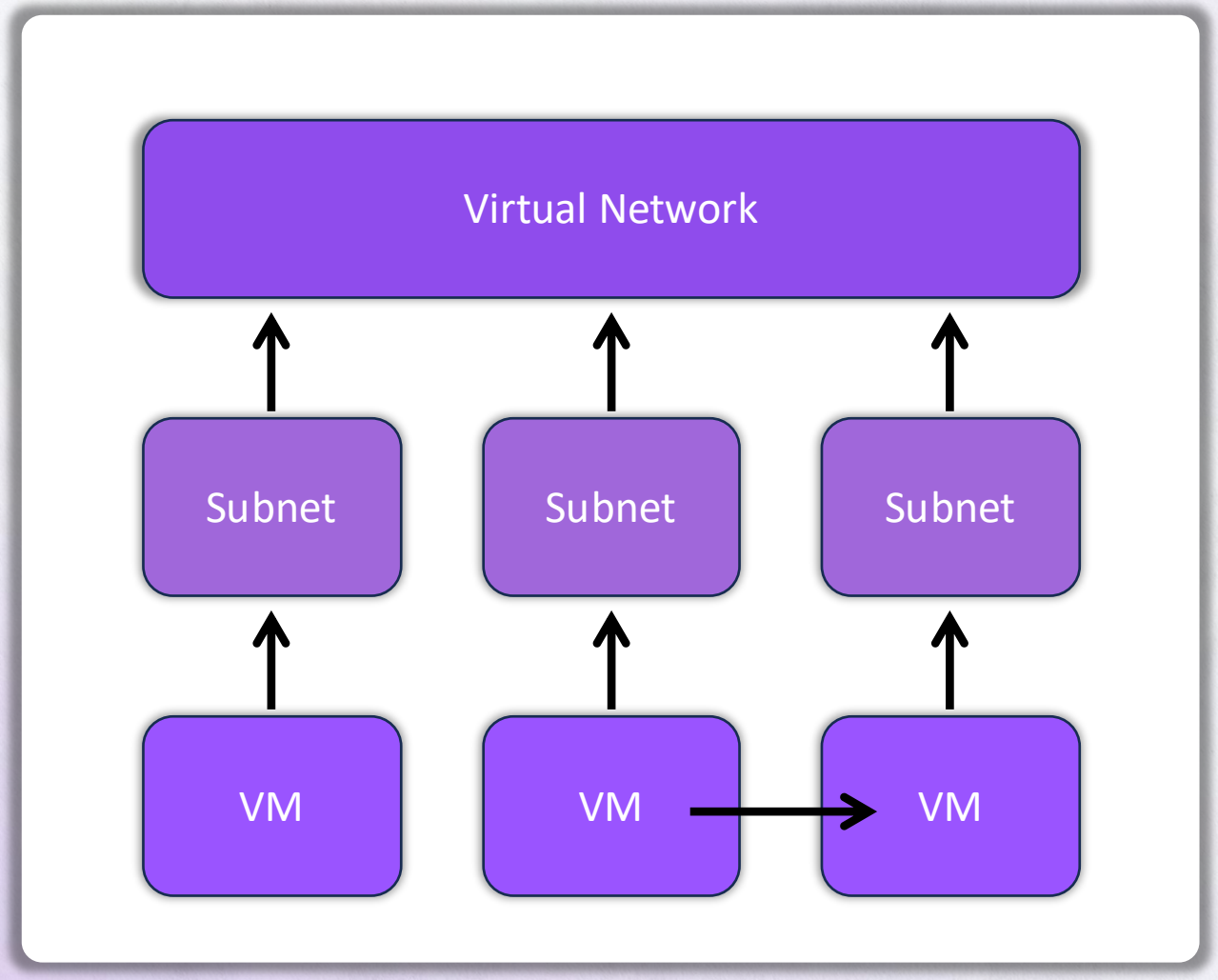
Terraform automatically builds a dependency graph from your configuration to determine the order in which to create, update, or destroy resources.

- Resource order in your .tf files doesn't matter
- Resource graph is how Terraform achieves parallel execution
- Foundation for understanding resource dependencies



Key Concept

The graph is built from resource references - when one resource refers to another's attributes, Terraform automatically creates a dependency.



Dependencies

Two types of dependencies:

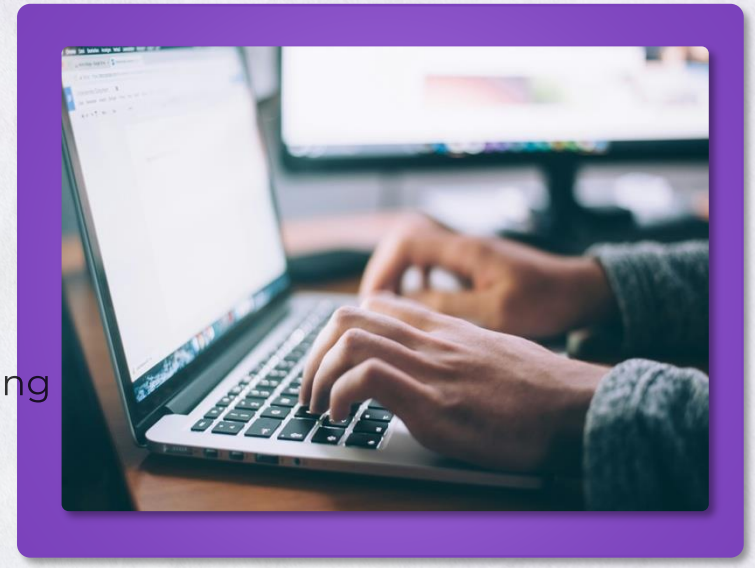
- **Implicit Dependencies (automatic)** – a reference to another resource that Terraform already knows about
- **Explicit Dependencies (Manual)** – Used when there's a dependency that Terraform can't automatically detect from resource references alone. Use `depends_on` for create an explicit dependency

Parallel Resource Operations

How Terraform Walks the Graph

Terraform Processes the Graph with Parallel Execution

Processes nodes as soon as their dependencies are satisfied. Non-dependent resources are created/modified simultaneously



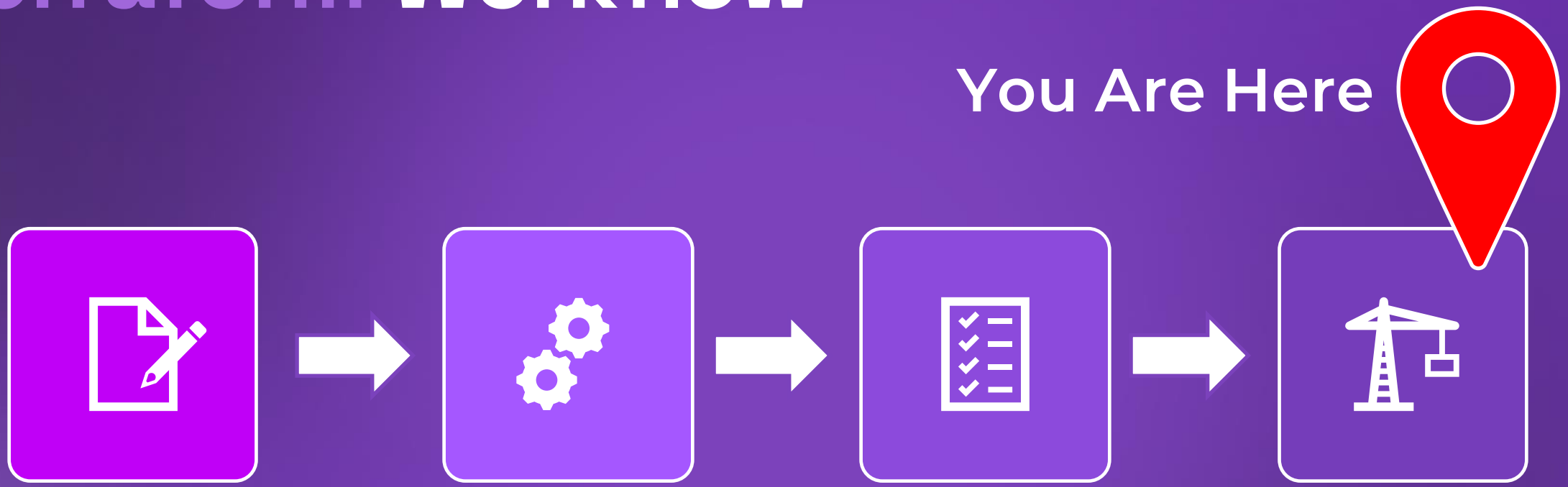
Default Configuration

By default, Terraform manages **10** operations in parallel but can be modified using the `-parallelism` flag (`terraform apply -parallelism=5`)

When to Change Parallelism

If you're running into API rate limiting issues, or for debugging/troubleshooting purposes. Most users never need to change this setting.

Terraform Workflow



`terraform apply`

`terraform apply` **tells Terraform to provision real infrastructure**

What Does It Do?

It takes your configuration and creates the resources as defined in your configuration. This is where Terraform makes changes to your infrastructure.

Refreshes State

`terraform apply` runs a plan first to compare your existing infrastructure to your desired state. By default, Terraform will prompt you to proceed with the proposed changes.

When Do You Use It?

Anytime you want to make changes to your infrastructure. This includes adding, modifying, or destroying real-world resources.

Understanding Apply Operations

Understanding what happens behind the scenes when Terraform modifies your infrastructure

State Locking

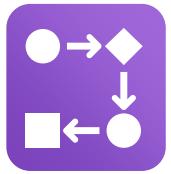
Terraform locks the state file during apply to prevent concurrent modifications.

State Updates

State file is updated after each resource is successfully created/modified/destroyed.

No Automatic Rollback

If a resource fails during apply, Terraform does not rollback previous successful changes. They remain in place.



Respects Dependencies

Commit to Version Control and Managed Automatically by Terraform.

terraform apply

Standard Apply →

TERMINAL

```
$ terraform apply
Confirm (Y/N):
```

Apply a Saved Plan →

TERMINAL

```
$ terraform apply myplan.tfplan
```

Apply & Auto Approve →

TERMINAL

```
$ terraform apply -auto-approve
(skips confirmation prompt - use carefully)
```

`-input=false` is also helpful for automation/CI/CD pipelines

Best Practices for `terraform apply`

1 Before You Apply

- ✓ Always review the plan
- ✓ Ensure You're in the Right Directory
- ✓ Test in Non-Production

2 During Apply

- ✓ Read the Plan Carefully
- ✓ Watch for Unexpected Changes
- ✓ Be Patient with Large Applies

3 After You Apply

- ✓ Verify the Changes
- ✓ Commit Your Configuration
- ✓ Document Significant Changes

`terraform destroy`

`terraform destroy` **t**erminates resources managed by Terraform in the current workspace or working directory.

Using the Destroy Command

Explicitly destroys all infrastructure being managed by Terraform. This operation does prompt for confirmation before proceeding. Shows destruction plan first.

Removing Individual Resources from your Configuration

Delete resource blocks from your .tf files and then run `terraform apply` Terraform detects removed resources and destroys the real-world resource.

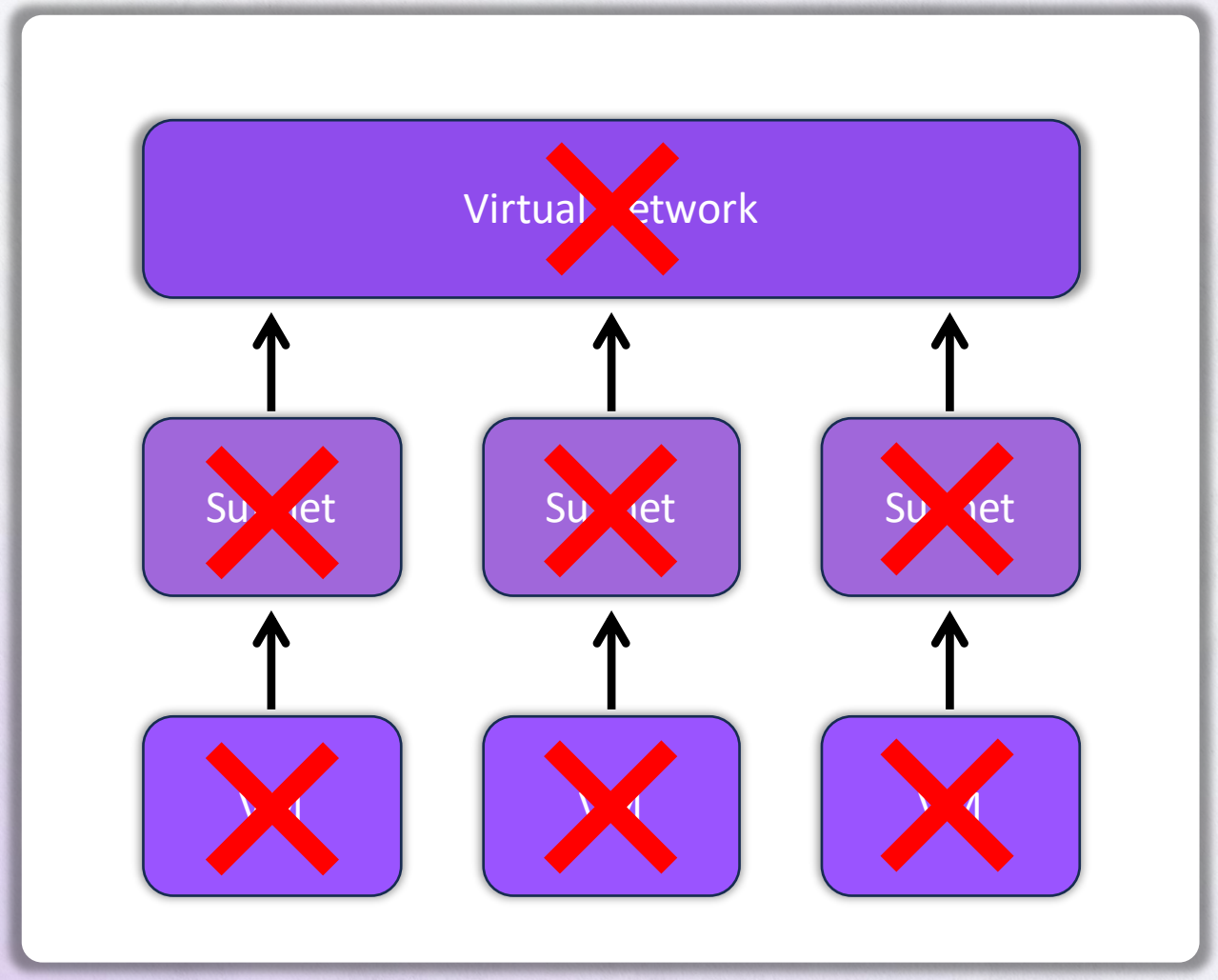
State File Updates

Once a resource is destroyed, Terraform removes it from the state file as well since it is no longer being managed.

WARNING!

Warning: Destroy operations are irreversible.

Always review the plan carefully



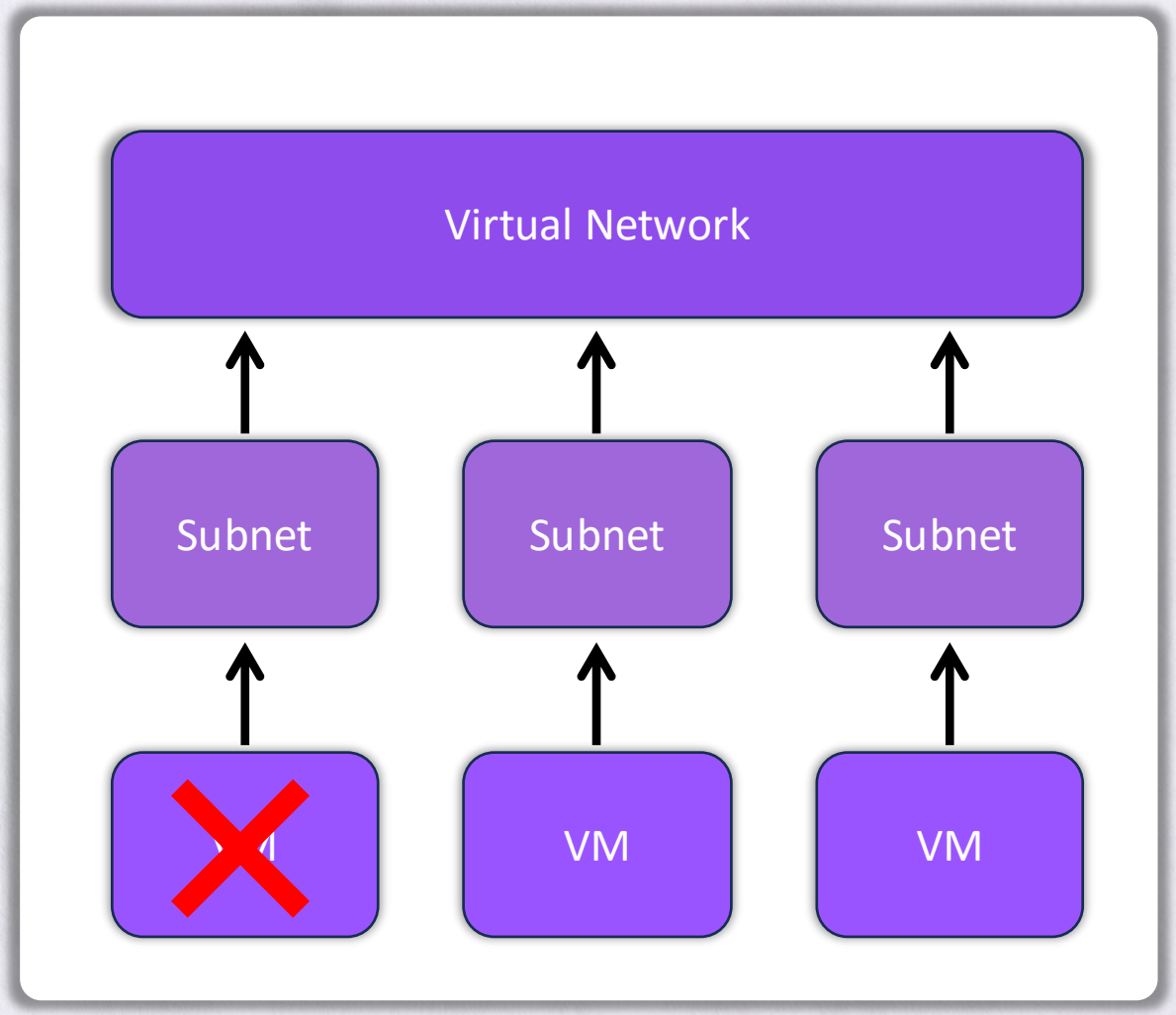
Destroying Resources

The default `terraform destroy` command will destroy all resources being managing by Terraform.

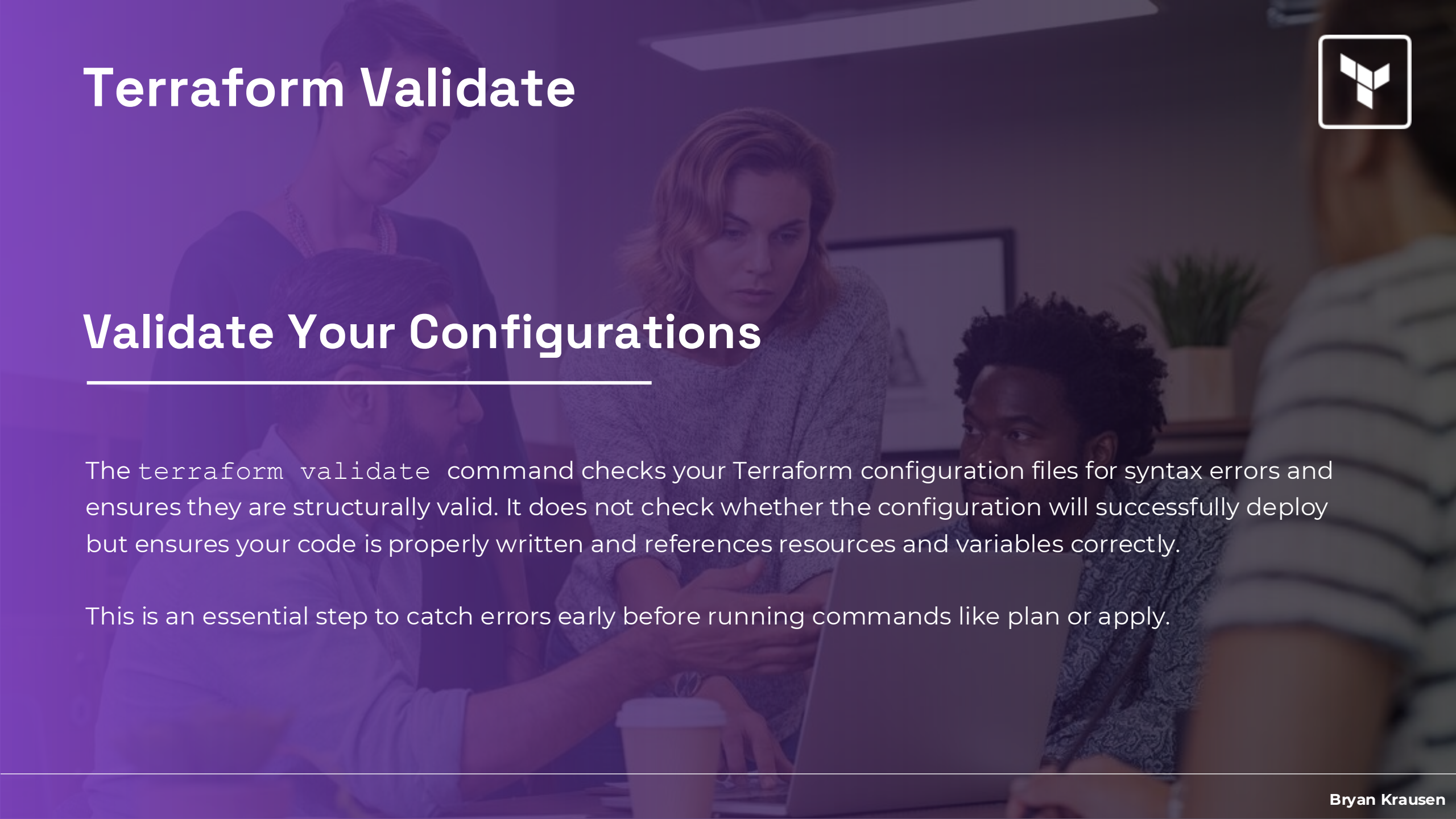
Destroying a Single Resource

You can target specific resources, if needed using the destroy command.

```
terraform destroy -target=<resource_name>
```



Terraform Validate

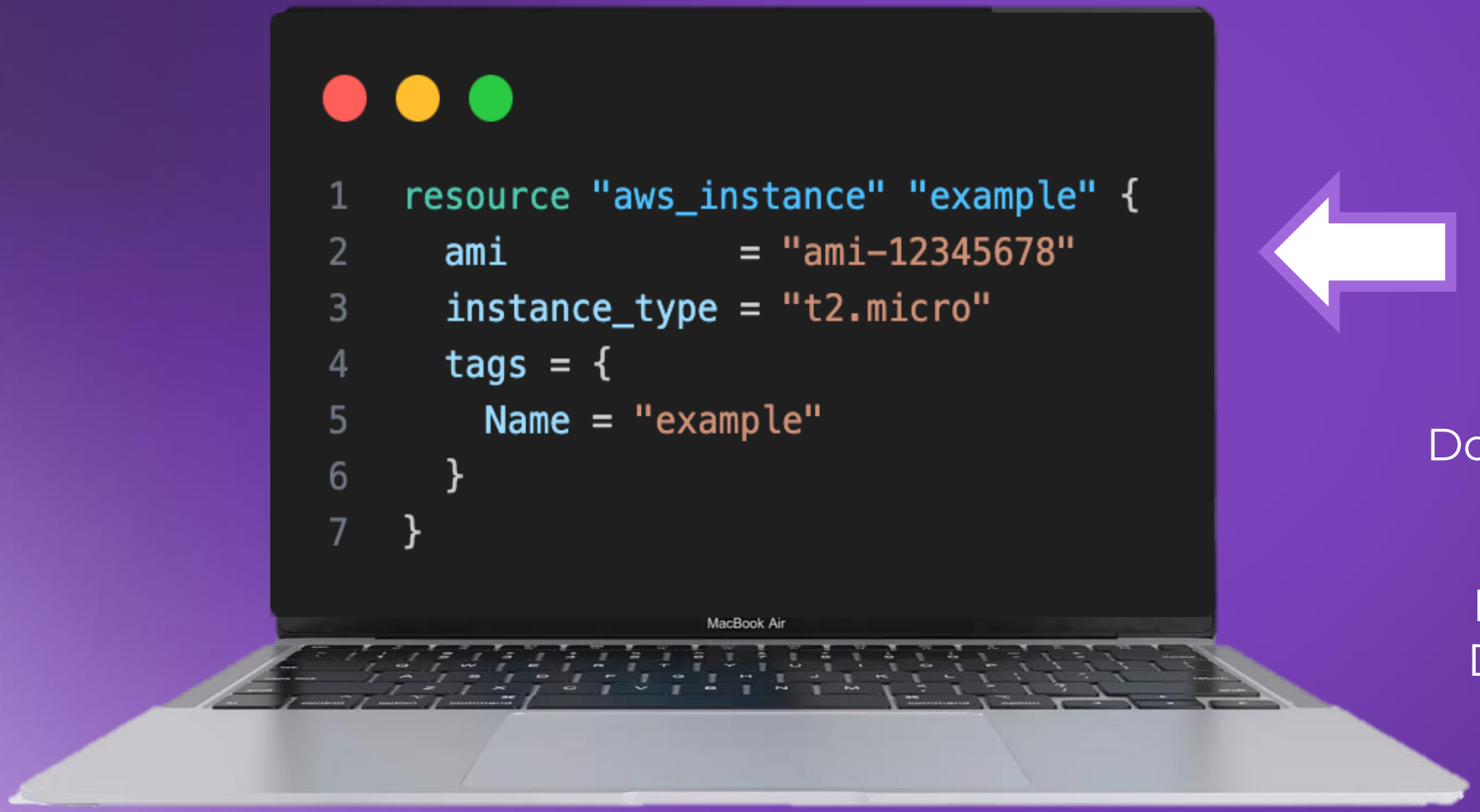


Validate Your Configurations

The `terraform validate` command checks your Terraform configuration files for syntax errors and ensures they are structurally valid. It does not check whether the configuration will successfully deploy but ensures your code is properly written and references resources and variables correctly.

This is an essential step to catch errors early before running commands like plan or apply.

Technical Details of Validate



`terraform validate`

Does Not Communicate with Providers

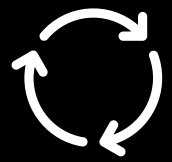
Because of That, Validate Doesn't Catch Every Error

Requirements

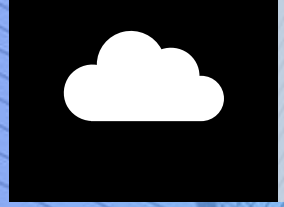
Initialize	Validate	Plan	Apply	Destroy
`terraform init`	`terraform validate`	`terraform plan`	`terraform apply`	`terraform destroy`

You MUST initialize your working directory first before running `terraform validate`

SUMMARY



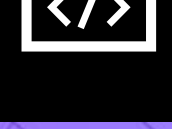
Terraform workflow begins with `terraform init`, which sets up the project and downloads necessary providers and modules.



`terraform plan` generates an execution plan, allowing you to preview changes before applying them

`terraform apply` is then used to make the planned changes to your infrastructure.

By default, Terraform will ask for confirmation before making any changes to real-world infrastructure



Each step in the workflow is crucial for managing infrastructure predictably and efficiently.

Following this workflow ensures consistency and control over infrastructure changes.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Using the Terraform CLI

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

What is the Terraform CLI?

The Terraform CLI is the primary way to interact with Terraform and execute the workflow steps

Unified Workflow

The CLI ensures consistency and repeatability across your workflow. – regardless of what platform that you're using Terraform

Command Interface

A command-line tool to manage infrastructure through commands like terraform init, plan, and apply

Consistent Pattern

Follows a consistent pattern:

```
terraform <command> [options]

tofu <command> [options]
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Getting to Know the Terraform CLI

Terraform commands follow this structure:

→ `terraform <subcommand> [options or flags]`

```
terraform   plan   -out=planfile
```

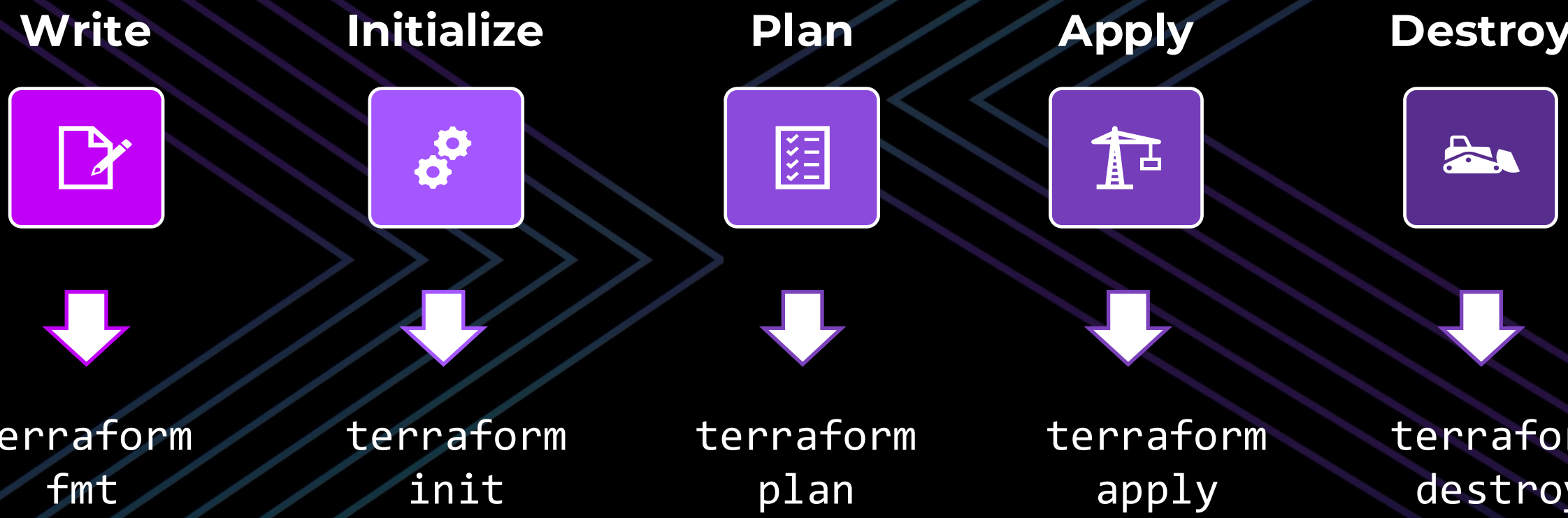
Base command to invoke the CLI

Subcommand

Optional Flag

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

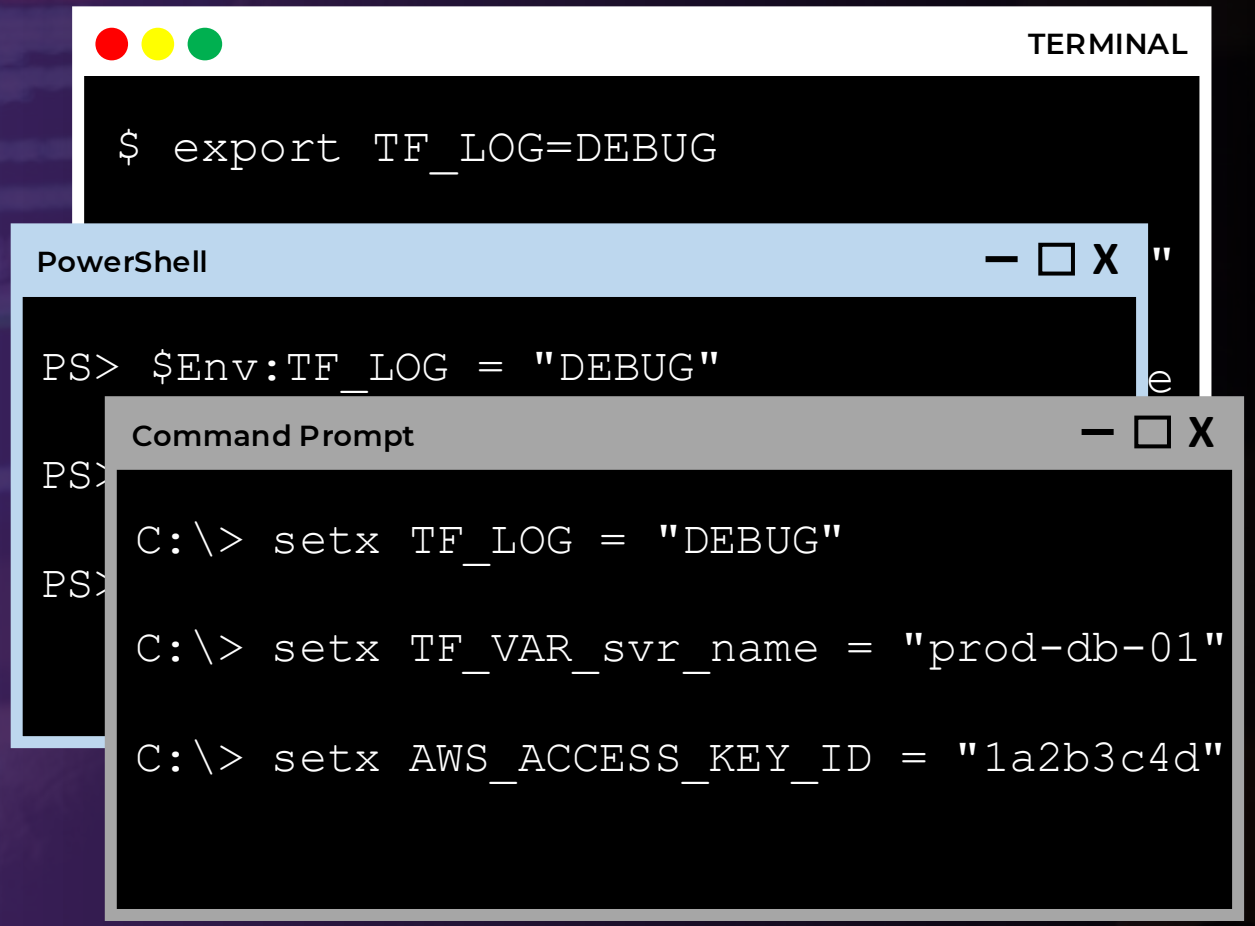
Mapping the Workflow to Terraform Commands



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Using Environment Variables with the CLI

- Environment variables provide a way to pass configuration settings and credentials to Terraform without hardcoding them into your files.
- They increase security by keeping sensitive data like API keys out of `.tf` files.



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Making the Most of the Terraform CLI

Auto-Complete

Tab completion helps you quickly fill in subcommands, flags, and file paths by pressing the **Tab** key.

It reduces typing errors and ensures you use valid options for Terraform commands.

Available for most shell environments (like Bash or Zsh) after enabling it with terraform -install-autocomplete.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Making the Most of the Terraform CLI

Using the `help` feature

TERMINAL

```
$ terraform --help
Usage: terraform [global options] <subcommand> [args]

The available commands for execution are listed below.
The primary workflow commands are given first, followed by
less common or more advanced commands.

Main commands:
  init          Prepare your working directory for other commands
  validate      Check whether the configuration is valid
  plan          Show changes required by the current configuration
  apply         Create or update infrastructure
  destroy       Destroy previously-created infrastructure
...
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Making the Most of the Terraform CLI

Using the **`help`** feature

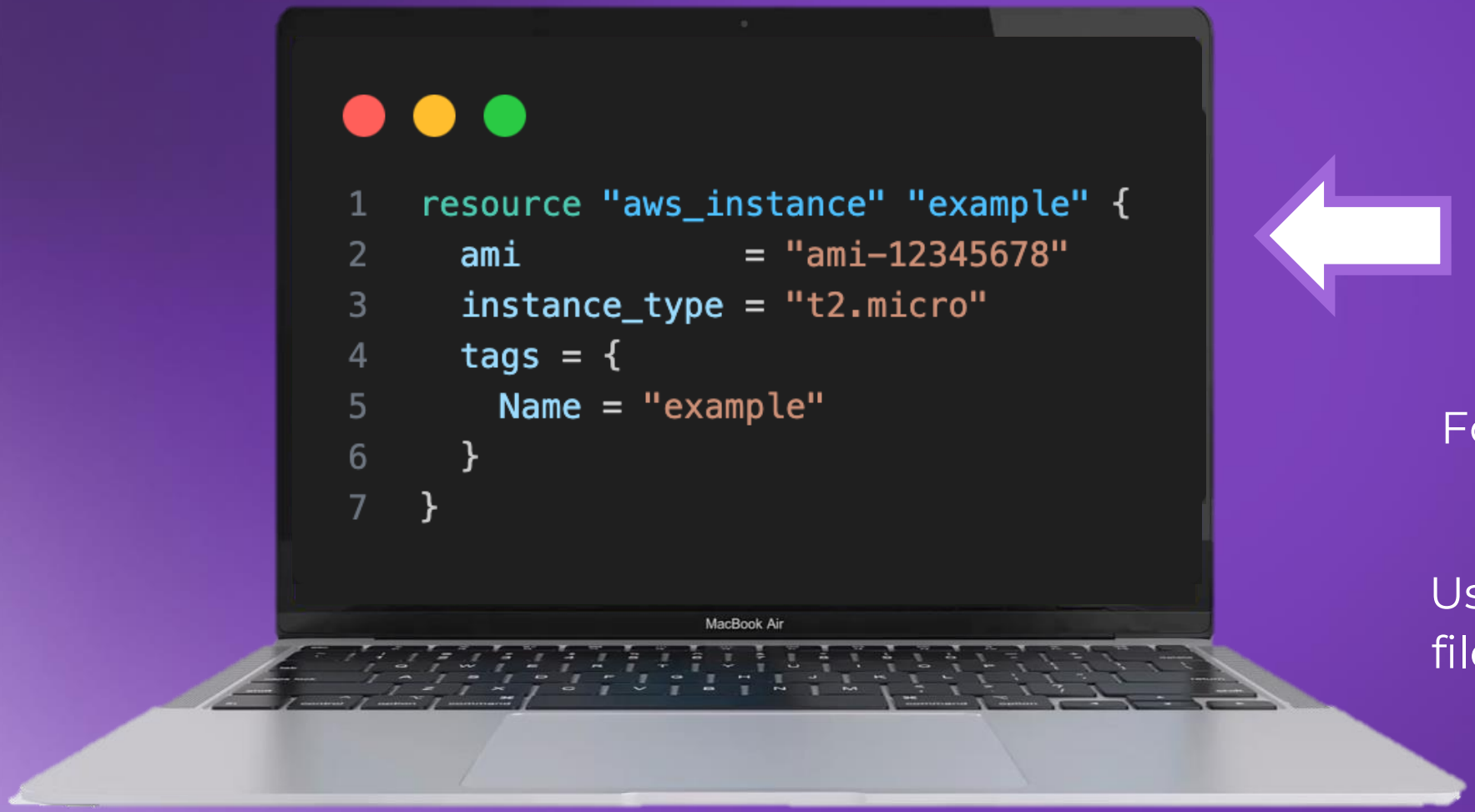
TERMINAL

```
$ terraform plan --help
Usage: terraform [global options] plan [options]

  Generates a speculative execution plan, showing what actions Terraform
  would take to apply the current configuration. This command will not
  actually perform the planned actions.

  You can optionally save the plan to a file, which you can then pass to
  the "apply" command to perform exactly the actions described in the plan.

Plan Customization Options:

  The following options customize how Terraform will produce its plan. You
  can also use these options when you run "terraform apply" without passing
  it a saved plan, in order to plan and apply in a single command.

  -destroy           Select the "destroy" planning mode, which creates a plan
                     to destroy all objects currently managed by this
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Making the Most of the Terraform CLI

Easily Format Your Code To a Canonical Format and Style



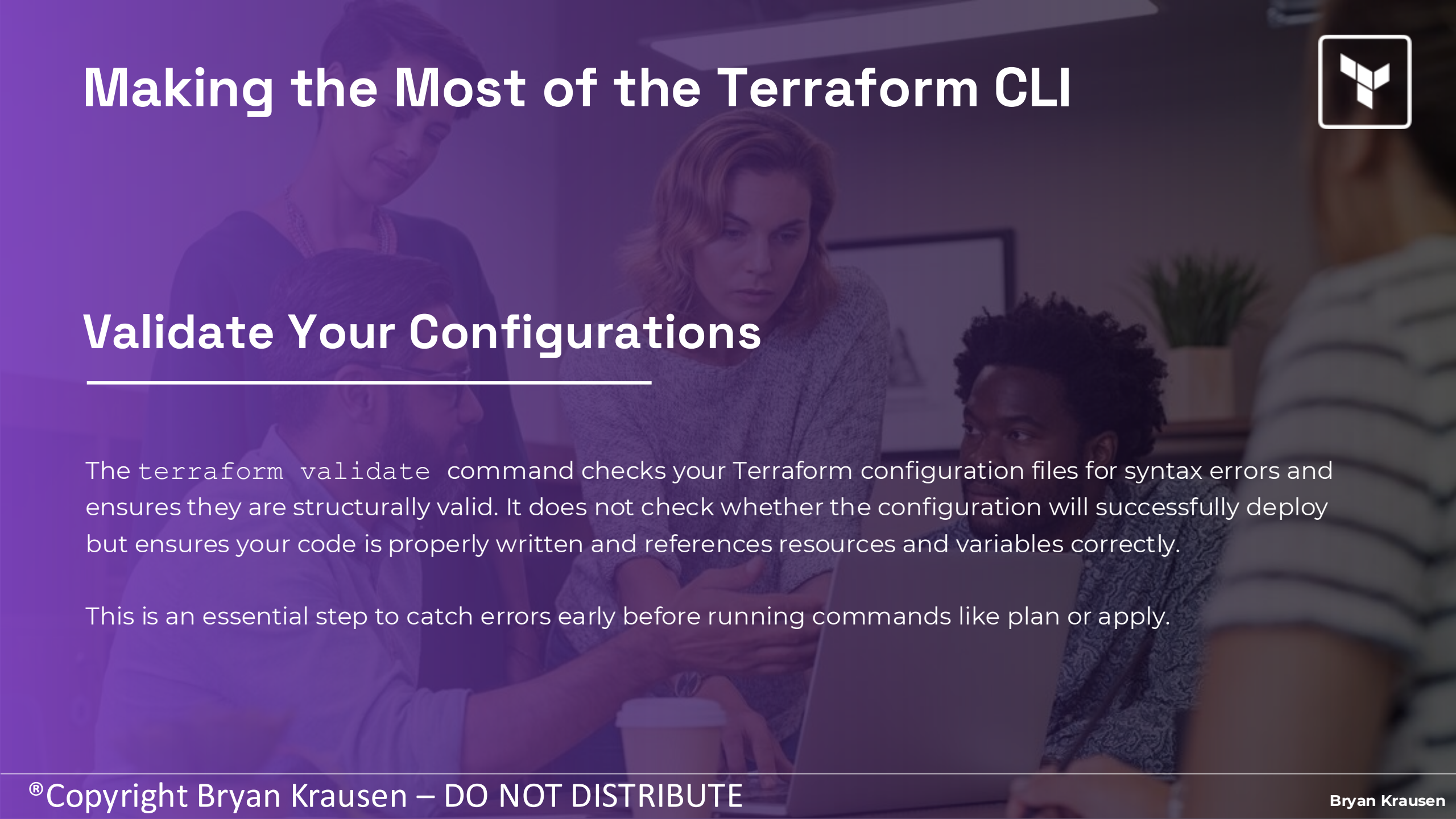
`terraform fmt`

Format all Terraform files in the current directory

Use `-recursive` to format files in subdirectories as well

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Making the Most of the Terraform CLI



Validate Your Configurations

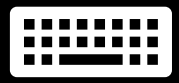
The `terraform validate` command checks your Terraform configuration files for syntax errors and ensures they are structurally valid. It does not check whether the configuration will successfully deploy but ensures your code is properly written and references resources and variables correctly.

This is an essential step to catch errors early before running commands like plan or apply.

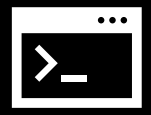
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

SUMMARY

Use `terraform <subcommand>` to manage each step of the workflow

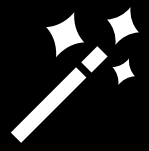


Enable `terraform -install-autocomplete` to speed up typing and reduce errors



Customize commands with options and flags depending on the subcommand you're using.

Run `terraform validate` to catch syntax errors early



Use `terraform fmt` to ensure clean and consistent formatting

Pass sensitive data and reusable defaults securely via `TF_VAR_*` and other variables.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

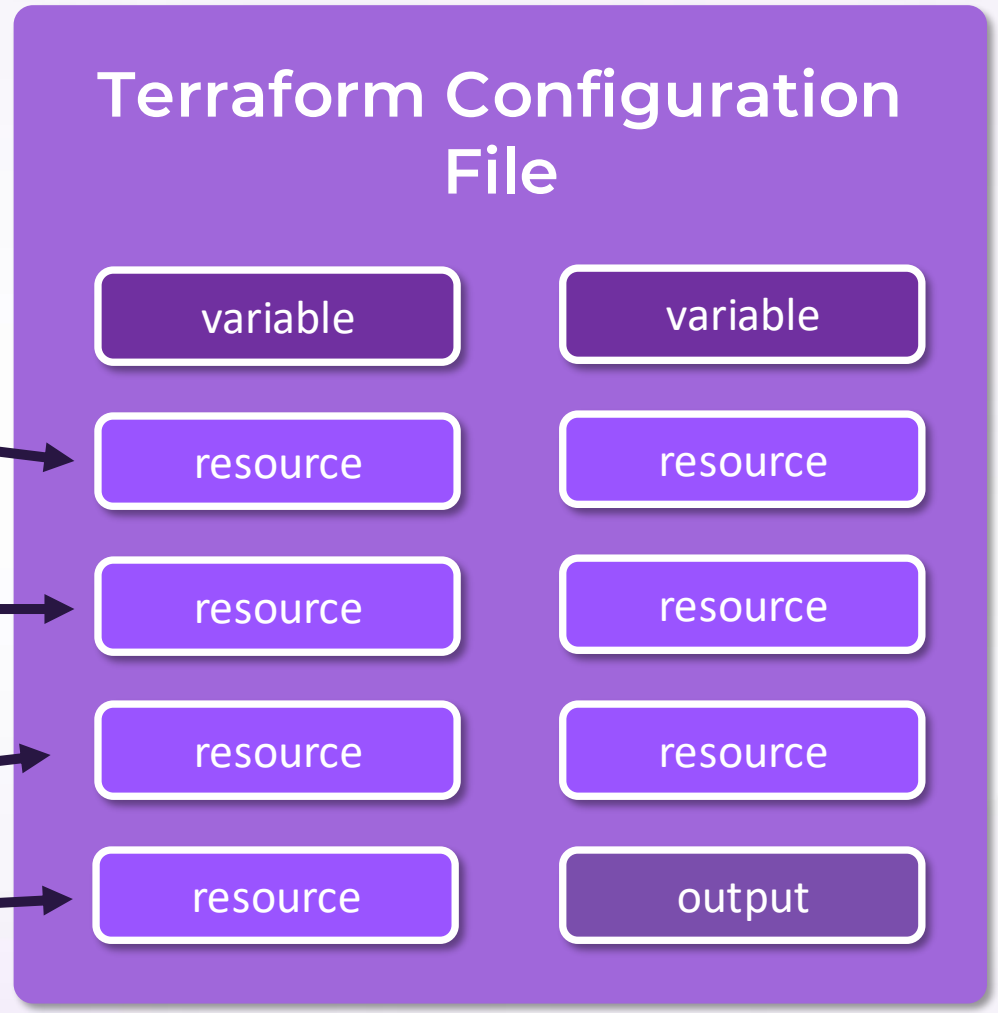
File Structure and Organization

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Organizing Code

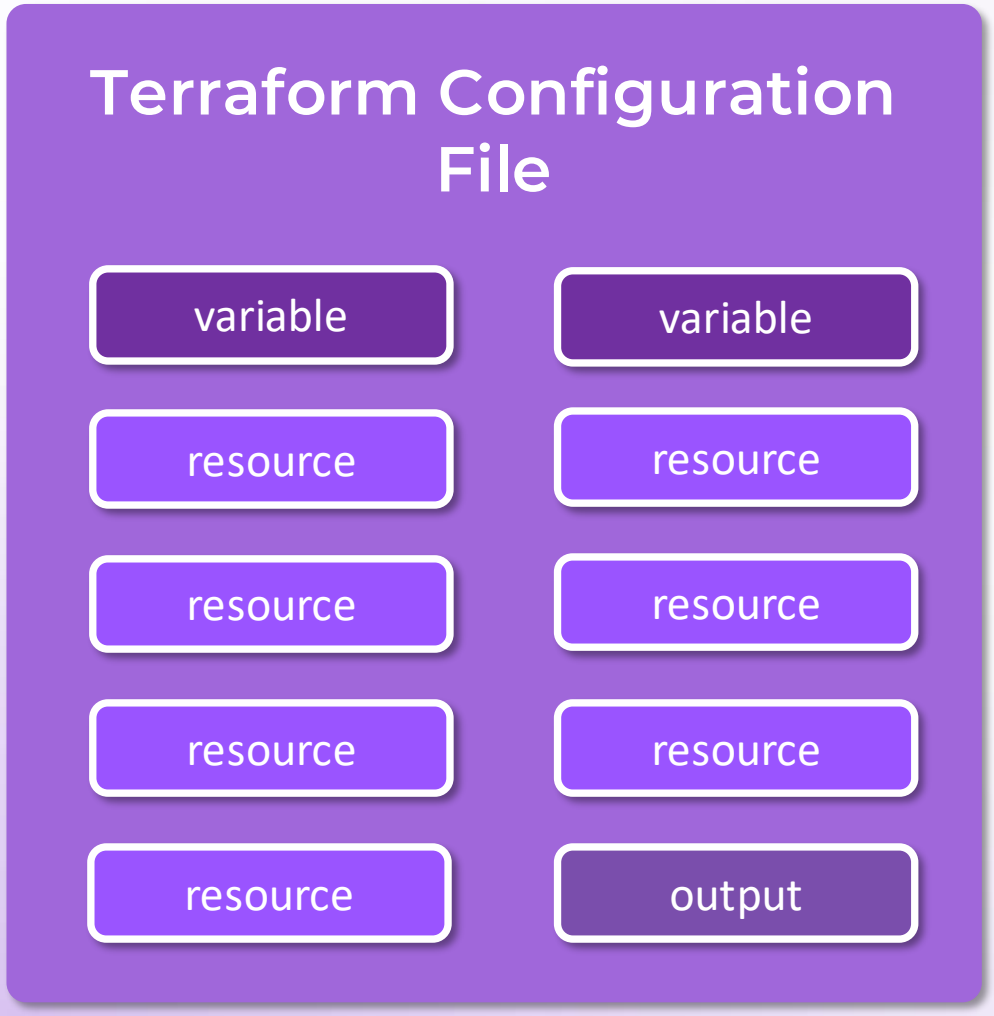
Infrastructure Requirements:

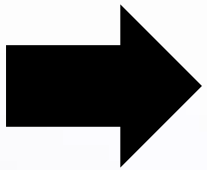
- ❑ S3 Bucket
- ❑ Virtual Machine
- ❑ Subnet
- ❑ Firewall Configuration
- ❑ DNS Record
- ❑ Load Balancer
- ❑ Kubernetes Cluster

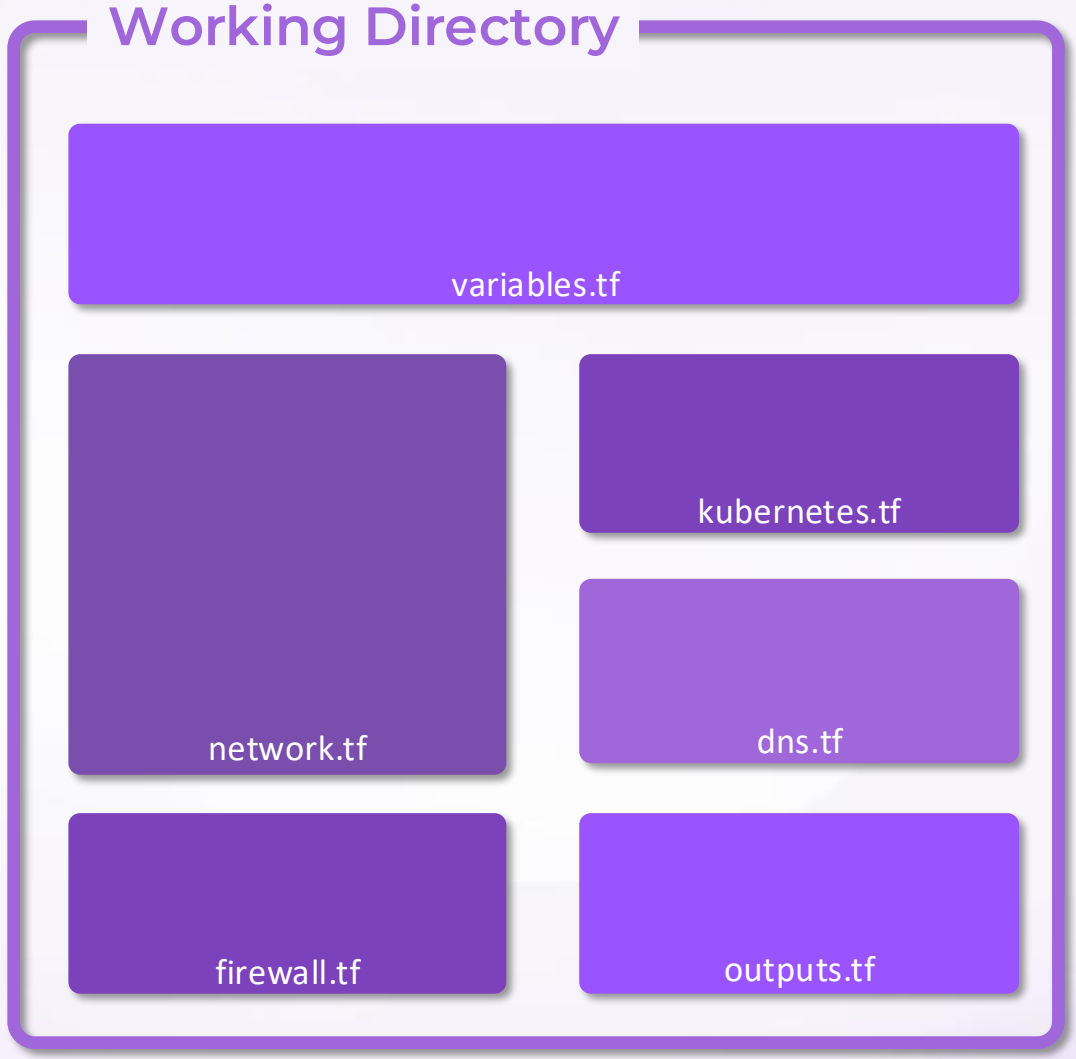


®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Organizing Code

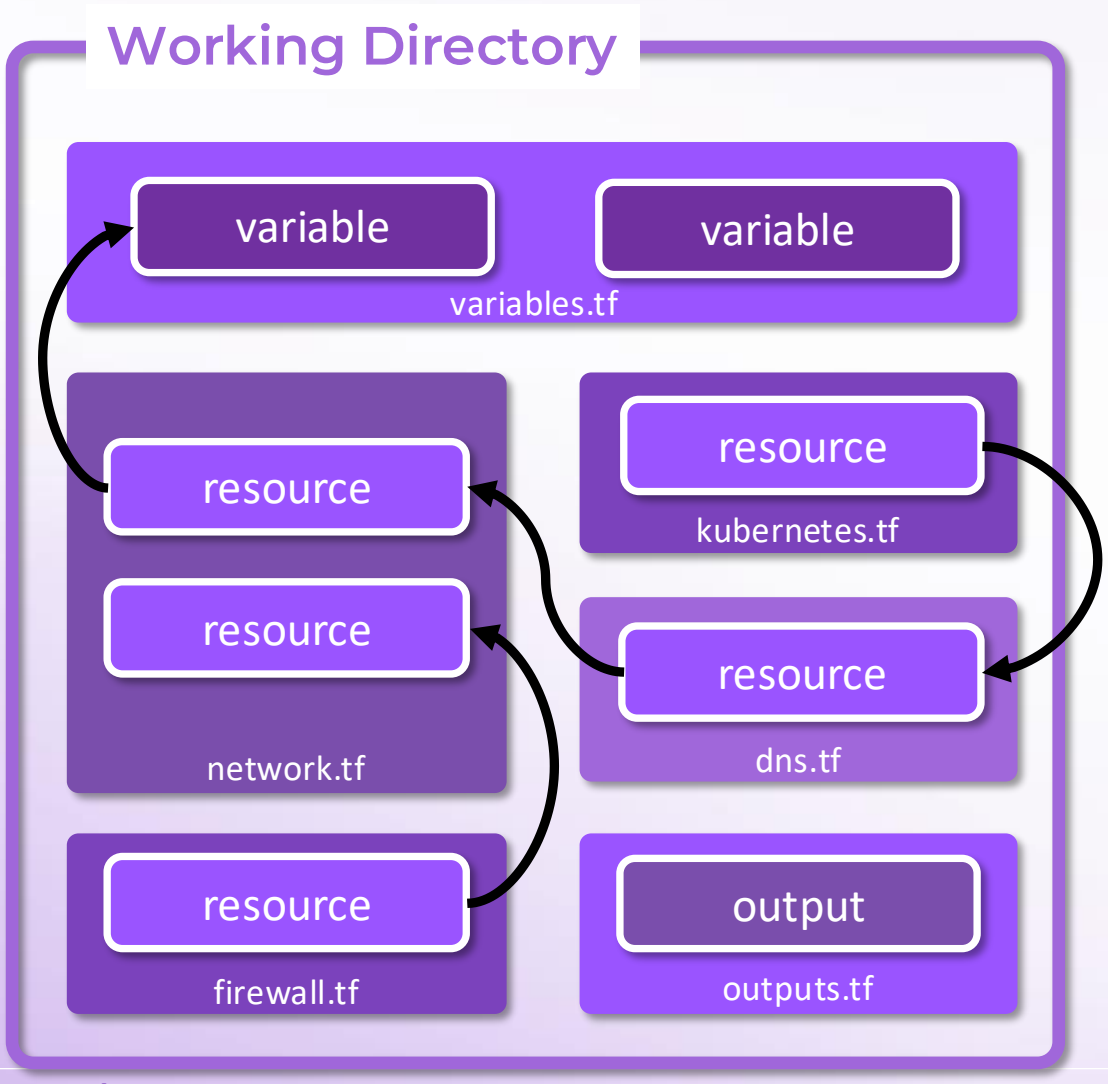




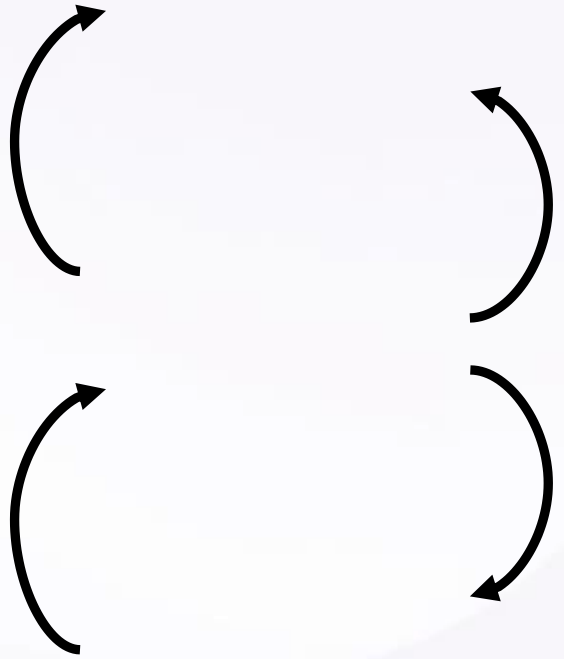


®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Executing Code



Terraform Processes All Files Together



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Common Terraform Files

main.tf
Commonly used for primary infrastructure components

variables.tf
Variable definitions used throughout your infrastructure

outputs.tf
Output blocks to output values to the terminal or to use for inputs to other modules

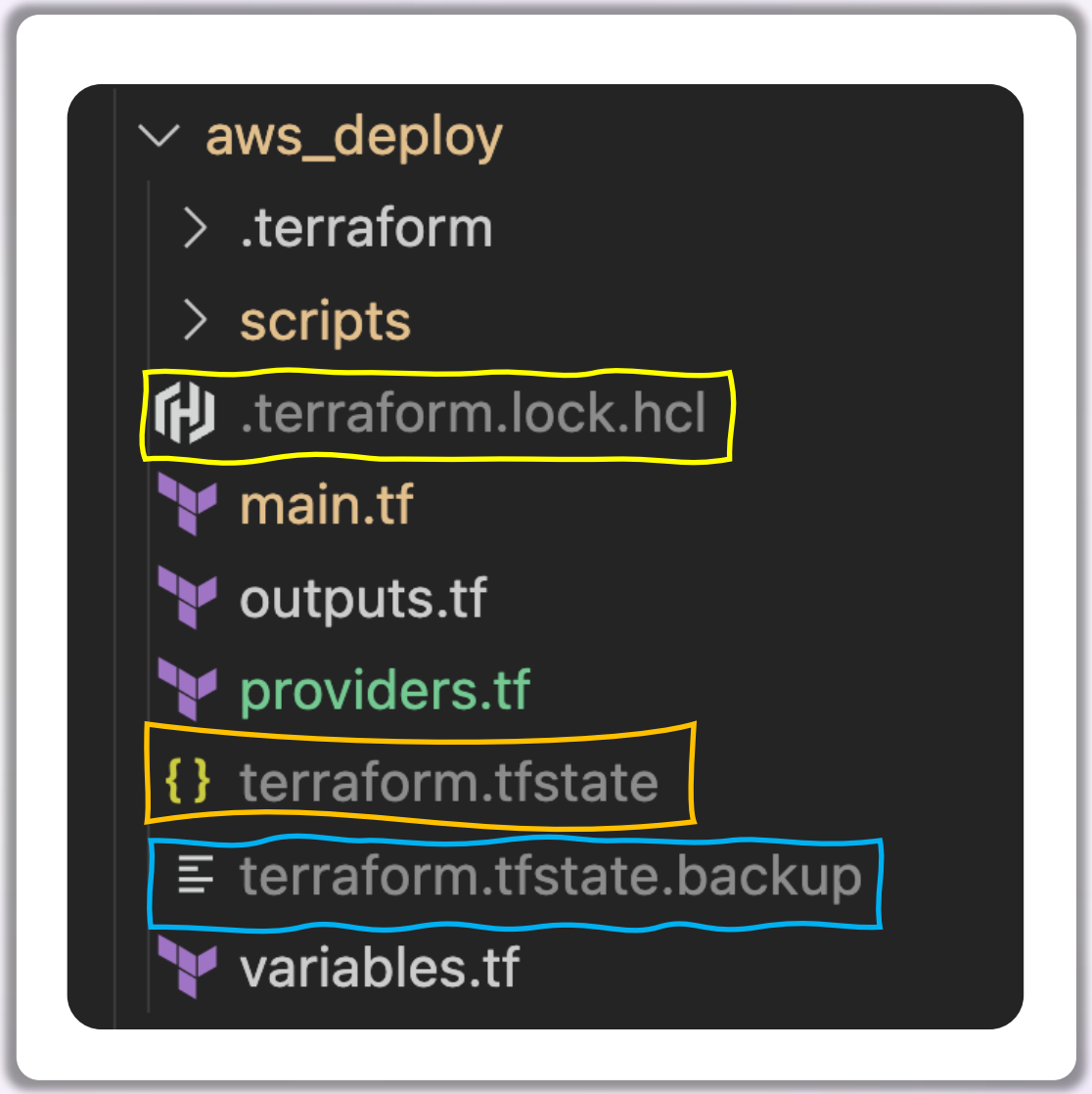
providers.tf
Provider configurations and requirements

terraform.tfvars
Define the values for your variables – usually ignored by .gitignore

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

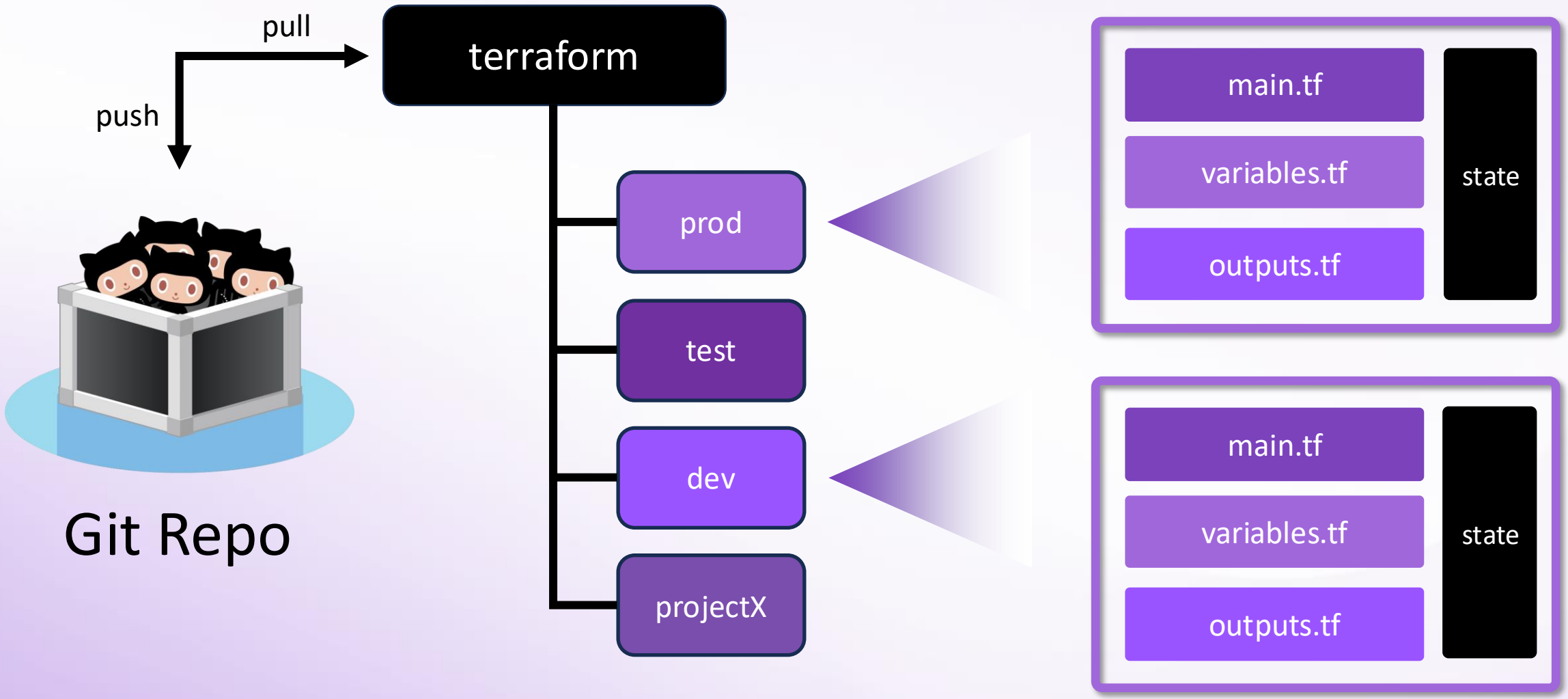
Additional Files

- `terraform.tfstate`
  The file that Terraform uses to store state
- `terraform.tfstate.backup`
  Backup of the previous state file before Terraform writes anything new to the state file
- `.terraform.lock.hcl`
  Used to track and select provider versions
- `.gitignore`
  Instructs Git to ignore files included in this file



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Working Directory

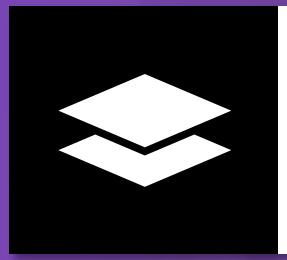


®Copyright Bryan Krausen – DO NOT DISTRIBUTE

SUMMARY

Terraform only processes the files in the current working directory unless you reference other files

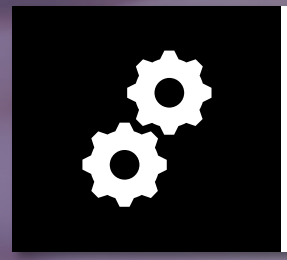
While not required, there are common file names for Terraform configurations



You can organize your files across multiple .tf files

Terraform creates and manages files for state and other configurations

Terraform combines all the files in the working directory when executing



Use subdirectories to break up your Terraform to better organize and reduce the blast radius of changes

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Understanding the Terraform Blocks

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

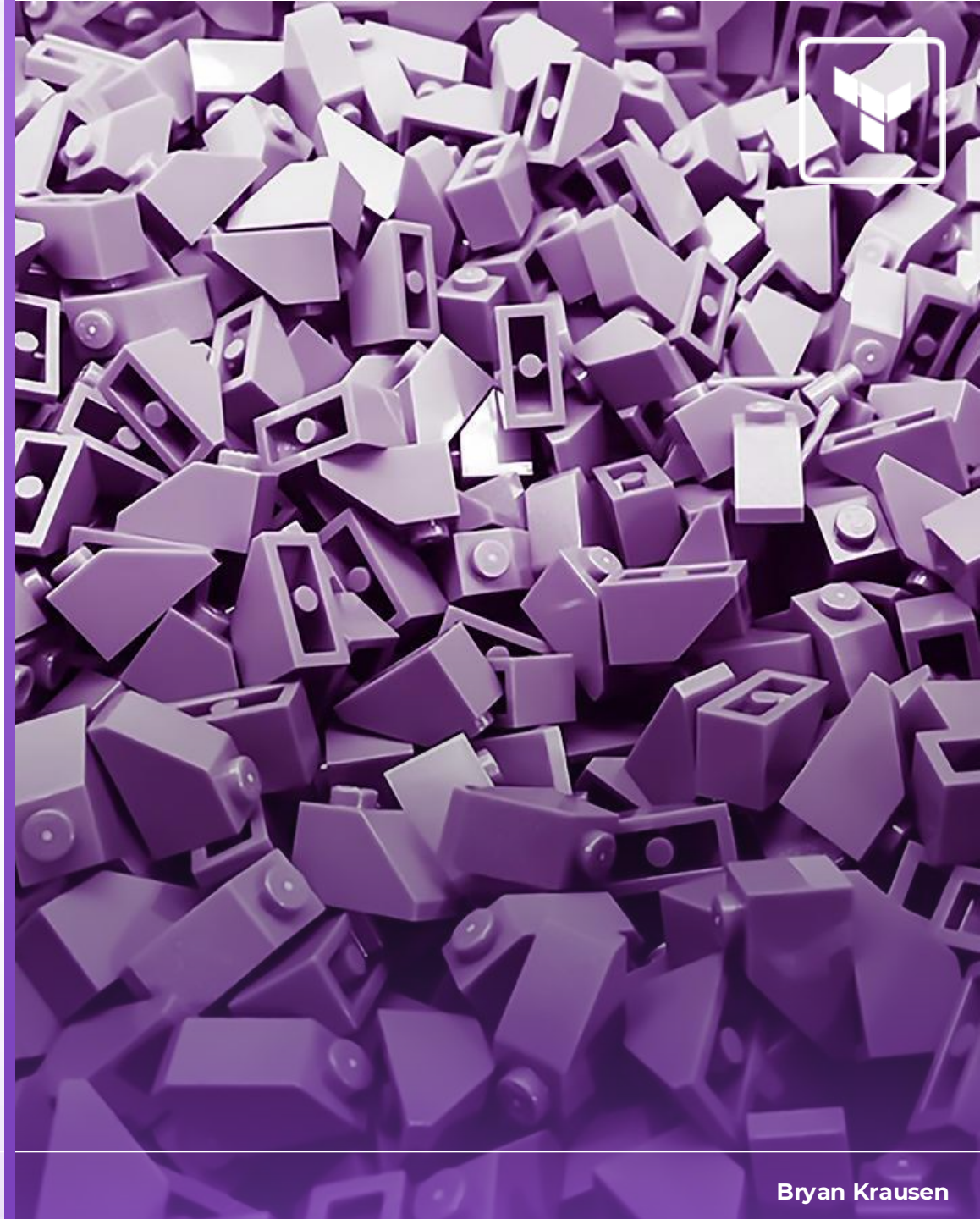
Terraform Blocks

The Building Blocks or Infrastructure

Clear Structure
Each component has a specific purpose, making it easy to organization and understand your configurations

Flexible and Modular
Terraform is designed to be flexible. You don't need to use all the components to build infrastructure

Reusable Patterns
Using components thoughtfully helps create reusable, modular configurations that are easy to maintain and scale



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Block Types

provider

Connect Terraform to cloud platforms or services (e.g., AWS, Azure, GCP, VMware)

resource

The infrastructure that Terraform creates, updates, or deletes (e.g., VMs, storage)

data

Retrieve information about existing resources to use in your configuration

variable

Make code reusable by defining values that can be customized for each run

output

Display or share essential information about resources after deployment

terraform

Define specific settings or resources in a configuration, like req. Terraform version

module

Reusable configurations that group related resources – makes it easy to share configs

import

Pull in existing resources into Terraform management

*not an exhaustive list of block types

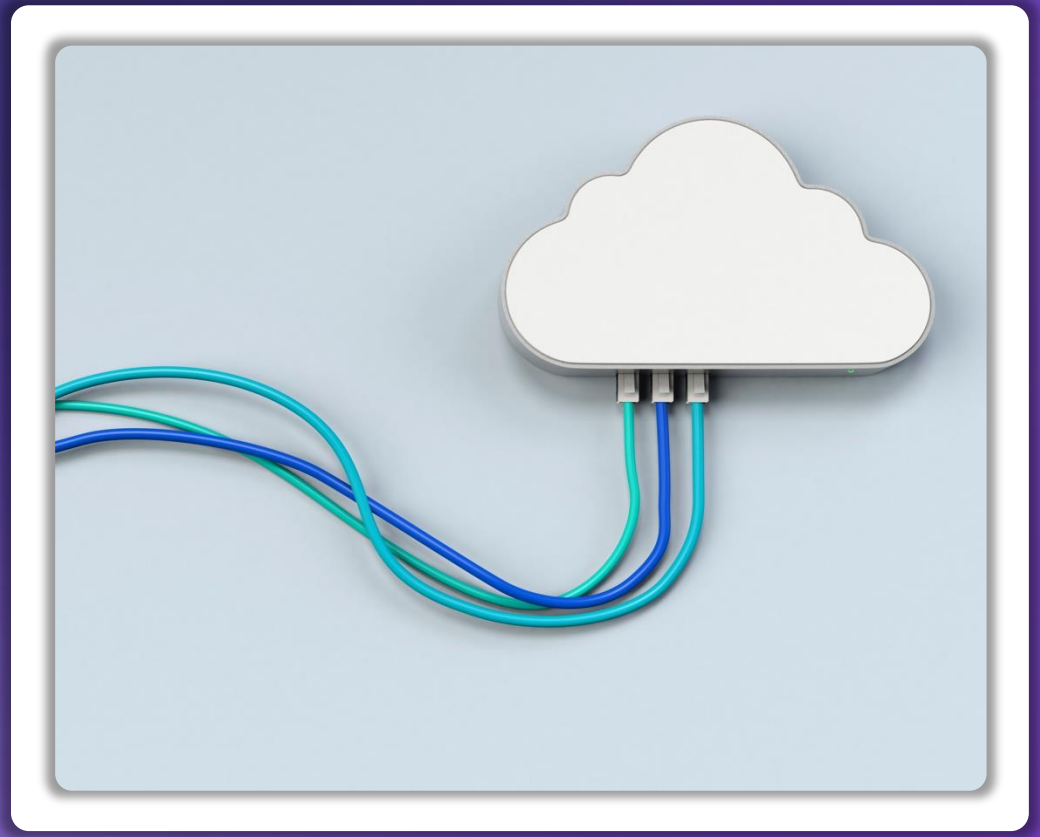
LECTURE

Provider Block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Providers

®Copyright Bryan Krausen – DO NOT DISTRIBUTE



Provider Block

The `provider` **block** is used to extend Terraform's functionality. It allows you to connect Terraform to any API-driven platform, such as cloud platforms, SaaS applications, on-premises hardware solutions, and more.

Provider blocks define where and what Terraform can manage, such as AWS for cloud resources or GitHub for repositories.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Provider Block

Plugins that extend the functionality of Terraform and enable it to interact with cloud providers, SaaS providers, and other API-supported apps.

Resources are Implemented by a Provider

Without providers, Terraform would not be able to provision or manage any infrastructure.

Who Publishes Providers?

HashiCorp maintains some providers, but some are managed by partners and the Terraform community.

Developed Separately From Terraform Core

This means functionality can be added/changed per version.

providers.tf

```
provider "aws" {
  region  = "us-east-2"
  profile = "prd-workload"
}


provider "azurerm" {
  features {}

  tenant_id       = "tenant-id"
  subscription_id = "sub-id"
  client_id       = "client-id"
  client_secret   = "client-secret"
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Breaking Down the Provider Block

providers.tf

```
provider "aws" {
  region  = "us-east-2"
  profile = "prd-workload"
}

provider "azurerm" {
  features {}

  tenant_id       = "tenant-id"
  subscription_id = "sub-id"
  client_id       = "client-id"
  client_secret   = "secret"
}
```

The name of the provider defined by the developer or maintainer

Arguments specific to the provider – found in documentation

Authentication arguments – should be provided via Environment Variables instead due to security concerns

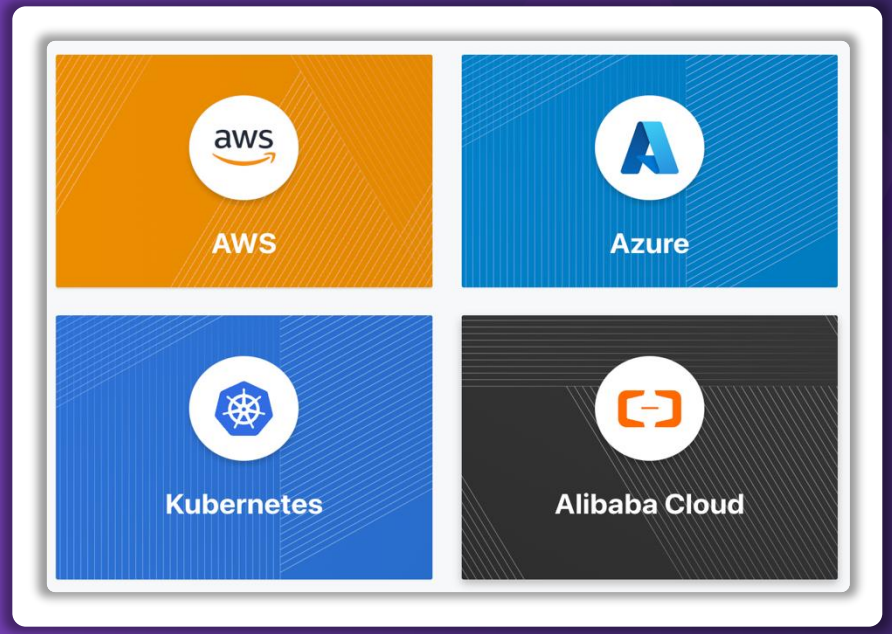
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Provider Block

- Some providers need credentials or access tokens to authenticate and manage resources on the target platform (e.g., AWS access keys, GitHub API token)
- A single provider block enables Terraform to manage multiple resources on that platform (e.g., a single AWS provider enables EC2 instances, S3 buckets, EKS clusters)

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Provider Block



- While providers are stored in GitHub, the documentation can be found on the Terraform Registry → registry.terraform.io
- When declared in your configuration, the provider is downloaded to the machine executing Terraform

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

`provider` Meta-Argument

Explicitly Assign Resources to a Specific Provider

- Explicitly assigns a resource to a specific provider configuration.
- Most often used for multi-cloud or multi-account deployments.
- Use it when deploying resources across multiple cloud accounts or regions or managing various instances of the same provider (*e.g., separate AWS accounts for dev and prod*).
- Uses the `provider = <provider_alias> syntax`

```
1  provider "aws" {
2    region = "us-east-1"
3  }
4
5  provider "aws" {
6    alias  = "prod"
7    region = "us-west-1"
8  }
9
10 resource "aws_s3_bucket" "dev_bucket" {
11   bucket   = "my-dev-bucket"
12 }
13
14 resource "aws_s3_bucket" "prod_bucket" {
15   provider = aws.prod
16   bucket   = "my-prod-bucket"
17 }
```

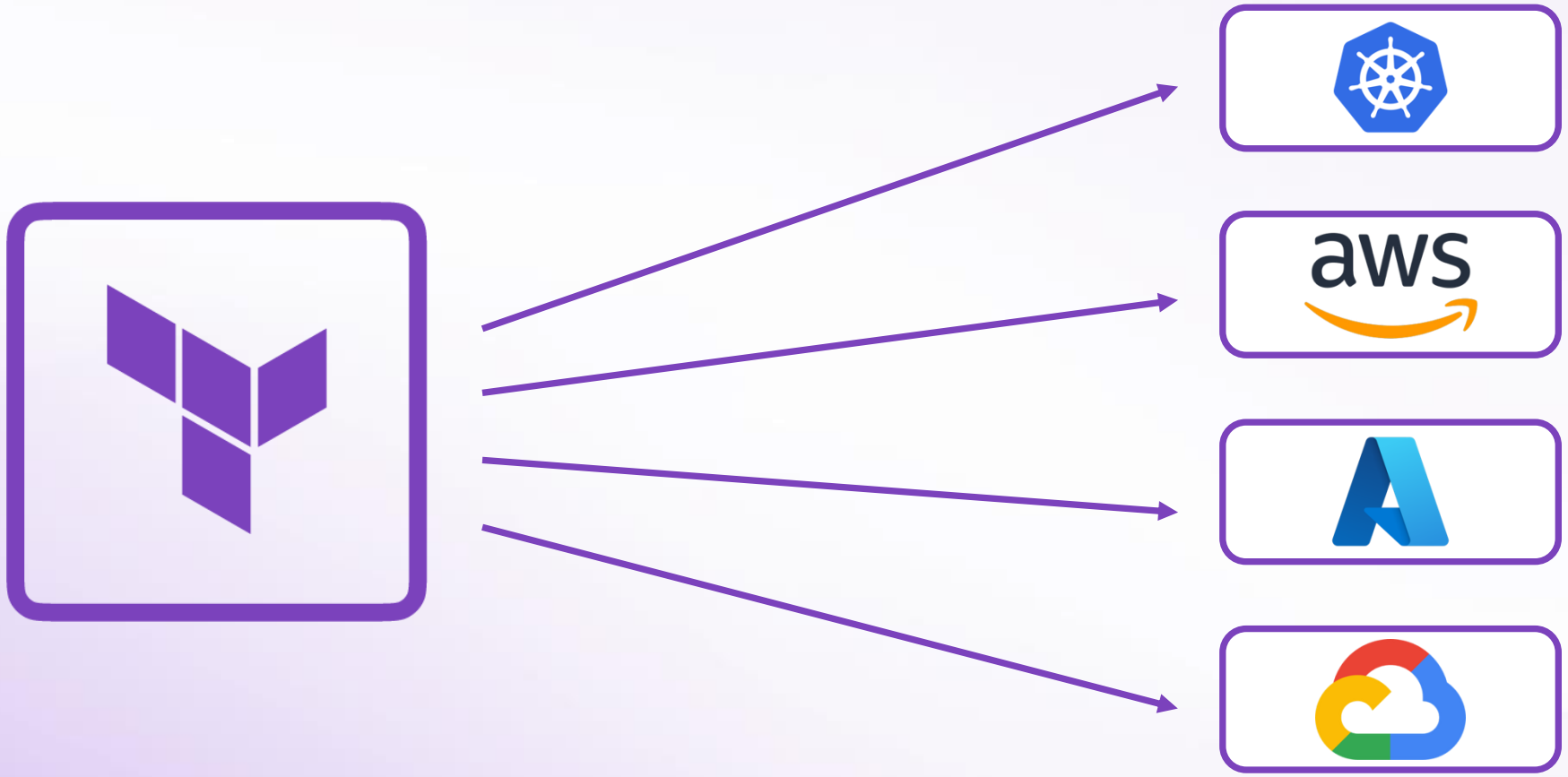
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Resource Block

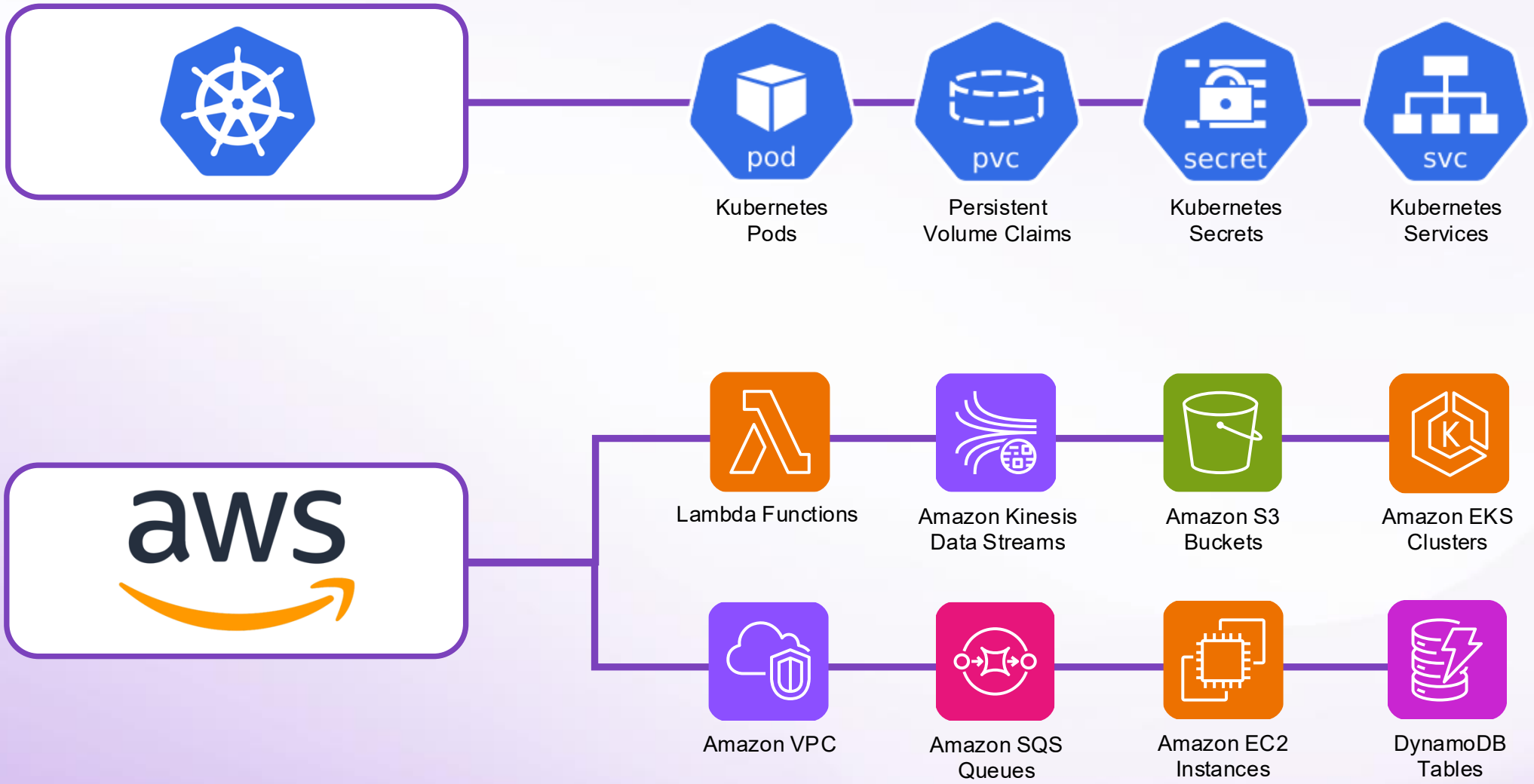
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Resources



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Resources



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Block

The `resource` **block** is the core element in Terraform. It defines the specific infrastructure pieces you want to create, manage, or modify, such as virtual machines, databases, or networks.

Resource blocks use a resource type (e.g., `aws_instance`) and a unique name. It acts as a blueprint with the settings you need to configure each resource.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Block

A resource block tells Terraform what to build and how it should be configured.

What Type of Resource is Needed?

The resource type identifies the type of resource that you need. Value is determined by the provider depending on the desired resource.

How Should the Resource be Configured?

Each resource block includes settings (or arguments) that configure its behavior.

Resource Identifiers

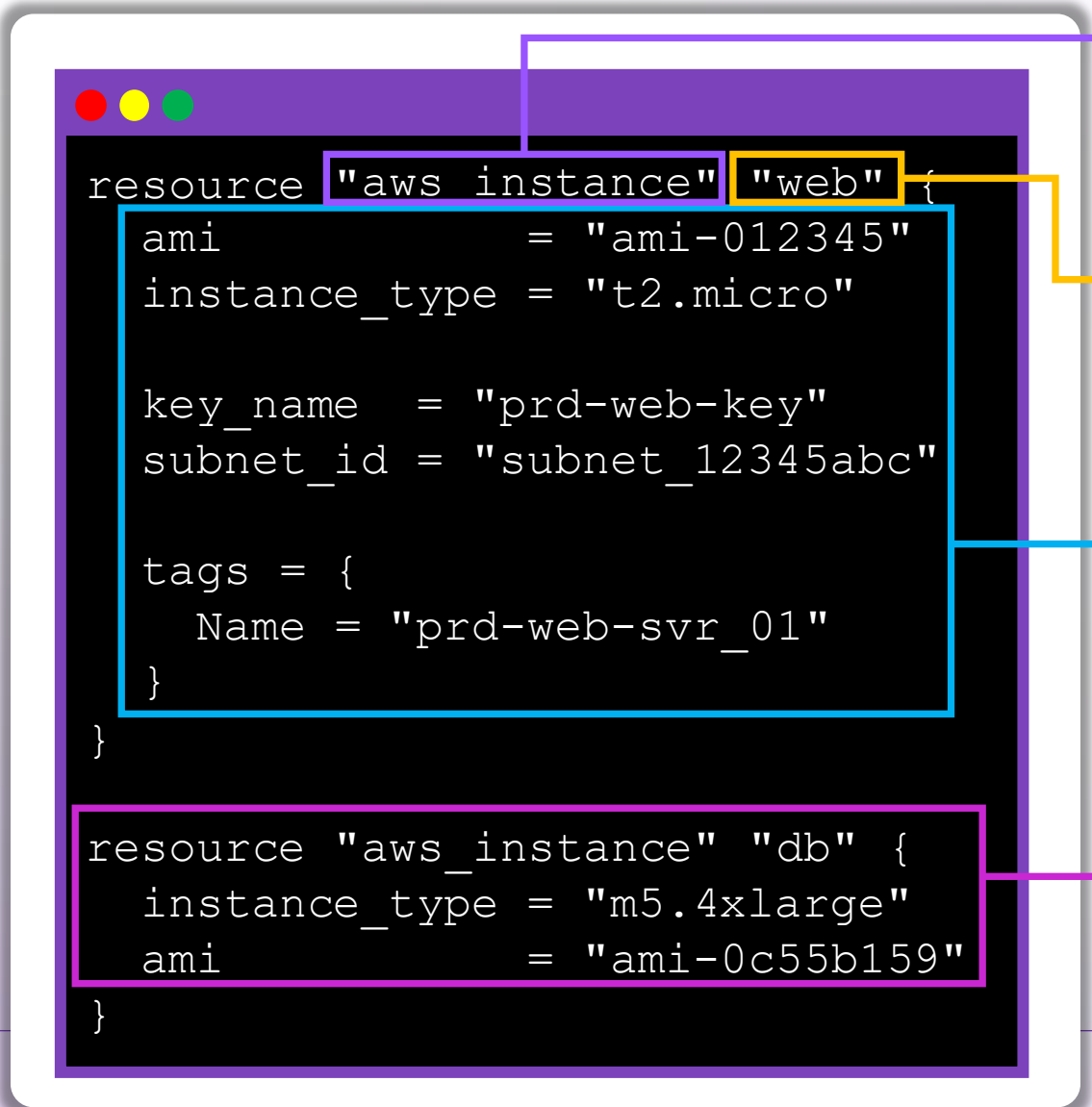
Each resource block is identified by a **type** and a **name**, which makes it easy to reference in other parts of your configuration

```
resource "aws_instance" "web" {
  ami           = "ami-012345"
  instance_type = "t2.micro"

  key_name  = "prd-web-key"
  subnet_id = "subnet_12345abc"

  tags = {
    Name = "prd-web-svr_01"
  }
}

resource "aws_instance" "db" {
  instance_type = "m5.4xlarge"
  ami           = "ami-0c55b159"
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUT

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Breaking Down the Resource Block



The type of resource to be created/ managed - defined within the provider

The resource name – user-provided string - must be unique per configuration.

Arguments specific to the resource type Can be found in provider documentation

A second resource block within the same Terraform configuration file

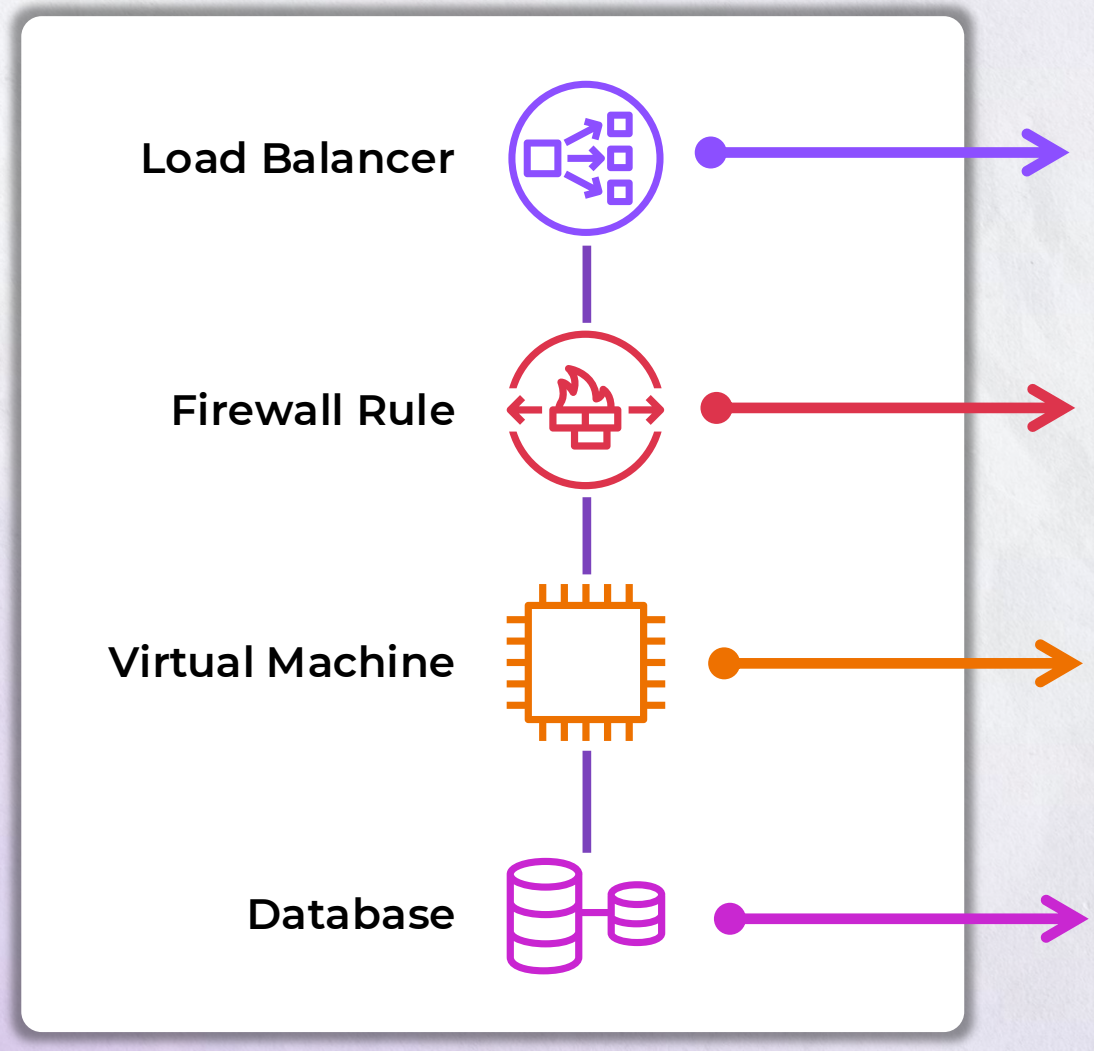
Resource Block

Resource Arguments

- The Image to Use
- The Size of the Instance
- Key Name for SSH Access
- The Subnet used to Provision the Instance
- Add Tags for the Resource

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Block



web_app.tf

```
resource "aws_lb" "public" {
  name = "prd-web-lb"
  load_balancer_type = "network"
}

resource "fortios_firewall_policy" "db" {
  action = "accept"
  name   = "allow_web_443"
}

resource "aws_instance" "web" {
  instance_type = "t3.large"
  ami           = "ami-0c55b159"
}

resource "aws_db_instance" "prd-db" {
  engine         = "mysql"
  instance_class = "db.t3.large"
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Referencing

Dynamically pull information from one resource block to another, creating a connected configuration

Reference a Resource block by using the resource **type** and the **name** of the resource.

1. `aws_lb.public_load_balancer`
2. `fortios_firewall_policy.db`
3. `aws_instance.web`
4. `aws_db_instance.prd_db`

web_app.tf

```
resource "aws_lb" "public_load_balancer" {
  name = "prd-web-lb"
  load_balancer_type = "network"
}

resource "fortios_firewall_policy" "db" {
  action = "accept"
  name   = "allow_web_443"
}

resource "aws_instance" "web" {
  instance_type = "t3.large"
  ami           = "ami-0c55b159"
}

resource "aws_db_instance" "prd_db" {
  engine         = "mysql"
  instance_class = "db.t3.large"
}
```

1
2
3
4

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Block

main.tf

```
resource "github_repository" "prod_repo" {
  name         = "prod-app-xyz-repo"
  visibility   = "private"
}

resource "github_branch" "default" {
  repository = github_repository.prod_repo.name
  branch     = "main"
}

resource "github_branch_default" "default" {
  repository = github_repository.prod_repo.name
  branch     = github_branch.default.branch
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Block

- Resource names must start with a letter or underscore, and may contain only letters, digits, underscores, and dashes.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Data Block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Accessing Existing Resources

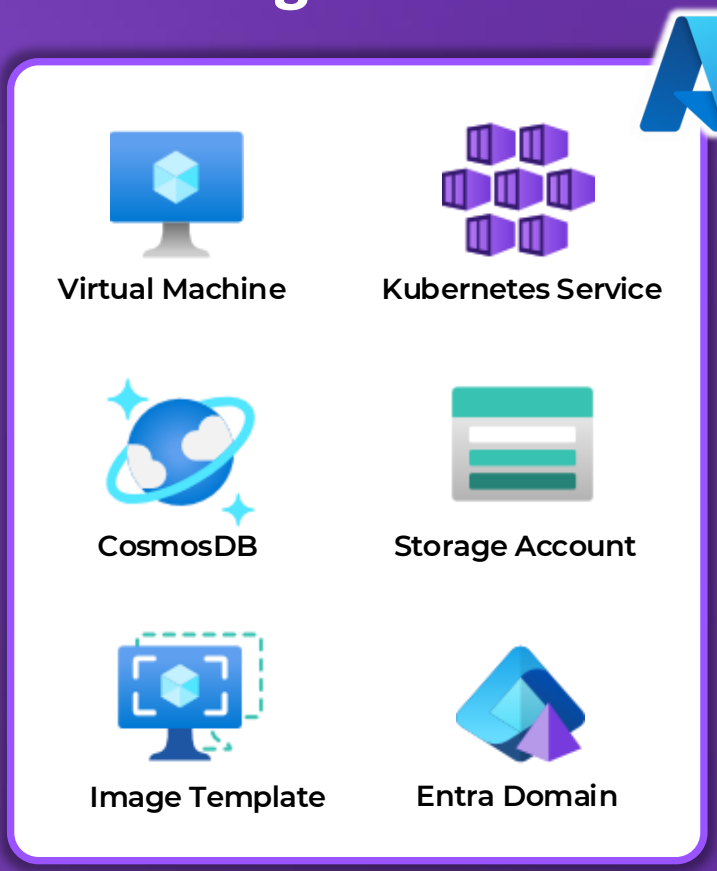
```
resource "azurerm_resource_group" "prd" {
  name     = "example-resources"
  location = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_mssql_database" "db1" {
  name      = "example-db"
  server_id = "server_db"
  collation = "sql_general"


data "azurerm_kubernetes_cluster" "aks" {
  name                = "prod-cluster"
  resource_group_name = "prod-group"
}
```

Provision Resources

Retrieve Information

Use Data Block to Retrieve Information

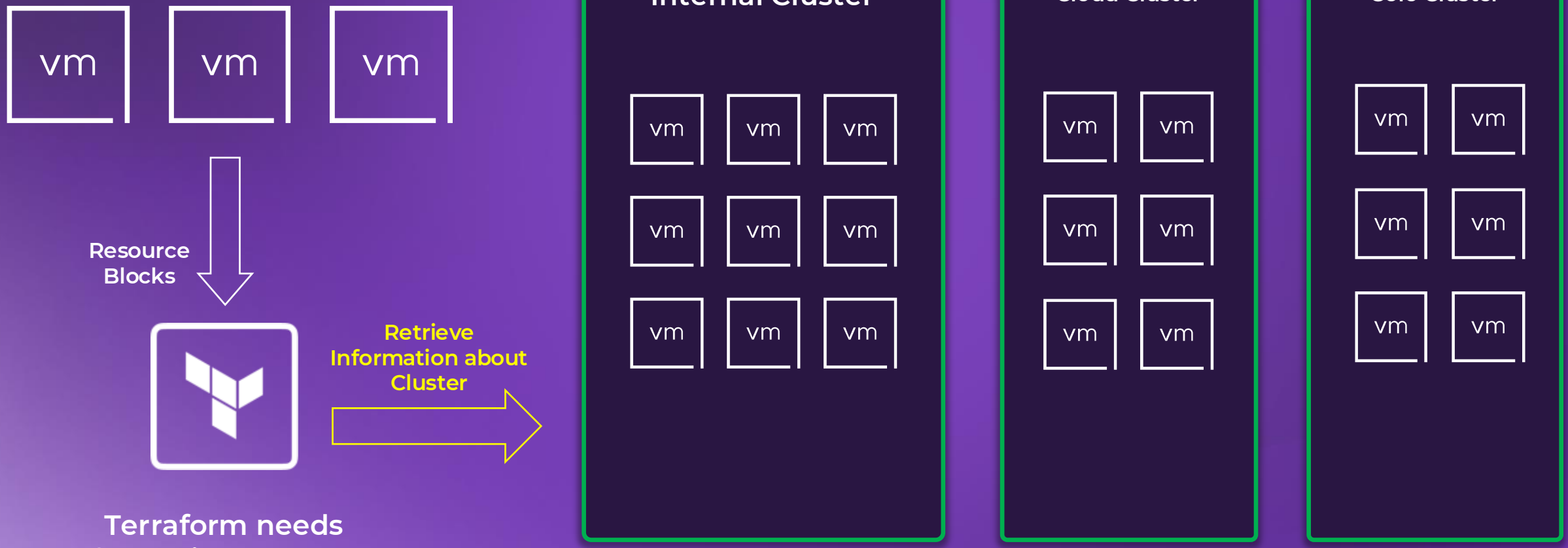
Existing Resources



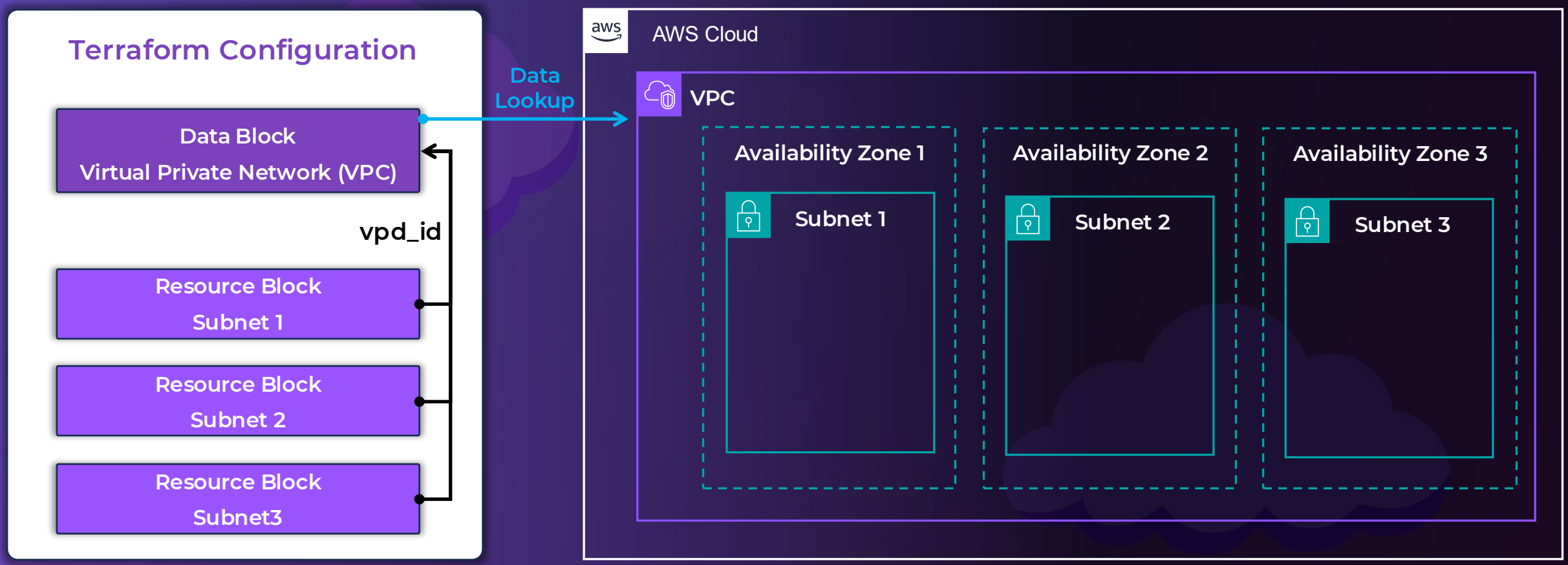
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

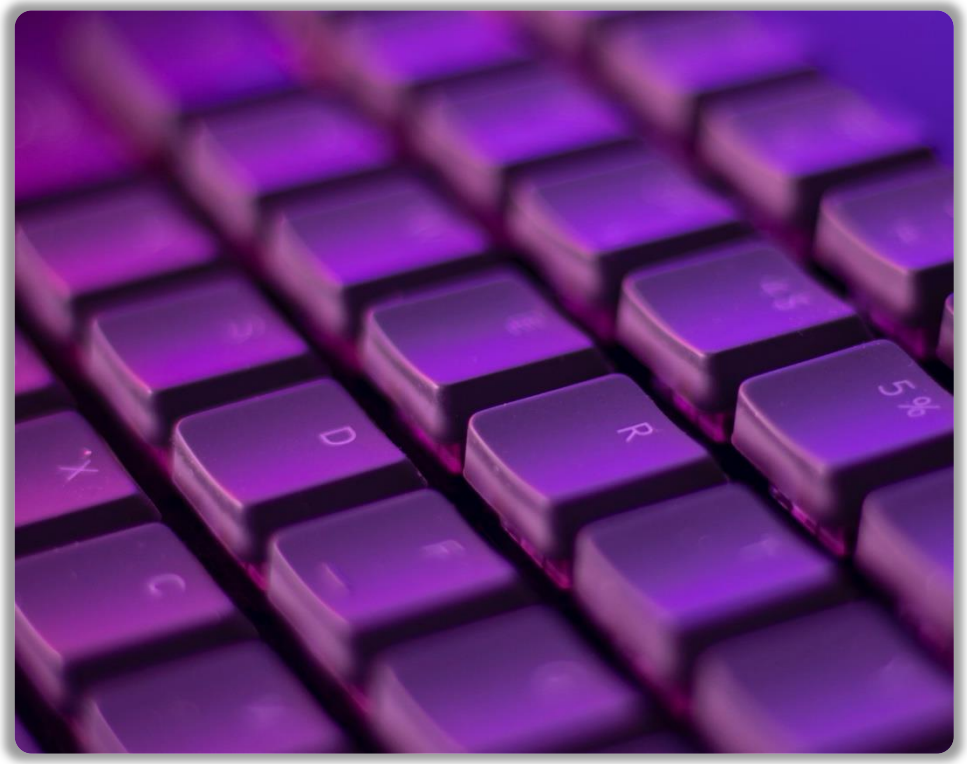
Existing Resources - Example



Existing Resources - Example



®Copyright Bryan Krausen – DO NOT DISTRIBUTE



Data Block

A **data** block retrieves information about existing resources or external data without creating new infrastructure. It allows you to dynamically reference attributes, such as the ID of an existing subnet or the details of a Kubernetes cluster, making configurations flexible and reusable.

By using **data** blocks, you can integrate Terraform with resources managed outside of its configuration.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Data Block

Data blocks allow Terraform to access information about existing resources rather than creating new ones.

Access Existing Infrastructure

Retrieve the details of existing resources so they can be referenced in other Terraform blocks

Dependency Resolution

Data blocks help when one resource relies on the properties of another existing resource

Avoid Hardcoding Values

Data blocks prevent the need for hardcoded values by dynamically pulling in details, which makes configurations more adaptable

network.tf

```
data "aws_vpc" "prd" {
  filter {
    name   = "tag:Name"
    values = ["prd-vpc"]
  }
}

resource "aws_subnet" "pub" {
  vpc_id     = data.aws_vpc.prd.id
  cir_block = "10.0.6.0/24"
  ...
}

output "prd_vpc_subnet" {
  value = data.aws_vpc.prd.cidr_block
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Breaking Down the Data Block

The given name used to reference the data block – user-defined string

The type of the resource to obtain data about - defined by provider

Query constraints defined by the data source to get the data you want

Get data about an Azure Resource Group named `dev-resource-group`

Get data about the default namespace for a Kubernetes cluster

main.tf

```
data "aws_vpc" "prd" {
  filter {
    name   = "tag:Name"
    values = ["prd-vpc"]
  }
}

data "azurerm_resource_group" "dev" {
  name = "dev-resource-group"
}

data "kubernetes_namespace" "app" {
  metadata {
    name = "customer-app"
  }
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Referencing Data Blocks

Get information from a `data` block to use elsewhere within the configuration.

`data` blocks retrieve a wide range of information about the specified resource – called **`attributes`**.

Data block attributes are referenced just like resource attributes: `data.<type>.<name>.<attribute>`

- `data.aws_vpc.prd.arn`
- `data.aws_vpc.prd.cidr_block`
- `data.aws_vpc.prd.id`
- `data.aws_vpc.prd.owner_id`

network.tf

```
data "aws_vpc" "prd" {
  filter {
    name   = "tag:Name"
    values = ["prd-vpc"]
  }
}
```

This data block gets all this data

```
{
  "arn": "arn:aws:ec2:us-west-2:xxx:vpc/vpc-0ab",
  "cidr_block": "10.0.0.0/16",
  "default": false,
  "dhcp_options_id": "dopt-0abcdef1234567890",
  "enable_dns_hostnames": true,
  "enable_dns_support": true,
  "id": "vpc-0abcd1234efgh5678",
  "instance_tenancy": "default",
  "ipv6_association_id": null,
  "ipv6_cidr_block": null,
  "main_route_table_id": "rtb-0abcdef1234567890",
  "owner_id": "123456789012",
  "tags": {
    "Name": "prd-vpc"
  }
}
```

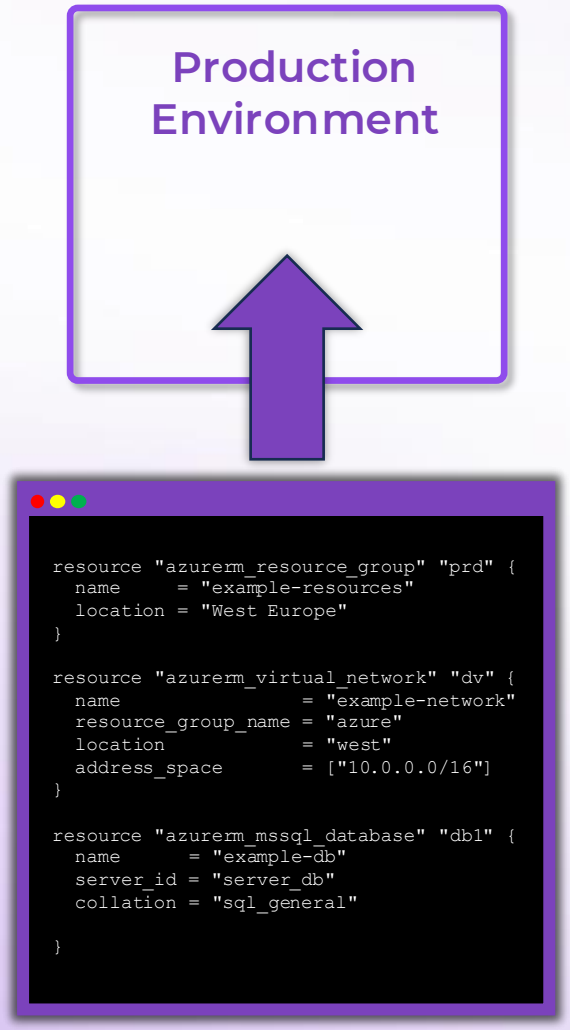
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

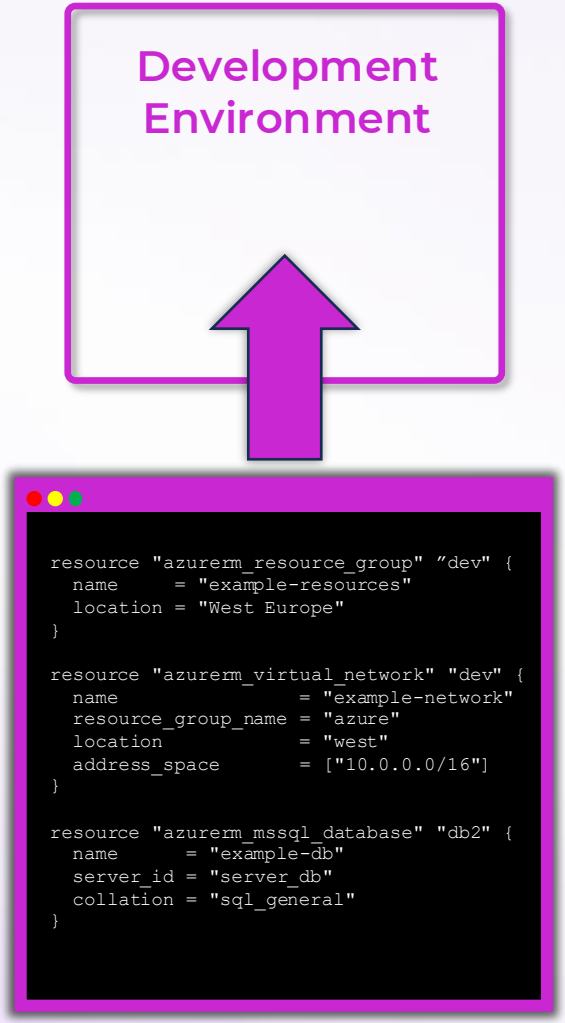
LECTURE

Variable Block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

The Problem: Hardcoding Values in Terraform





®Copyright Bryan Krausen – DO NOT DISTRIBUTE

The Solution: Using Variables in Terraform

Cloud Ops Team

Prod Env Variables

QA Team

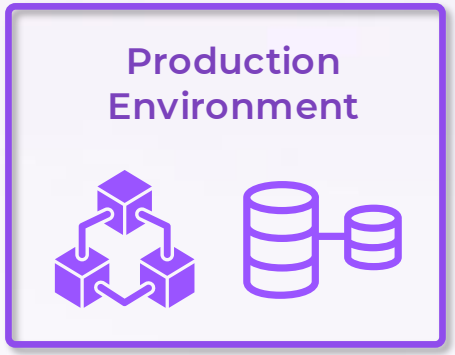
QA Env Variables

Dev Team

Dev Env Variables

```
resource "azurerm_resource_group" "prd" {
  name     = var.rg_name
  location = var.rg_location
}

resource "azurerm_virtual_network" "dv" {
  name                = var.net_name
  resource_group_name = var.net_rg
  location            = var.net_location
  address_space       = var.net_cidr
}

resource "azurerm_mssql_database" "db1" {
  name      = var.mssql_name
  server_id = var.server_id
  collation = var.collation
```

A single Terraform configuration can result in many unique environments



Larger VMs
Multi-Database
Complex Compute
Highly Redundant

Medium VMs
Larger Database
More Compute

Small VMs
Single Database
Little Compute
No Redundancy

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Variable Block

A **`variable`** block is used to define inputs that can be reused and customized, making your configurations more flexible and dynamic. Variables allow you to parameterize values instead of hardcoding them directly into resource blocks.

This approach improves reusability, simplifies managing multiple environments, and helps maintain cleaner, more maintainable configurations.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Variable Block

Define an input that can be reused and customized to make configurations more flexible and dynamic

Dynamic Inputs for Dynamic Code

Variables allow configurations to adapt by using different values for different environments or scenarios.

Centralized Management of Values

By defining variables in one place, you can update values easily without modifying multiple resource blocks.

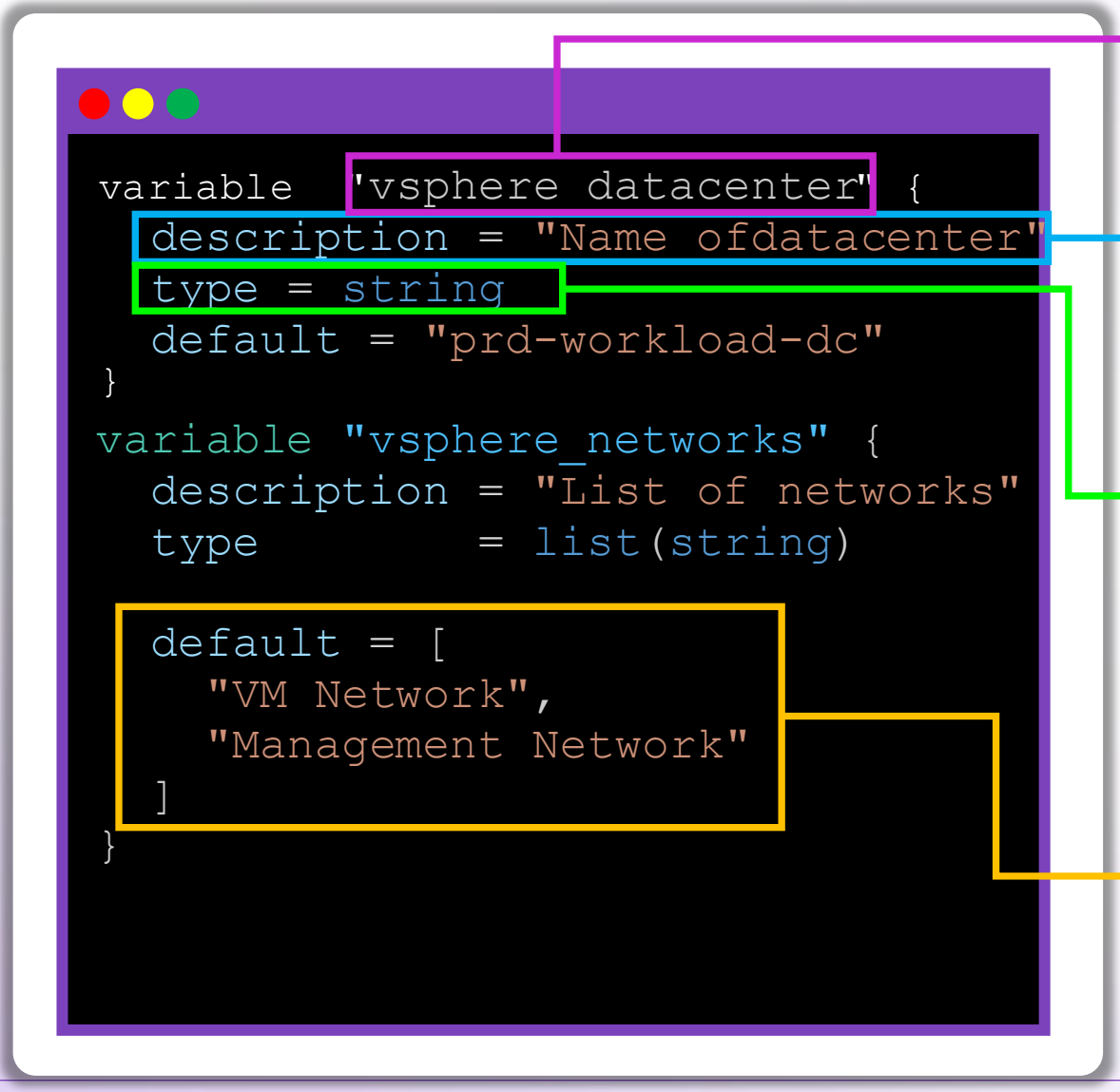
Improved Reusability of Code

Variable blocks make Terraform configurations modular and reusable across projects and teams.

```
variable "vsphere_datacenter" {
  description = "Name of datacenter"
  type        = string
  default     = "prd-workload-dc"
}

variable "vsphere_networks" {
  description = "List of networks"
  type        = list(string)

  default     = [
    "VM Network",
    "Management Network"
  ]
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUT

Breaking Down the Variable Block



The name for the variable – a user-provided string

Friendly description of the variable and what it's used for...

Define what value types are accepted for the variable

The default value of the variable if not set anywhere else in Terraform

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Types of Variables

Primitive Variable Types

String

a sequence of characters representing some text

```
variable "region" {
  type    = string
  default = "us-east-2"
}
```

Number

a numeric value

```
variable "num_of_vms" {
  type    = number
  default = 3
}
```

Bool

a boolean value – either True or False

```
variable "enable_ha" {
  type    = bool
  default = true
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Types of Variables

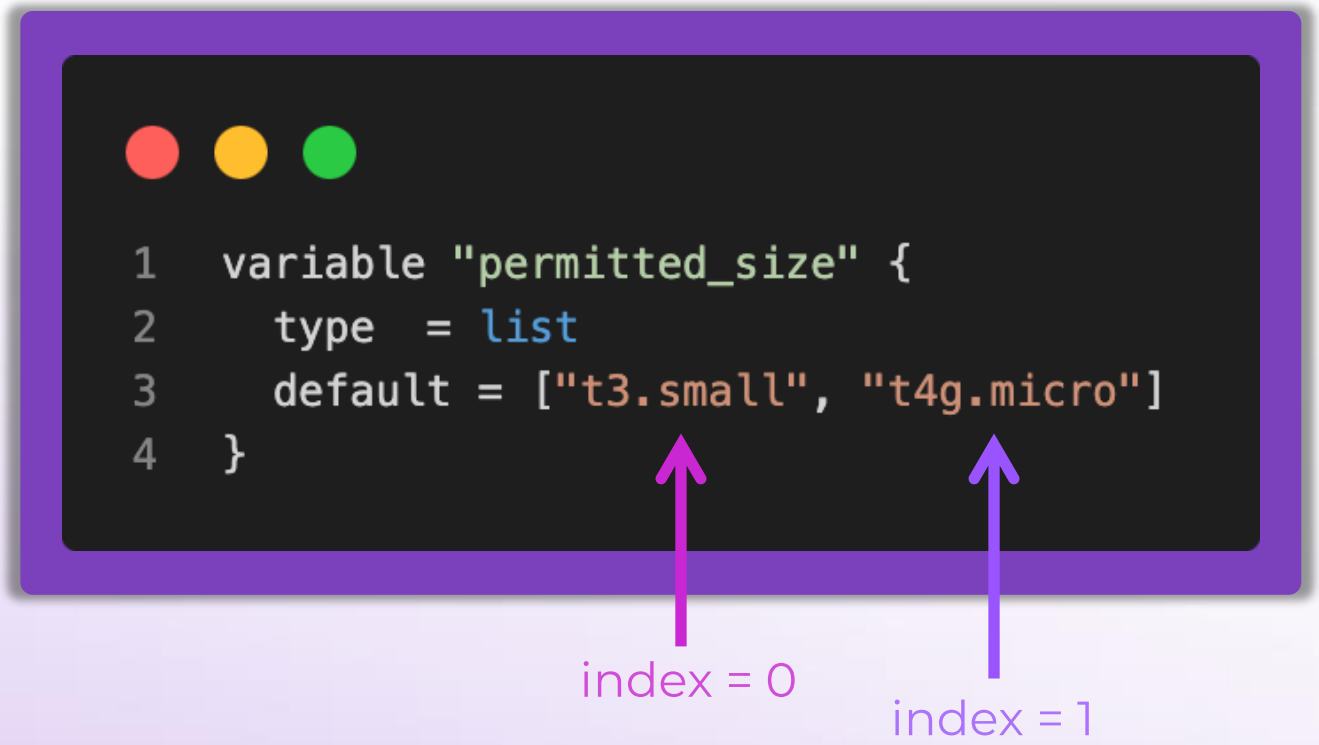
Complex Variable Types

List

A sequence of similar values

Presented by a pair of square brackets [ ] containing a comma-separated sequence of values

Note: lists start at 0



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

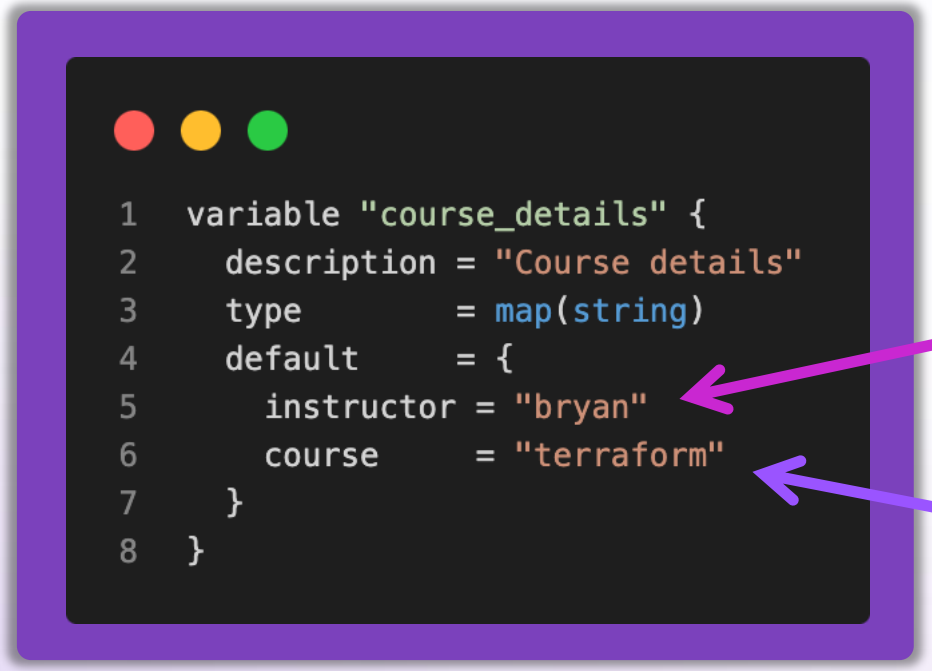
Types of Variables

Complex Variable Types

Map

A group of values identified by named labels

Presented by a pair of curly braces { } and contains a series of key=value pairs



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Types of Variables

Complex Variable Types

Set

A collection of guaranteed unique values

Terraform does not support accessing elements of a set using an index because sets are unordered collections.

Note: Convert to a list to access

```
variable "pub_subnet_ids" {
  type    = set(string)
  default = ["subnet-12345", "subnet-67890"]
}
```

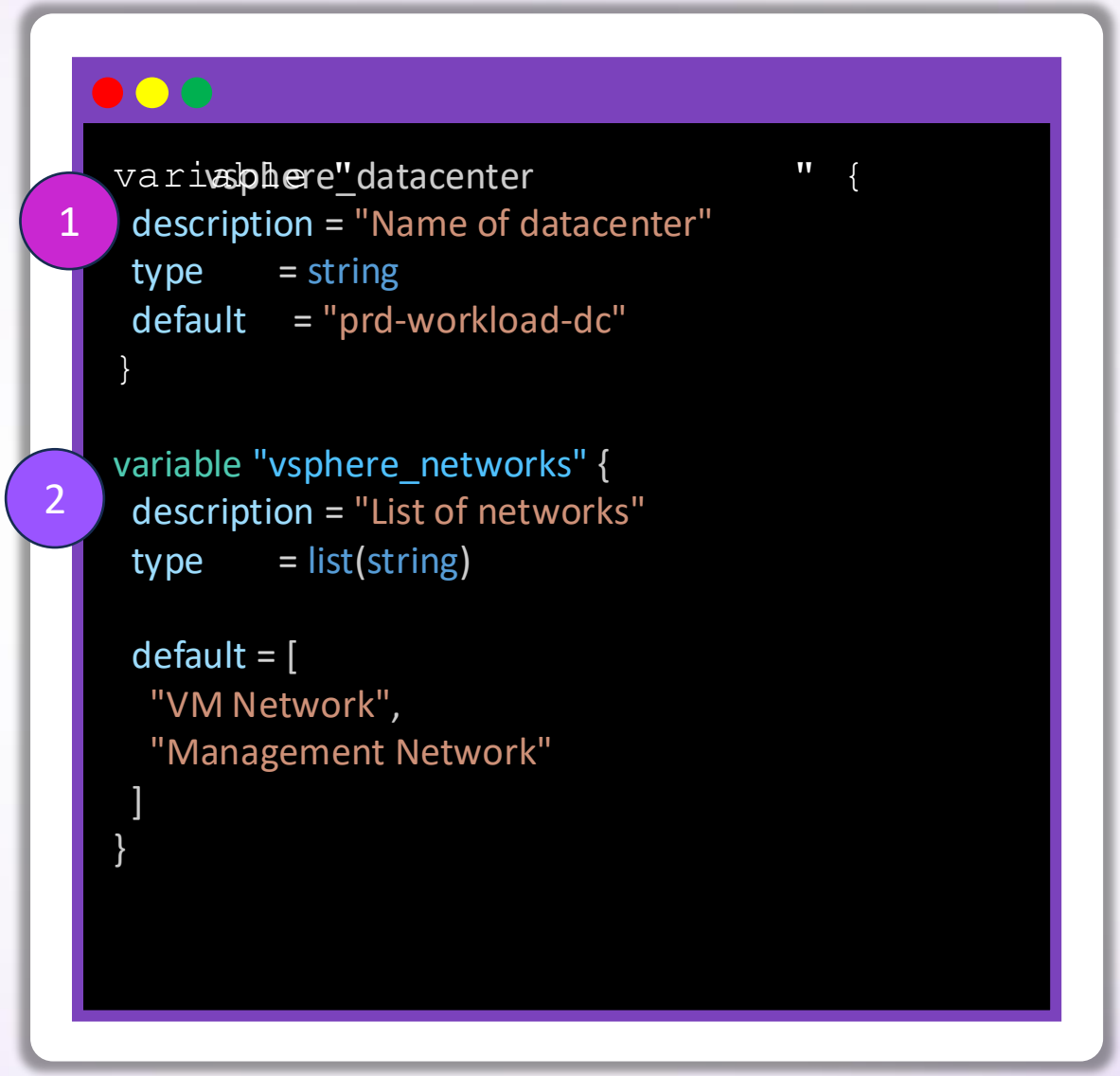
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Resource Referencing

Reference a Variable within the Terraform configuration to use the value

Reference a Variable by using the **name** of the variable.

1. `var.vsphere_datacenter`
2. `var.vsphere_networks[0]`



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Assigning Values to Variables

After variables are declared in your Terraform configuration, the values can be set in many different ways...

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Variable Defaults

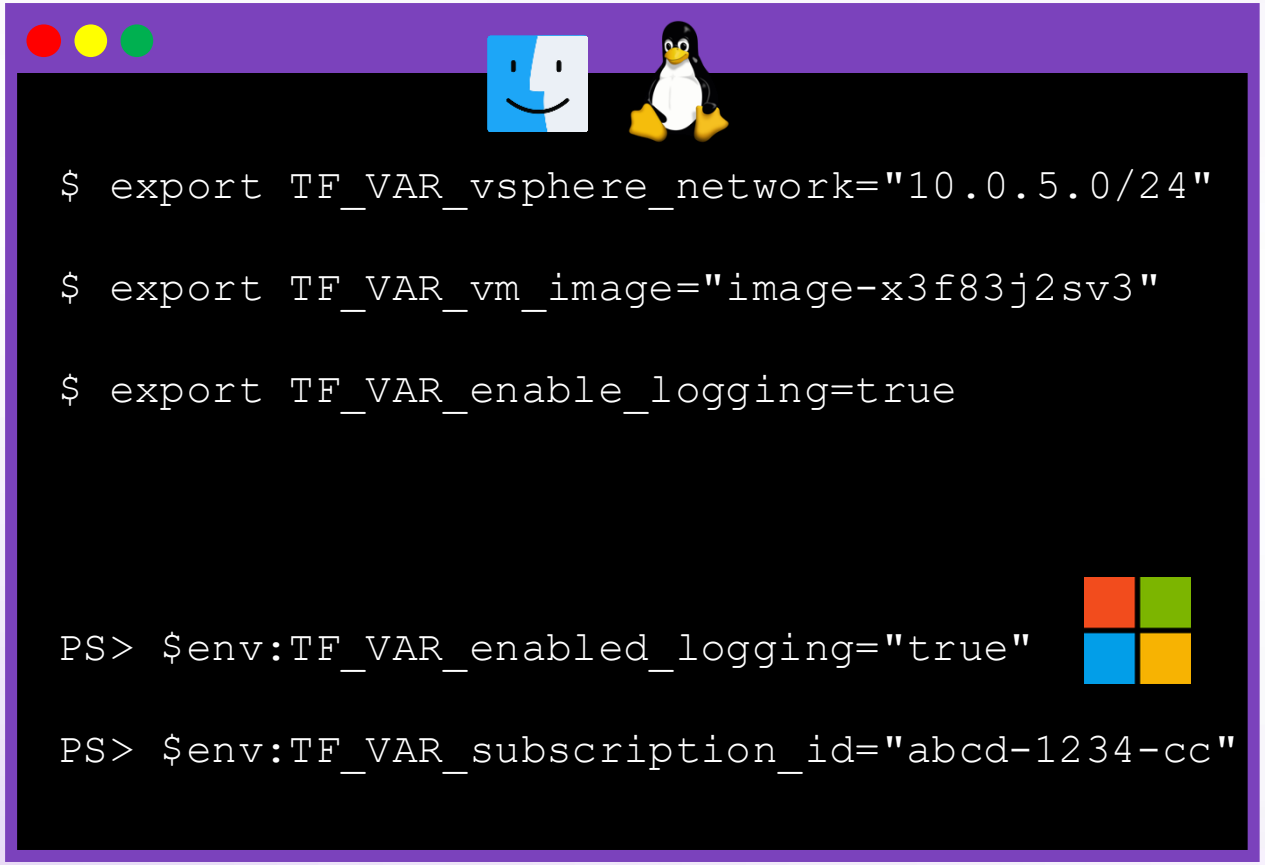
Set a value directly in the variable block declaration

```
variable "pub_subnet_ids" {
  type    = set(string)
  default = ["subnet-12345", "subnet-67890"]
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Environment Variables

- Use an environment variable to set the value
- Must start with `TF_VAR_` followed by the variable name
- Useful for when you don't want to commit the value to a repo



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

tfvars File

- Use a `.tfvars` file to set variable values
- Terraform automatically loads a `.tfvars` file if they exist
- Set values using a `key=value` format using the variable names
- A `.tfvars` file is usually excluded from version control (.gitignore)

terraform.tfvars

```
# VM-level configurations
vsphere_network = "10.0.5.0/24"
vm_image        = "image-x3f83j2sv3"

# Application Configurations
enable_logging = true


# Cloud Account Configurations
subscription_id = "abcd-1234-cc"
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Command Line Flags

- Pass variable values directly from the command line.
- Use `-var="key=value"` to set the value for a specific variable.
- Command-line `-var` values override both default values and values in `.tfvars` files.
- Useful for quick, temporary changes or testing without modifying files.

TERMINAL

```
$ terraform plan -var="enable_logging=true"

$ terraform apply -var="region=us-west-2" -var="vm_image=image-x3f83j2sv3"
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Order of Precedence

What hierarchy does Terraform follow when resolving variable values from different sources?

Flexibility for Values

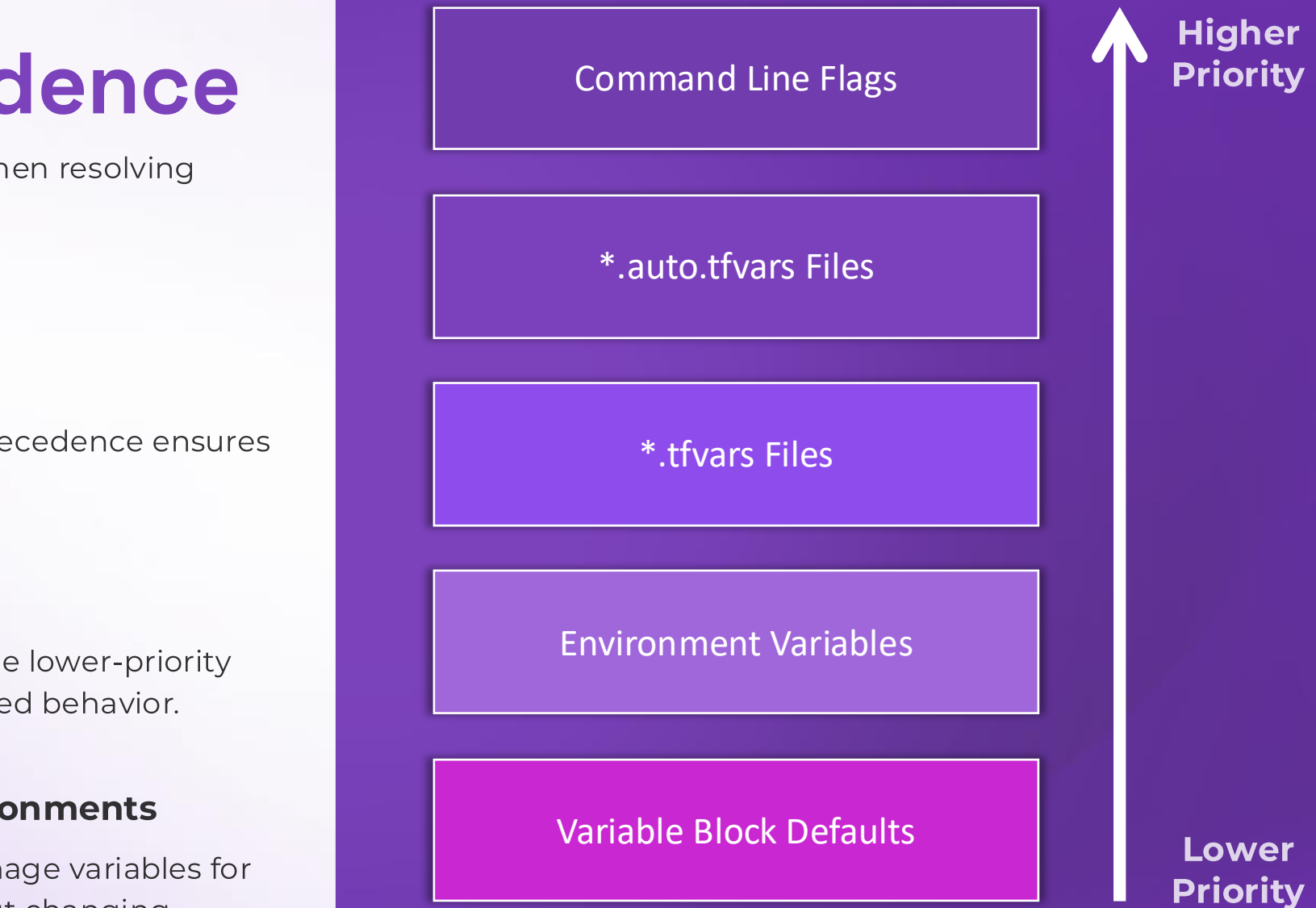
Understanding the order of precedence ensures the right value is applied

Clear Override Rules

Higher-priority sources override lower-priority defaults, preventing unexpected behavior.

Adaptability Across Environments

Precedence allows you to manage variables for different environments without changing configurations.



®Copyright Bryan Krausen – DO NOT DISTRIBUT

LECTURE

Output Block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Accessing New Resources

```
resource "azurerm_resource_group" "prd" {
  name     = "example-resources"
  location = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}
```

Write Code

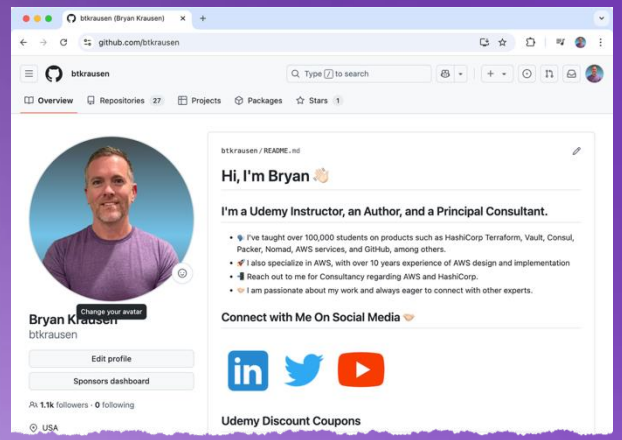
Provision Resources

New Resources

- Virtual Machine
- Kubernetes Service
- CosmosDB
- Storage Account
- Image Template
- Entra Domain

Output Information to Access Resources

https://btk.me/btk



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Output Block

An **`output`** block in Terraform is used to display key information about your infrastructure after a deployment, such as IP addresses, DNS names, or resource IDs. Outputs make it easy to retrieve and share important details without manually looking them up in the cloud provider's console.

They're also useful for passing data between modules or automating workflows.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Output Block

An output block displays important information about your infrastructure, like resource IDs or URLs, after deployment.

Display Critical Information

Outputs make it easy to retrieve important details like IP addresses, DNS names, or resource IDs after deployment.

Enable Module Integration

Outputs can pass data between Terraform modules, simplifying workflows and making configs reusable.

Improve Workflow Efficiency

Outputs automate the retrieval of key values, reducing the need for manual lookups in provider consoles.

outputs.tf

```
# Output Instance IP Address
output "instance_public_ip" {
  description = "Public IP of Server"
  value       = aws_instance.web.public_ip
}

# Output DNS Name for Load Balancer
output "website_dns" {
  description = "Website DNS Record"
  value       = aws_elb.web_app.dns_name
}

# Output Friendly URL of Website
output "website_url" {
  value = "https://${aws_alb.web.dns_name}"
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Breaking Down the Output Block

outputs.tf

```
# Output Instance IP Address
output "instance_public_ip" {
  description = "Public IP of Web server"
  value       = aws_instance.web.public_ip
}

# Output DNS Name for Load Balancer
output "website_dns" {
  description = "Website DNS Record"
  value       = aws_elb.web_app.dns_name
}

# Output Friendly URL of Website
output "website_url" {
  value = "https://${aws_alb.web.dns_name}"
}
```

The name of the output – user-defined string

The value of the output

Another output block – a more advanced value combining the value of a resource within a string

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Terraform Block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Imagine This Real-World Scenario...

Your Workstation

```
resource "azurerm_resource_group" "prd" {
  name     = "example-resources"
  location = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_mssql_database" "db1" {
  name      = "example-db"
  server_id = "server_db"
  collation = "sql_general"

}
```

Commit Code

Git Repo

Pull Code

Colleague's Workstation

```
resource "azurerm_resource_group" "prd" {
  name     = "example-resources"
  location = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_mssql_database" "db1" {
  name      = "example-db"
  server_id = "server_db"
  collation = "sql_general"

}
```

Terraform Version 1.10

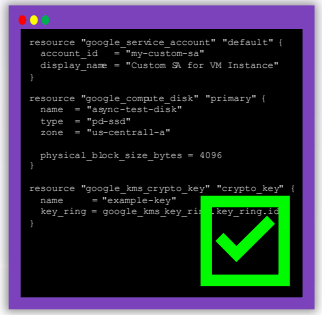
Terraform Version 1.11

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

The Risk of Mismatched Provider Versions

CloudOps Team



Commit Code

Git Repo

Pull Code

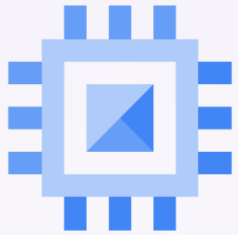
Data Research Team

Adds Newly Released Service

Downloads Latest GCP Provider Version

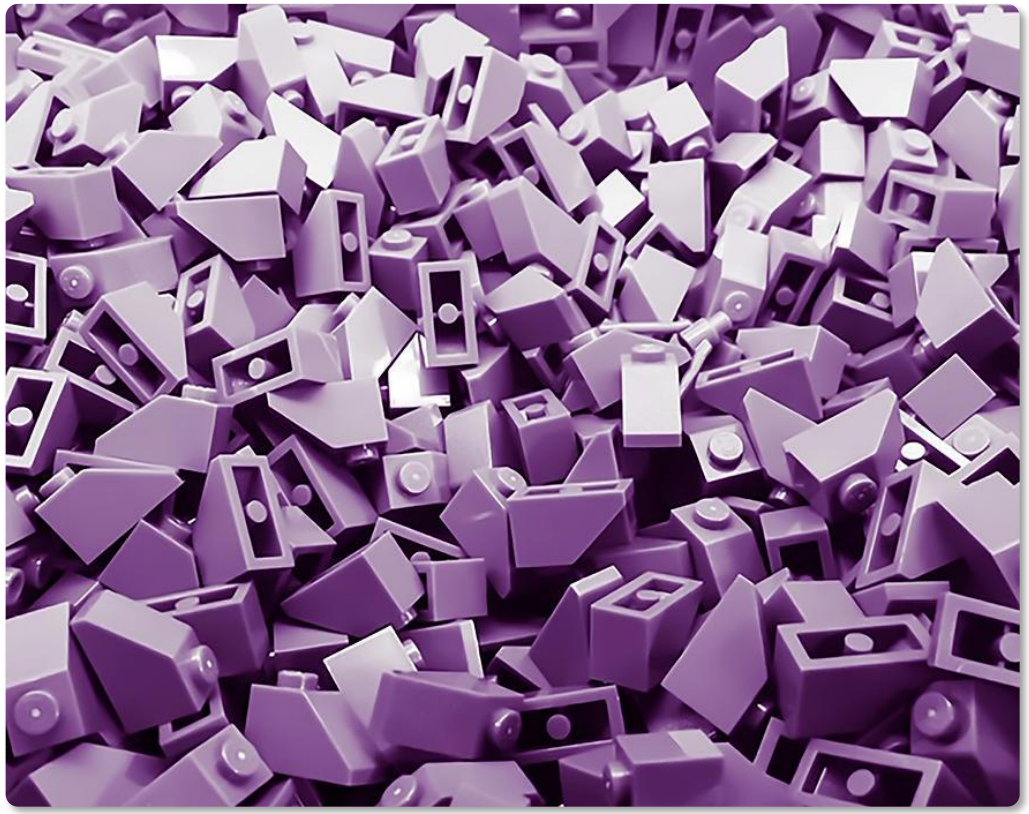
Google Cloud Platform

Google Cloud Platform



GCP Provisioned Resources

Bryan Krausen



Terraform Block

The **`terraform`** block sets global configurations for your Terraform project, such as required provider versions, Terraform version constraints, and backend settings for state management. It ensures consistency across teams and environments by enforcing specific versions and configurations.

Using the **`terraform`** block helps avoid errors and provides a solid foundation for managing infrastructure effectively.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Block

The `terraform` block ensures your Terraform setup is consistent, and scalable across teams and environments.

Version Control

Specifies the required Terraform and provider versions to avoid compatibility issues

Backend Configuration

Configures where Terraform state is stored to ensure secure and reliable state management

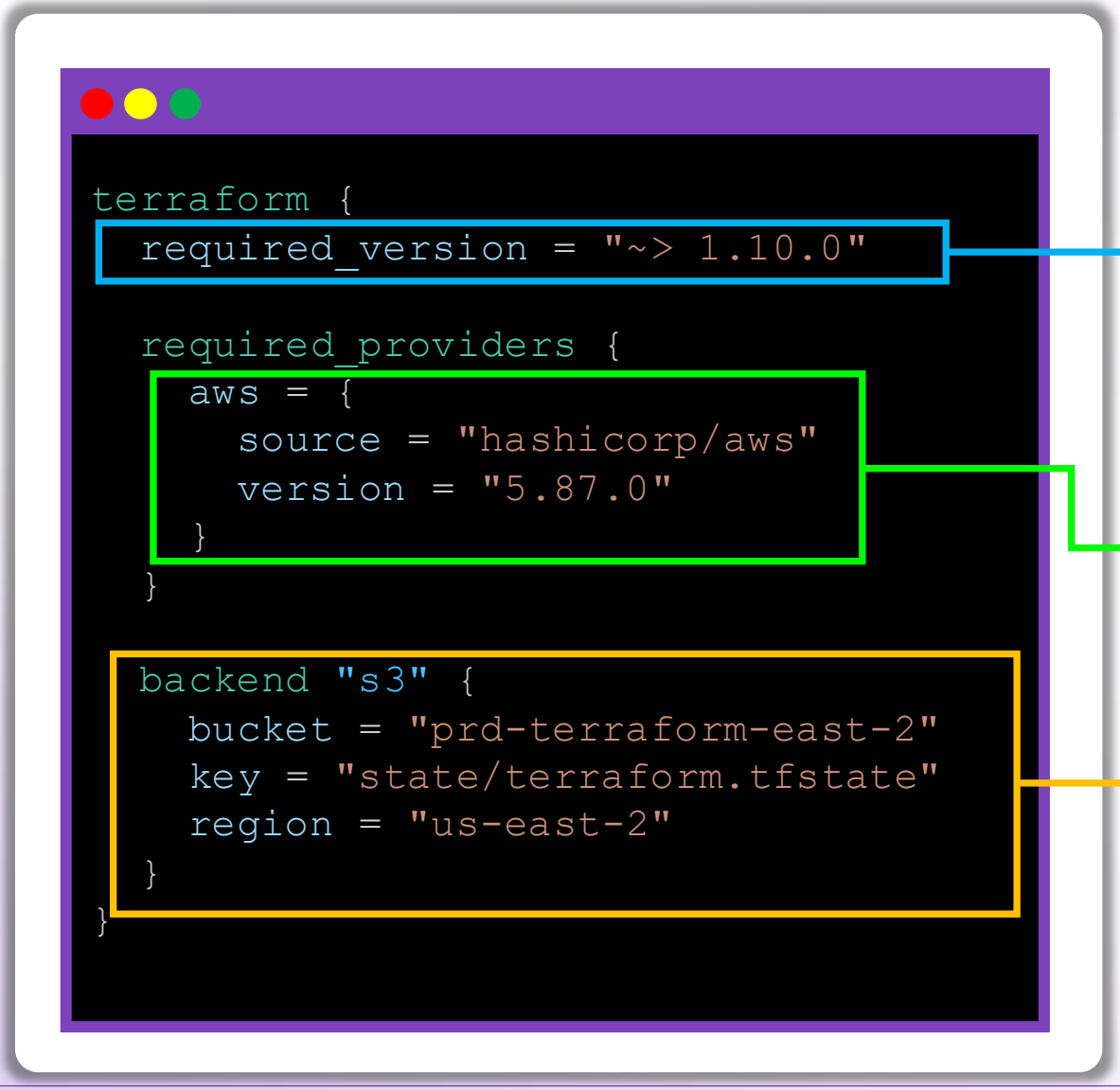
Provider Requirements

Defines the providers and the versions needed for your infrastructure

```
terraform {
  required_version = "~> 1.10.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket = "prd-terraform-east-2"
    key    = "state/terraform.tfstate"
    region = "us-east-2"
  }
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Breaking Down the Terraform Block



```
terraform {
  required_version = "~> 1.10.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "5.87.0"
    }
  }

  backend "s3" {
    bucket = "prd-terraform-east-2"
    key = "state/terraform.tfstate"
    region = "us-east-2"
  }
}
```

Ensures Terraform version 1.10.x is used to run this configuration

Specify a required provider and require version 5.87.0

The backend configuration of where you want to store Terraform state

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Version Constraints

```
terraform {
  required_version = "1.9.8"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "5.77.0"
    }
  }
}
```

- Only Terraform v1.9.8 can be used to run this configuration
- Installs/uses AWS provider version 5.77.0

```
terraform {
  required_version = ">= 1.9.8"

required_providers {
    aws = {
      source = "hashicorp/aws"
      version = ">= 5.77.0"
    }
  }
}
```

- Terraform v1.9.8 or greater can be used to run this configuration
- Installs/uses AWS provider version greater or equal to 5.77.0

```
terraform {
  required_version = "~> 1.11.0"

required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.87.0"
    }
  }
}
```

- Terraform v1.11.x or greater can be used to run this configuration
- Installs/uses AWS provider version greater or equal to 5.87.x

~ allows only the right-most version component to increment

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

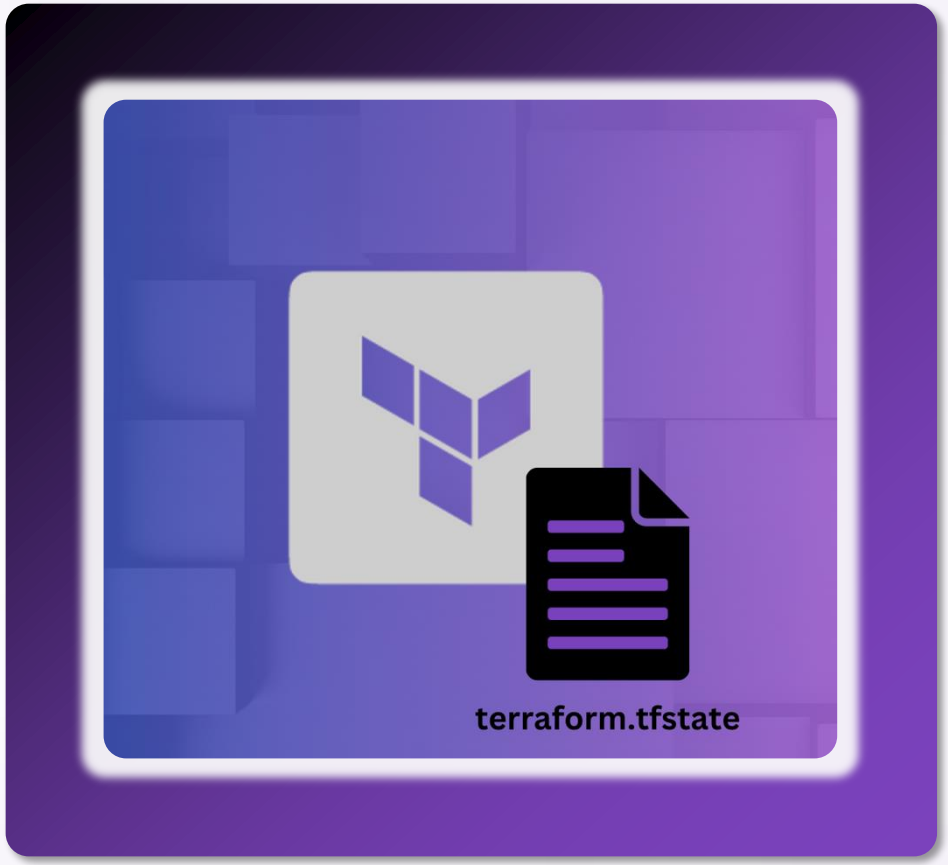
LECTURE

Understanding Terraform State

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Introduction to Terraform State

Terraform state is a file that tracks the resources managed by Terraform, including their IDs and relationships.



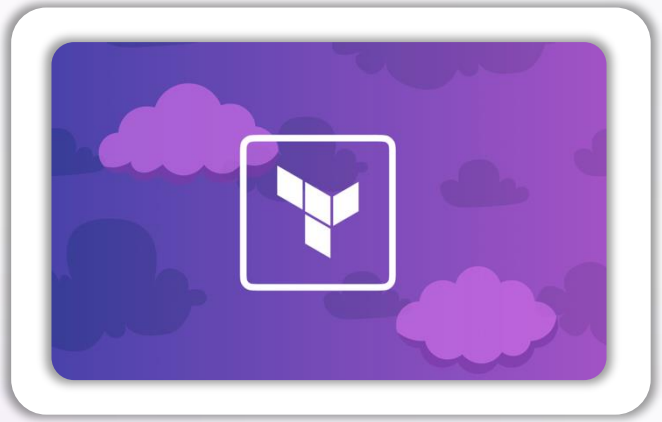
- It acts as a single source of truth for your infrastructure. Without the state file, Terraform would not know its current state.
- Terraform cannot operate without state
- Before any operation, Terraform does a refresh to update the state with the real-world infrastructure

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Why Terraform State is Important

Resource Management

Enables Terraform to determine the difference between the desired and current state.



Dependency Management

Tracks resource relationships to apply updates in the correct order.

Terraform does a refresh to update the state before any operations

Code Collaboration

Remote state enables secure, shared workflows for teams.

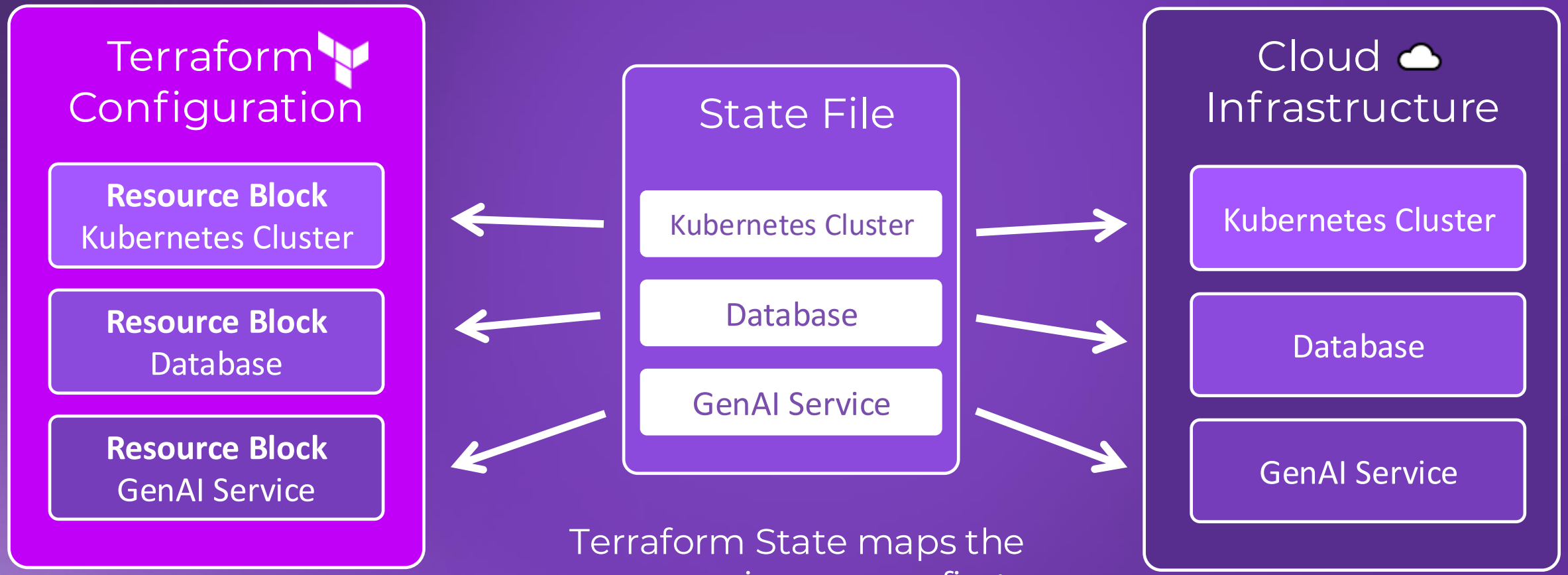
State File

- Terraform stores state in file, which acts as a sort of database to map Terraform configurations to real-world resources
- Alongside the mapping between resources and remote objects, Terraform also tracks metadata for internal purposes.
- This metadata allows Terraform to track order of operations, the provider used, etc.

```
{
  "module": "module.nomad-client[\"nomad-client-3\"]",
  "mode": "managed",
  "type": "aws_instance",
  "name": "this",
  "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
  "instances": [
    {
      "index_key": 0,
      "schema_version": 1,
      "attributes": {
        "ami": "ami-03a6eaae9938c858c",
        "arn": "arn:aws:ec2:us-east-1:634211456147:instance/i-0eb4591447d4fdf7c",
        "associate_public_ip_address": true,
        "availability_zone": "us-east-1c",
        "capacity_reservation_specification": [
          {
            "capacity_reservation_preference": "open",
            "capacity_reservation_target": []
          }
        ],
        "cpu_core_count": 1,
        "cpu_options": [
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform State



Terraform State maps the resources in your config to the real-world infrastructure

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

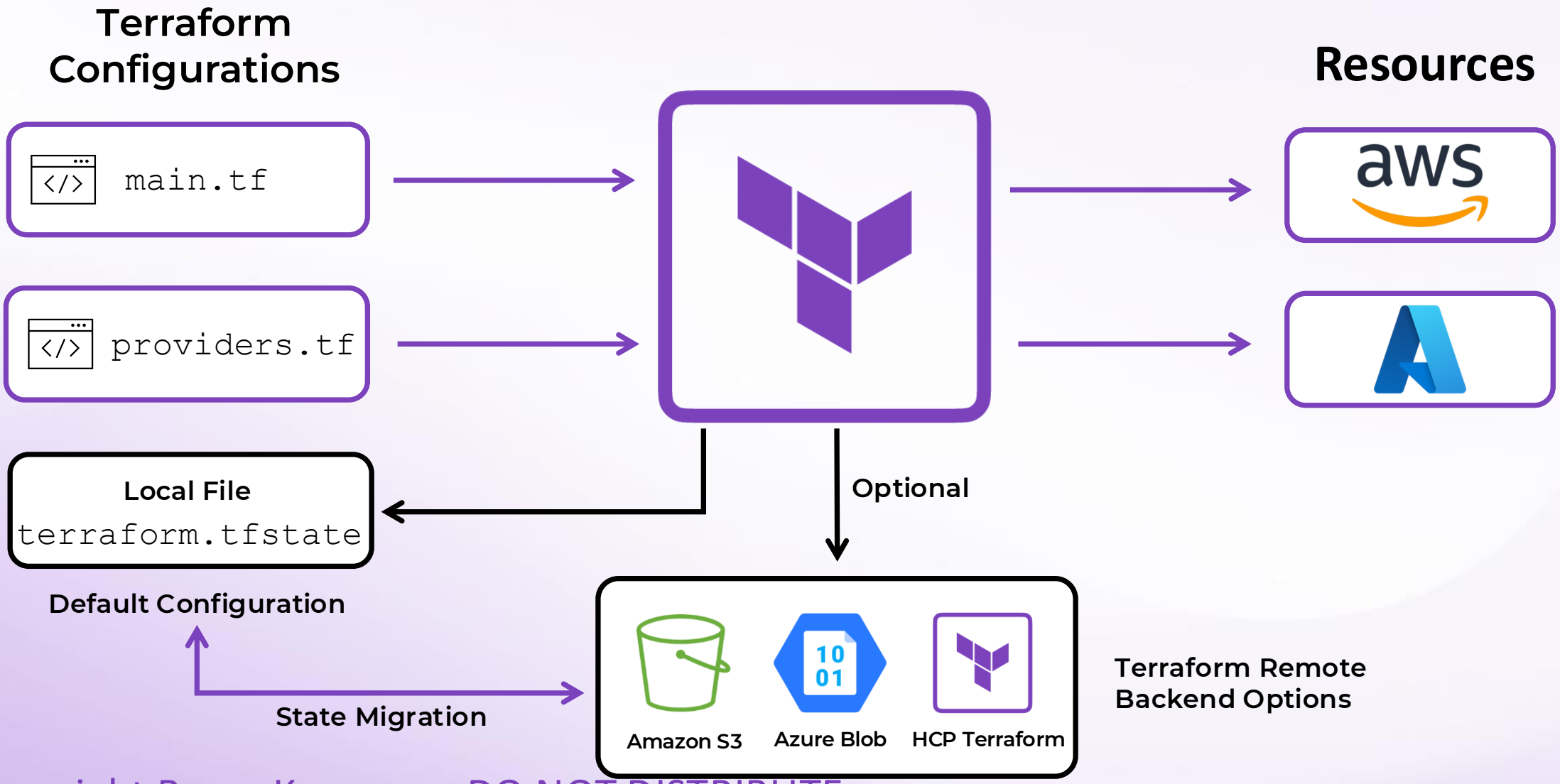
State Storage

- By default, Terraform stores state in the current working directory in a JSON-formatted file named `terraform.tfstate`
- Terraform needs to "lock" the state file when performing any operations that might write to the file (to avoid conflicts)
- Terraform can be configured to store state in a shared location. A backend configuration will determine where state is stored

```
{
  "module": "module.nomad-client[\"nomad-client-3\"]",
  "mode": "managed",
  "type": "aws_instance",
  "name": "this",
  "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
  "instances": [
    {
      "index_key": 0,
      "schema_version": 1,
      "attributes": {
        "ami": "ami-03a6eaae9938c858c",
        "arn": "arn:aws:ec2:us-east-1:634211456147:instance/i-0eb4591447d4fdf7c",
        "associate_public_ip_address": true,
        "availability_zone": "us-east-1c",
        "capacity_reservation_specification": [
          {
            "capacity_reservation_preference": "open",
            "capacity_reservation_target": []
          }
        ],
        "cpu_core_count": 1,
        "cpu_options": [
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform – Remote State



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Remote State Configuration

```
terraform {
  backend "type" {
    # Backend-specific configuration
  }
}
```

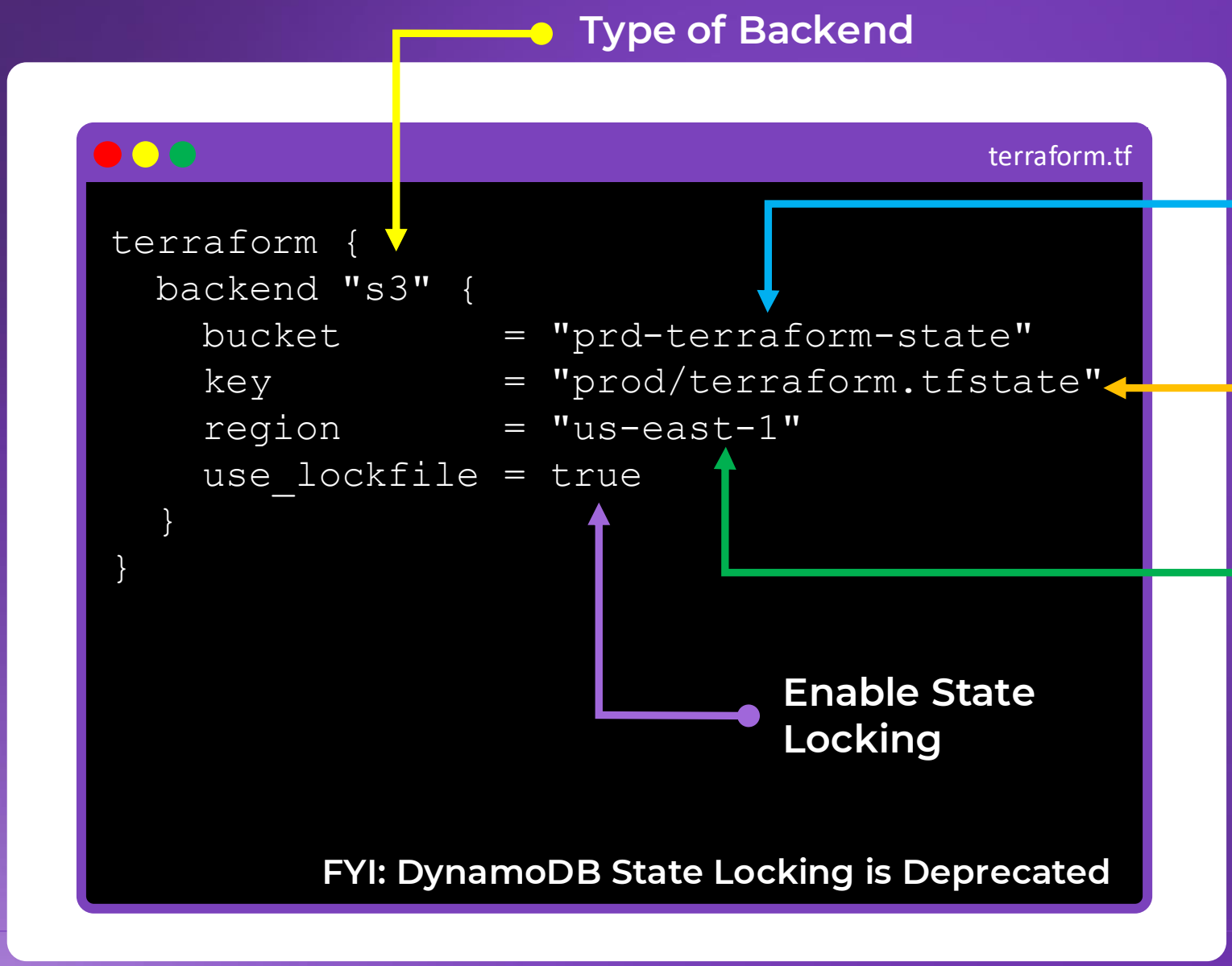
Key Components

- Backend Type - Specifies where state is stored (s3, azurerm, gcs, etc.)
- Backend Configuration - Provider-specific settings for accessing the backend
- State Locking - Optional but recommended (Lock file for S3, automatic for others)

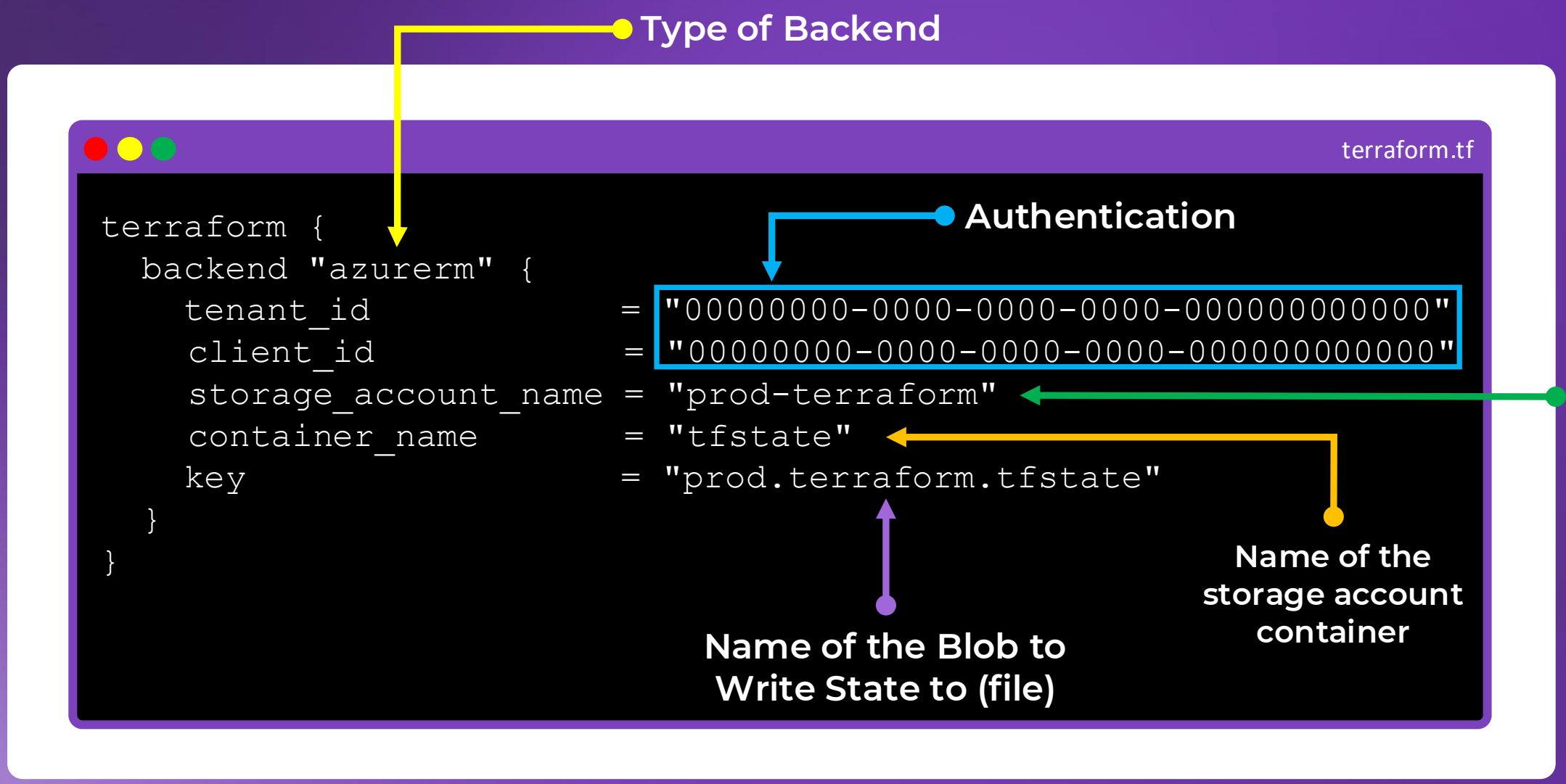
Important Notes

- Backend configuration goes in a `terraform` block, not a `provider` block
- You must run `terraform init` after adding or changing backend configuration

Remote State – Amazon S3

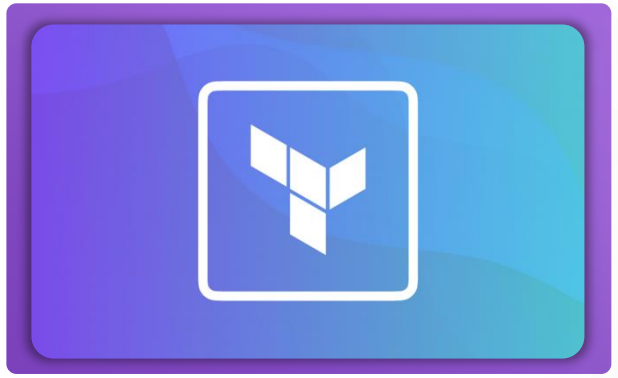


Remote State – Azure Blob Storage



Do NOT set your authentication in this file – use roles or environment variables instead

State Storage Best Practices



Local State

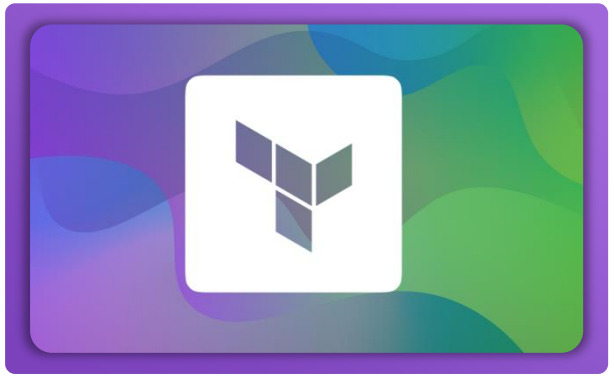
Local State is the default, storing `terraform.tfstate` in your working directory. Perfect for learning and solo projects.

Limitations:

- no team collaboration
- no locking
- plain text security risks
- no automatic backups

Remote State

- Remote State uses backends like S3 or Azure Storage for centralized storage.
- Provides team collaboration, state locking, encrypted storage, and automatic versioning.
- Requires initial setup and may incur costs.



Best Practices

- Use local state for learning and experimentation only.
- Use remote state with locking for all team projects and production.
- This is the recommended approach for the exam and industry standard.

Why Inspect Terraform State?

Understanding What Terraform "Knows"

You've created infrastructure with Terraform. Now what?

Sometimes you might need to:

- See what resources Terraform is managing
- Check the current recorded state of a resource
- Troubleshoot why a plan shows unexpected changes
- Verify what Terraform "thinks" exists

Inspecting State using the Terraform CLI

Primary CLI Commands:

- `terraform state list` – list all resources in Terraform state
- `terraform show` – display state or plan details

Other subcommands available:

- `terraform state mv` – move a resource
- `terraform state rm` – remove a resource from state
- `terraform state pull` – download state from current location
- `terraform state push` – push state to the configured backend

Important:

- ✅ Most have been replaced by config-driven blocks (`moved`, `removed`)
- ✅ They're not emphasized on the 004 exam

Listing Resources

terraform state list

Quick Inventory of Managed Resources

See everything that Terraform is Managing or check if a resource is already managed

Find the Exact Address of a Resource

Maybe you need the address for other commands or for troubleshooting

Verify a Resource

Make sure the resource was successfully created and is being managed.

TERMINAL

```
$ terraform state list

data.aws_availability_zones.available
data.aws_region.current
aws_eip.nat_gateway_eip
aws_internet_gateway.internet_gateway
aws_nat_gateway.nat_gateway
aws_route_table.private_route_table
aws_route_table.public_route_table
aws_subnet.public[0]
aws_subnet.public[1]
aws_vpc.vpc
```

Only returns the address of resources

TERMINAL

```
$ terraform show
# aws_eip.nat_gateway_eip:
resource "aws_eip" "nat_gateway_eip" {
    allocation_id            = "eipalloc-077b7c1af359aa7"
    association_id           = null
    carrier_ip               = null
    customer_owned_ip        = null
    customer_owned_ipv4_pool = null
    domain                   = "vpc"
    id                       = "eipalloc-077b7c1af359a6e"
    instance                 = null
    network_border_group     = "us-east-1"
    network_interface        = null
    private_ip               = null
    public_dns               = "ec2-34-197-203-182.....
    public_ip                = "34.197.203.182"
    public_ipv4_pool         = "amazon"
    tags                     = {
        "Name" = "demo_igw_eip"
    }
    tags_all                 = {
        "Name" = "demo_igw_eip"
    }
    vpc                      = true
}

# aws_internet_gateway.internet_gateway:
```

terraform show

Displays human-readable output of your state or plan file

When to use it:

- See all attributes of your resources
- Review planned changes in detail
- Inspect computed values and metadata
- Understand the current recorded state

Bryan Krausen

What If You Need a Bit More Focus?

Want details about just one resource? Use `terraform state show <resource>`

TERMINAL

```
$ terraform state show aws_vpc.vpc
# aws_vpc.vpc:
resource "aws_vpc" "vpc" {
    arn                              = "arn:aws:ec2:us-east-1:1234:vpc/vpc-07462485"
    assign_generated_ipv6_cidr_block = false
    cidr_block                       = "10.1.0.0/16"
    default_network_acl_id           = "acl-01b3609f712b682d0"
    default_route_table_id           = "rtb-0be178d4fc093ed83"
    default_security_group_id        = "sg-07996016eacf3b0c6"
    dhcp_options_id                  = "dopt-019aeb64"
    enable_classiclink               = false
    enable_classiclink_dns_support   = false
    enable_dns_hostnames             = false
```

Output Abbreviated

Pro Tip: Use `terraform state list` **to get exact address, then** `terraform state show` **to inspect it**

Review Legacy State Commands

- `terraform state mv` → **moved block**
- `terraform state rm` → **removed block**
- `terraform state pull/push` → **Manual state operations (rare)**

HashiCorp is moving toward "configuration as code" for all state operations - it's version-controlled, reviewable, and collaborative.

For the 004 exam: Know these exist, but focus on the config-driven alternatives

I've Detected State Drift – How Do I Fix It?

01

Revert the Changes

Force infrastructure back to match your configuration file using a regular `terraform apply`.

Your `.tf` files are the source of truth - overwrite the real-world changes.

02

Accept the Changes

Update state to match reality using **Refresh-Only Mode**.

The real infrastructure becomes the source of truth - update state to accept it.

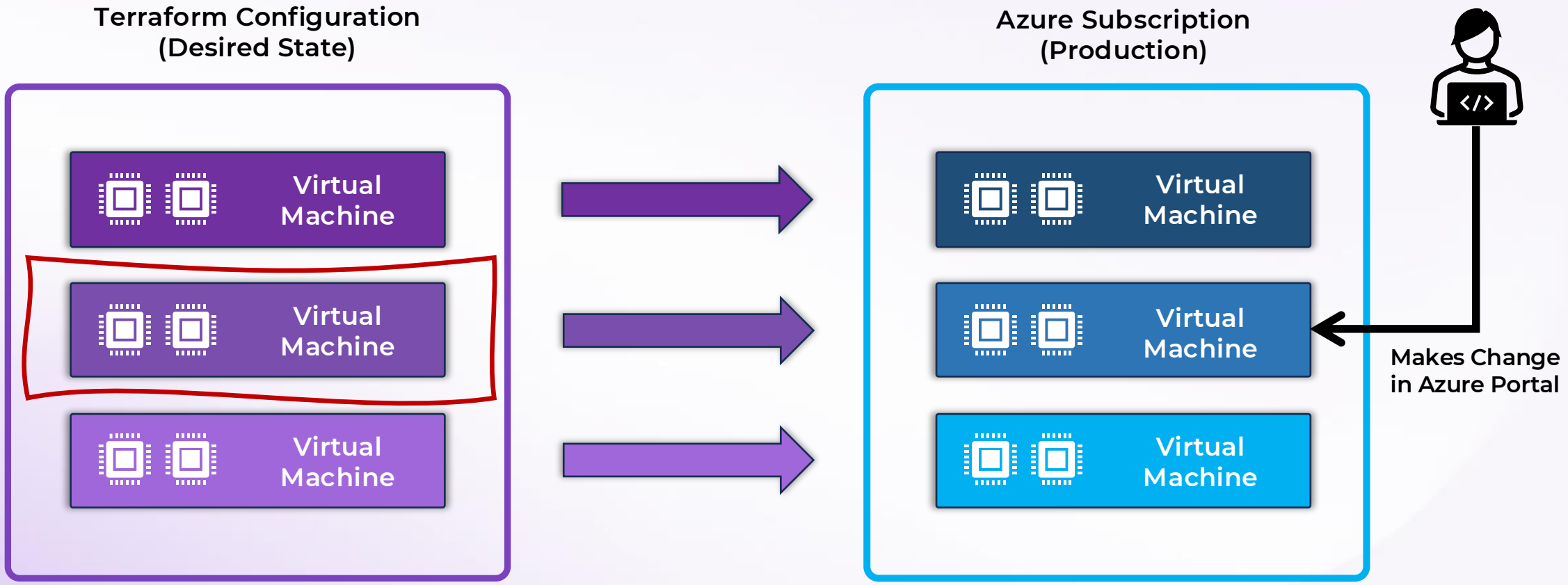
DECIDE

Which Should You Choose?

Revert it if the manual change was a mistake or unauthorized.

Accept it if the change is valid and you want to keep it.

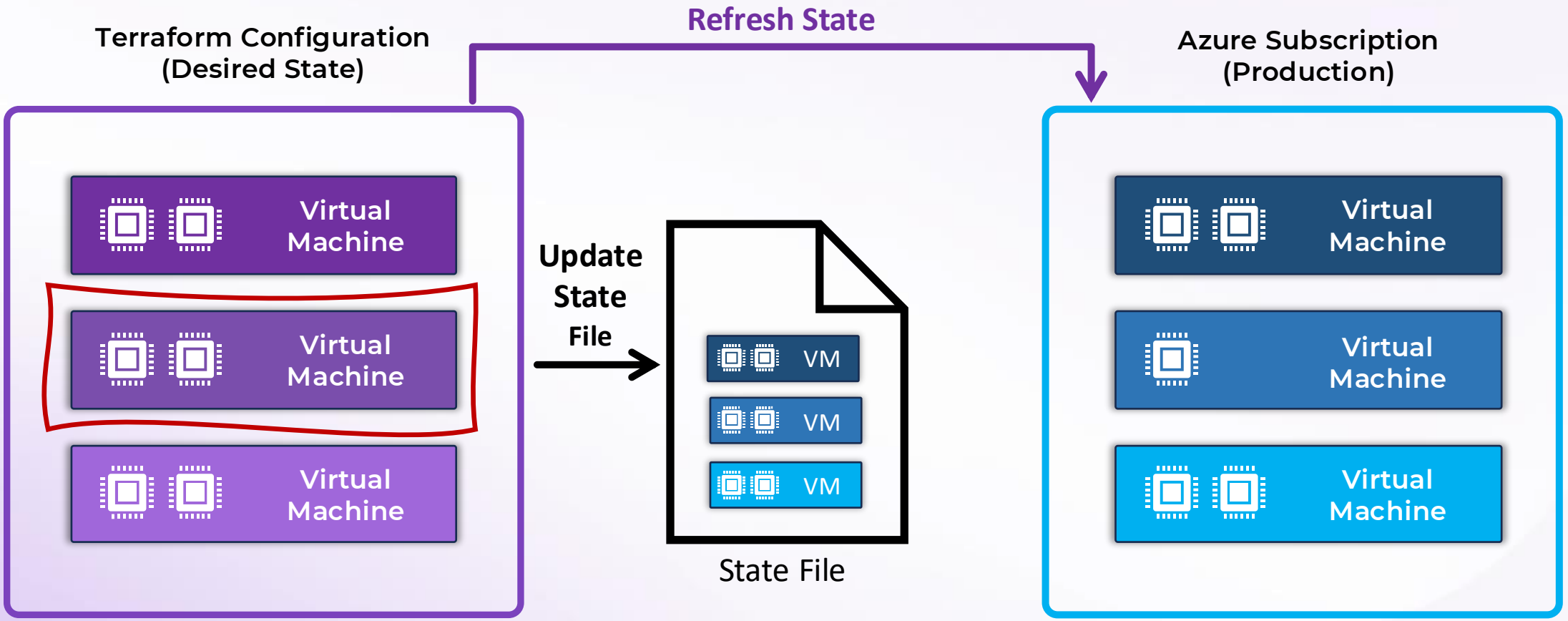
Revert the Changes



Use `terraform apply` to revert the changes

Because you're desired configuration hasn't changed

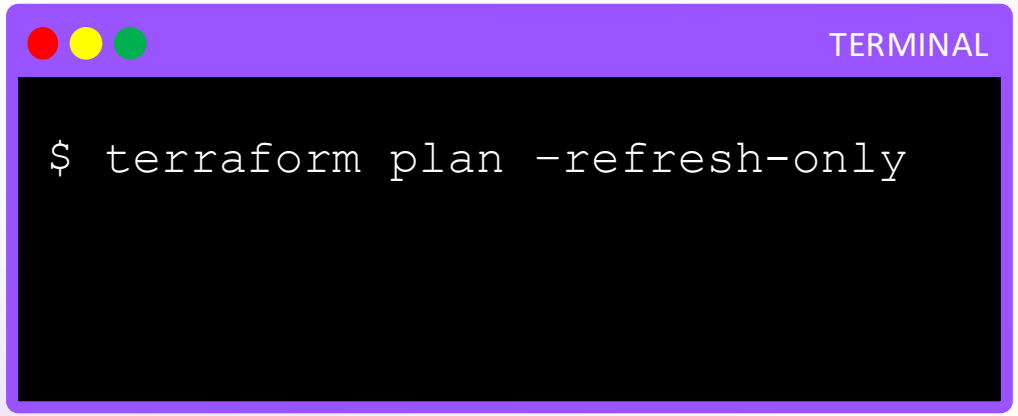
Accept the Changes using Refresh-Only Mode



Use `terraform apply -refresh-only` to update state to match changes

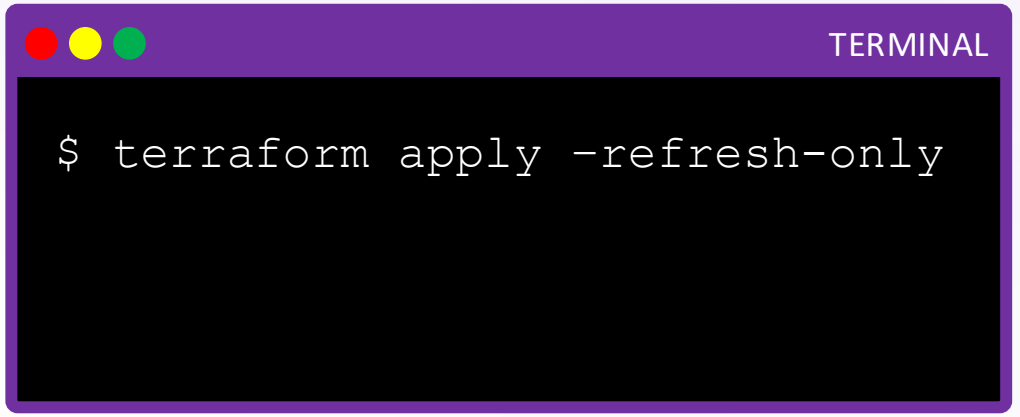
Reconcile your state file to match the drifted remote objects

Refresh-Only Mode



Preview Only - No Changes Made

- Shows you what WOULD change in state
- Safe - makes no modifications
- Use this to investigate drift before committing



Actually Updates State File

- Actually updates the state file
- Makes the changes permanent
- Use after you've reviewed and decided to accept

State Refactoring

Reorganizing how Terraform tracks your infrastructure without changing the infrastructure itself

Common scenarios:

- You renamed a resource in your code
- You're reorganizing modules
- You need to adopt existing infrastructure
- You want to stop managing something but keep it running

Without refactoring tools, Terraform thinks you're destroying old resources and creating new ones

The Old Way

VS

The New Way

- Used separate CLI commands (`terraform state mv`, `terraform state rm`, `terraform import`)
- Commands are run outside your configuration
- Not version-controlled
- Hard to review in pull requests
- Risky - one wrong command and you've modified state permanently

- Use configuration blocks (`moved`, `removed`, `import`)
- Declared in your `.tf` files
- Version-controlled alongside your infrastructure
- Reviewable in PRs
- Part of the standard `plan`/`apply` workflow
- Can be tested before being applied

Core Refactoring Blocks

`moved` block

```
moved {
  from = azurerm_subnet.subnet1
  to   = azurerm_subnet.prod_private
}
```

Purpose
Rename or relocate resources in state

When to Use
Refactoring code structure, renaming resources

`removed` block

```
removed {
  from = aws_instance.production_db

  lifecycle {
    destroy = true
  }
}
```

Purpose
Stop managing a resource but keep it running

When to Use
Handing off ownership, removing from Terraform control

`import` block

```
import {
  to = aws_s3_bucket.terraform_state
  id = "terraform_state_bucket"
}
```

Purpose
Adopt existing infrastructure into Terraform

When to Use
Bringing manually-created resources under management

The Refactoring Workflow

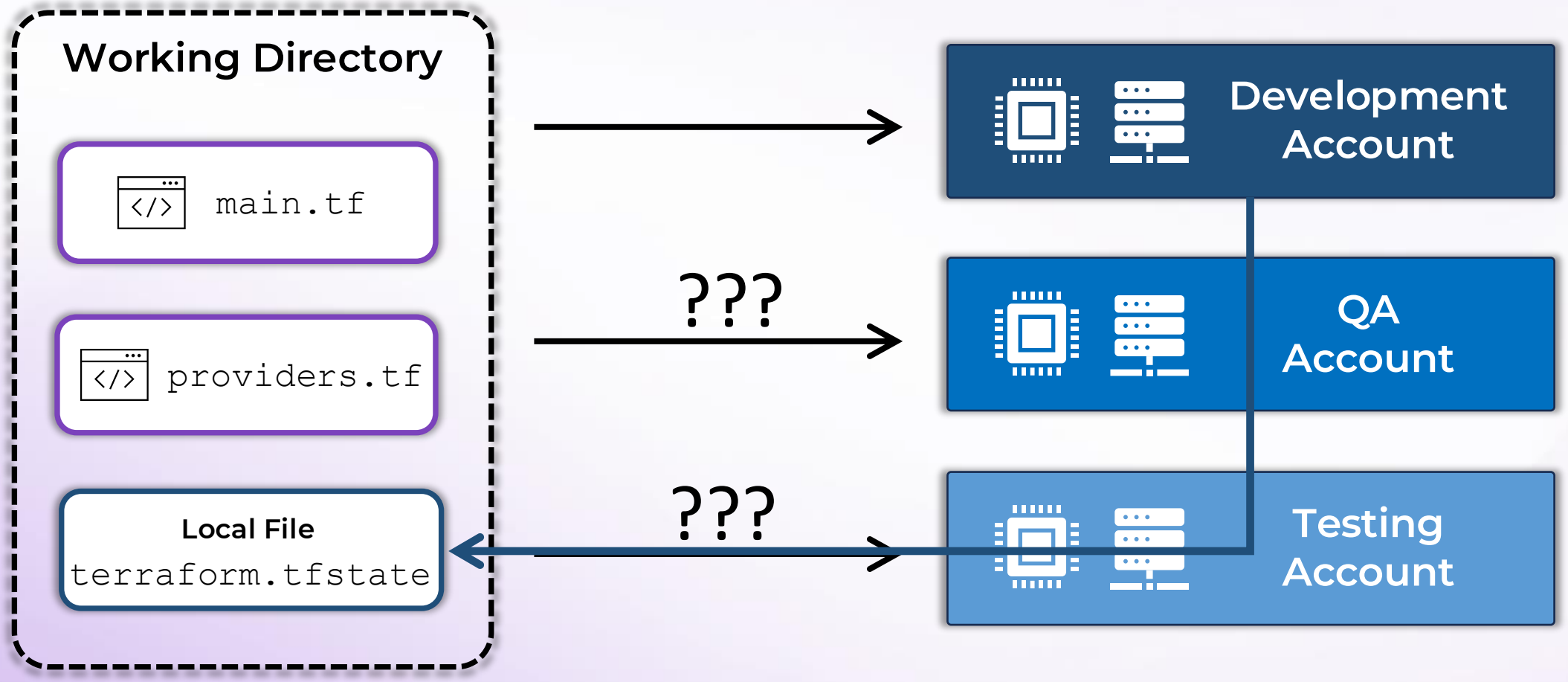
LECTURE

Understanding Terraform Workspaces

Bryan Krausen

Scenario: Managing Mulitple Environmnets

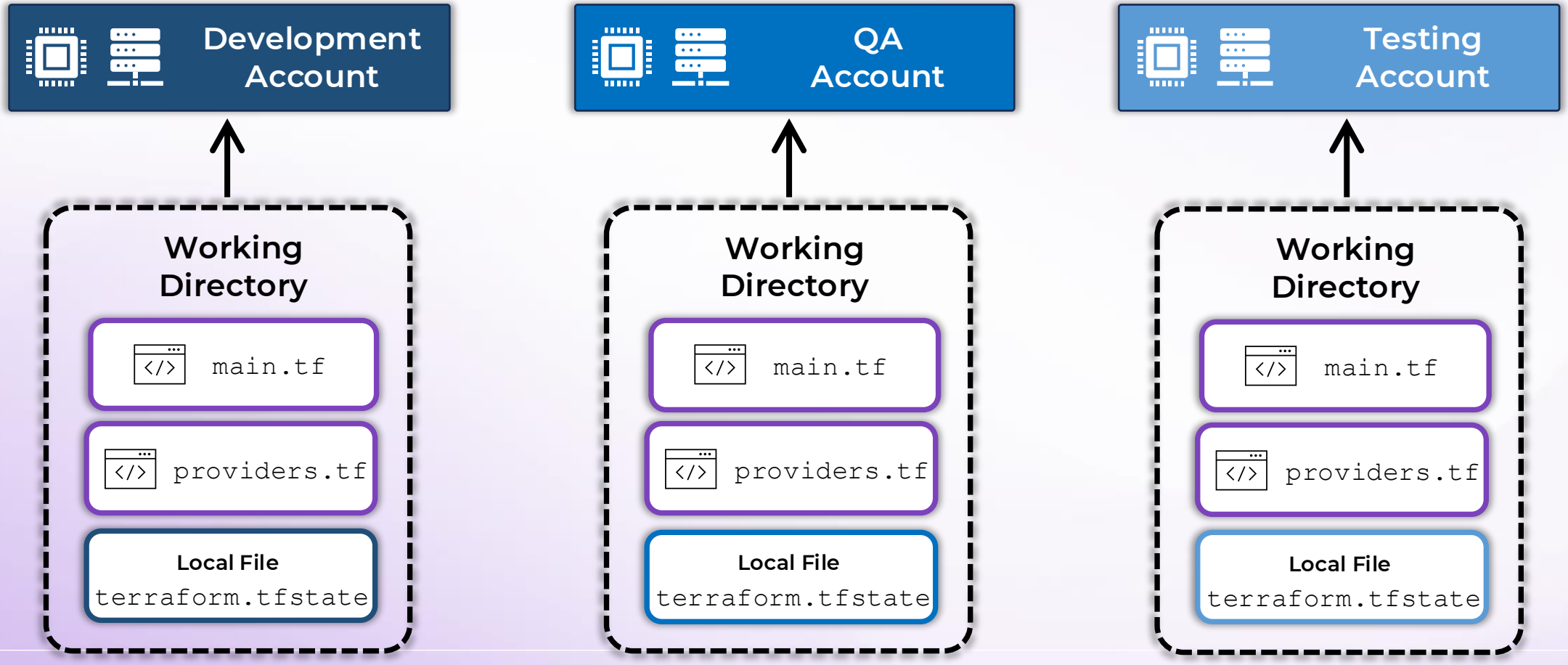
How do you manage three separate versions of the same configuration?



Scenario: Managing Mulitple Environmnets

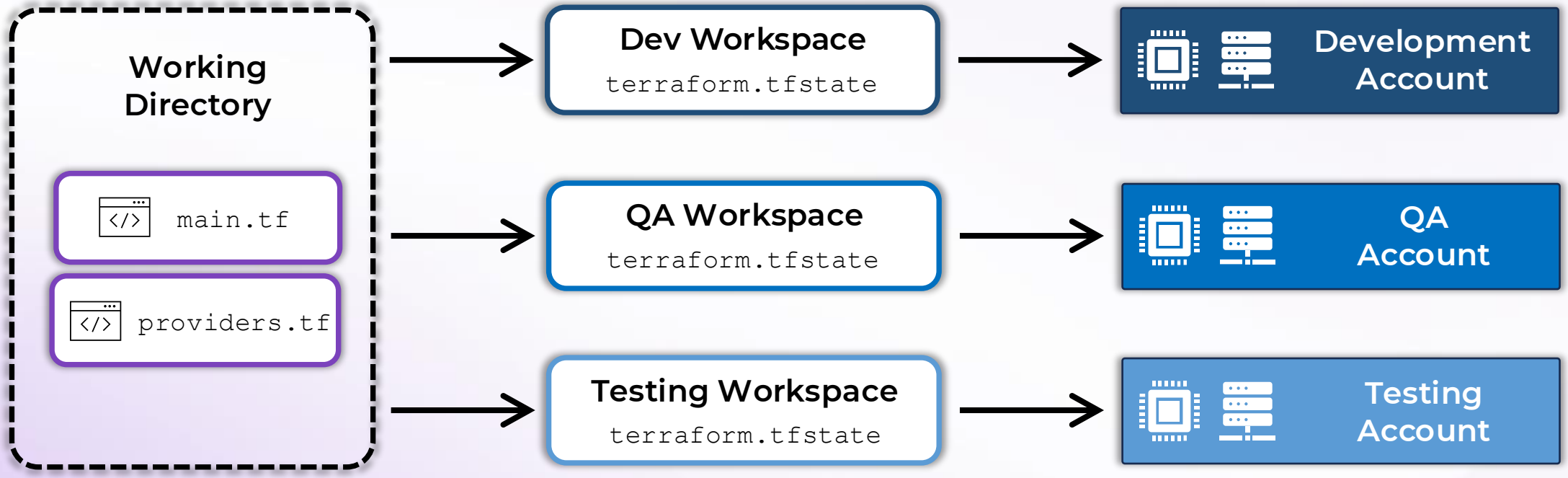
Create Multiple Working Directories

(Results in Code Duplication)



Scenario: Managing Mulitple Environmnets

Use Terraform Workspaces



- ✅ Same configuration code across all environments
- ✅ Each workspace maintains its own isolated state
- ✅ No code duplication or manual syncing required

What is a Terraform Workspace?

A workspace is a named container for a separate instance of your Terraform state.

Each workspace maintains its own isolated state file

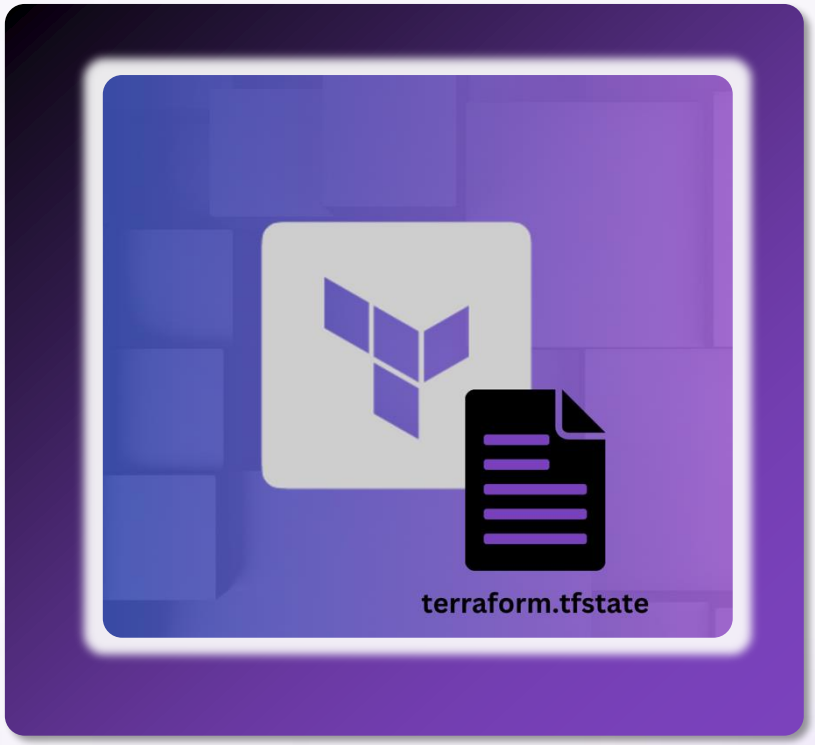
Each workspace tracks different infrastructure resources independently

You've already been using one!

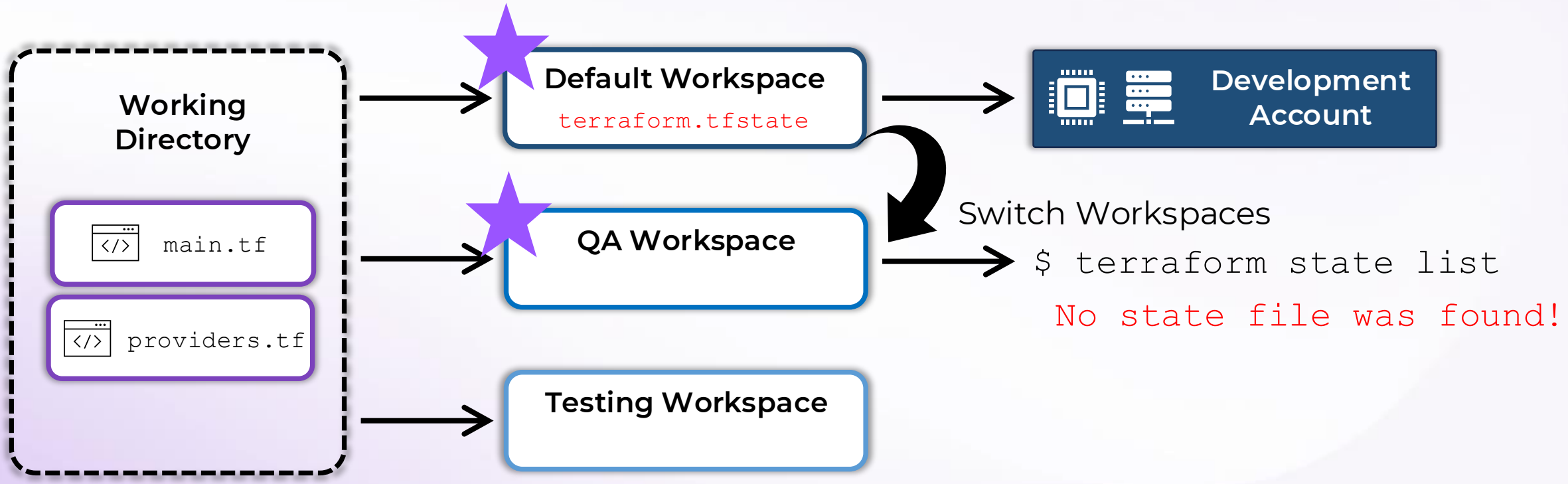
Terraform automatically creates a workspace called `default` when you run `terraform init`

Switch workspaces to switch environments

Same configuration code, same backend, but different state for dev, staging, and production



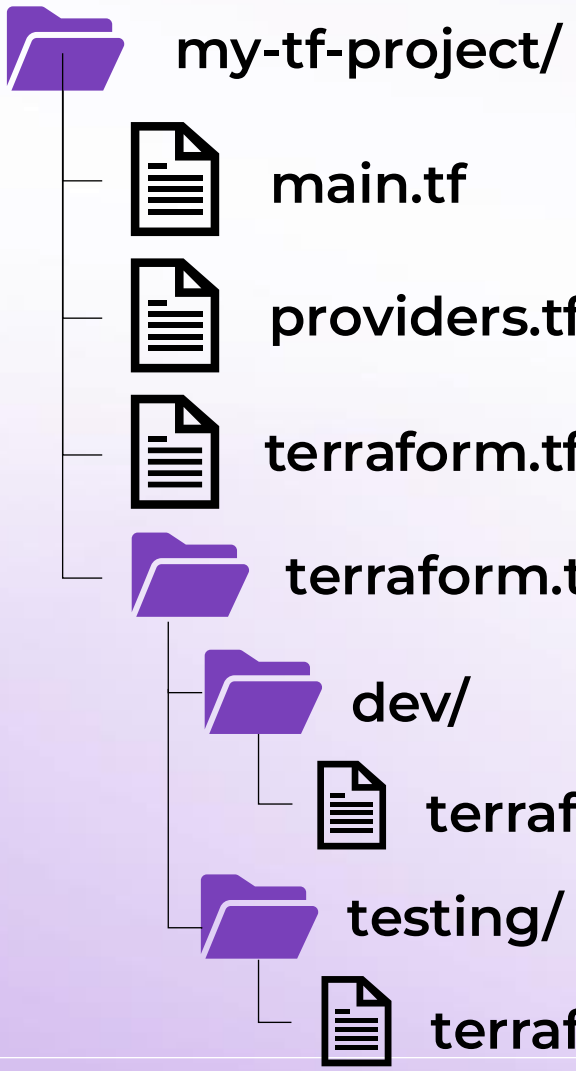
Working with Mulitple Workspaces



Each workspace maintains its own isolated state file

Each workspace tracks different infrastructure resources independently

How Workspaces Store State Files



- When you create additional workspaces, Terraform creates a `terraform.tfstate.d/` directory
- Each workspace gets its own subdirectory with an isolated state file
- The default workspace state remains in the root directory

`default` Workspace

`dev` Workspace

`testing` Workspace

Creating and Switching Workspaces

The `terraform workspace` command

- `list` → List all workspaces
- `new` → Create a new workspace
- `select` → Change the selected namespace
- `show` → Display the current workspace
- `delete` → Delete the specified workspace

TERMINAL

```
$ terraform workspace list
  default
  dev
  prod
* qa
  test
```

Currently selected workspace

TERMINAL

```
$ terraform workspace new dev
Created and switched to workspace "dev"!
```

Two Types of Workspaces: Don't Get Them Confused!

CLI Workspaces

(What we're learning in this lecture)

- Multiple state files in **one configuration**
- Switch between environments (dev, staging, prod)
- Same backend, same code
- Lives on your **local machine** or remote backend
- `terraform workspace select dev`

VS

HCP Terraform Workspaces

(HCP Terraform)

- A complete working directory in the cloud
- Each workspace has its own configuration, variables, and state
- Different workspaces can be connected to different repositories
- Includes VCS integration, run history, and team collaboration
- Managed through the HCP Terraform UI or API

LECTURE

Using Built-In Functions to Standardize Code

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Built-In Functions

Simplify infrastructure code by providing built-in helpers to manipulate strings, numbers, lists, and more.

- Reduces repetitive tasks
- Simplifies logic
- Ensures consistent patterns in deployments

Mastering core functions creates more maintainable, reliable, and readable Terraform configurations.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Core Terraform Functions

Numeric Functions

Quickly evaluate numeric values (e.g., finding the smallest subnet size).

Examples:

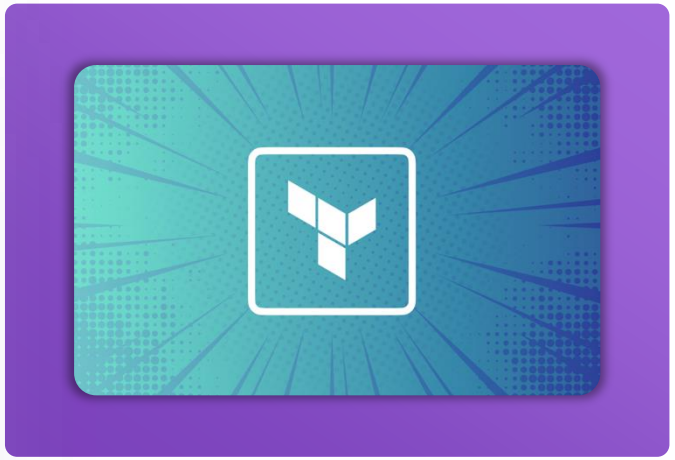
`mix` **&** `max`

String Functions

Concatenate or encode data for resource names, user data, etc.

Examples:

`join` **&** `split`



Type Conversions

Ensure consistent data structures across your configuration and avoid type mismatch issues.

Examples:

`toset` **&** `tolist`

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

General Syntax for Terraform Built-In Functions

Syntax: `function_name(argument1, argument2, ...)`

- **Example 1:** `upper("hello")` → **returns** `"HELLO"`
- **Example 2:** `min(4,7,2,9,5)` → **returns** `2`
- **Example 3:** `join("-", "hello", "terraform")` → `"hello-terraform"`

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Numeric Functions

Determining the largest or smallest value among different variables/outputs

- **Dynamic Resource Allocation:** Adjust CPU, memory, or storage sizes without hardcoding.
- **Automated Calculations:** Remove guesswork by letting Terraform handle numeric logic.

```
# Max: Returns the highest value in a list of numbers
max(10, 4, 7) → 10

# Min: Returns the lowest value in a list of numbers
min(10, 4, 7) → 4

# ####################

variable "number" {
  type    = number
  default = 15
}

max(10, 4, var.number) → 15
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

String Functions

Manipulate and transform textual data — enabling consistent naming, simplified automation, and more maintainable configurations.

```
# Join: Concatenates a list of strings into a single string.
join("-", ["prod", "web", "us-west-1"]) → prod-web-us-west-1

# upper / lower: Purpose: Converts a string to uppercase/lowercase
upper("example") → EXAMPLE
lower("HELLO") → hello

# replace: Substitutes part of a string with another value.
replace("123-abc", "abc", "xyz") → 123-xyz

# base64encode: Encodes string to Base64, often required for user data
base64encode("my-secret-data") → "bXktc2VjcmV0LWRhdGE="
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Network Functions

Specialized functions to work with IP addresses and subnets

- **Automate CIDR Calculations:** Build subnet blocks and host addresses seamlessly.
- **Reduce Manual Errors:** Eliminate hand-calculating IP ranges.
- **Scalable Network Design:** Easily create multiple subnets in large or complex deployments.

```hcl
# Create a VPC with a /16 CIDR
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# Create a public subnet in the first available range
resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  availability_zone = "us-east-1a"
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 3, 0)
}

# Create a private subnet in the second available range
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  availability_zone = "us-east-1b"
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 3, 1)
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Type Conversion Functions

Convert collections of data into different types for inputs or outputs

- `toset` / `tolist`
  Convert collections into lists or sets
- `tomap`
  Convert a list or object to a map
- `tostring`
  Convert the argument to a string

```hcl
variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1a", "us-east-1b"]
}

locals {
  unique_zones = toset(var.availability_zones)
}

resource "aws_subnet" "example" {
  for_each = local.unique_zones

  vpc_id            = aws_vpc.main.id
  availability_zone = each.value
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 2, each.key)
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Enhancing Code with Dynamic Values

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Variables Recap

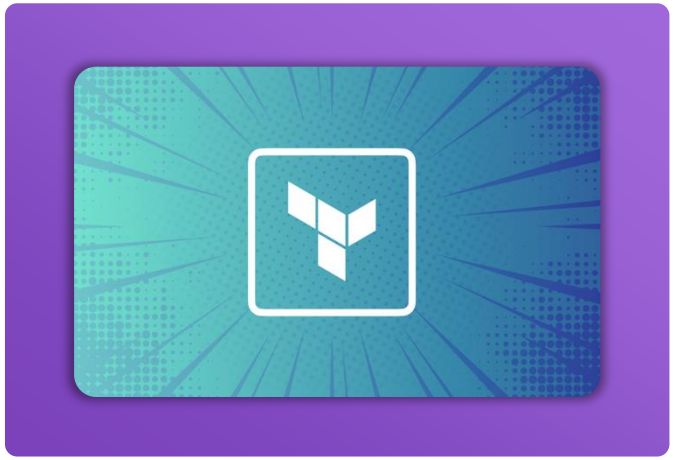
Dynamic Inputs for Terraform Code

Input variables let you customize Terraform configurations without changing the source code or hardcoding values

Variable Types

Variable types include string, number, bool, map, set, list.

Set values via defaults, ENV, .tfvars, and command line



Variables Keep Your Code DRY

Pass in different values to create unique resources

Source: HashiCorp State of Cloud Strategy Survey

Data Sources - Reap

Pull External Information into Terraform

Provider-Specific Info

Data sources (blocks) query provider-specific information about existing resources so we can use it in our code

Common Use Cases

Dynamic lookups, reference existing infrastructure, and avoid hardcoding or repeating information

No Changes

Data sources don't create or make any changes to infrastructure. They only read information and pull it into Terraform

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Combining Variables and Data Sources Can Create More Flexible Code

Dynamic code doesn't use static values – it uses interpolation, meta-arguments, functions, local values, and other methods to adapt to changing needs.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Static Inputs & Variables

main.tf

```
resource "aws_instance" "example" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
}

resource "aws_vpc" "example" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "hardcoded-vpc"
  }
}
```

main.tf

```
resource "github_repository" "example" {
  name        = "my-static-repo"
  description = "A static name"
  visibility  = "public"
}

resource "github_team" "example" {
  name        = "hardcoded-team"
  description = "A team with a name"
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Dynamic Code

```
resource "aws_instance" "example" {
  ami = data.aws_ami.ubuntu.id
  instance_type = local.instance_type

  tags = {
    Name = "${var.app}-${local.env}-server"
  }
}

resource "aws_vpc" "example" {
  cidr_block = var.vpc_cidr

  tags = {
    Name = "vpc-${local.env}-${data.aws_region}-${var.network}"
  }
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Interpolation

This ${ . . . } sequence is called an interpolation. It evaluates the expression between the markers, converts the result, and inserts it into the string

main.tf

```
variable "name" {
  type    = string
  default = "Bryan"
}


"Hello, ${var.name}!"
Hello, Bryan!


"My name is ${var.name}, I'm your instructor!"
My name is Bryan, I'm your instructor!
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Interpolation

main.tf

```
variable "environment" {
  type    = string
  default = "dev"
}

data "aws_region" "current"{}
data "aws_caller_identity" "current" {}


name = "server-${data.aws_region.current.name}-${var.environment}"
server-us-east-1-dev

bucket_name = "s3-${data.aws_caller_identity.current.account_id}-backups"
s3-9876543210-backups
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

LECTURE

Using Locals to Avoid Code Duplication

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

When Code Duplication Strikes

```hcl
resource "some_resource" "server-abc" {
  argument     = var.argument
  agument_type = var.argument_type

  tags = {
    Name        = "${var.app_na
    ManagedBy   = "Terraform"
    Team        = var.team
    Environment = var.environme
    ID          = "server-${loc
  }
}
```

```hcl
resource "some_resource" "server-xyz" {
  argument     = var.argument
  agument_type = var.argument_type

  tags = {
    Name        = "${var.app_name}-${var.server}-xyz"
    ManagedBy   = "Terraform"
    Team        = var.team
    Environment = var.environment
    ID          = "server-${local.env}-${data.aws_region}-${var.server[1]}"
  }
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

What are Local Values?

Named values that you can reference in other parts of your Terraform code

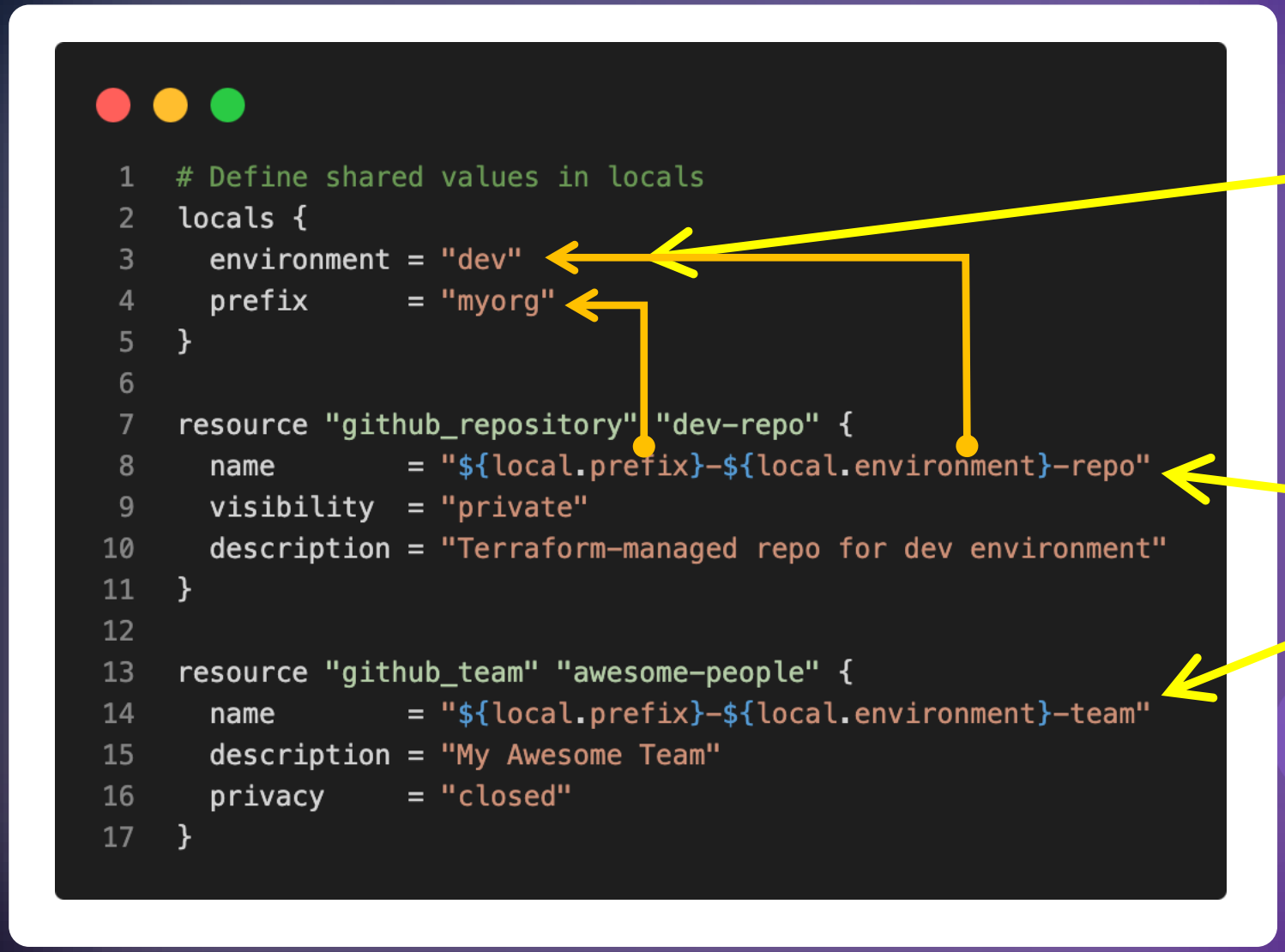
- Most often referred to as "locals," think of locals like short-term memory for your Terraform code
- They centralize and store values you use often, like names, environments, or shared tags
- Local values can be helpful to avoid repeating the same values or expressions multiple times in a configuration

```
1  locals {
2    app_name    = "my-app"
3    environment = "dev"
4  }
5
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Bryan Krausen

Basic Syntax - Local Values



```hcl
# Define shared values in locals
locals {
  environment = "dev"
  prefix      = "myorg"
}

resource "github_repository" "dev-repo" {
  name        = "${local.prefix}-${local.environment}-repo"
  visibility  = "private"
  description = "Terraform-managed repo for dev environment"
}

resource "github_team" "awesome-people" {
  name        = "${local.prefix}-${local.environment}-team"
  description = "My Awesome Team"
  privacy     = "closed"
}
```

locals block

Using a locals block

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Bryan Krausen

Basic Syntax - Local Values

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

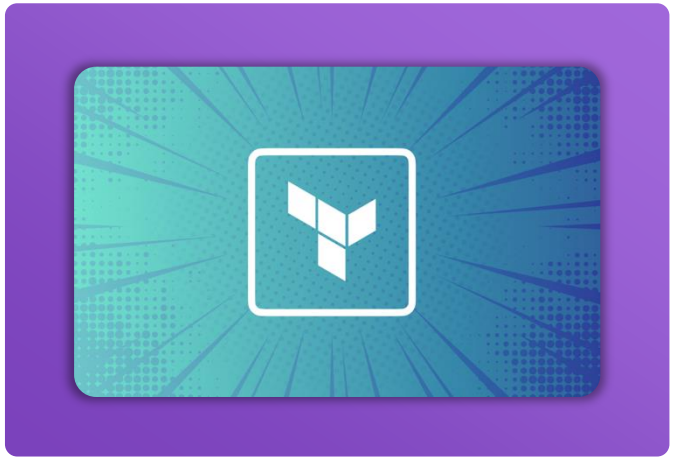
Benefits of Locals

Centralized Logic

The ability to easily change the value in a central place is the key advantage of local values

Clearer Code

Reduces clustter and makes it obvious where a value comes from



Less Risk

Changing a local is less error-prone than updating multiple hardcoded values

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

When Should I Use Local Values?

- When you need to combine variables or data sources to create dynamic values, locals help keep your code readable and consistent.

- If the same value appears multiple times (like a prefix, environment name, or common tag), define it once in locals instead of repeating it across multiple resources.

- Instead of embedding lengthy calculations or conditions directly in resource definitions, you can compute them in locals first and reference them where needed.

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

`count` Meta-Argument

Specify a number of instances for a resource

- Allows Terraform to create multiple identical resources using a single block
- Assigns an index (starts at 0) to reference each instance
- Use `count` to create resources that are identical but need to be repeated or when a variable controls the number of resources
- Reference the resource using `<type>.<name>[index]`

```hcl
variable "vm_count" {
  type    = number
  default = 3
}

resource "azurerm_virtual_machine" "example" {
  count               = var.vm_count
  name                = "vm-${count.index}"
  location            = "East US"
  resource_group_name = "my-resource-group"
  vm_size             = "Standard_D2s_v3"

  os_profile {
    computer_name  = "vm-${count.index}"
    admin_username = "adminuser"
  }
}
```

Variable controlling the number of resources

`azurerm_virtual_machine.example[0]`

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

`for_each` Meta-Argument

A More Flexible Way to Manage Multiple Resources

Creates Multiple Resources

Pulls in values from a map or set rather than a fixed number

Resources Get a Unique Key

Rather than an index, each resource gets a unique key as a reference, making it more stable than count

Ideal for Managing Unique Configurations

Use it when resources need different names, tags, or settings

```
1  variable "vms" {
2    type = map(string)
3    default = {
4      "vm1" = "Standard_D2s_v3"
5      "vm2" = "Standard_D4s_v3"
6      "vm3" = "Standard_B2ms"
7    }
8  }
9
10 resource "azurerm_virtual_machine" "example" {
11   for_each            = var.vms
12   name                = each.key
13   location            = "East US"
14   resource_group_name = "my-resource-group"
15   vm_size             = each.value
16
17   os_profile {
18     computer_name  = each.key
19     admin_username = "adminuser"
20   }
21 }
azurerm_virtual_machine.example["vm1"]
```

Variable controlling the resources

Key

Value

®Copyright Bryan Krausen – DO NOT DISTRIBUT

State Refactoring

Reorganizing how Terraform tracks your infrastructure without changing the infrastructure itself

Common scenarios:

- You renamed a resource in your code
- You're reorganizing modules
- You need to adopt existing infrastructure
- You want to stop managing something but keep it running

Without refactoring tools, Terraform thinks you're destroying old resources and creating new ones

The Old Way

VS

The New Way

- Used separate CLI commands (`terraform state mv`, `terraform state rm`, `terraform import`)
- Commands are run outside your configuration
- Not version-controlled
- Hard to review in pull requests
- Risky - one wrong command and you've modified state permanently

- Use configuration blocks (`moved`, `removed`, `import`)
- Declared in your `.tf` files
- Version-controlled alongside your infrastructure
- Reviewable in PRs
- Part of the standard `plan`/`apply` workflow
- Can be tested before being applied

Core Refactoring Blocks

`moved` block

```
moved {
  from = azurerm_subnet.subnet1
  to   = azurerm_subnet.prod_private
}
```

Purpose
Rename or relocate resources in state

When to Use
Refactoring code structure, renaming resources

`removed` block

```
removed {
  from = aws_instance.production_db

  lifecycle {
    destroy = true
  }
}
```

Purpose
Stop managing a resource but keep it running

When to Use
Handing off ownership, removing from Terraform control

`import` block

```
import {
  to = aws_s3_bucket.terraform_state
  id = "terraform_state_bucket"
}
```

Purpose
Adopt existing infrastructure into Terraform

When to Use
Bringing manually-created resources under management

The Refactoring Workflow

LECTURE

Moved Block

The Real-World Scenario...

- After building our infrastructure, we realize that our naming isn't clear...
- Blocks are very similar, but the names aren't descriptive in relation to our resources...
- We need to refactor, but what happens when we rename this?

```
resource "aws_vpc" "vpc" {
  cidr_block = var.sandbox_vpc_cidr
  tags = {
    Name        = var.sbx_vpc_name
    Environment = "sandbox_network"
    Terraform   = "true"
  }
}

resource "aws_vpc" "vpc1" {
  cidr_block = var.test_vpc_cidr
  tags = {
    Name        = var.test_vpc
    Environment = "testing_network"
    Terraform   = "true"
  }
}
```

You Decide to Rename Using a Naming Standard...

main.tf

```
resource "aws_vpc" "vpc" {
  cidr_block = var.sandbox_vpc_cidr
  tags = {
    Name        = var.sbx_vpc_name
    Environment = "sandbox_network"
    Terraform   = "true"
  }
}
```

main.tf

```
                                                    cidr

                                                    me
                                                    rk"
```

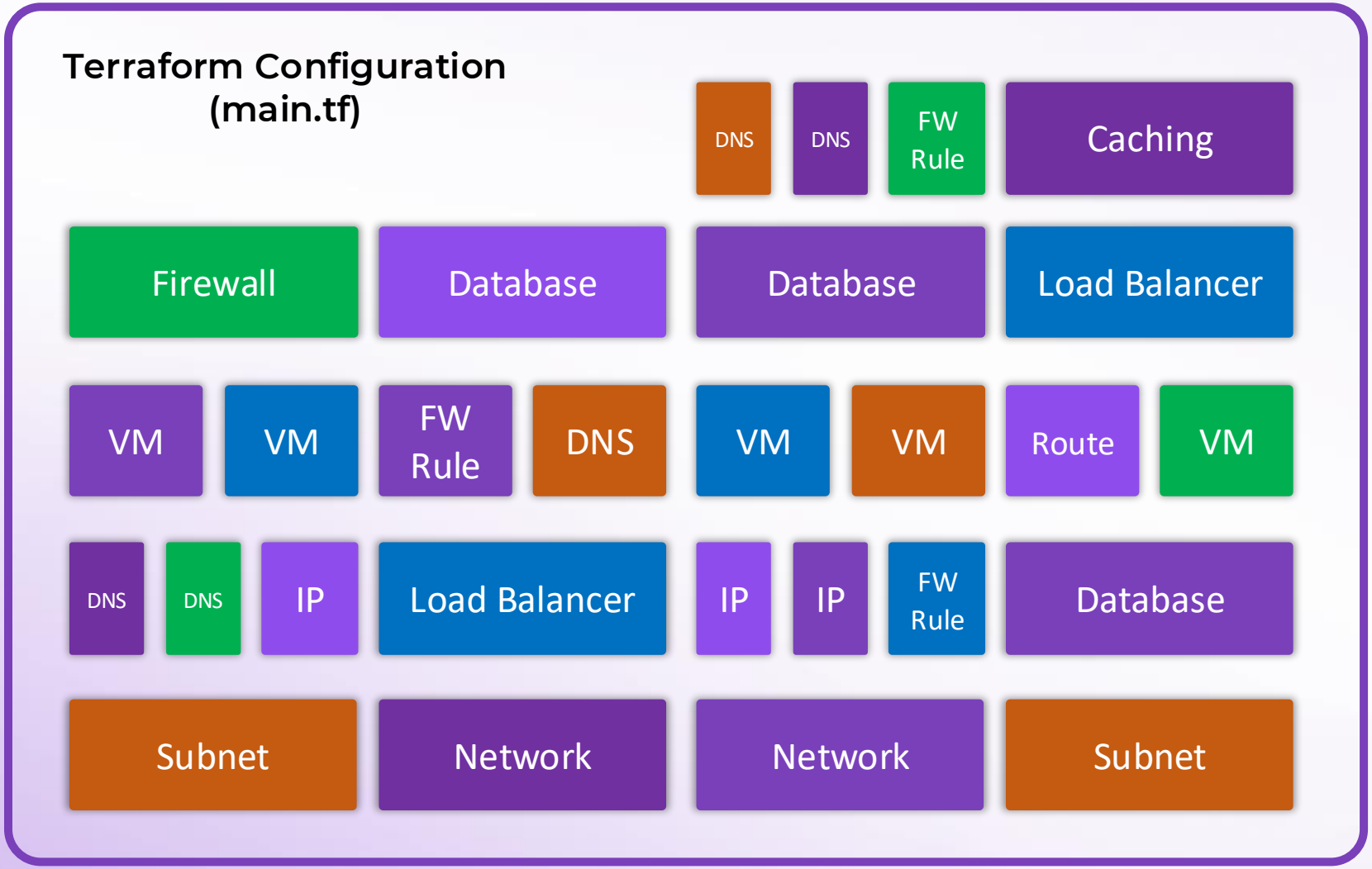
```
$ terraform plan
Terraform will perform the following actions:

  # aws_vpc.prd_vpc will be created
  + resource "aws_vpc" "prd_vpc" {
      + arn                                  = (know
 # aws_vpc.vpc will be destroyed
  # (because aws_vpc.vpc is not in configuration)
  - resource "aws_vpc" "vpc" {

Plan: 1 to add, 0 to change, 1 to destroy.
```

ERMINAL

ted Output

Bryan Krausen

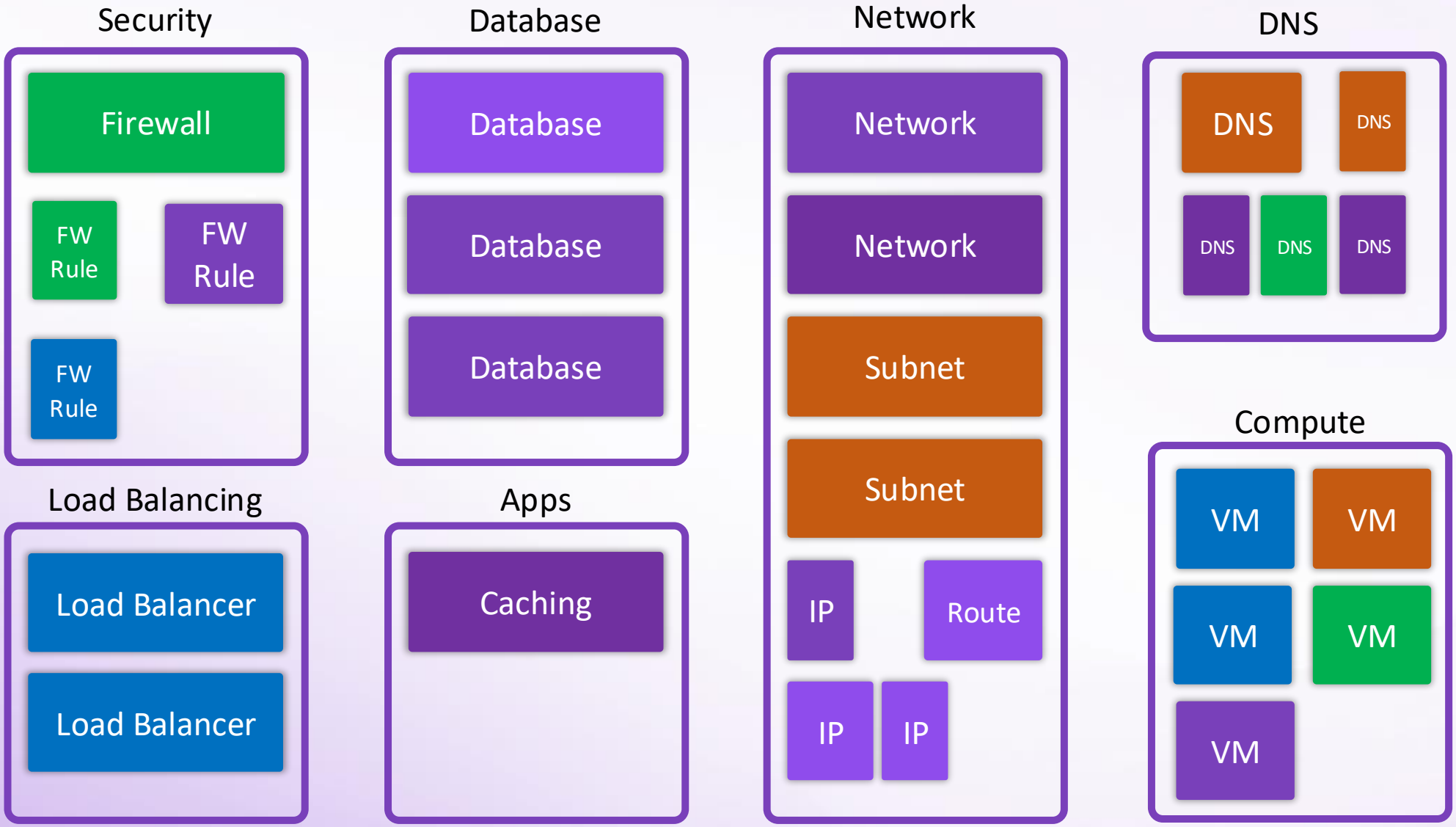
Another Scenario...



"I need to reorganize this"

Bryan Krausen

Another Scenario...



Bryan Krausen

Moved Block

The `moved` block tells Terraform you've renamed or relocated a resource in your code. It updates the state file to reflect the new address without destroying and recreating your actual infrastructure.

Using the `moved` block prevents downtime and data loss during refactoring. It's your safety net when cleaning up resource names or restructuring your configuration.

Moved Block

The `moved` block updates state to reflect refactored resource addresses without destroying infrastructure.

Two Parameters

`from` = specify the original resource address
`to` = specify the new resource address

State Updates Only

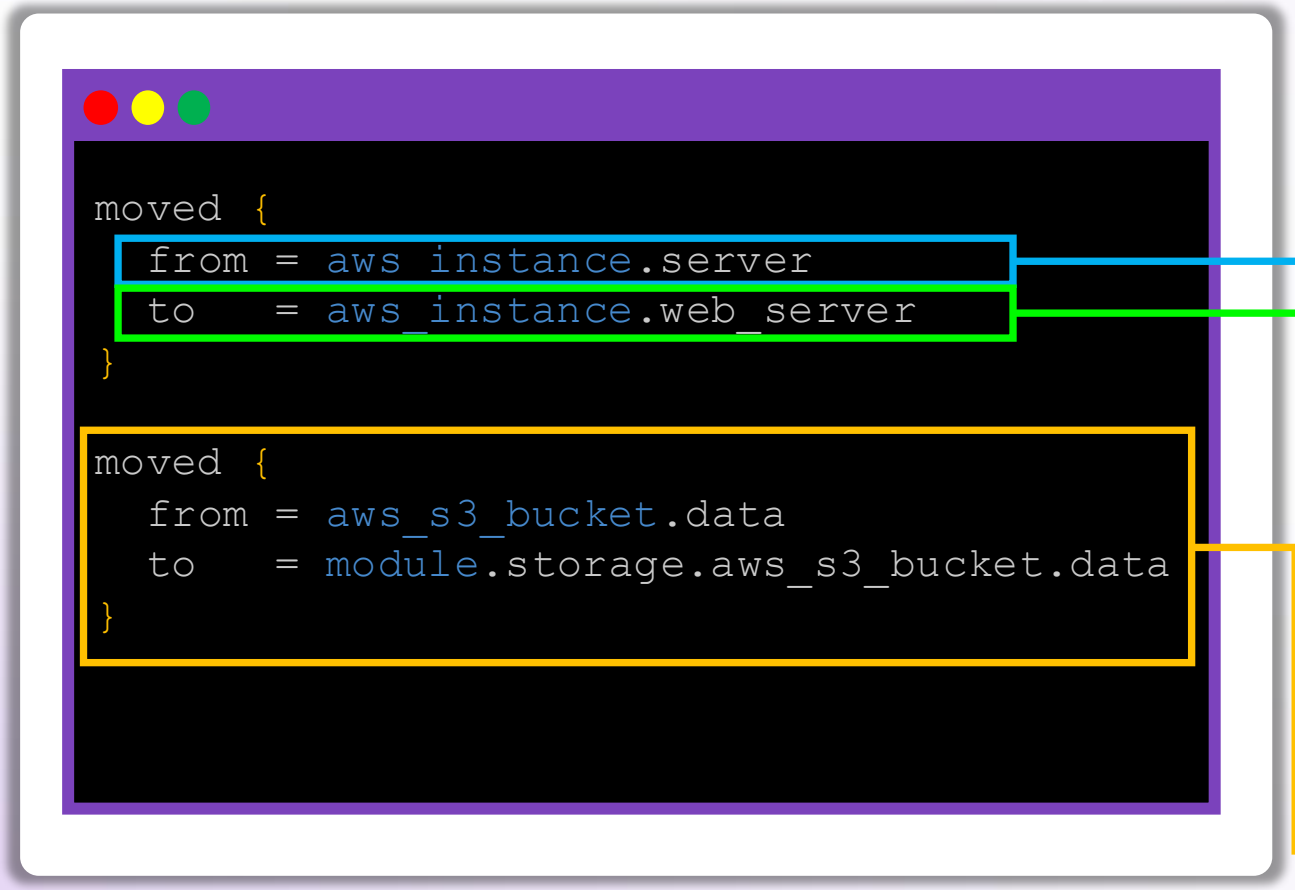
Updates only the state file - no infrastructure changes are made

Resource Blocks Must Exist

You MUST write the new `resource` blocks before refactoring resource address

```
moved {
  from = aws_instance.server
  to   = aws_instance.web_server
}

moved {
  from = aws_s3_bucket.data
  to   = module.storage.aws_s3_bucket.data
}

moved {
  from = module.old_app
  to   = module.new_app
}
```

Breaking Down the Moved Block



Where was the original resource block managing this infrastructure?

Where is the NEW resource block that will manage this infrastructure?

An example showing that you want to move the resource block into a new module.

The Refactoring Workflow

Write

Add new `resourse` **blocks to Terraform config**

Write

Add new `moved` **blocks to Terraform config**

Plan

`terraform plan` **(validate)**

Apply

`terraform apply` **(updates state)**

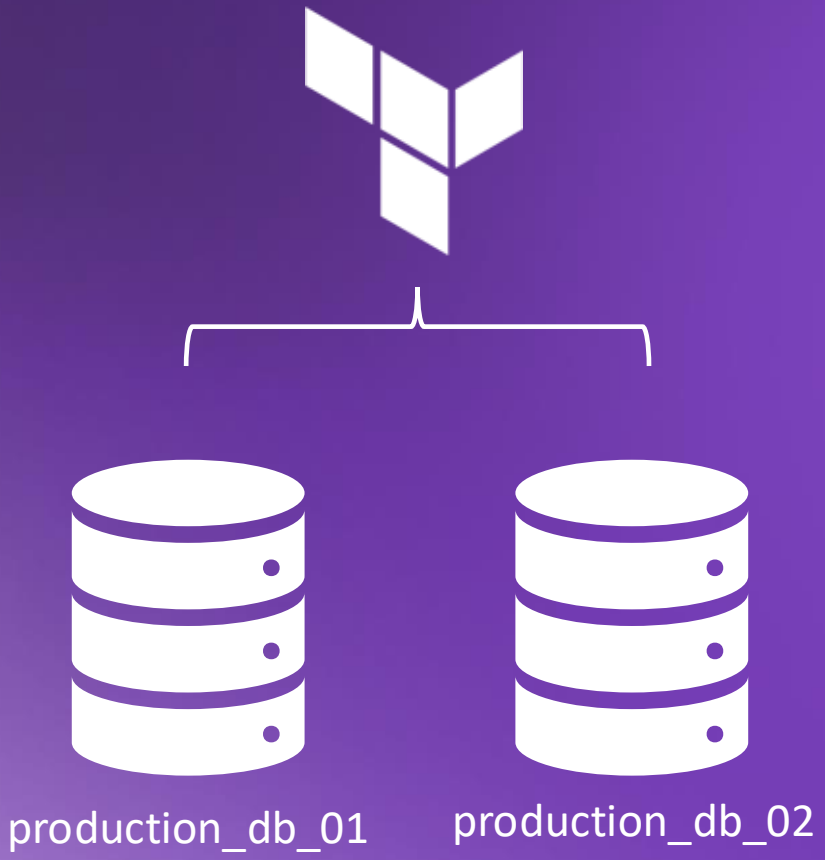
Delete

Delete the `moved` **blocks**

LECTURE

Removed Block

Real-World Scenario



database.tf

```
resource "google_sql_database_instance" "prd_db_1" {
  name             = "production-db-01"
  database_version = "POSTGRES_15"
  region           = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}

resource "google_sql_database_instance" "prd_db_2" {
  name             = "production-db-02"
  database_version = "POSTGRES_15"
  region           = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}
```

The database is now critical to production and the DBA team wants manual control over schema changes

What NOT To Do!

database.tf

```
resource "google_sql_database_instance" "prd_db_1" {
  name             = "production-db-01"
  database_version = "POSTGRES_15"
  region           = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}

resource "google_sql_database_instance"
  name             = "production-db-02"
  database_version = "POSTGRES_15"
  region           = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}
```

WAIT! We just want to stop managing it, not DELETE IT!

Removed Block

The `removed` block instructs Terraform to remove a resource from state.

- **The `from` argument**

  Create standardized infrastructure patterns that can be used across multiple projects and teams.

- **Lifecycle Argument: `destroy = false`**

  By default, Terraform removes the resource from state and destroys the actual resource.

- **State Removal Only**

  Removes from state file - infrastructure remains untouched

database.tf

```
removed {
  from = google_sql_database_instance.prd_db_1
  lifecycle {
    destroy = false
  }
}

removed {
  from = google_sql_database_instance.prd_db_2
  lifecycle {
    destroy = false
  }
}
```

You must also remove the resource block from your config

Key Differences

Compring the Removed Block to the Moved Block

`moved` Block

vs

`removed` Block

"I'm reorganizing my code, update state to match new addresses"

"I'm done managing this with Terraform, remove it from state"

The Refactoring Workflow

Decision Tree

What Do You Need To Do?	What Block to Use?
Renaming resources?	`moved` block
Reorganizing into modules?	`moved` block
Stopping Terraform management?	`removed` block
Handing off to another team?	`removed` block
Splitting configurations?	`removed` block

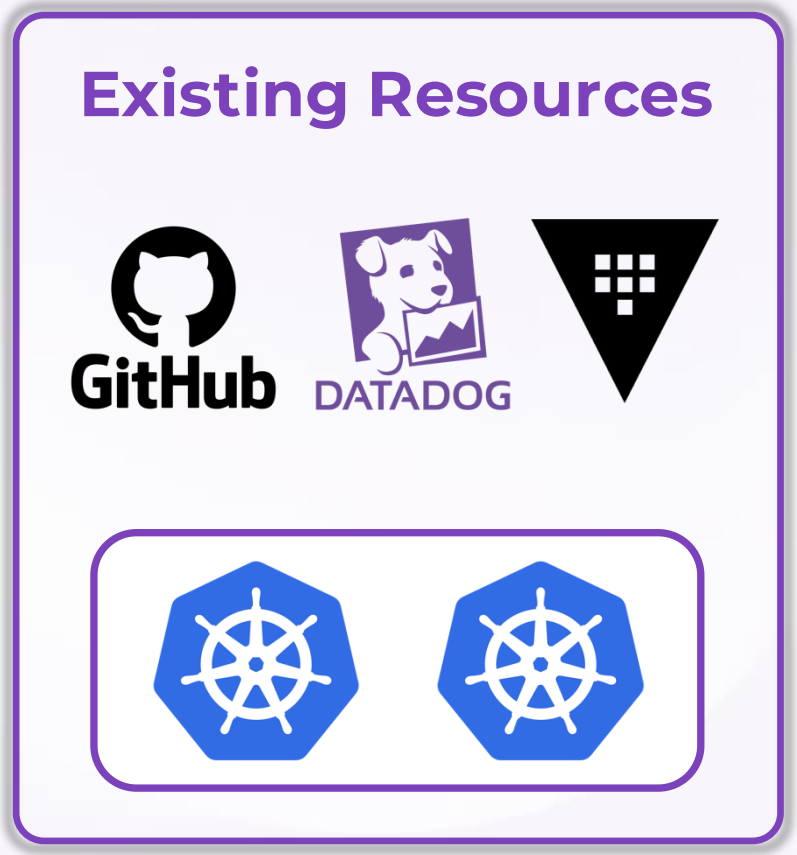
LECTURE

Import Block

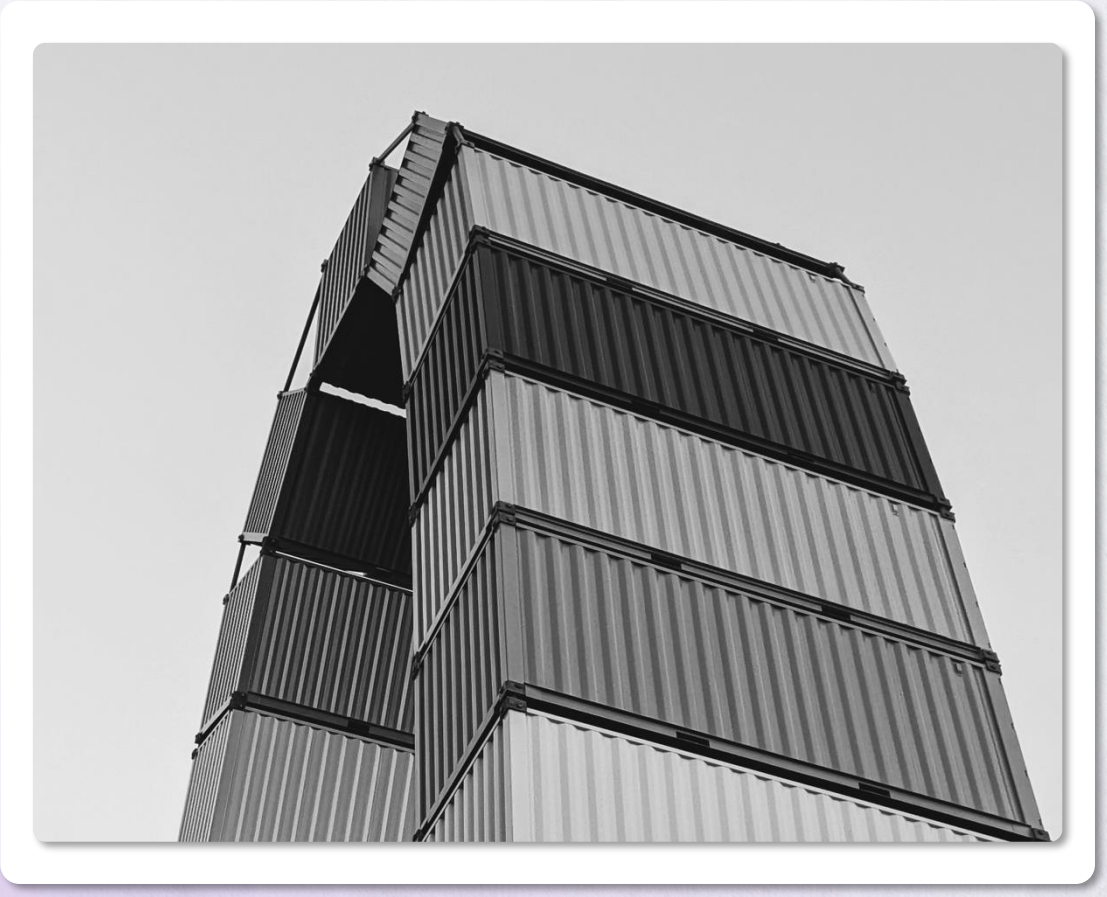
Manage all the Things with Terraform...

```
resource "aws_instance" "web" {
  ami           = "ami-012345"
  instance_type = "t2.micro"

  key_name   = "prd-web-key"
  subnet_id = "subnet_12345abc"

  tags = {
    Name = "prd-web-svr_01"
  }
}

resource "aws_instance" "db" {
  instance_type = "m5.4xlarge"
  ami           = "ami-0c55b159"
}
```

After learning Terraform, you want to start provisioning all resources using Infrastucture as Code



...but what about these resources already running in the cloud/on-premises?



Import Block

The `import` block in Terraform lets you bring existing infrastructure under Terraform management by mapping real-world resources to your configuration. This allows Terraform to track and manage them without manual setup.

It simplifies migration, reduces effort, and ensures infrastructure is versioned and controlled.

Import Block

Define an input that can be reused and customized to make configurations more flexible and dynamic

Map Existing Resources

Links real-world infrastructure to Terraform without changing it

Simplifies Migration

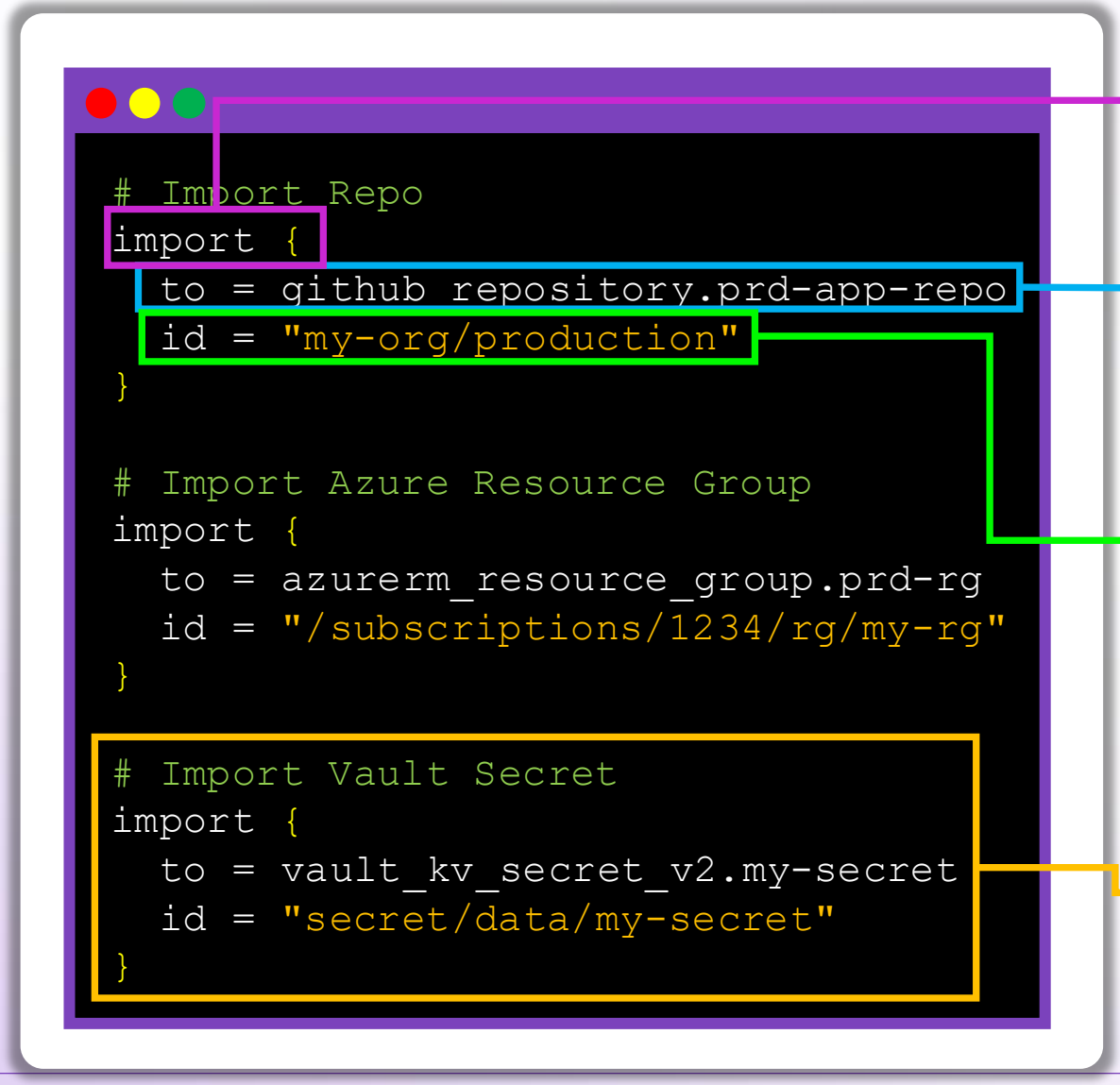
Reduces manual setup when adopting Terraform

Enables State Management

Bring unmanaged resources under Terraform control

```
# Import Repo
import {
  to = github_repository.prd-app-repo
  id = "my-org/production"
}

# Import Azure Resource Group
import {
  to = azurerm_resource_group.prd-rg
  id = "/subscriptions/1234/rg/my-rg"
}

# Import Vault Secret
import {
  to = vault_kv_secret_v2.my-secret
  id = "secret/data/my-secret"
}
```

Breaking Down the Import Block



```
# Import Repo
import {
  to = github_repository.prd-app-repo
  id = "my-org/production"
}

# Import Azure Resource Group
import {
  to = azurerm_resource_group.prd-rg
  id = "/subscriptions/1234/rg/my-rg"
}

# Import Vault Secret
import {
  to = vault_kv_secret_v2.my-secret
  id = "secret/data/my-secret"
}
```

The import block & curly braces

What resource block in Terrform will this be mapped to?

What is the actual resource that you want to import? A VM? A repo? A network?

Another example of importing a resource from Vault

Import Block - Requirements

- Before importing a resource, you need to create the resource block that will manage the imported resource moving forward.
- You can write it manually or use `terraform plan -generate-config-out=PATH`

```
# Import Repo
import {
  to = github_repository.prd-app
  id = "my-org/production"
}

# Import Vault Secret
import {
  to = vault_kv_secret_v2.secret
  id = "secret/data/my-secret"
}
```

```
resource "github_repository" "prd-app" {
  name        = "production"
  description = "Production Codebase"
}


resource "vault_kv_secret_v2" "secret" {
  mount     = "secret"
  name      = my-secret
  data_json = jsoncode ({admin=pass"})
}
```

`terraform import` Command

The original CLI command for importing existing resources into Terraform state

How it works:

- Run command with resource address and cloud provider ID
- Resource added to state immediately
- Still required writing configuration manually
- No plan step - direct state modification

For the exam: Know that `terraform import` exists, but focus on the `import` block as the modern approach.

You MUST write the resource block first before importing

main.tf

```
resource "aws_instance" "web" {
  ami           = var.prod_ami
  instance_type = m5.medium

  tags = {
    Name = "Production Web"
  }
}
```

TERMINAL

```
$ terraform import aws_instance.web i-12367890abcdef0

aws_instance.web: Importing from ID "i-12367890abcdef0"...
aws_instance.web: Import prepared!
aws_instance.web: Refreshing state...

Import successful!
```

LECTURE

Intro to Modules

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

What are Modules?

A module is a container for related resources that are used together.

They are the primary way to package and reuse configurations.

Improved Organization

Break large configurations into logical units to easily reuse over and over again.

Easier Collaboration

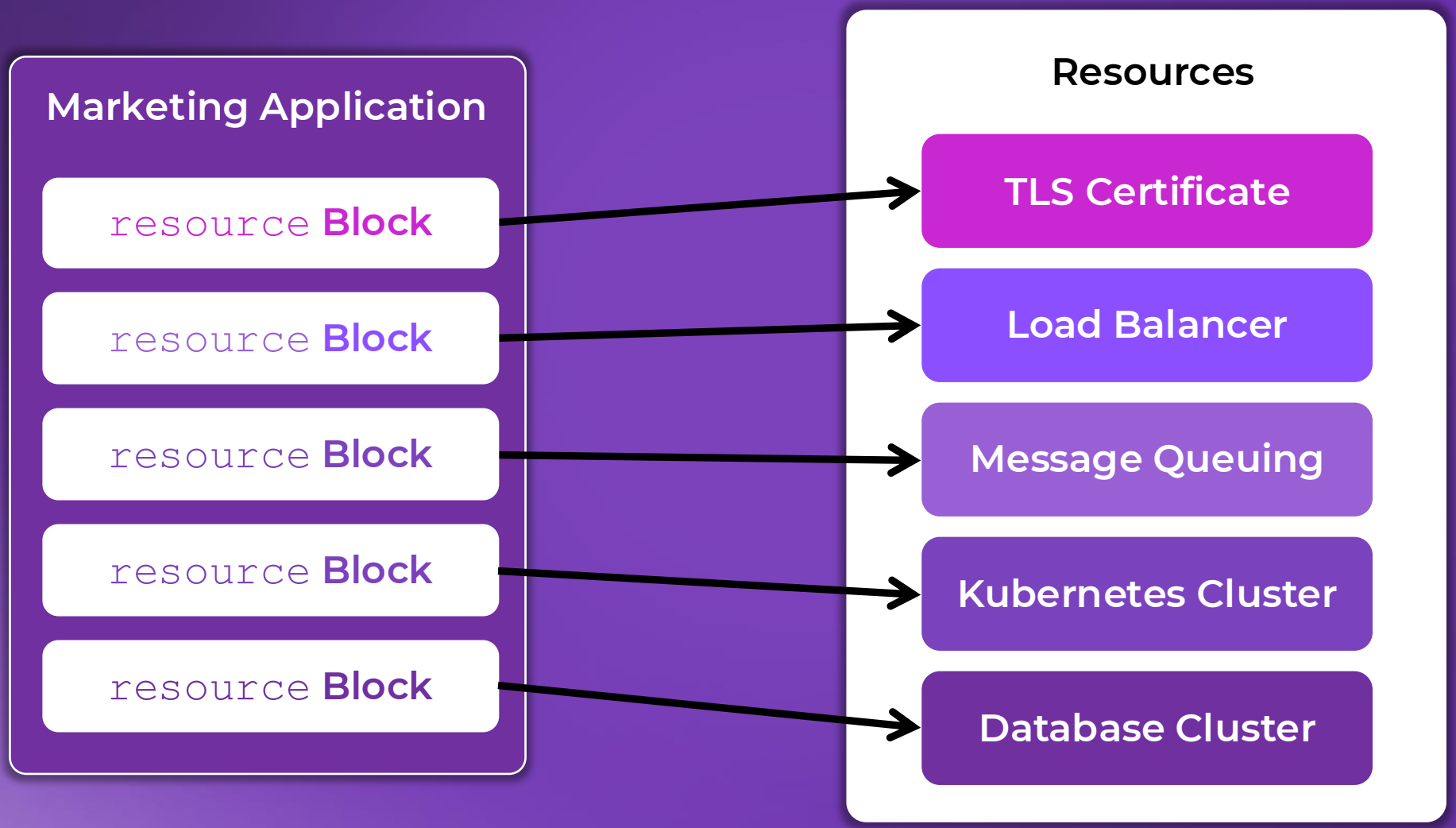
Teams can share and maintain standardized building blocks or publish approved modules for organizational use.

Consistent Patterns

Enforce naming conventions, security controls, and best practices across deployments.

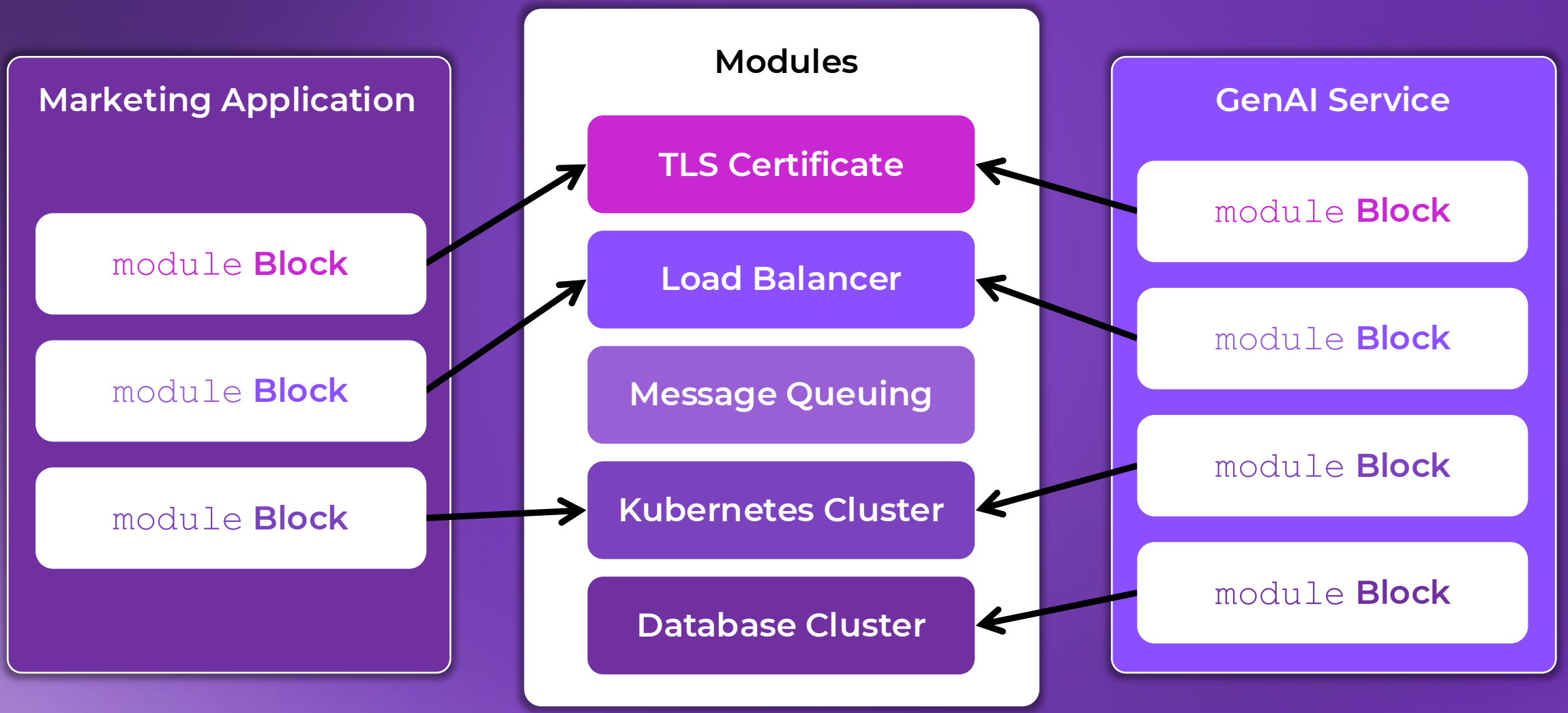
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Building Blocks



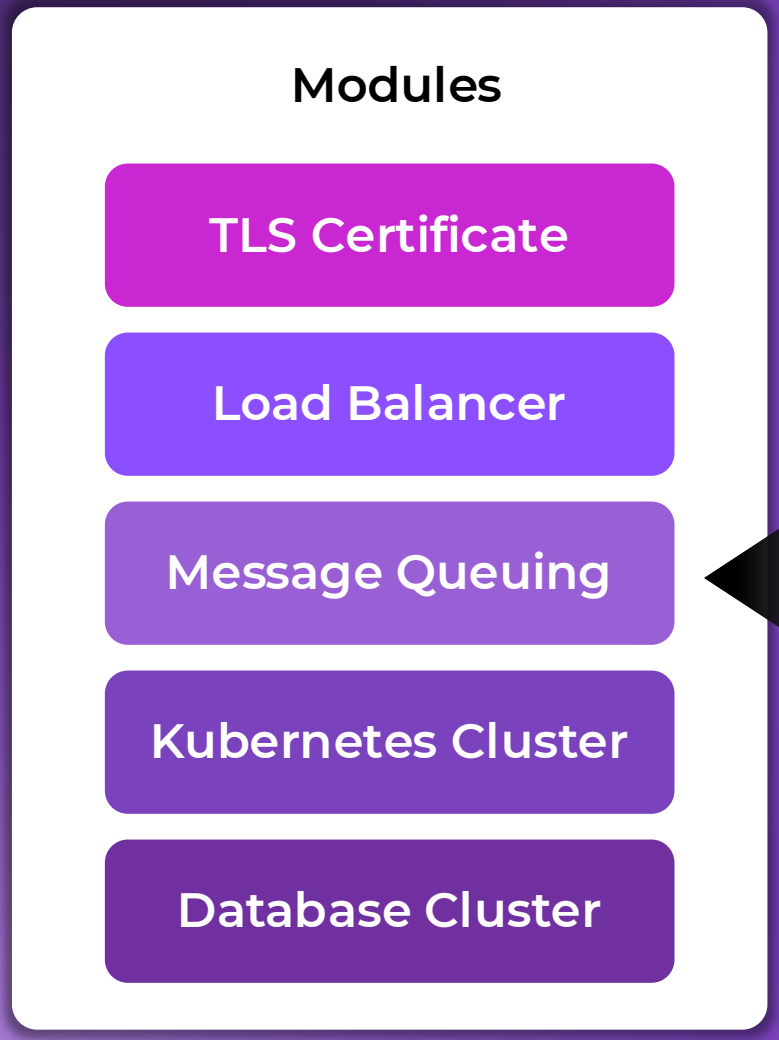
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

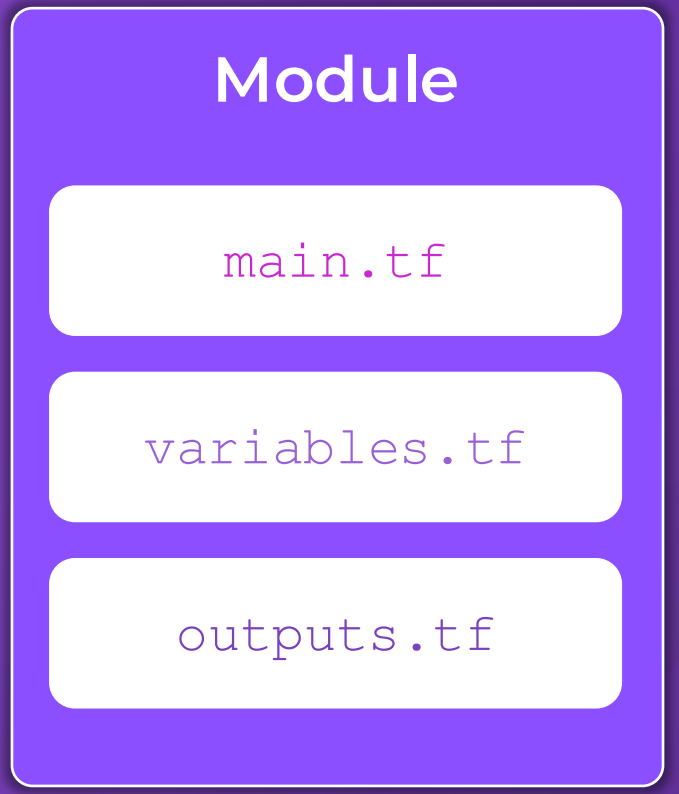
Terraform Building Blocks



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Building Blocks



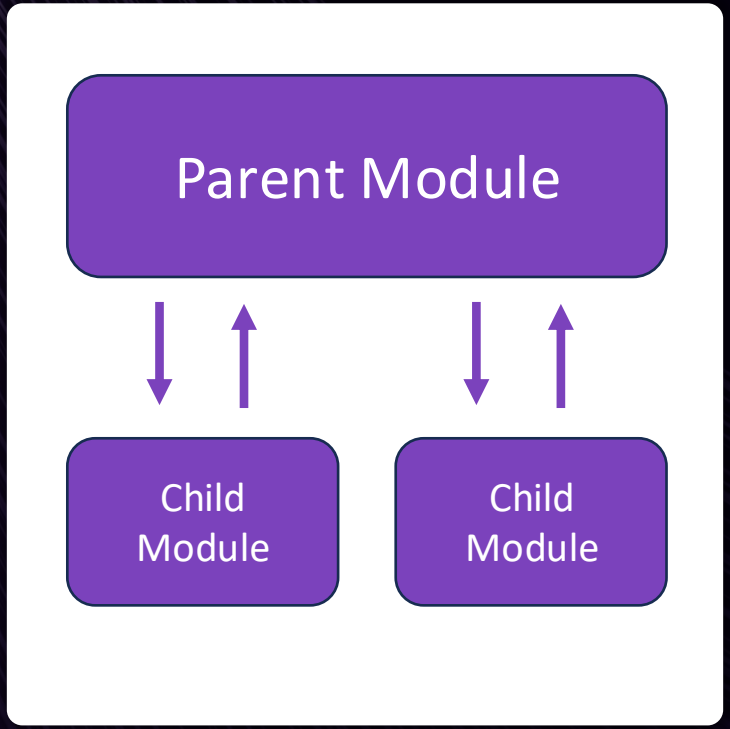


®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Module Block

Parent Module vs. Child Module

- A parent (calling) module references and configures other modules, while a child module is the reusable infrastructure component being called.
- Modules can be retrieved from a registry, a repo, or stored locally.



®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Parent/Root/Calling Module

```hcl
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs             = data.aws_availability_zone.east.name_suffix
  private_subnets = var.private_subnets
  public_subnets  = var.public_subnets

  enable_nat_gateway = true
  enable_vpn_gateway = true

  tags = {
    CreatedBy   = "terraform"
    Environment = "prod"
  }
}
```

Module Blocks Call Child Modules
Defines that this module is the calling module of the child module

Must Define Where the Module Lives
Path to local directory or remote module source to download from

Variables Defined in Root Module
Values must be passed to the module to be used

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Very Simple Child Module

```hcl
variable "cidr_block" {
  type    = string
  default = "10.0.0.0/16"
}

variable "name" {
  type    = string
  default = ""
}

resource "aws_vpc" "vpc" {
  name       = var.name
  cidr_block = var.cidr_block

  tags = {
    Name = var.name
  }
}

output "vpc_id" {
  value = aws_vpc.vpc.id
}
```

Modules Should Look Familiar

Modules are just regular Terraform configurations that are variablized

Must Define One or More Resources

The child module is where the resource blocks are defined

Child Modules Have Outputs

Outputs allow you to pass information from one module to another

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Root Module

```hcl
variable "cidr_block" {
  type    = string
  default = "192.168.0.0/16"
}

module "prod_vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "my-vpc"
  cidr = var.cidr_block
}

output "vpc_id" {
  module.prod_vpc.vpc_id
}
```

Values we want to pass to our module

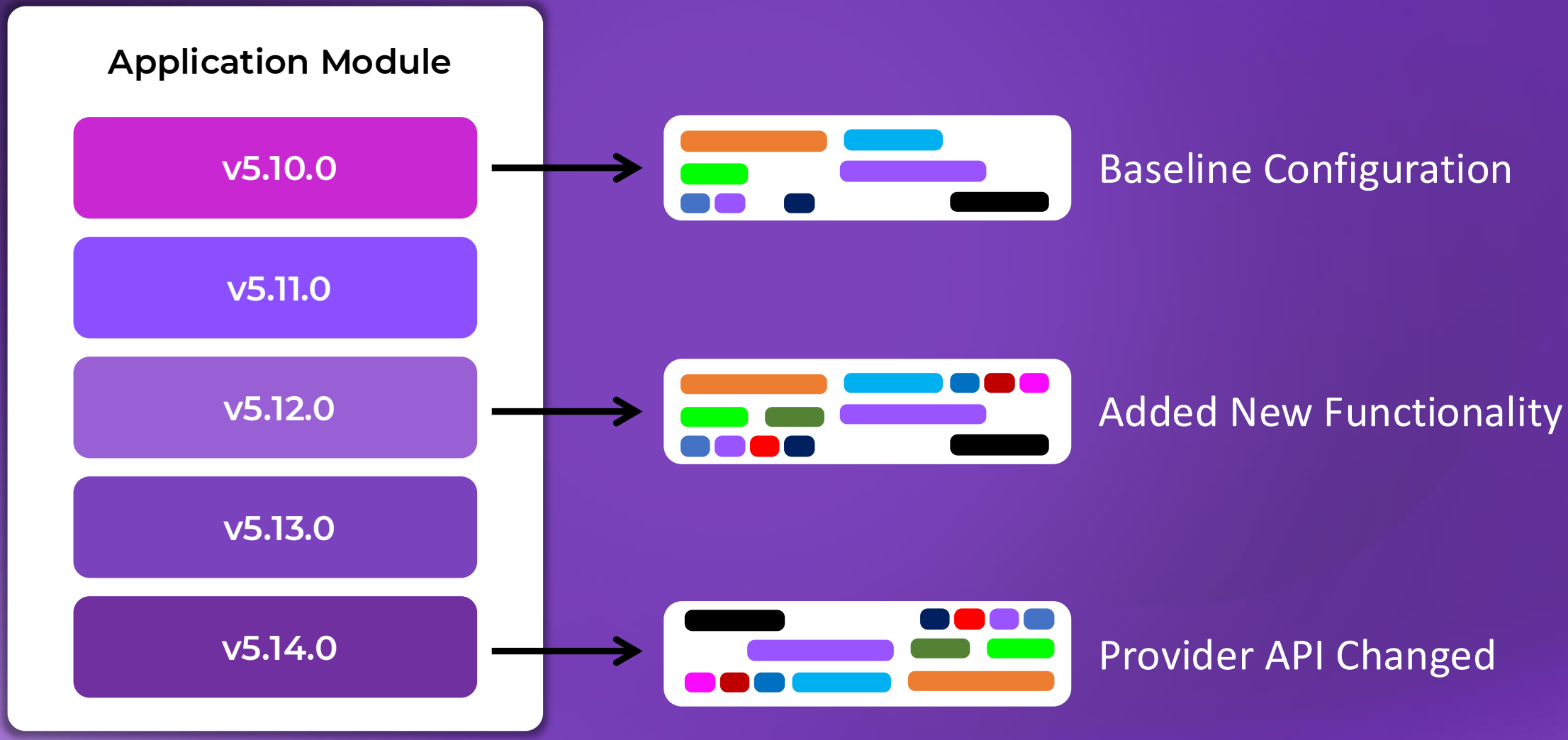
Child Module

```hcl
variable "cidr" {
  type    = string
  default = "10.0.0.0/16"
}

variable "name" {
  type    = string
  default = ""
}

resource "aws_vpc" "vpc" {
  name       = var.name
  cidr_block = var.cidr

  tags = {
    Name = var.name
  }
}

output "vpc_id" {
  value = aws_vpc.vpc.id
}
```

Export values to be accessed by calling the module

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Module Versioning



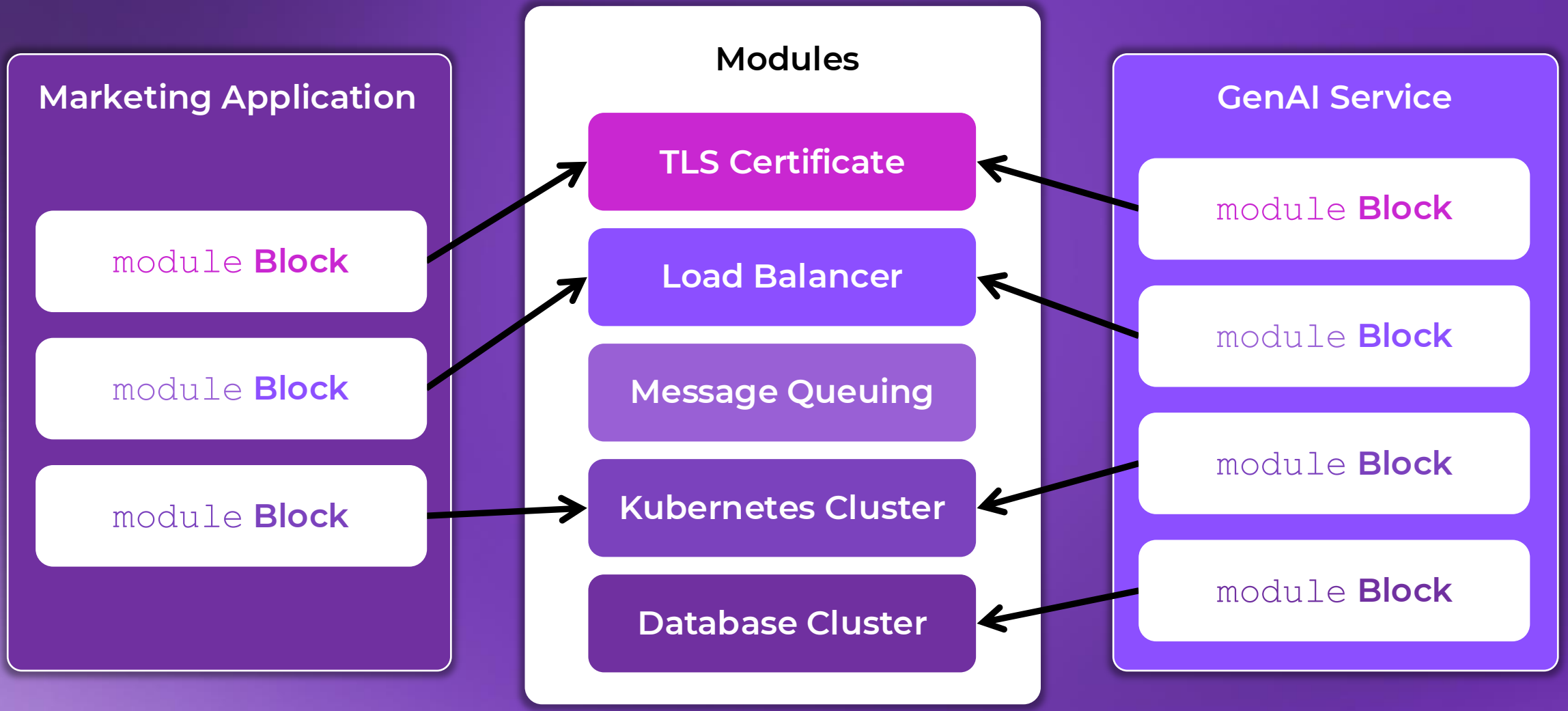
®Copyright Bryan Krausen – DO NOT DISTRIBUTE

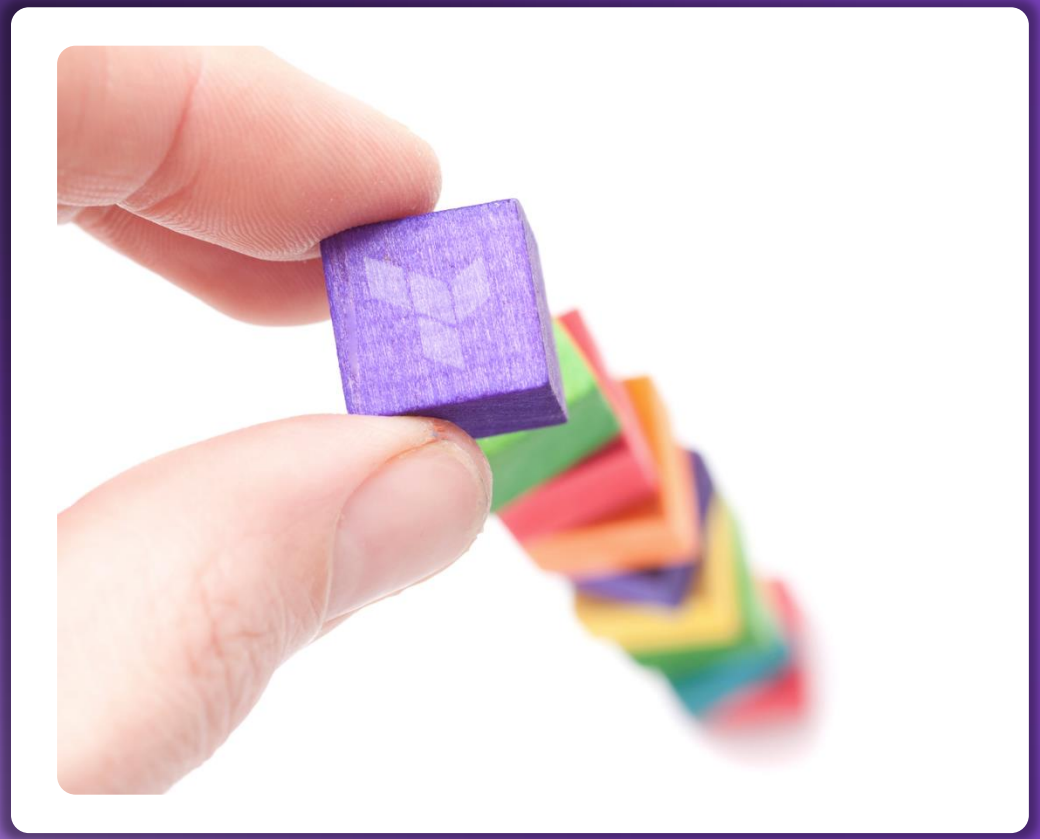
Pinning to a Specific Module Version

```hcl
variable "cidr_block" {
  type    = string
  default = "192.168.0.0/16"
}

module "prod_vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.12.0"

  name = "my-vpc"
  cidr = var.cidr_block
}

output "vpc_id" {
  module.prod_vpc.vpc_id
}
```

®Copyright Bryan Krausen – DO NOT DISTRIBUTE

Terraform Building Blocks





Module Block

The **`module`** block allows you to reuse and organize Terraform configurations by calling pre-defined modules, which will simplify infrastructure management. Modules allow you to group resources into reusable components, such as a network or database module.

By passing variables to modules and using outputs, you can integrate them into your configuration for more dynamic and flexible deployments.

Module Block

Modules help you create maintainable and standardized infrastructure by packaging commonly used resources together.

Code Reusability

Create standardized infrastructure patterns that can be used across multiple projects and teams.

Simplified Management

Break complex infrastructure into smaller, manageable components that are easier to maintain and update.

Consistent Deployment

Ensure infrastructure consistency by using tested, version-controlled module configurations across environments.

main.tf

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "4.0.1"

  name = var.vpc_name
  cidr = var.vpc_cidr_block

  azs = ["us-west-2a", "us-west-2b"]
  private_subnets = ["10.0.1.0/24"]
  public_subnets  = ["10.0.101.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true
}
```

Module Block

Modules help you create maintainable and standardized infrastructure by packaging commonly used resources together.

Code Reusability

Create standardized infrastructure patterns that can be used across multiple projects and teams.

Simplified Management

Break complex infrastructure into smaller, manageable components that are easier to maintain and update.

Consistent Deployment

Ensure infrastructure consistency by using tested, version-controlled module configurations across environments.

main.tf

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "4.0.1"

  name = var.vpc_name
  cidr = var.vpc_cidr_block

  azs = ["us-west-2a", "us-west-2b"]
  private_subnets = ["10.0.1.0/24"]
  public_subnets  = ["10.0.101.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true
}
```

Breaking Down the Module Block

main.tf

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "4.0.1"

  name = var.vpc_name
  cidr = var.vpc_cidr_block

  azs = ["us-west-2a", "us-west-2b"]
  private_subnets = ["10.0.1.0/24"]
  public_subnets  = ["10.0.101.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true
}
```

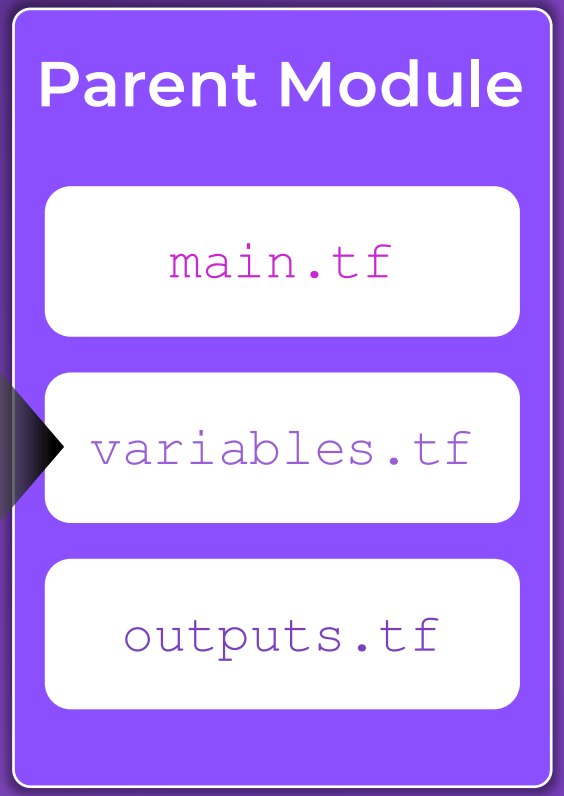
Where the module is located to download

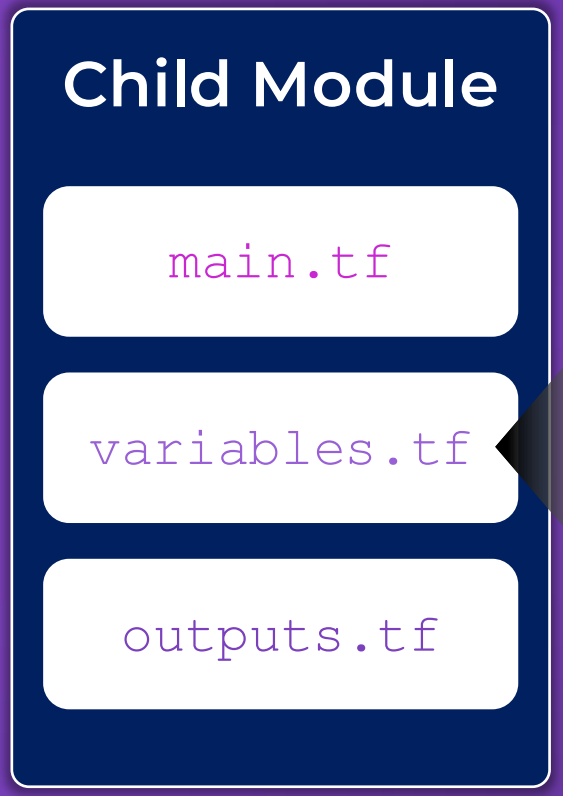
The version of the module to use

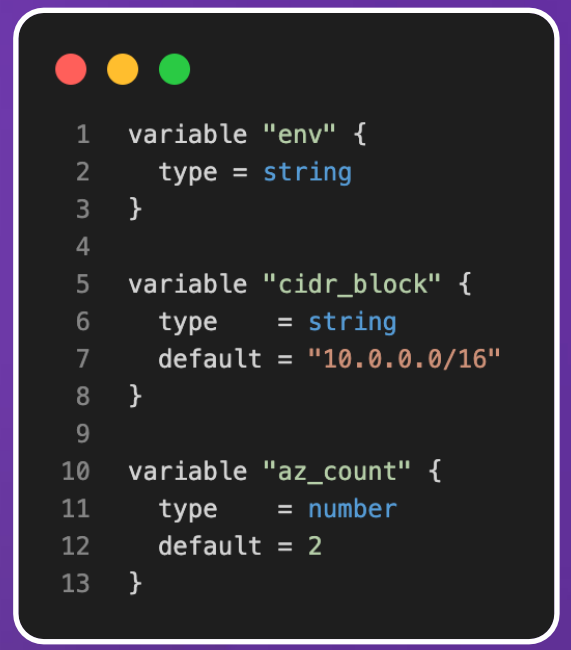
Local variables/values to pass into the module

Understanding Variable Scope in Modules

```
variable "environment" {
  type    = string
  default = "prod"
}

variable "region" {
  type    = string
  default = "us-east-1"
}

variable "project" {
  type    = string
  default = "myapp"
}
```



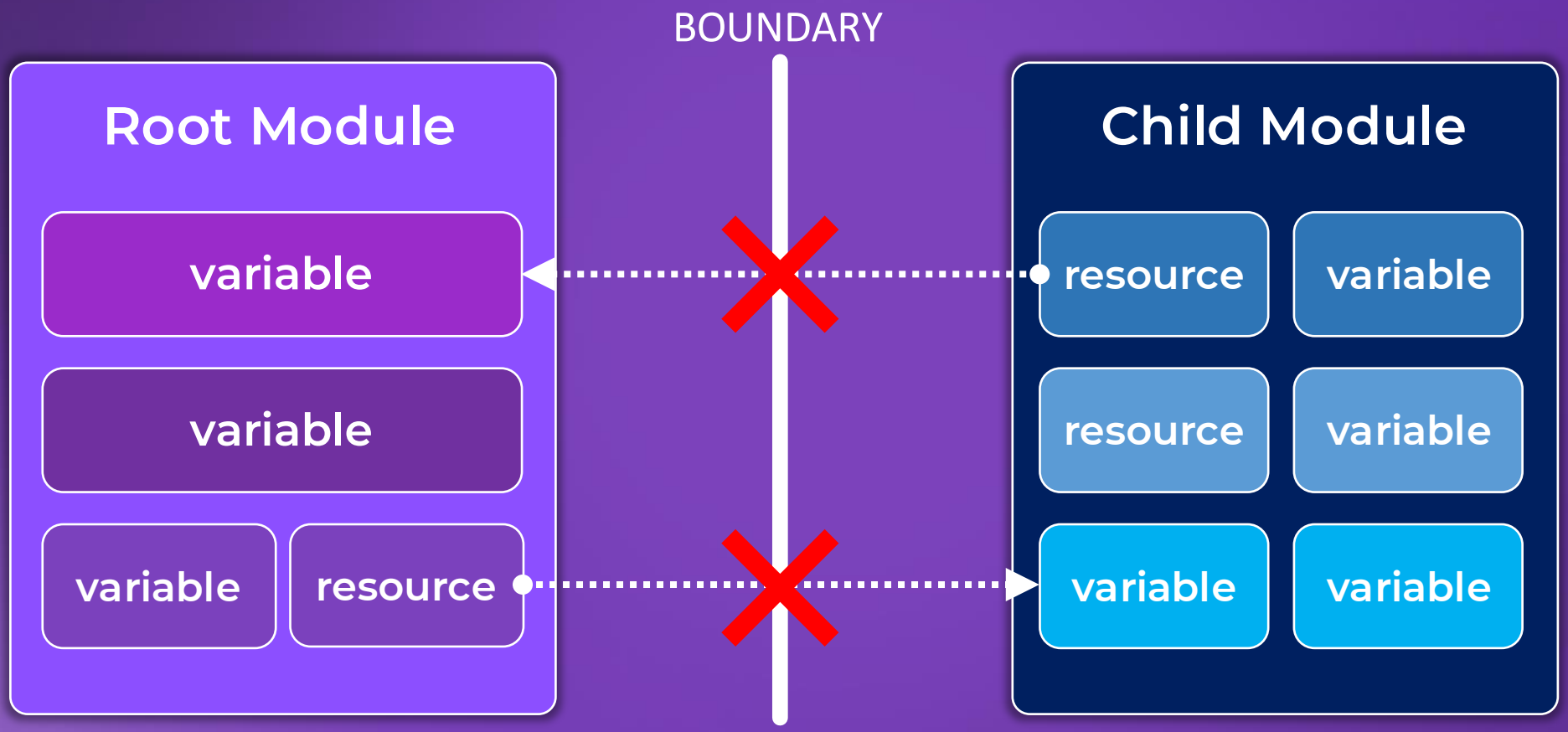




```
variable "env" {
  type = string
}

variable "cidr_block" {
  type    = string
  default = "10.0.0.0/16"
}

variable "az_count" {
  type    = number
  default = 2
}
```

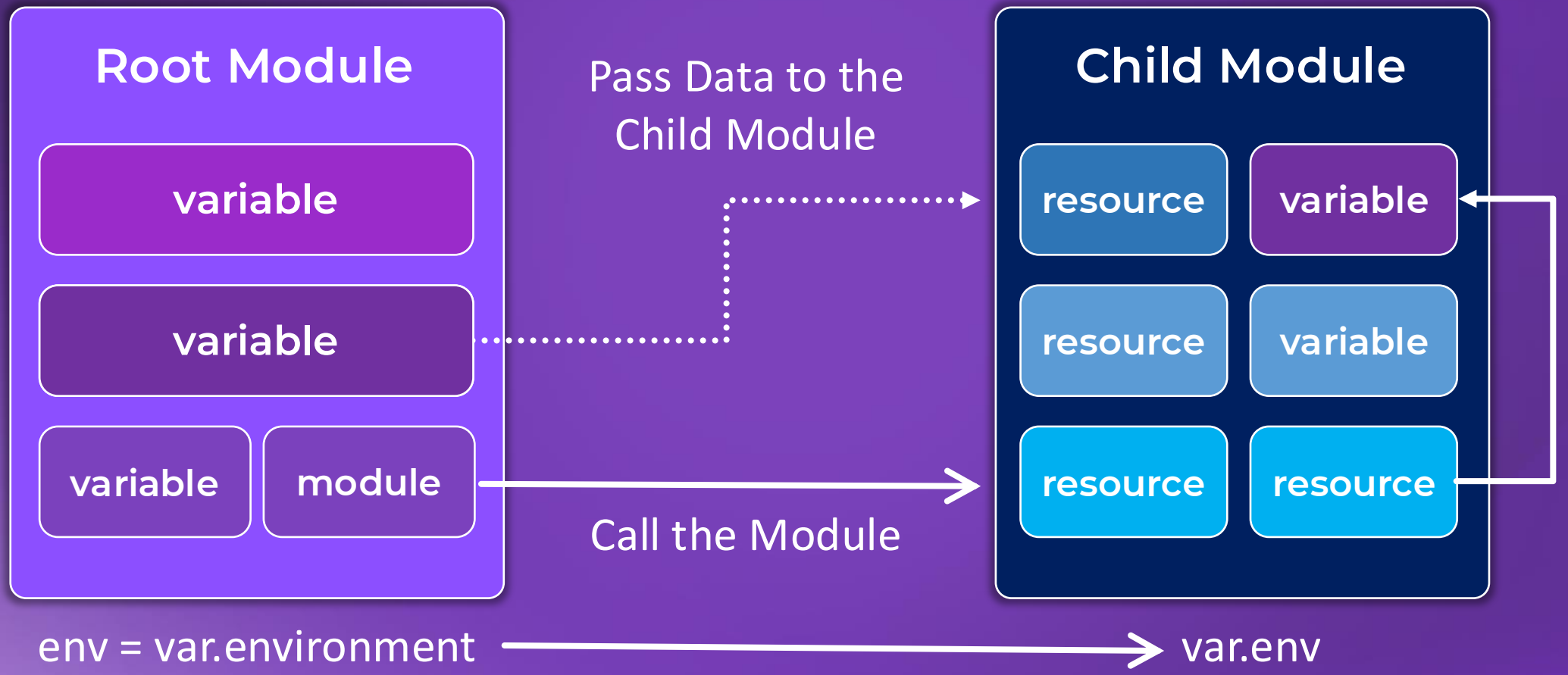
Both the parent and child module(s) may have variables defined

Understanding Variable Isolation



Each module has its own isolated variable scope

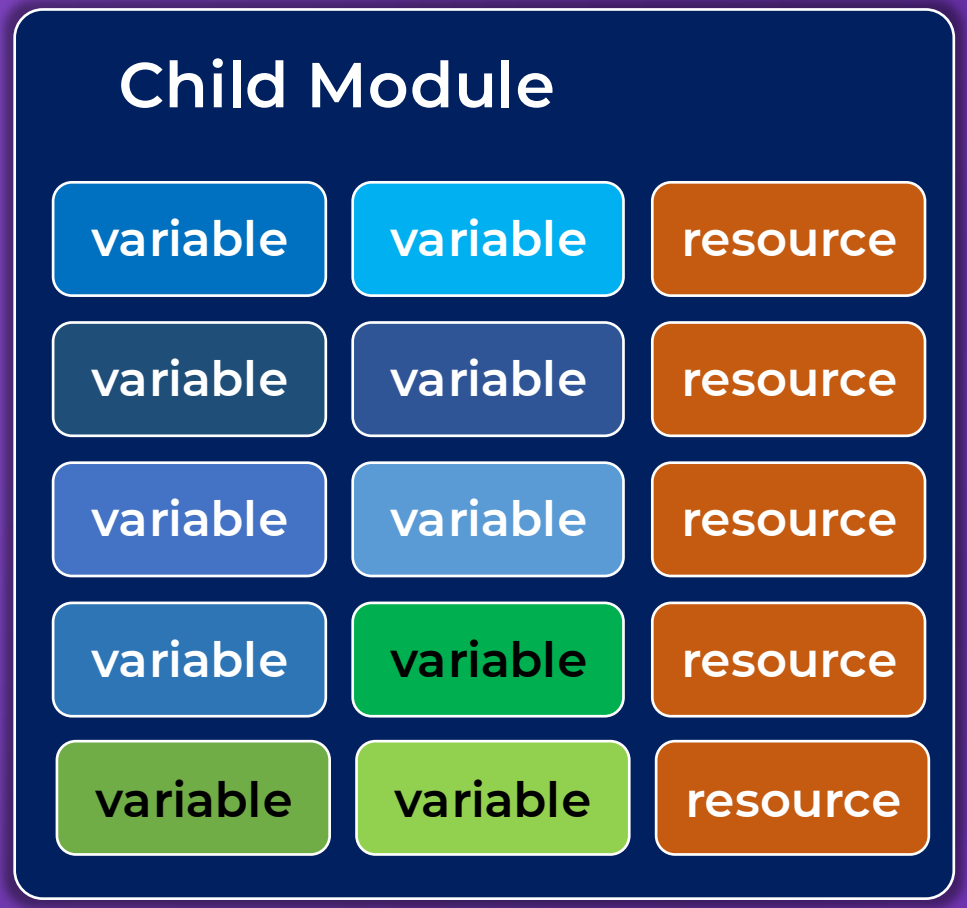
Passing Data to a Child Module



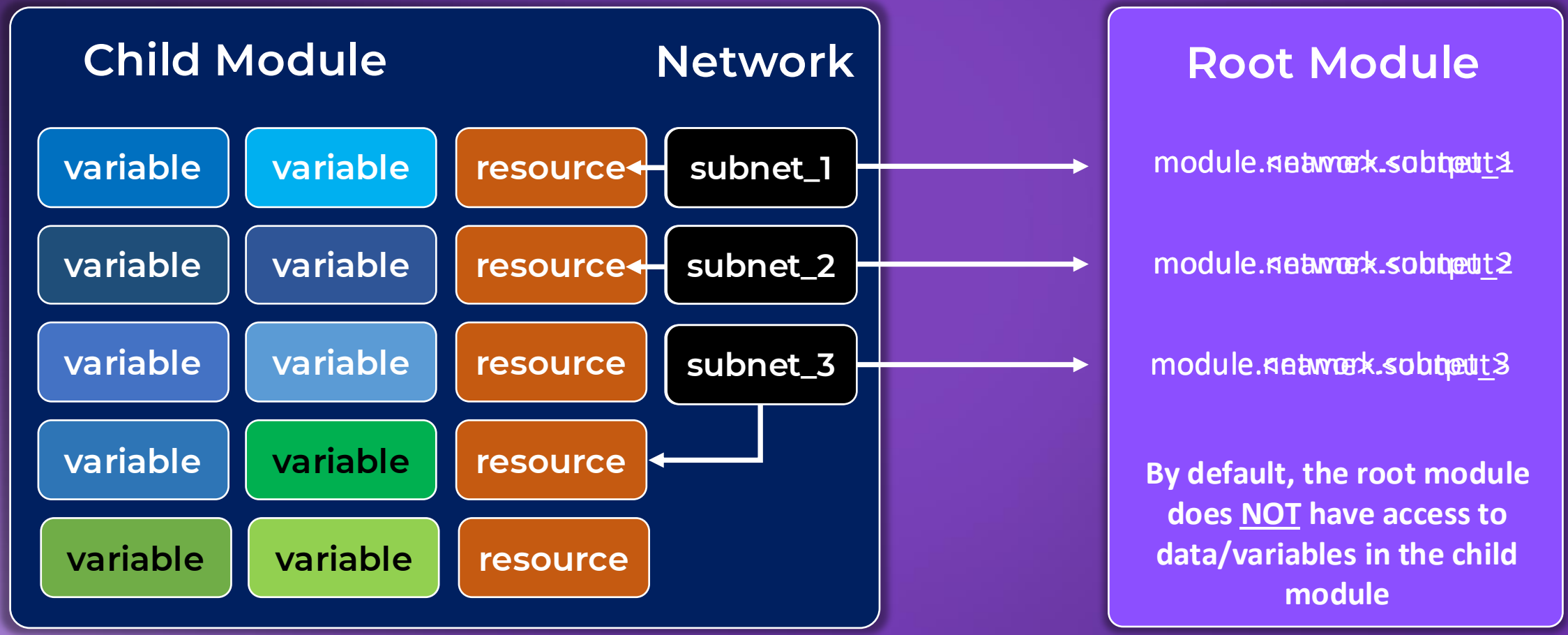
Passing Data to a Child Module

main.tf

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "4.0.1"

  name = var.vpc_name
  cidr = var.vpc_cidr_block

  azs = ["us-west-2a", "us-west-2b"]
  private_subnets = ["10.0.1.0/24"]
  public_subnets  = ["10.0.101.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true
}
```



Output Data Back to a Root Module



depends-on Meta-Argument

Ensuring Resources Are Created in the Right Order

Explicitly Define Dependencies

Terraform may not automatically detect or know about dependencies needed for your deployment

Ensure Correct Provisioning Order

Useful for resources that rely on others to exist first

Prevents Race Conditions

Complex deployments may require one resource to be created before another

```
resource "aws_db_instance" "db" {
  identifier          = "mydb"
  engine              = "mysql"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  username            = "admin"
  password            = "SuperSecurePassword123!"
  skip_final_snapshot = true
}

resource "aws_instance" "app_server" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"

  depends_on = [aws_db_instance.db]
}
```

Dependencies

Implicit vs. Explicit Dependency

- Implicit dependency is when one resource references another resource's attribute. When you use an expression like `aws_instance.web.id`, Terraform knows it must create the referenced resource first - no manual configuration needed.

- Sometimes resources need to be created in a specific order, but there's no attribute reference to create an implicit dependency. In these cases, you use the `depends_on` argument to manually create an explicit dependency to tell Terraform "create this other thing first.

Implicit Dependency

```
1  resource "aws_security_group" "web" {
2    name = "web-sg"
3  }
4
5  resource "aws_instance" "app" {
6    ami             = "ami-12345"
7    instance_type = "t2.micro"
8    security_groups = [aws_security_group.web.id]
9  }
```

resource reference

Explicit Dependency

```
1  resource "aws_iam_role" "lambda" {
2    name = "lambda-role"
3    # role configuration...
4  }
5
6  resource "aws_lambda_function" "app" {
7    function_name = "my-function"
8    role          = aws_iam_role.lambda.arn
9
10   depends_on = [aws_iam_role_policy_attachment.lambda_logs]
11 }
```

explicit depends_on

`lifecycle` Meta-Argument

Controlling How Terraform Manages Resources

- **Customizes how Terraform Manages Resources**

  Change the default behavior during your Terraform workflow.

- **You can protect resources or avoid changes**

  Customizes how Terraform creates, updates, and destroys specific resources.

- **Uses the `lifecycle` block inside of your resource**

  Options include `prevent_destroy`, `create_before_destroy`, and `ignore_changes`

```
1  resource "azurerm_resource_group" "example" {
2    name     = "my-resource-group"
3    location = "East US"
4
5    lifecycle {
6      prevent_destroy = true
7    }
8  }
9
10 resource "azurerm_virtual_machine" "example" {
11   name                = "my-vm"
12   location            = "East US"
13   resource_group_name = "my-resource-group"
14   vm_size             = "Standard_D2s_v3"
15
16   lifecycle {
17     ignore_changes = [vm_size]
18   }
19 }
```

Lifecycle Options

main.tf

```
resource "aws_instance" "web" {
  ami           = "ami-12345"
  instance_type = "t2.micro"

  lifecycle {
    create_before_destroy = true
  }
}
```

create_before_destroy

Tells Terraform to create the replacement resource before destroying the old one.

Essential for zero-downtime updates when resources must be replaced.

Lifecycle Options

prevent_destroy

Blocks Terraform from destroying the resource.

Provides protection against accidental deletion of critical infrastructure like databases.

main.tf

```
resource "aws_db_instance" "prod" {
  identifier = "production-db"
  # ...

  lifecycle {
    prevent_destroy = true
  }
}
```

Lifecycle Options

main.tf

```
resource "aws_instance" "app" {
  ami   = "ami-12345"

  tags = {
    Name = "app-server"}

  lifecycle {
    ignore_changes = [tags]
  }
}
```

ignore_changes

Tells Terraform to ignore changes to specific attributes.

Perfect when external systems modify resources outside of Terraform.

Lifecycle Options

replace_triggered_by

Forces resource replacement when specified resources or attributes change.

Useful for dependencies that don't trigger automatic replacement.

main.tf

```
resource "aws_instance" "web" {
  ami = "ami-12345"

  lifecycle {
    replace_triggered_by = [
      aws_security_group.web
    ]
  }
}
```

Lifecycle Options

main.tf

```
resource "azurerm_virtual_machine" "web" {
  name     = "web-vm"
  location = var.location
  vm_size  = var.vm_size

  lifecycle {
    precondition {
      condition     = var.vm_size != "Basic_A0"
      error_message = "Basic_A0 is deprecated"
    }
  }
}
```

`precondition`
`postcondition`

Validate resource state before/after operations.

Advanced validation for complex deployments.

Validating Your Configuration

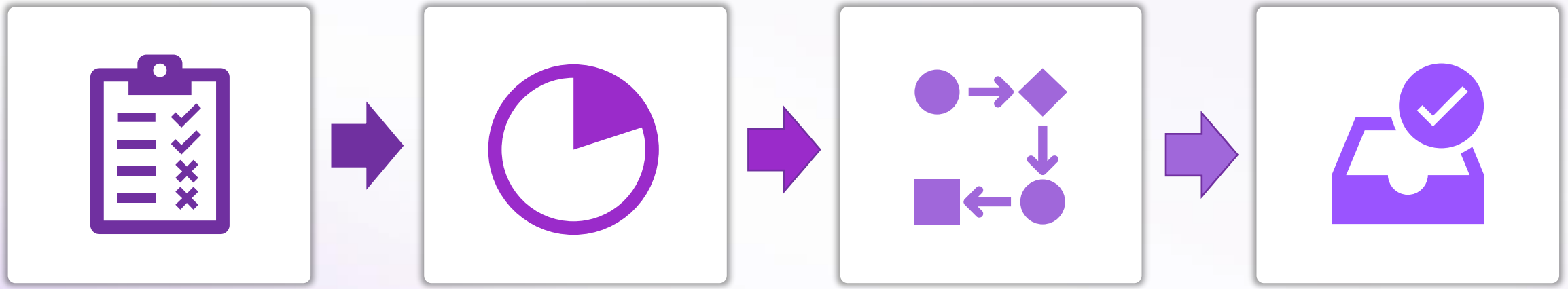
Terraform has four validation mechanisms, each serving different purposes

- Variable Validation
- Preconditions
- Postconditions
- Checks

Think of these as defensive layers protecting your infrastructure at different stages

Validating Your Configuration

Order of Operations



Variable Validation

Verify your configuration parameters when Terraform creates a plan

- Verify input variables meet specific format requirements
- Verify input values fall within acceptable ranges
- Prevent Terraform operations if a variable is misconfigured
- Failure results is displaying the configured `error_message`, and stops the operation from proceeding

```
variable "variable_name" {
  value  = <expression-to-evaluate>

  validation {
    condition     = <expression-to-verify>
    error_message = "<error message string>"
  }
}
```

Configured inside the `variable` block you want validated

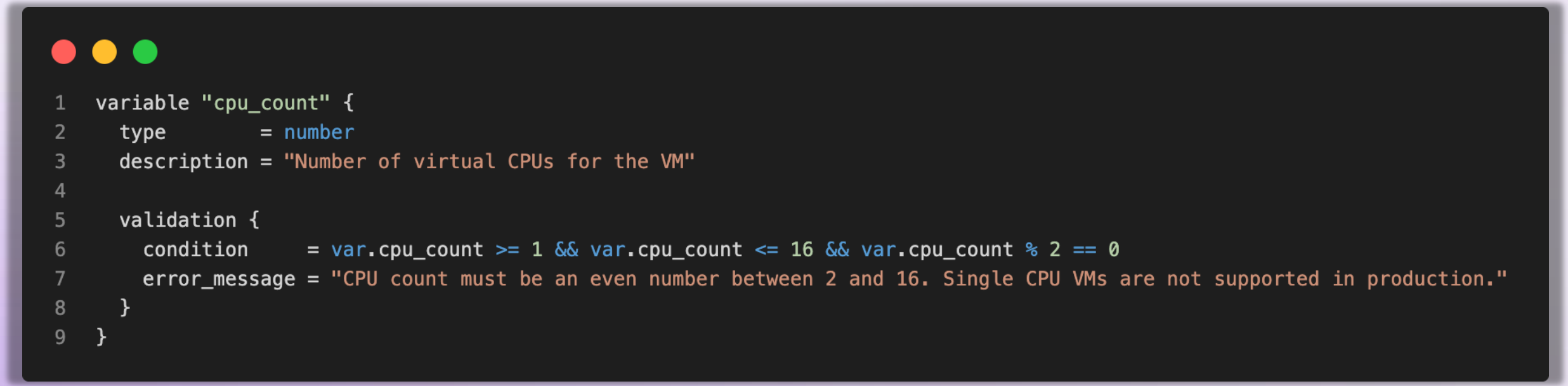
Variable Validation

Verify your configuration parameters when Terraform creates a plan

```
variable "location" {
  type        = string
  description = "Azure region where resources will be deployed"

  validation {
    condition = contains([
      "eastus",
      "westus2",
      "centralus"
    ], var.location)
    error_message = "Location must be one of the approved US regions: eastus, westus2, or centralus."
  }
}
```

Variable Validation

Verify your configuration parameters when Terraform creates a plan



```
variable "cpu_count" {
  type        = number
  description = "Number of virtual CPUs for the VM"

  validation {
    condition     = var.cpu_count >= 1 && var.cpu_count <= 16 && var.cpu_count % 2 == 0
    error_message = "CPU count must be an even number between 2 and 16. Single CPU VMs are not supported in production."
  }
}
```

Variable Validation

Verify your configuration parameters when Terraform creates a plan

```hcl
variable "instance_type" {
  type        = string
  description = "EC2 instance type for the application server"

  validation {
    condition = contains([
      "t3.micro", "t3.small", "t3.medium",
      "t3.large", "t3.xlarge", "t3.2xlarge",
      "m5.large", "m5.xlarge", "m5.2xlarge",
      "c5.large", "c5.xlarge", "c5.2xlarge"
    ], var.instance_type)
    error_message = "Instance type must be an approved type from the t3, m5, or c5 families. Graviton instances require architecture approval."
  }

  validation {
    condition = (
      var.instance_type == "t3.micro" ||
      var.instance_type == "t3.small" ||
      !startswith(var.instance_type, "t3")
    )
    error_message = "T3.micro and t3.small are only allowed for development environments. Use t3.medium or larger for staging and production."
  }

  validation {
    condition     = !can(regex("\\.(24|32)xlarge$", var.instance_type))
    error_message = "Extra-large instances (24xlarge and above) require infrastructure team approval. Please submit a request ticket."
  }
}
```

Preconditions/Postconditions

Verify your resources before and after Terraform creates them

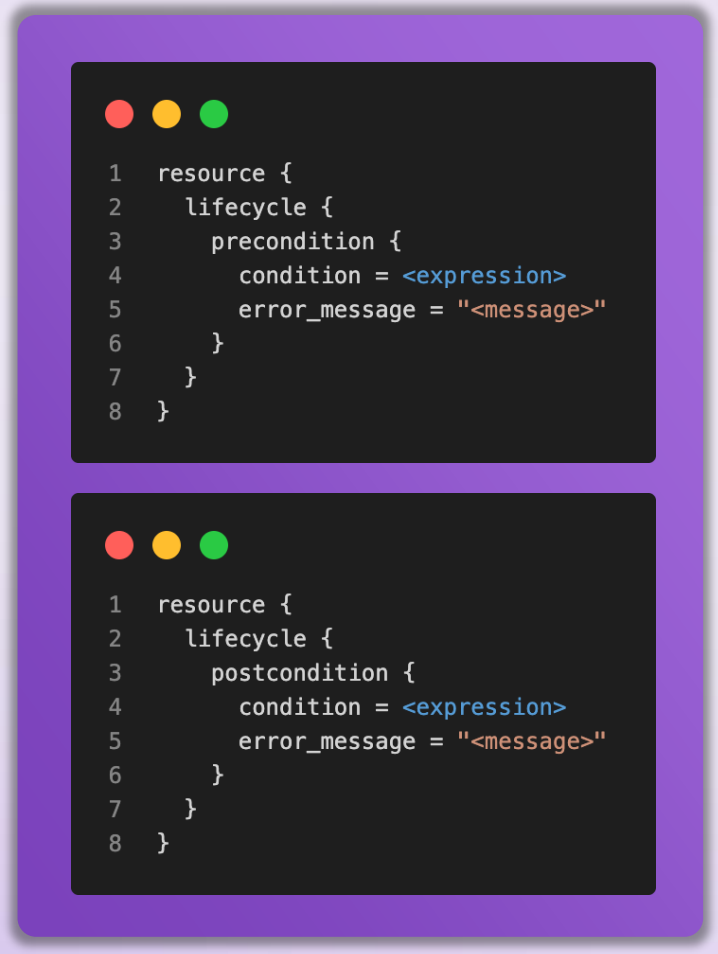
Preconditions:

- Verify BEFORE Terraform creates something (e.g., checking that an AMI has the right architecture before launching an instance)

Postconditions:

- Verify AFTER Terraform creates something (e.g., ensuring an instance actually got a private DNS record after launch)

Configured inside the `lifecycle` block for the `resource` you want to verify



```
resource {
  lifecycle {
    precondition {
      condition = <expression>
      error_message = "<message>"
    }
  }
}
```

```
resource {
  lifecycle {
    postcondition {
      condition = <expression>
      error_message = "<message>"
    }
  }
}
```

Preconditions/Postconditions

Verify your resources before and after Terraform creates them

```
resource "azurerm_app_service" "webapp" {
  name                = var.app_service_name
  location            = var.location
  resource_group_name = var.resource_group_name
  app_service_plan_id = azurerm_app_service_plan.plan.id

  lifecycle {
    precondition {
      condition     = contains(data.azurerm_subnet.app_subnet.service_endpoints, "Microsoft.Web")
      error_message = "The subnet must have the Microsoft.Web service endpoint enabled before deploying App Services."
    }
  }
}
```

Preconditions/Postconditions

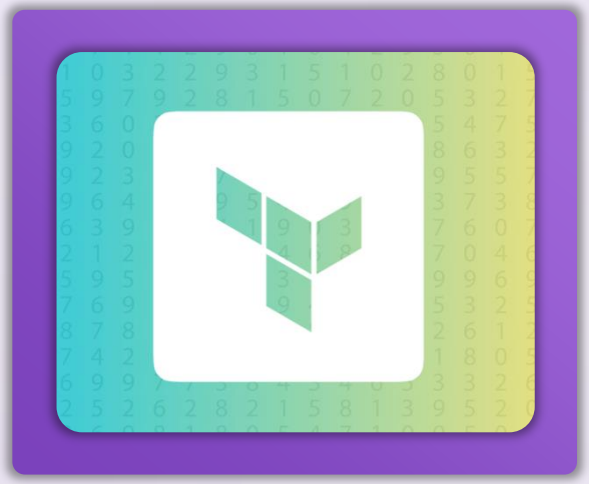
Verify your resources before and after Terraform creates them

```
resource "google_compute_instance" "web_server" {
  name         = var.instance_name
  machine_type = var.machine_type
  zone         = var.zone

  boot_disk {
    initialize_params {
      image = var.boot_image
    }
  }

  lifecycle {
    postcondition {
      condition     = self.network_interface[0].access_config[0].nat_ip != ""
      error_message = "The instance must have a public IP address assigned. Check your network configuration."
    }
  }
}
```

Checks (via `check` Blocks)

Validate your infrastructure outside of the typical resource lifecycle

- Validate resources, data sources, variables, or outputs in your configuration.
- Validate the behavior of your infrastructure as a whole.
- Verify infrastructure configuration without blocking operations.
- The `check` block executes as the last step of `plan` or `apply` operation, after Terraform has planned or provisioned your infrastructure.
- When a `check` block's assertion fails, Terraform reports a warning and continues executing the current operation.



Check blocks are standalone, top-level blocks in your `.tf` files - they're NOT nested inside resources, data sources, or any other block type.

Checks (via `check` Blocks)

Validate your infrastructure outside of the typical resource lifecycle

```
check "web_health" {
  data "http" "web_health_endpoint" {
    url = "https://my-awesome.website.com/health"
  }

  assert {
    condition     = data.http.web_health_endpoint.status_code == 200
    error_message = "Web health endpoint returned ${data.http.web_health_endpoint.status_code}. Expected 200."
  }
}
```

Choosing the Right Validation

What Do You Need To Do?	Validation Type
Validate user input before planning?	**Variable validation**
Verify assumptions before creating a resource?	**Precondition**
Verify results after creating a resource?	**Postcondition**
Monitor infrastructure health without blocking operations?	**Check**

LECTURE

Why Secrets in Terraform Are Risky

Scenario

1. You're writing Terraform to create an RDS database, and it needs a password

2. You might think: "I'll just put it in a variable and move on"

3. But here's the reality - that password is about to show up in at least 3 places you might not expect

The Core Problem

Terraform needs secrets to manage infrastructure, but those secrets can be exposed in multiple places

Secrets Stored as Plaintext

API Keys, Passwords, Account Numbers, etc. could end up in files (not good).

Secrets Shared Across Teams

Remote backends make state accessible to multiple people and logs and outputs are captured.

Secrets Need Protection at Every Layer

Multiple exposure points means multiple protections are needed. Understanding WHERE secrets appear is the first step.

terraform.tfstate

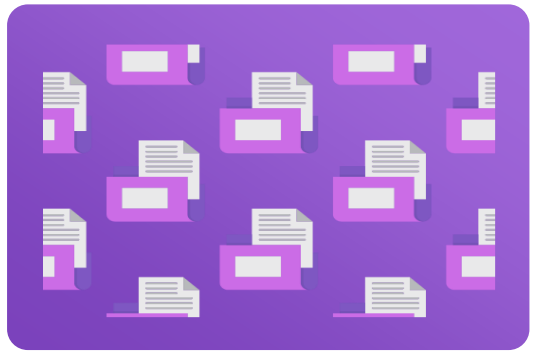
```
{
  "version": 4,
  "terraform_version": "...",
  "serial": ...,
  "lineage": "...",
  "outputs": {
    "database_password_output": {
      "value": "MySuperSecretPassword123!",
      "type": "string",
      "sensitive": true
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "aws_db_instance",
      "name": "main",
      "provider": "...",
      "instances": [
        {
          "schema_version": ...,
          "attributes": {
            // ... other attributes
            "password": "MySuperSecretPassword123!",
            // ...
          },
```

“ ”

This is not a bug – Terraform NEEDS this information to manage your infrastructure.

We just ways to protect it and prevent it.

Where Senstive Data Can End Up



State Files

- terraform.tfstate (local or remote)
- Plaintext JSON with resource attributes

Logs & Console

- terraform plan output
- Provider error messages
- CI/CD build logs

Version Control

- Committed .tfvars files
- Hardcoded values in .tf files
- Git history (permanent)

Plan Files

- Saved terraform plan files
- Contain proposed changes with values
- Often stored in CI/CD artifacts

LECTURE

Protecting Secrets in Your Configuration

Protecting Secrets in Your Configuration

Overview

Sensitive Variables

Hide Values in Output and Logs

Environment Variables

Keep Secrets out of Version Control

External Secret Sources

Don't Store Secrets in Terraform at all

These Techniques Work Together for Security in Layers

The Problem:

Passwords in Plain Sight

```
1  variable "db_password" {
2    description = "Database password"
3    type        = string
4  }
5
6  resource "aws_db_instance" "main" {
7    identifier     = "mydb"
8    engine         = "postgres"
9    instance_class = "db.t3.micro"
10   username       = "admin"
11   password       = var.db_password
12 }
```

TERMINAL

```
$ terraform plan

Terraform will perform the following
  actions:

  # aws_db_instance.main will be created
  + resource "aws_db_instance" "main" {
      + password = "MyS3cr3tP@ssw0rd!"
      + username = "admin"
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

This information up in CI/CD logs, console output, and anywhere `terraform plan` runs

The Solution:

Mark Variables as Sensitive

```
1
2
3
4
5
6
7  resource "aws_db_instance" "main" {
8    identifier     = "mydb"
9    engine         = "postgres"
10   instance_class = "db.t3.micro"
11   username       = "admin"
12   password       = var.db_password
13 }
```

TERMINAL

```
$ terraform plan

Terraform will perform the following
  actions:

  # aws_db_instance.main will be created
  + resource "aws_db_instance" "main" {
      + password       = (sensitive value)
      + username       = "admin"
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

What Sensitive Variables Protect

✅ What It DOES Protect

- ✅ Hides values in `terraform plan` output
- ✅ Hides values in `terraform apply` output
- ✅ Prevents values from appearing in logs
- ✅ Redacts values in error messages
- ✅ Works for variables and outputs

❌ What It DOESN'T Protect

- ❌ Still stored in state file (plaintext)
- ❌ Still visible if you open state directly
- ❌ Still transmitted to providers
- ❌ Not encryption - just output masking

Think of sensitive variables as 'hide from logs' and not 'encrypt the data'

Sensitive Outputs

```hcl
output "database_endpoint" {
  description = "Database connection endpoint"
  value       = aws_db_instance.main.endpoint
}

output "database_password" {
  description = "Database password"
  value       = aws_db_instance.main.password
  sensitive   = true  # ← Prevents output from showing in console
}
```

TERMINAL

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

database_endpoint = "mydb.abc123.us-east-1.rds.amazonaws.com:5432"
database_password = <sensitive>
```

Sensitive Outputs

Regular outputs show their values

Sensitive outputs are hidden

Still accessible programmatically or using `terraform output <name>` command

```hcl
output "database_endpoint" {
  description = "Database connection endpoint"
  value       = aws_db_instance.main.endpoint
}

output "database_password" {
  description = "Database password"
  value       = aws_db_instance.main.password
  sensitive   = true
}
```

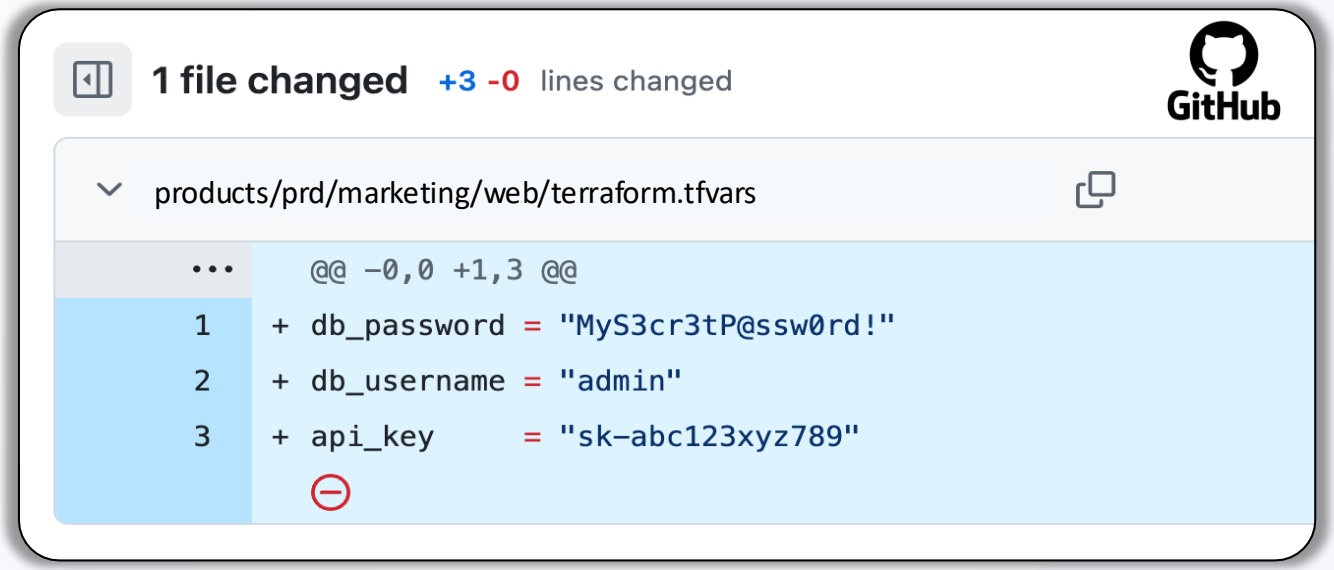
The Problem:

Committing Secrets to Version Control

❌ Don't Do This

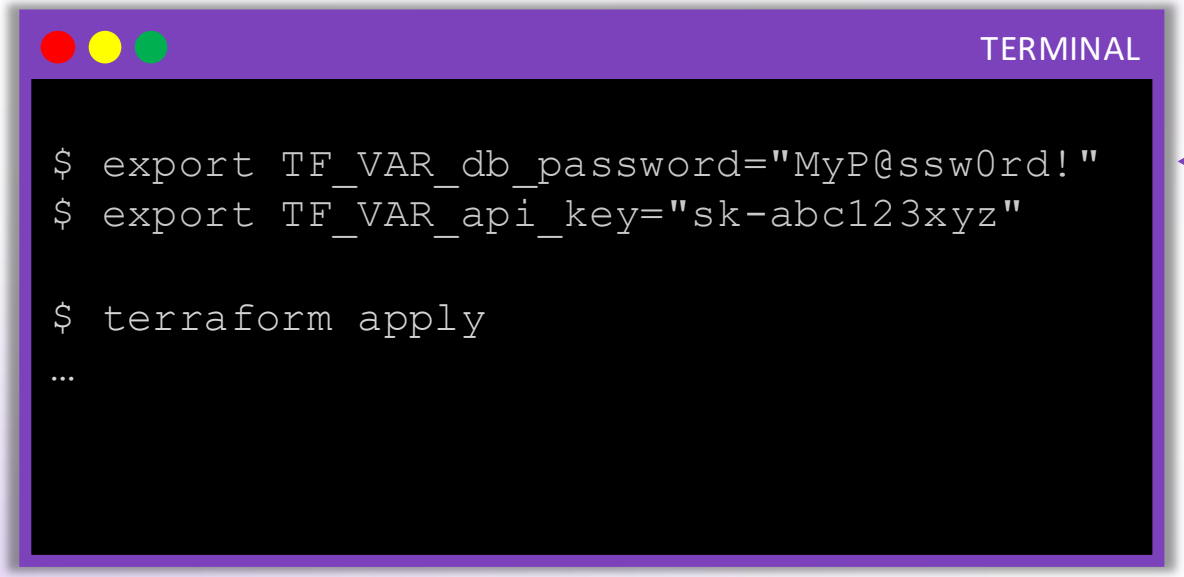
terraform.tfvars

```
1  db_password = "MyS3cr3tP@ssw0rd!"
2  db_username = "admin"
3  api_key     = "sk-abc123xyz789"
```



because it ends up here...forever

The Solution: Environment Variables

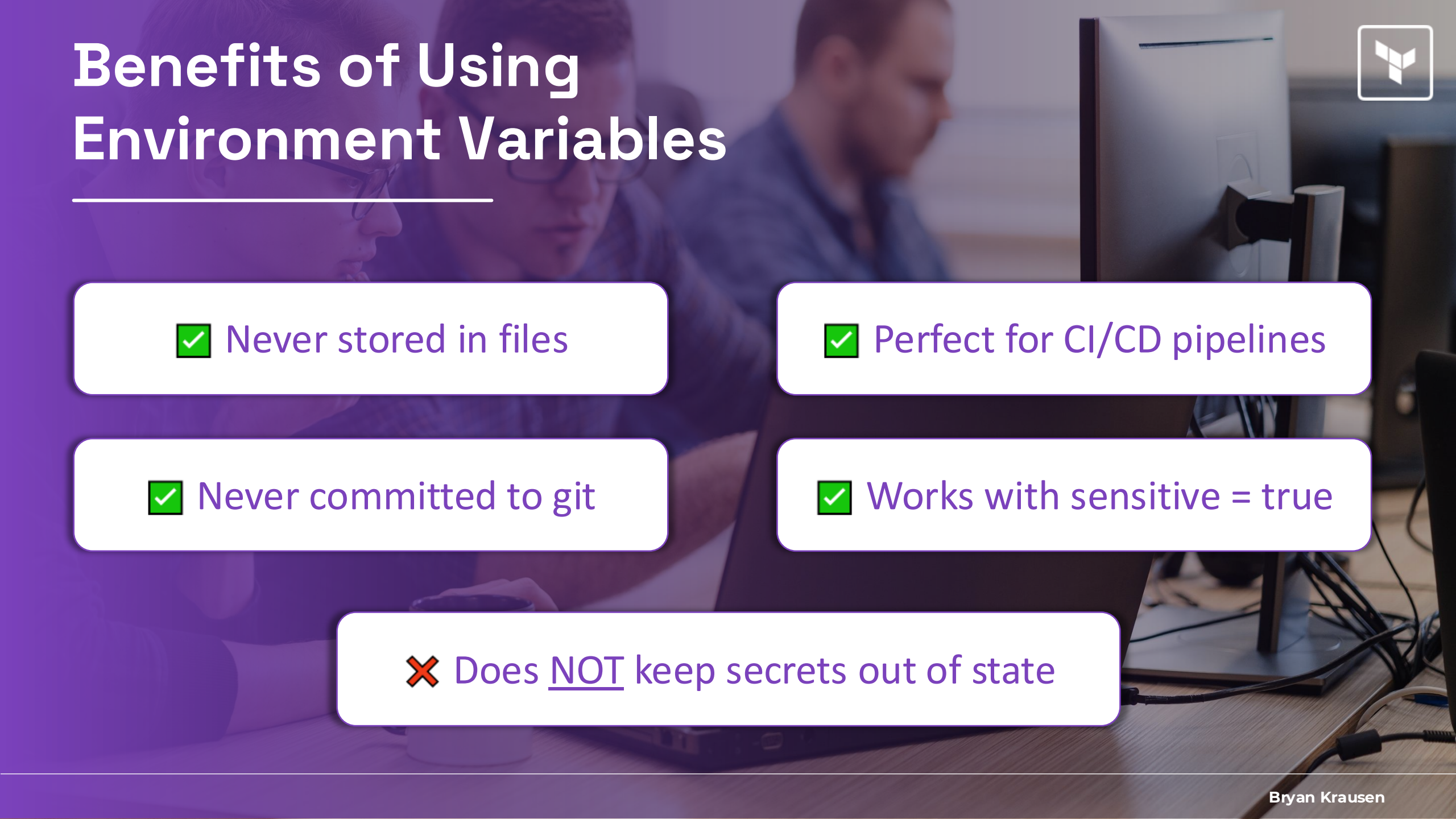



```hcl
variable "db_password" {
  description = "Database password"
  type        = string
  sensitive   = true
}
```

✅ Set the Value using an Environment Variable Instead

- No default value needed!
- Terraform automatically reads `TF_VAR_db_password`

Benefits of Using Environment Variables



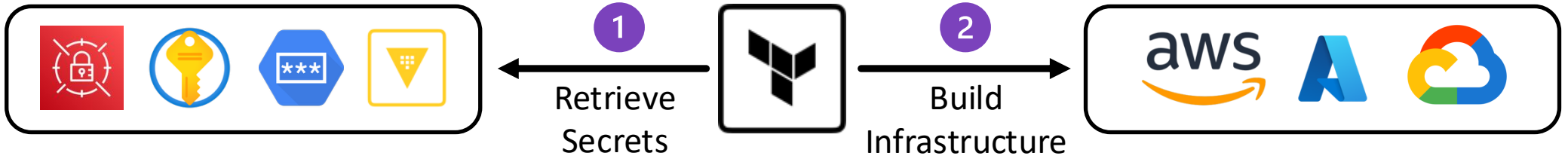
- ✅ Never stored in files
- ✅ Perfect for CI/CD pipelines
- ✅ Never committed to git
- ✅ Works with sensitive = true
- ❌ Does NOT keep secrets out of state

Environment Variables in GitHub Actions

```yaml
name: Terraform Apply
on: [push]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Terraform Apply
        env:
          TF_VAR_db_password: ${{ secrets.DB_PASSWORD }}
          TF_VAR_api_key: ${{ secrets.API_KEY }}
        run: |
          terraform init
          terraform apply -auto-approve
```

- Secrets stored in GitHub Secrets
- Injected as env vars at runtime
- Never appear in logs or code

The Best Practice: Don’t Store Secrets in Terraform



Secrets Managed by Dedicated Tools

Centralized Rotation and Access Control

Reduced Secrets in Terraform State

One Source of Truth for Secrets

Reading Secrets from AWS Secrets Manager

```hcl
# Read the secret from AWS Secrets Manager
data "aws_secretsmanager_secret_version" "db_pass" {
  secret_id = "production/database/password"
}

# Use it in your resource
resource "aws_db_instance" "main" {
  identifier     = "mydb"
  engine         = "postgres"
  instance_class = "db.t3.micro"
  username       = "admin"
  # Secret is fetched at apply time
  password = data.aws_secretsmanager_secret_version.db_pass.secret_string
}
```

- Terraform reads the secret when it runs
- Password is NOT in your .tf files
- Still ends up in state
- Uses a provider for the target secrets manager

Similar patterns work with Azure Key Vault, GCP Secret Manager, and HashiCorp Vault

Protection Quick Reference

Technique	Protects Logs/Output?	Protects State?	Protects Git Commits?	Best For
Sensitive Variables	✅ Yes	❌ **No**	❌ **No**	Hiding output, marking intent
Environment Variables	✨ Partial	❌ **No**	✅ Yes	Local dev, CI/CD, keeping secrets out of files
External Sources	✨ Partial	‼️ Reduces	✅ Yes	Production secrets, centralized management

- Create environment vars to get secrets into Terraform
- Store secrets in external sources where possible
- Use sensitive variables to hide output

LECTURE

Securing State Files

The State File Problem

- Terraform state contains complete resource configuration
- Includes all sensitive values (passwords, keys, tokens)
- Stored as plaintext JSON by default
- No file encryption unless you configure it yourself

terraform.tfstate

```
{
  "version": 4,
  "terraform_version": "...",
  "serial": ...,
  "lineage": "...",
  "outputs": {
    "database_password_output": {
      "value": "MySuperSecretPassword123!",
      "type": "string",
      "sensitive": true
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "aws_db_instance",
      "name": "main",
      "provider": "...",
      "instances": [
        {
          "schema_version": ...,
          "attributes": {
            // ... other attributes
            "password": "MySuperSecretPassword123!",
            // ...
          },
```

Why Local State is Risky

Local State

(terraform.tfstate in your directory)

VS

Remote State

(S3, HCP Terraform, etc.)

- ✗ Stored on your laptop unencrypted
- ✗ Easy to accidentally commit to git
- ✗ Hard to share with teams safely
- ✗ No access controls
- ✗ No encryption at rest

- ✓ Stored in a centralized location for team access
- ✓ Never stored on a local filesystem (laptop)
- ✓ Can enable encryption at rest (most backends)
- ✓ Access is controlled via IAM/RBAC systems
- ✓ Audit Logging in generally available

Always use remote state with encryption for production workloads

Configuring Encrypted Remote State

```
terraform {
  backend "s3" {
    bucket       = "terraform-state"
    key          = "prod/network/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true
    encrypt      = true       # server-side encryption
    kms_key_id   = "<kms_id>" # kms key encyption
  }
}
```

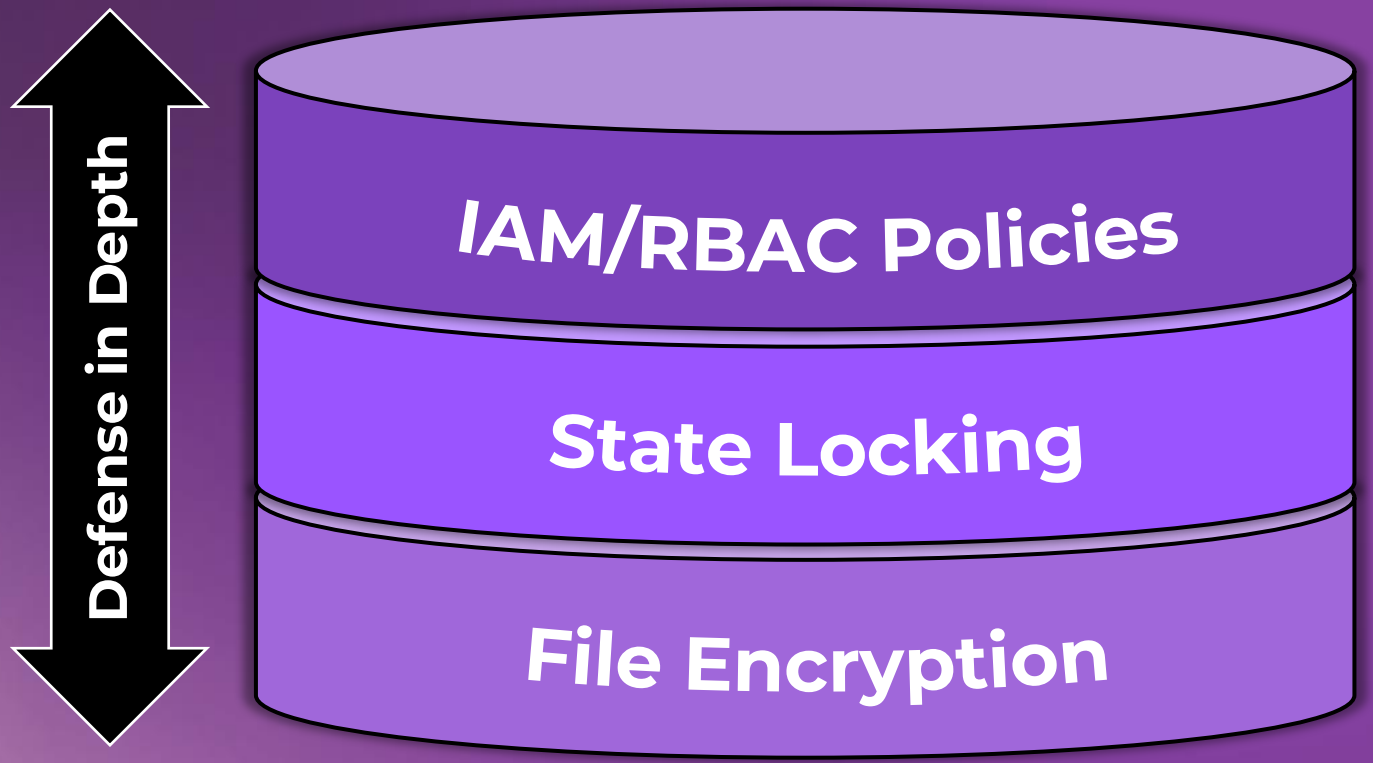
`encrypt = true` **enables AES-256 encryption**

State is encrypted before being written to S3

State locking enabled to prevent corruption

Access controlled via IAM policies

Controlling Who Can Access Terraform State



- Only authorized users/roles can read state
- Separate read vs write permissions
- Audit logging for all access

- Prevents concurrent modifications
- Lockfile for S3, built-in for others
- Protects against corruption

- KMS for additional encryption layer
- Customer-managed keys for compliance
- Automatic key rotation

Defense in Depth: Encryption + Access controls + Audit logging

State File Security Checklist

- [x] Always use remote state for production
- [x] Enable encryption at rest
- [x] Implement access controls
- [x] Enable state locking

State files will contain secrets. Your job is to protect where they're stored and who can access them.

LECTURE

Keeping Secrets Out of State

Ephemeral Values and Write-only Arguments

Even with All Our Protections…

- ✅ Sensitive variables → Hide output
- ✅ Environment variables → Avoid git
- ✅ Encrypted state → Protect storage
- ✅ External sources → Best practice

⚠ Sensitive Values still end up in state.
But…what if we could change that?

terraform.tfstate

```
{
  "version": 4,
  "terraform_version": "...",
  "serial": ...,
  "lineage": "...",
  "outputs": {
    "database_password_output": {
      "value": "MySuperSecretPassword123!",
      "type": "string",
      "sensitive": true
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "aws_db_instance",
      "name": "main",
      "provider": "...",
      "instances": [
        {
          "schema_version": ...,
          "attributes": {
            // ... other attributes
            "password": "MySuperSecretPassword123!",
            // ...
          },
```

Ephemeral Values

Secrets That Don't Persist

Introduced in Terraform 1.10, ephemeral values are a new approach to handling secrets. They exist only in memory during Terraform operations and are never written to state files or plan files. Values are calculated fresh on each Terraform run and discarded immediately after use.

Ephemeral variables must be marked with `ephemeral = true`

TERRAFORM 1.10+ REQUIRED

```
variable "database_password" {
  description = "Password for the database"
  type        = string
  sensitive   = true
  ephemeral   = true
}
```

Omit Value from State and Plans

Set the `ephemeral` argument to `true` in a `variable` block containing temporary sensitive data to omit that value from state and plan files.

Note: The value is also redacted in the Terraform CLI and the HCP Terraform UI

Where Can I Use Ephemeral Values?

- Local Values
- Provider Blocks
- Ephemeral Resources
- Ephemeral Variables
- Ephemeral Outputs
- Provisioner Blocks
- Write-Only Arguments

Important Restriction

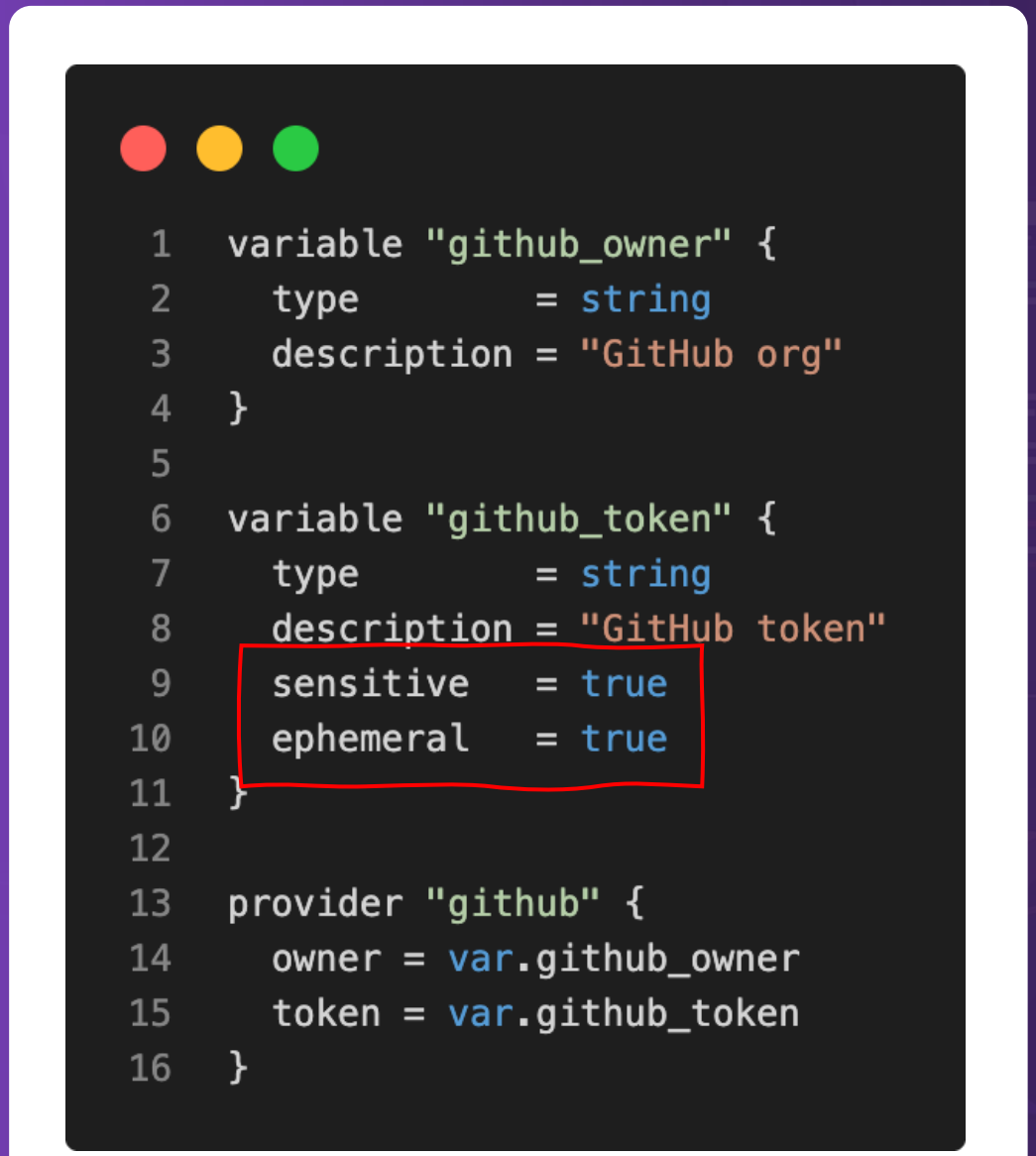
Where You CAN'T Use Ephemeral Values

TERMINAL

```
# This DOESN'T work ❌
resource "azurerm_mssql_server" "db" {
  administrator_login_password = var.db_password  # ephemeral
  # ...
}

Error: Invalid use of ephemeral value
| ...
|
| Ephemeral values are not valid in resource arguments,
| because resource instances must persist between
| Terraform phases.
```

- Resources need values that persist in state.
- But...we know that ephemeral values disappear after use.
- So how do we use secrets in resources?

This is where write-only arguments come in...



```
variable "github_owner" {
  type        = string
  description = "GitHub org"
}

variable "github_token" {
  type        = string
  description = "GitHub token"
  sensitive   = true
  ephemeral   = true
}

provider "github" {
  owner = var.github_owner
  token = var.github_token
}
```

Use Case for Ephemeral Values

The sensitive helps prevent the token from being displayed in output.

Ephemeral is the key part: Terraform omits the value from state and plan files entirely.

Ephemeral Resources

- New block type (not data or resource)
- Fetches data at runtime and stored in memory
- Never persisted in state and flushed when Terraform operations are completed
- Opened and closed during execution

```hcl
# Regular data source (stored in state)
data "azurerm_key_vault_secret" "db" {
  name         = "database-password"
  key_vault_id = var.key_vault_id
}

# Ephemeral resource (NOT stored in state)
ephemeral "azurerm_key_vault_secret" "db" {
  name         = "database-password"
  key_vault_id = var.key_vault_id
}
```

- Uses `ephemeral` keyword instead of `data`
- Syntax is nearly identical to data sources

When Should I Use an Ephemeral Resource?

- Fetching secrets from Vault
- Reading temporary credentials
- Retrieving dynamic tokens

The provider developer determines which arguments you can define for an ephemeral resource.

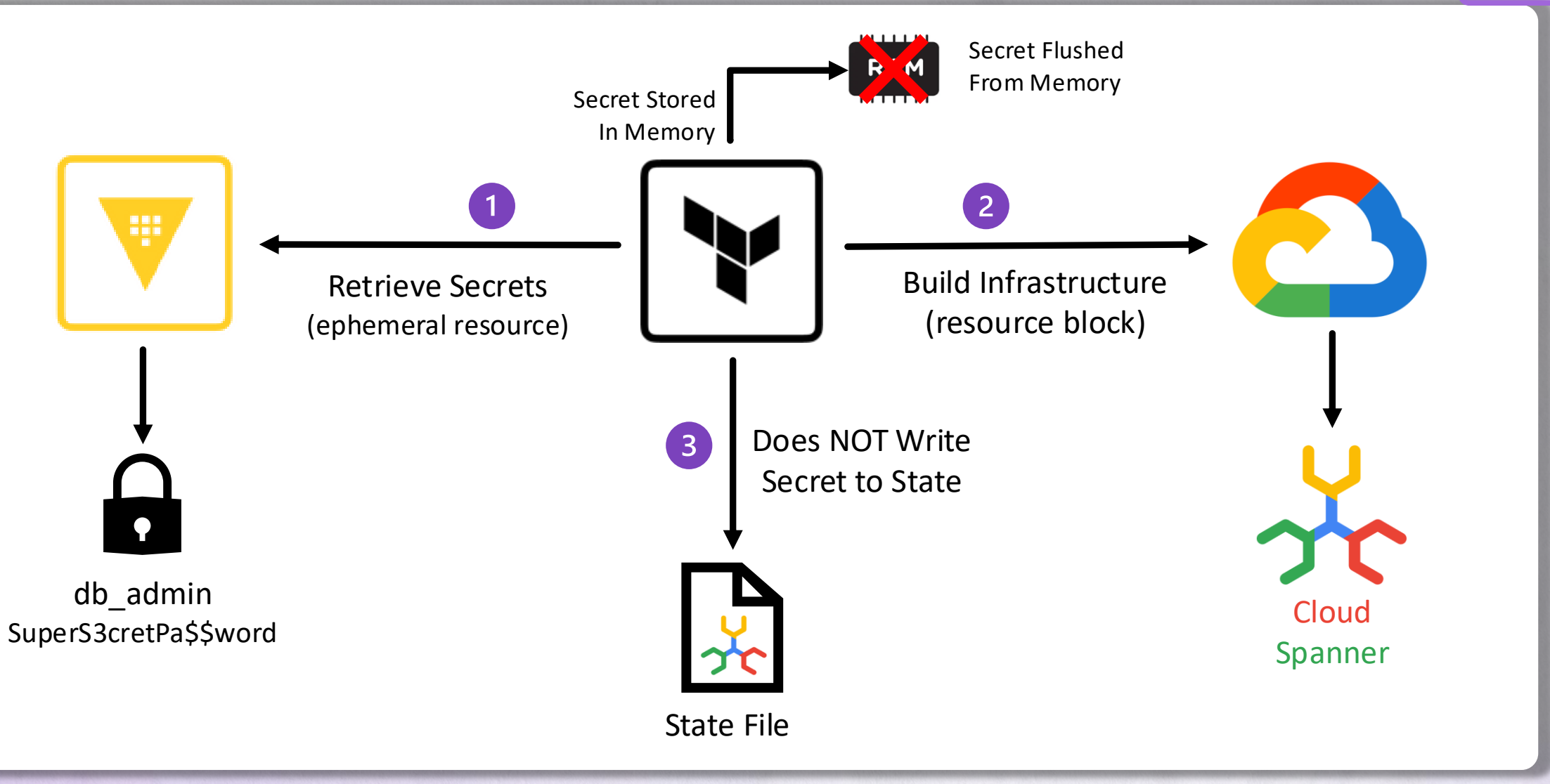
```
resource "aws_db_instance" "test" {
  instance_class      = "db.t3.micro"
  allocated_storage   = "5"
  engine              = "postgres"
  username            = "example"
  skip_final_snapshot = true
  password_wo         = "old-password-here"
  password_wo_version = 1
}
```

Write-Only Arguments

Write-only arguments let you securely pass temporary values to Terraform's managed resources during an operation without persisting those values to state or plan files.

Each provider defines any available write-only arguments on their managed resources.

Write-Only Arguments



Comparison

Regular Resource Block

```
1  resource "aws_db_instance" "test" {
2    instance_class      = "db.t3.micro"
3    allocated_storage   = "5"
4    engine              = "postgres"
5    skip_final_snapshot = true
6    username            = "db-admin"
7    password            = var.database_password
8  }
9
```

Secret Would Be Stored in State

Write-only Argument with Versions

```
1  resource "aws_db_instance" "test" {
2    instance_class      = "db.t3.micro"
3    allocated_storage   = "5"
4    engine              = "postgres"
5    skip_final_snapshot = true
6    username            = "db-admin"
7    password_wo         = var.ephemeral_pass
8    password_wo_version = 1
9  }
```

Secret Would NOT Be Stored in State

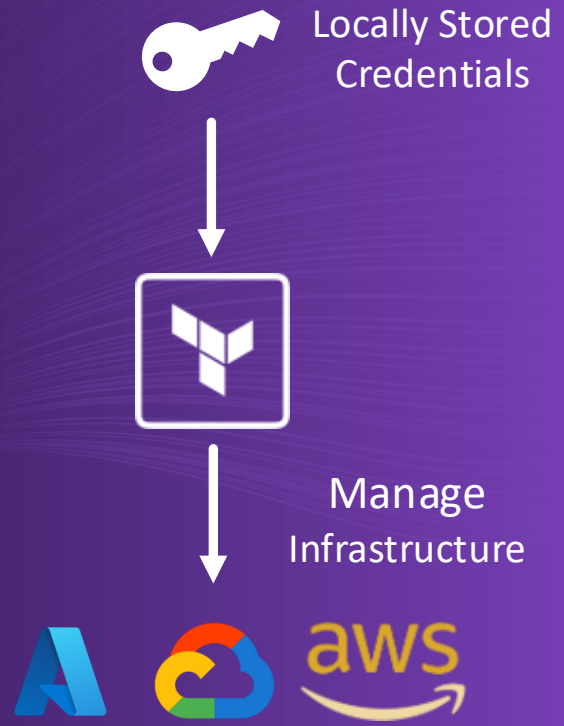
LECTURE

Working with HashiCorp Vault for Dynamic Credentials

The Problem with Using Static Credentials

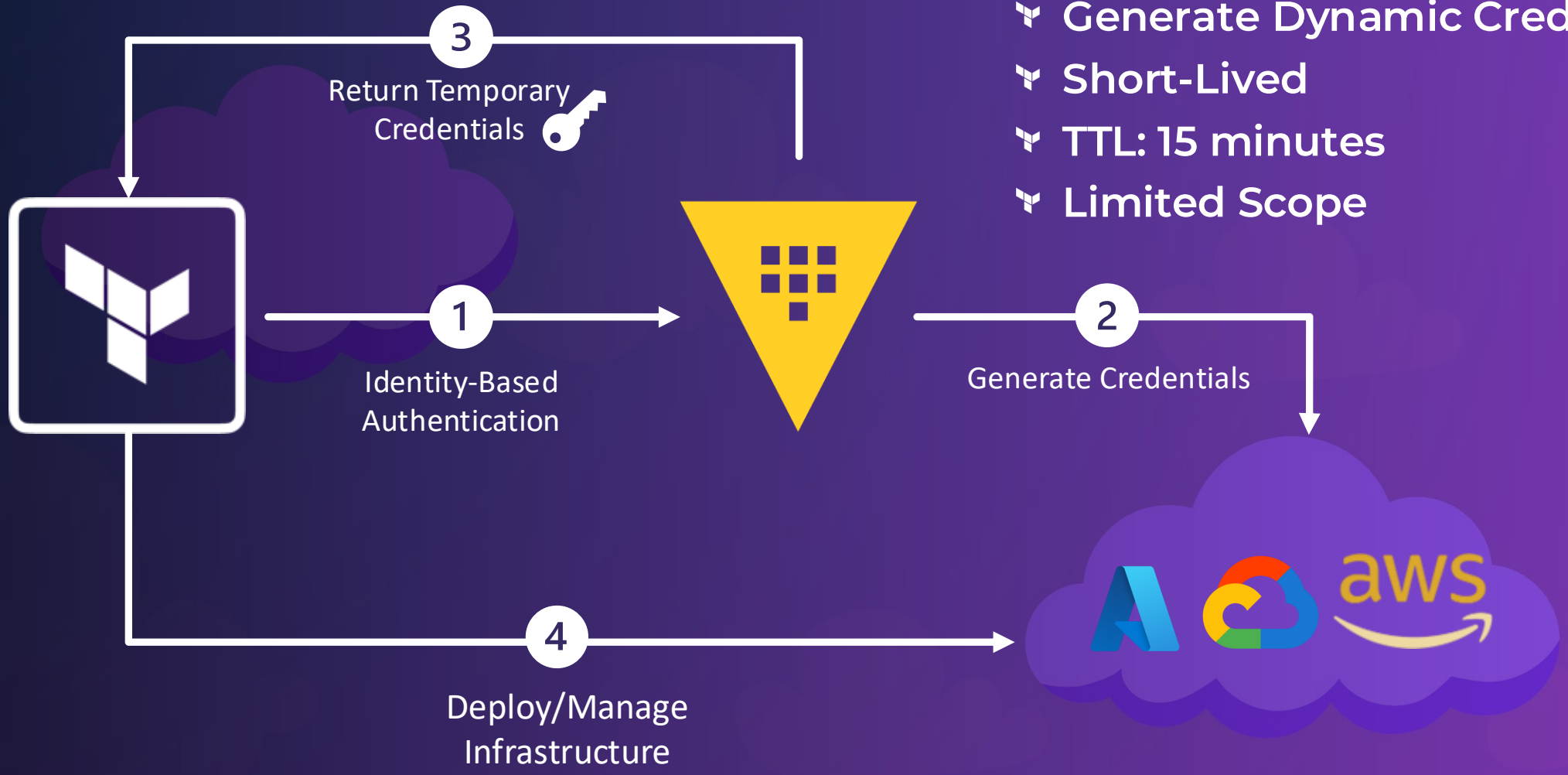
Traditional Approach:

- Terraform users get their own, long-lived credentials
- Stored locally on machines/laptops (security risk)
- Difficult to rotate, revoke, and audit
- Large attack surface because they are highly privileged

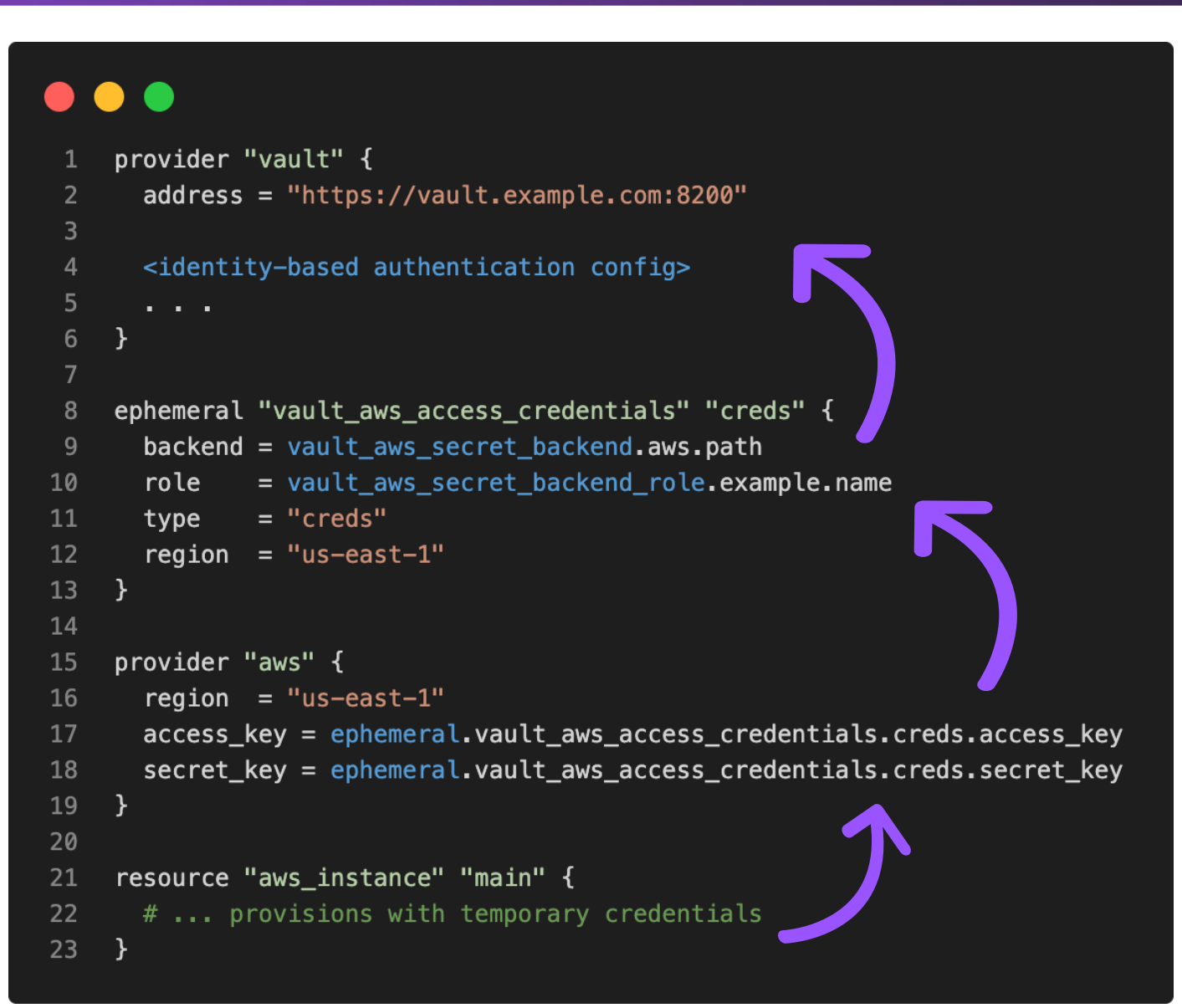


What if credentials were short-lived and generated on-demand?

Dynamic Credential Generation



- Generate Dynamic Credentials
- Short-Lived
- TTL: 15 minutes
- Limited Scope



The Vault Provider Workflow

- Credentials generated when requested
- Valid for 120 seconds
- Automatically revoked after TTL
- Creds not written to state when using ephemeral resources

Summary: Why Use Vault for Terraform

Centralized Secret Management

- One place to manage platform credentials
- Easy to rotate long-lived credentials

Reduced Attack Surface

- Credentials expire automatically
- Compromised creds only valid for minutes

No Local Credential Storage

- Developers never touch AWS credential
- No credential redistribution

LECTURE

Debugging Terraform

Why Would You Need Detailed Logs?

- Investigating unexpected Terraform behavior
- Preparing bug reports for HashiCorp or provider teams
- Understanding exactly what Terraform is doing under the hood

The TF_LOG Environment Variable

Setting TF_LOG Enables Detailed Logging

TERMINAL

```
$ export TF_LOG=TRACE     //Linux/MacOS/Bash
> $env:TF_LOG="TRACE"     //Windows Powershell

$ terraform plan
<detailed logs>
```

Most Verbose (Detailed) → Less Verbose

JSON → TRACE | DEBUG | INFO | WARN | ERROR

Separate Core and Provider Logging

```
TF_LOG_CORE=TRACE
```

- Logging can be enabled separately for Terraform Core itself
- Takes the same level arguments (trace, debug, info, etc.)
- Provide logs to the Terraform core team (support)

```
TF_LOG_PROVIDER=TRACE
```

- Enable logging for only the providers
- Takes the same level arguments (trace, debug, info, etc.)
- Offer logs to the provider developers to troubleshoot bugs

Saving Logs to a File

Persist Logs using the TF_LOG_PATH env

TERMINAL

```
$ export TF_LOG=TRACE
$ export TF_LOG_PATH=terraform.log
```

TF_LOG (or TF_LOG_CORE/TF_LOG_PROVIDER) must also be set

Logs are appended to the file

File is created automatically if it doesn't exist

LECTURE

Introduction to HCP Terraform

What is HCP Terraform?

HCP Terraform is a hosted service that helps teams use Terraform to provision infrastructure. HCP Terraform can help you collaborate, manage remote state, and automate workflows.

The Value of HCP Terraform

Secure Remote State Storage

Single source of truth

Encrypted at rest

State locking built-in

Team Collaboration

Shared workspaces

Role-based access control

Audit logs

Remote Execution

Consistent environment

No local dependencies

Reliable runs

Governance & Policy

Policy enforcement

Cost estimation

Drift detection

Comparing Terraform with HCP Terraform

Terraform Community

- ✅ Define your infrastructure as code
- ✅ Free and "code-available"
- ✅ Rich set of integrations (providers)
- ✅ Reusable modules (public registry)
- ✅ Ability to plan out your changes
- ✅ Local execution and control

☁ HCP Terraform adds...

- ➕ Managed remote state storage
- ➕ Consistent execution environments
- ➕ Built-in team collaboration
- ➕ Role-based access control (RBAC)
- ➕ Policy enforcement (Sentinel/OPA)
- ➕ Cost estimation before apply
- ➕ Audit logs and compliance
- ➕ Private module registry

How Does HCP Terraform Impact Me?

🤝 Your Code Still Works!!

- Everything you've learned about Terraform - the `resource` blocks, the `data` sources, the modules you've created - it all works exactly the same with HCP Terraform
- HCP Terraform doesn't change the Terraform language - it changes where and how your code runs

💻 Leverage existing Workflows:

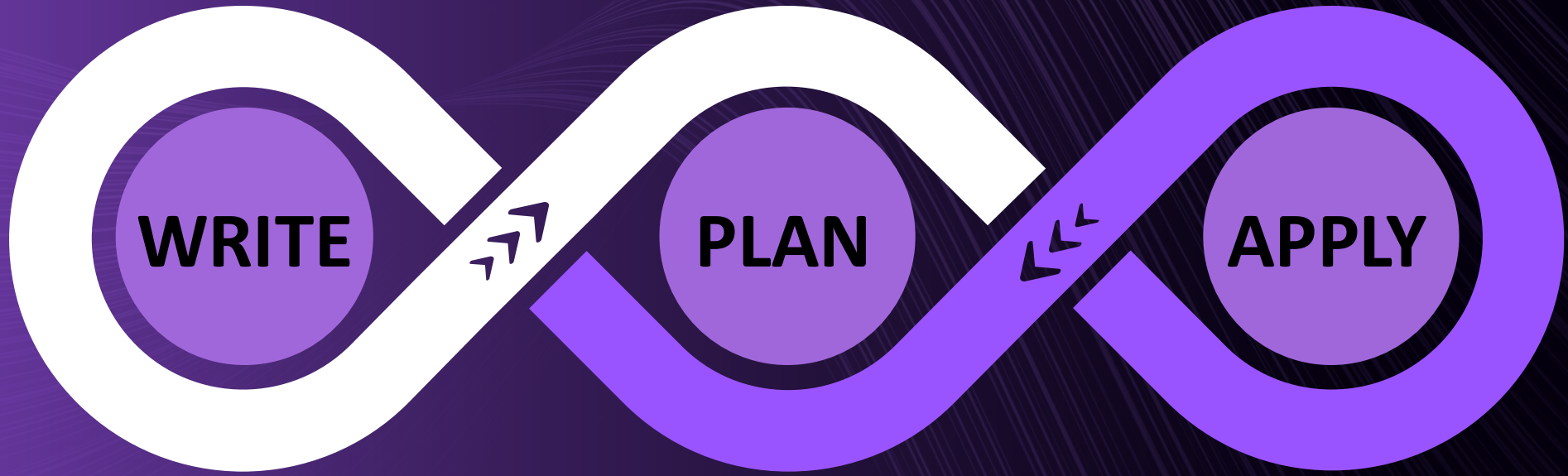
- Your CLI commands (`terraform plan`, `terraform apply`) work the same way
- Version control, CI/CD pipelines, and automation all integrate seamlessly
- You gain collaboration features, audit logs, and managed state without changing your daily workflow

🚀 Adopt at Your Own Pace:

- Start with just remote state storage and add features over time
- Keep using local execution if you want - or switch to remote when you're ready
- Whether you're a solo practitioner or part of a large team, HCP Terraform scales with you

The HCP Terraform Workflow

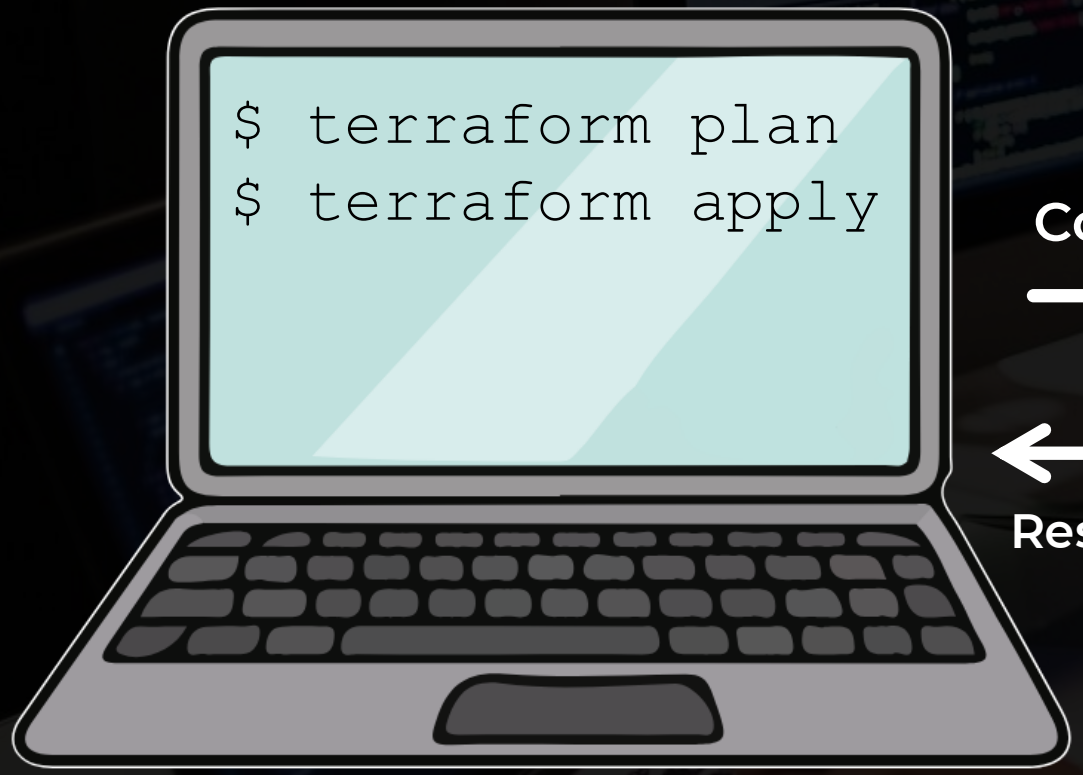
Plan runs remotely in HCP Terraform
See changes before applying



Write Terraform configuration locally
Same .tf files you already know

Apply runs remotely in HCP Terraform
Infrastructure created/updated

Where Does Code Execution Happen?



Your Local Machine

Commands Sent

Results Returned to CLI

HCP Terraform

Execution Happens Here

Execution Modes: Where Runs Happen

Remote Execution

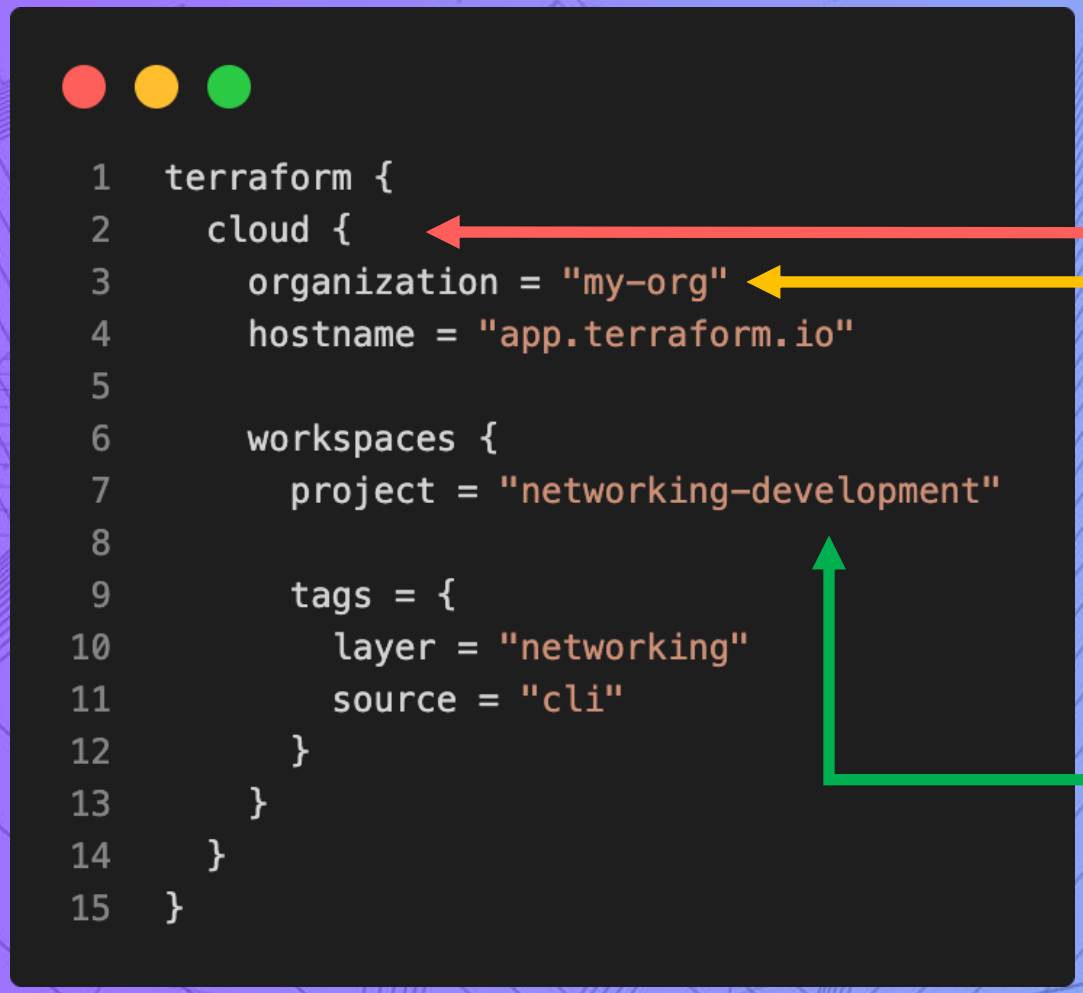
- Plans and applies run in HCP Terraform
- Recommended for teams
- Most HCP features available
- Results streamed back to local CLI

VS

Local Execution

- Plans and applies run on your machine
- State still stored remotely in HCP Terraform
- Limited access to HCP features

Connecting to HCP Terraform



```
terraform {
  cloud {
    organization = "my-org"
    hostname = "app.terraform.io"

    workspaces {
      project = "networking-development"

      tags = {
        layer = "networking"
        source = "cli"
      }
    }
  }
}
```

`cloud` block tells Terraform to use HCP

Point to your organization (auth required)

Configure what workspace/project to use

HCP Terraform Tiers

Free Tier

State storage and basic features

Includes 500 managed resources at no charge

Essentials

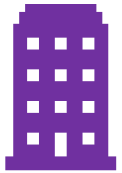
Access to More Features

Includes Stacks, Unlimited Applies, etc.

Standard

For Enterprises

Drift Detection, Golden Patterns, Audit Events, Policy Enforcement



Premium

Multi-Cloud Enterprise

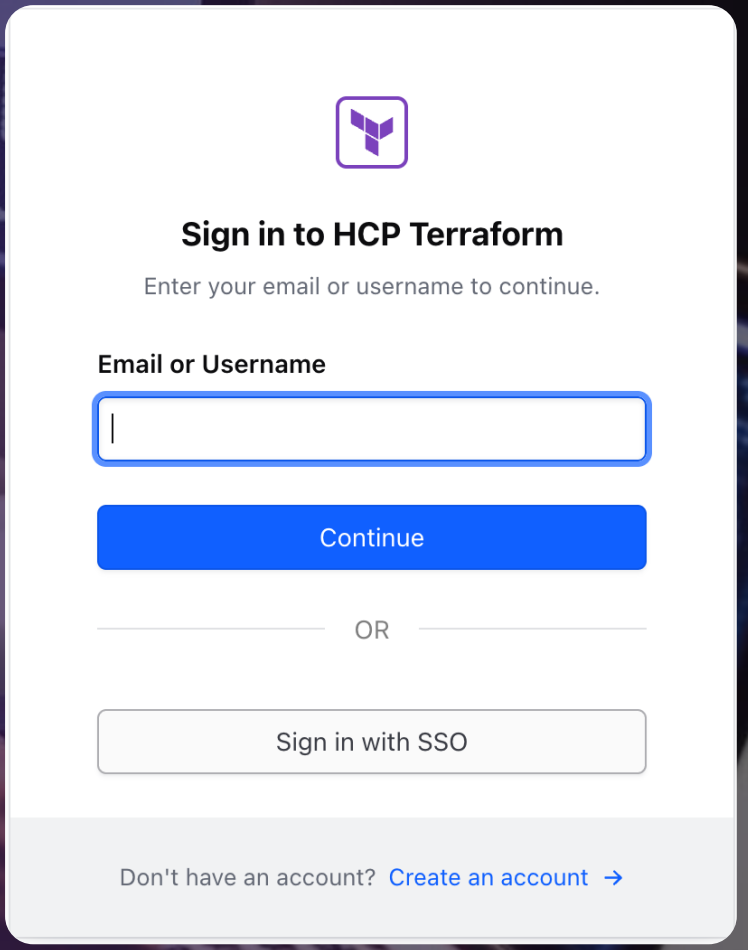
Private VCS, Run Tasks, Module Revocation

*Subject to Change – Check for Updates

How Do I Sign Up?

Visit the HCP Terraform sign-up page and follow the prompts to create a free HCP Terraform account.

https://app.terraform.io



Key Takeaways

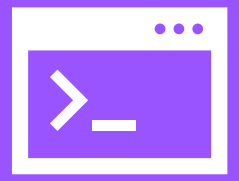
01

HCP Terraform is a managed platform that extends Terraform

02

Same Terraform code - different execution environment

03



Remote execution is default; local execution available for flexibility

LECTURE

Getting Connected and Authenticated

The Gap We Need to Bridge

You need to authenticate before HCP Terraform trusts your CLI

Connecting to HCP Terraform

```
terraform {
  cloud {
    organization = "my-org"
    hostname = "app.terraform.io"

    workspaces {
      project = "networking-development"

      tags = {
        layer = "networking"
        source = "cli"
      }
    }
  }
}
```

Step 1

Add your `cloud` block to your Terraform configuration

`organization` **argument = your org name**

`workspaces` **block with the two approaches:**

- `name/project` **= target a specific workspace/project**
- `tags` **= targets workspaces matching tag(s)**

Authenticating with `terraform login`

Step 2

TERMINAL

```
$ terraform login
Terraform will request an API token for app.terraform.io using your
  browser.

If login is successful, Terraform will store the token in plain text in
the following file for use by subsequent commands:
    /Users/bk/.terraform.d/credentials.tfrc.json

Do you want to proceed?
  Only 'yes' will be accepted to confirm.

  Enter a value: yes
```

Authenticating with `terraform login`

Tokens

Create an API token

Your API tokens can be used to access the HCP Terraform API and perform all the actions your user account is entitled to. For more information, see the user API tokens documentation.

Treat these tokens like passwords, as they can be used to access your account without a username, password, or two-factor authentication.

Tokens (1)

terraform login Expires today at 10:40 AM

Copy the token, then return to your terminal and paste it into your Terraform login prompt. HCP Terraform will not di… token again, so store it securely.

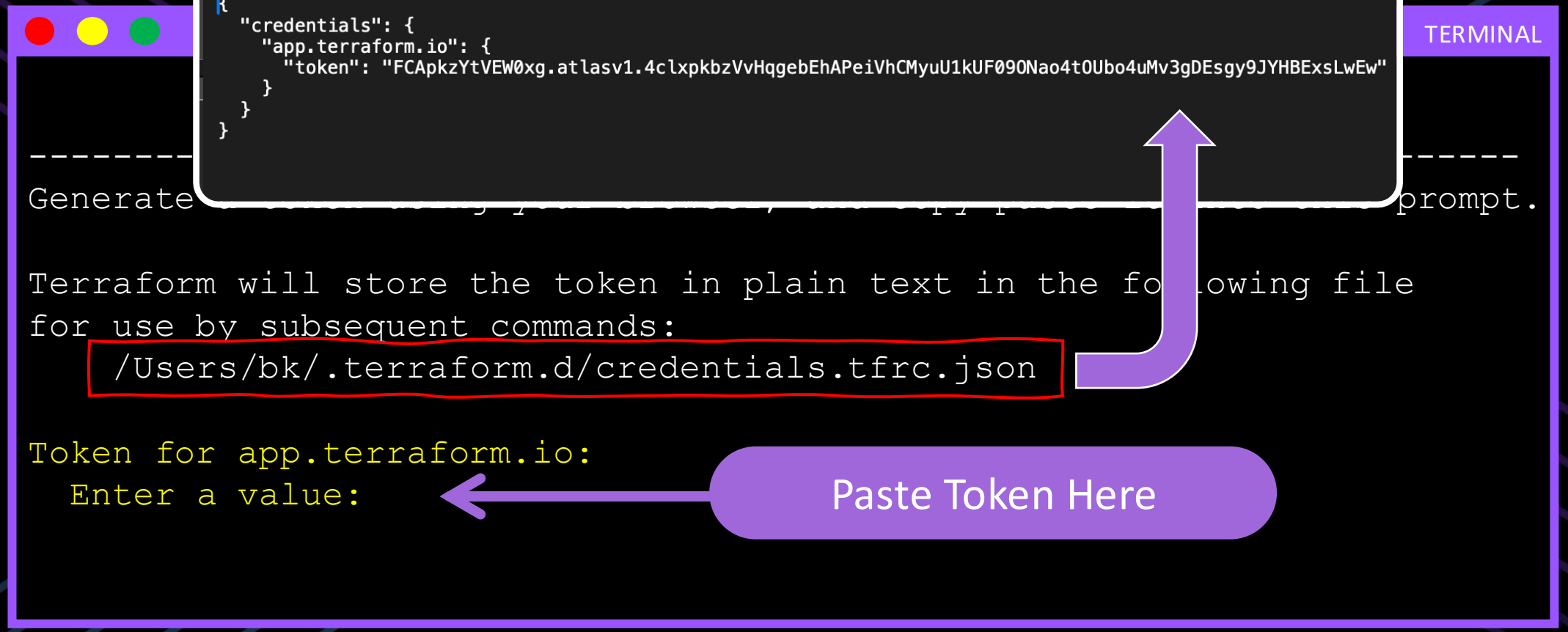
COPY

FCApkzYtVEW0xg.atlasv1.4clxpkbzVvHqgebEhAPeiVhCMyuU1kUF090Nao4tOUbo4uMv3gDEsgy9JYHBExsLwEw

Created just now by user bkrausen

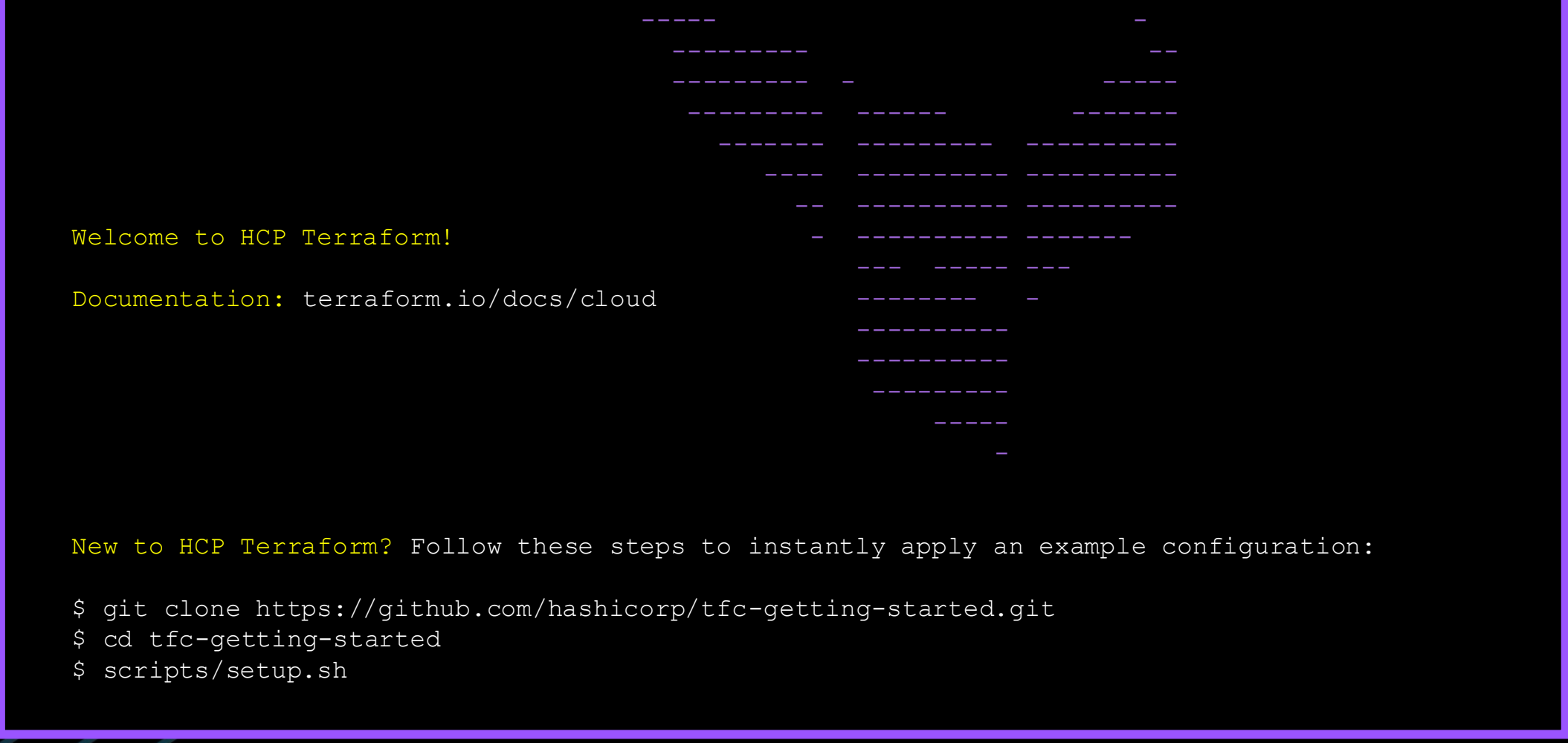
Token has not been used

Authenticating with `terraform login`

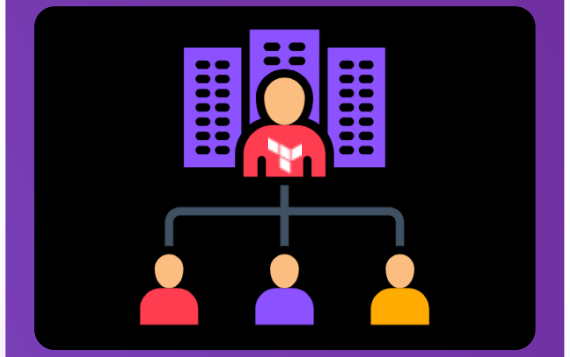


Authenticating with `terraform login`

TERMINAL



HCP Terraform - API Token Types



Organization Tokens

- Used to Manage Teams, Membership, and Workspaces
- Does not have permission to run plans and applies

Team Tokens

- Often used by automated services, like CI/CD pipelines
- Can run Terraform operations (plan/apply)

User Tokens

- Same permission level as your user account
- Only type of token that can access multiple organizations

Audit Tokens

- Provides read-only access
- Used for accessing the audit trail API

HCP Terraform - Environment Variable Alternative

As an alternative to using `terraform login`, you can set the token using the env:

```
TF_TOKEN_app_terraform_io=<token>
```

Why is this important to know?

CI/CD pipelines and automation will likely need this where interactive browser login isn't possible

LECTURE

Understanding HCP Terraform Workspaces

These Are NOT the Workspaces You Know

CLI Workspaces

- Same Configuration, Multiple State Files
- Managed locally with `terraform workspace`
- Lightweight state isolation

HCP Terraform Workspaces

- Complete working environments
- Config + state + variables + settings + credentials + run history
- Managed through HCP Terraform (UI, CLI, or API)

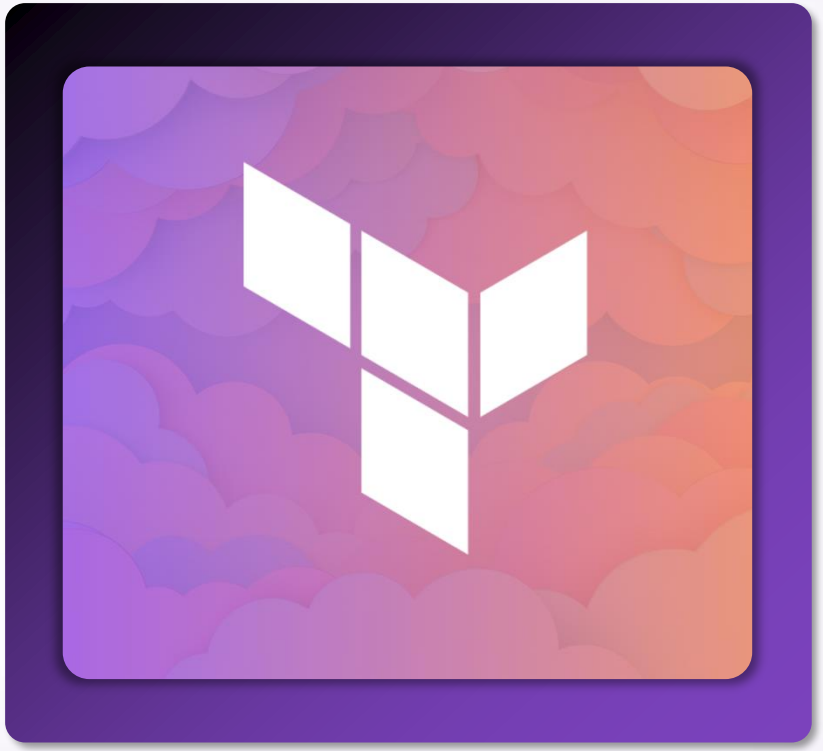
Same Name, Fundamentally Different Concept

Bryan Krausen

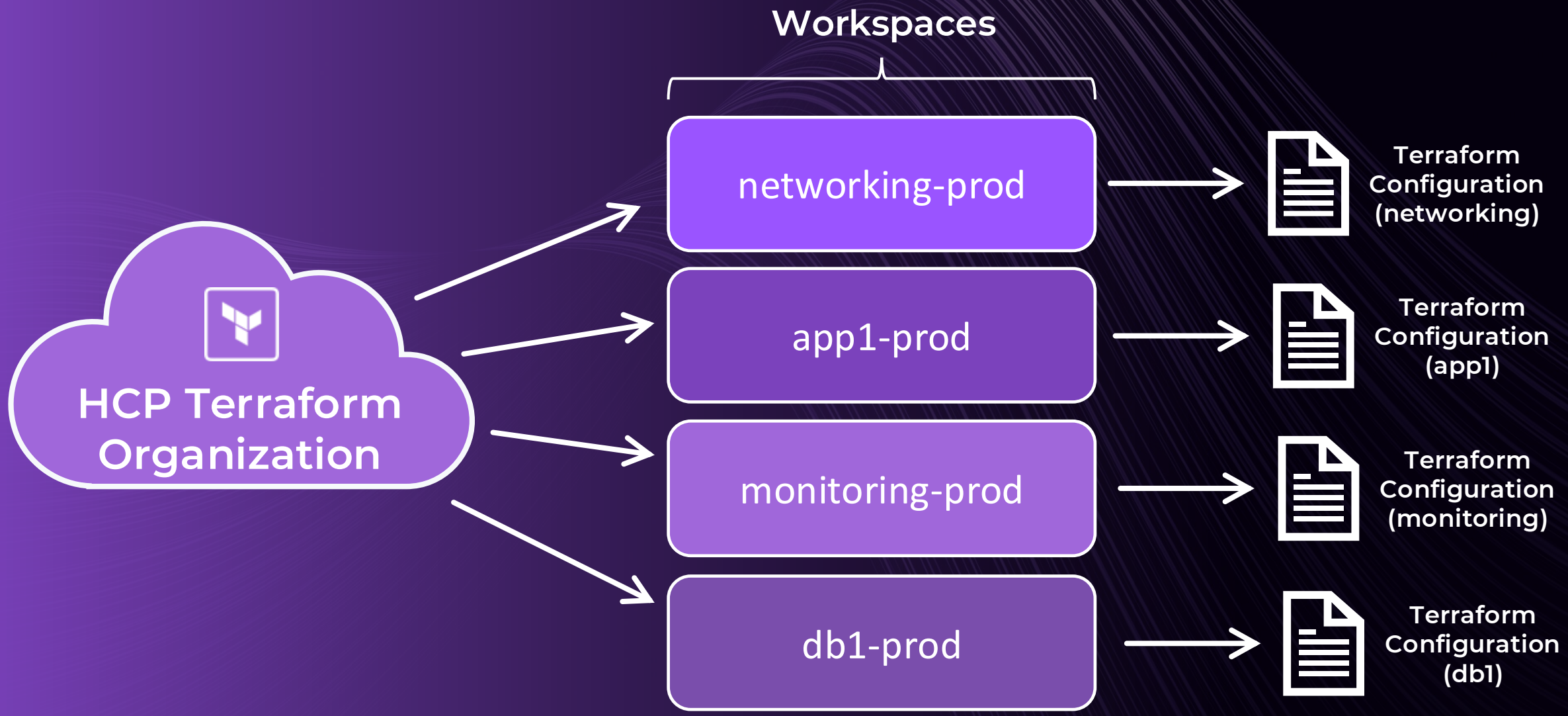
Introduction to HCP Terraform Workspaces

A workspace is everything Terraform needs to manage a collection of infrastructure.

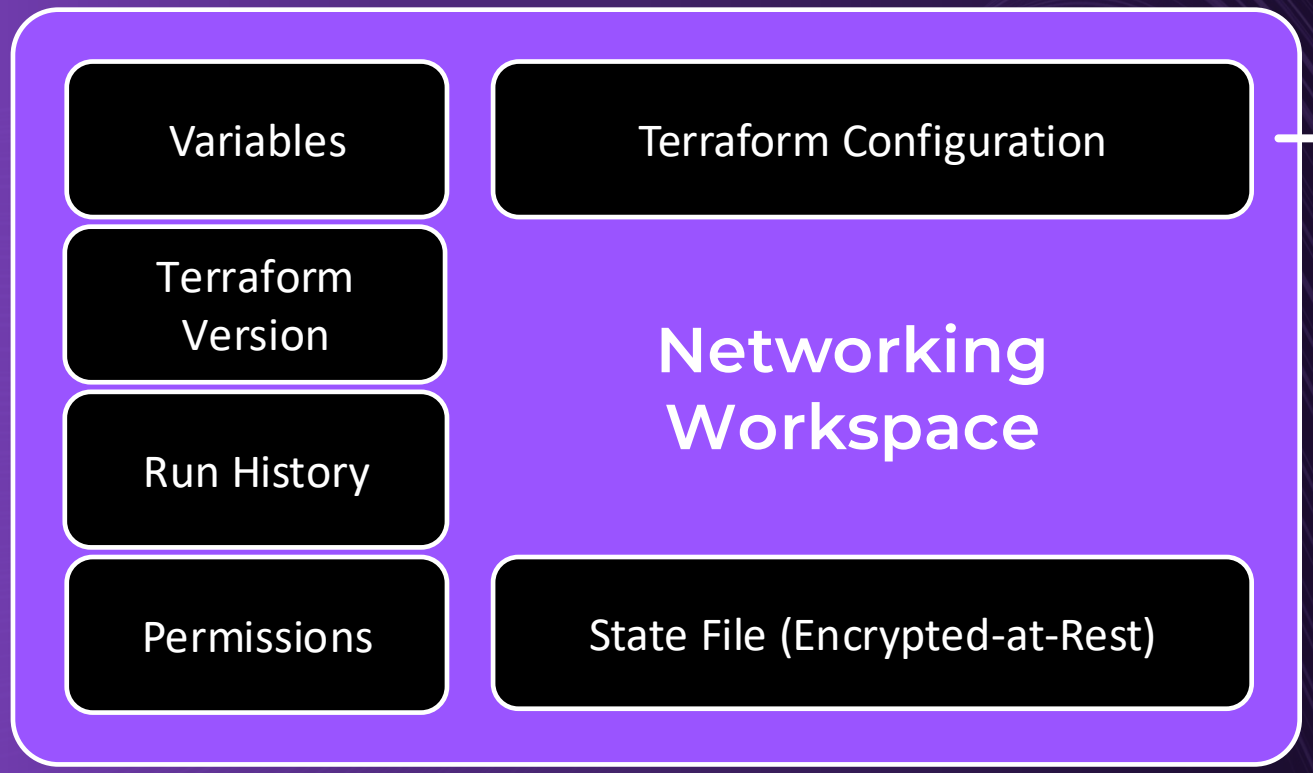
- Terraform configuration (linked via VCS or uploaded via CLI)
- State file (with version history and rollback capability)
- Variables (Terraform variables + environment variables, with sensitive marking)
- Run history (full audit trail of every plan/apply)
- Execution settings (mode, Terraform version, auto-apply)
- Team access and permissions



HCP Terraform Workspaces



HCP Terraform Workspaces



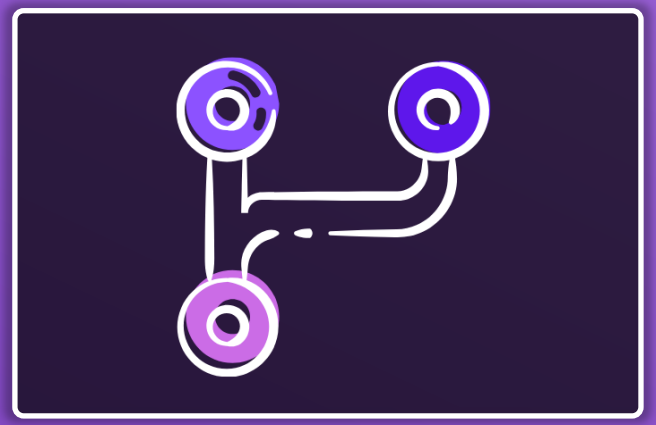
VCS Connection

Git Repository
(GitHub, BitBucket, etc.)

Commits
Pull Requests

Developer/DevOps Engineer

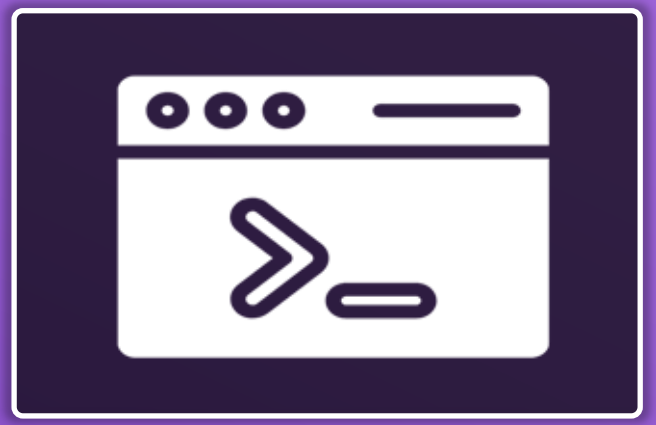
Workspace Workflow Types



Version Control Workflow

Trigger runs based on changes to configuration in repositories

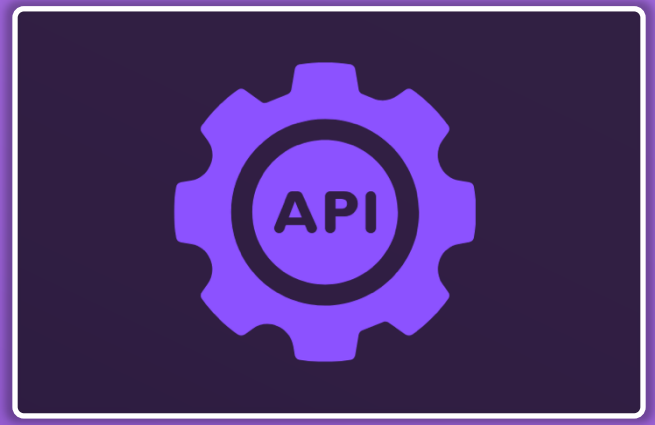
Best for those who need traceability and transparency



CLI-Driven Workflow

Trigger runs in a workspace using the Terraform CLI

Best for those comfortable with Terraform CLI



API-Driven Workflow

Trigger runs using the HCP Terraform API

Best for those with custom integrations and pipelines

Remote Operations

Terraform runs managed by HCP Terraform are called remote operations

- Your plans and applies occur on HCP Terraform's infrastructure.
- You and your team have the ability to review and collaborate on runs within the app.
- Full feature access (Sentinel, cost estimation, notifications)

Local Execution Mode

You can disable remote operations for any workspace by changing its Execution Mode to Local

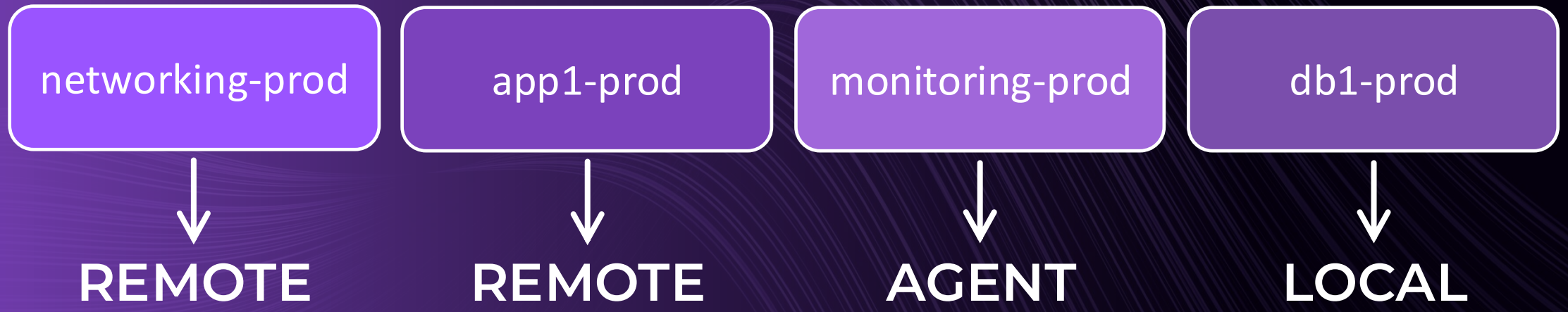
- Plan and apply run on your machine
- HCP Terraform is only used to store and synchronize state.
- Workspace variables and variable sets are NOT evaluated

Agent Mode

Run your own lightweight HCP Terraform agents to run Terraform in isolated, private, or on-premises infrastructure.

- Plan and apply run on a self-hosted agent
- For private/on-premises infrastructure HCP Terraform can't reach directly
- HCP Terraform and Terraform Enterprise use the same agent instance to evaluate policies and run Terraform operations

Execution Mode

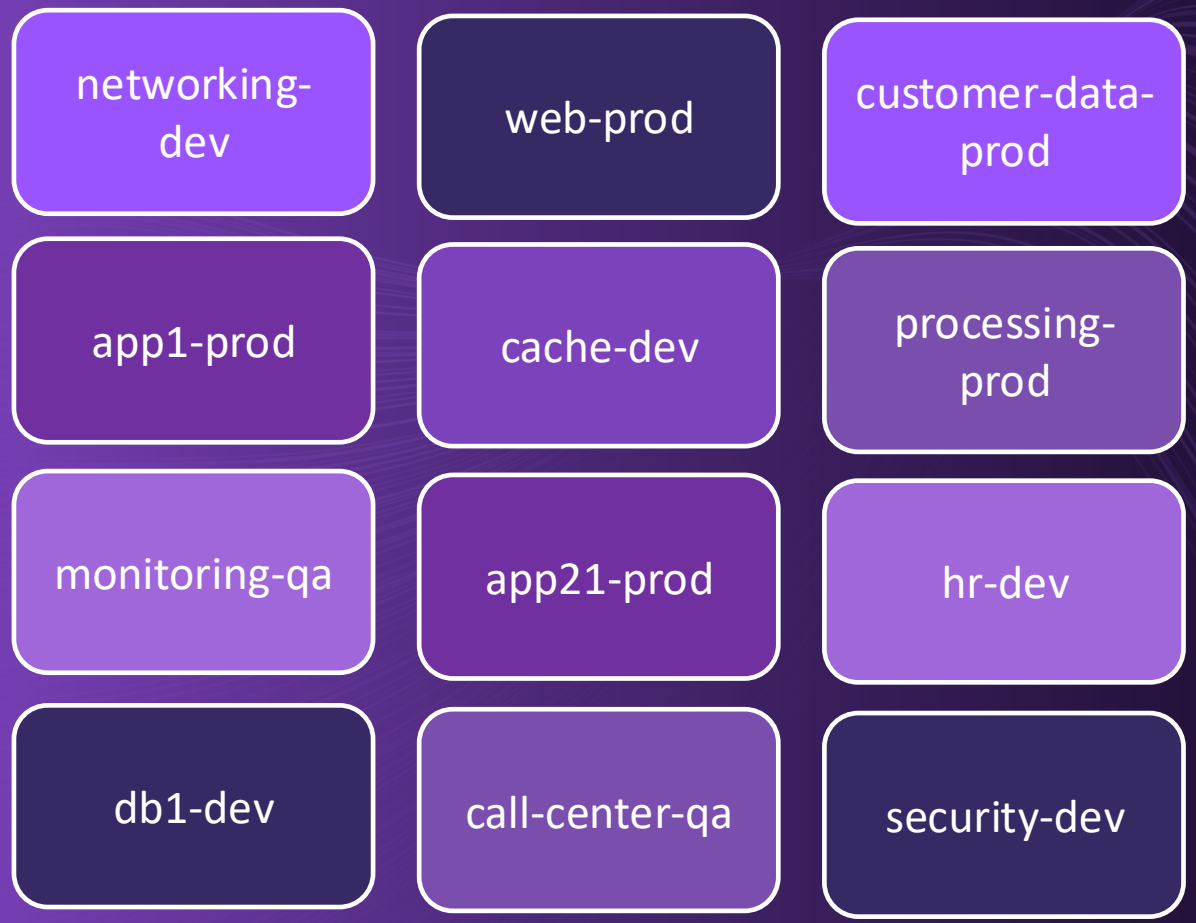


Execution mode is configured per workspace

LECTURE

Organizing Workspaces with Projects

Workspace Sprawl



As Your Organizations Grow:

- Workspaces Become Unmanageable
- Making Changes One Workspace at a Time
- No Clean Way to Separate Team Boundaries
- Duplicating Efforts

Workspace Organization

Development Project

- cache-dev
- networking-dev
- hr-dev
- db1-dev
- security-dev

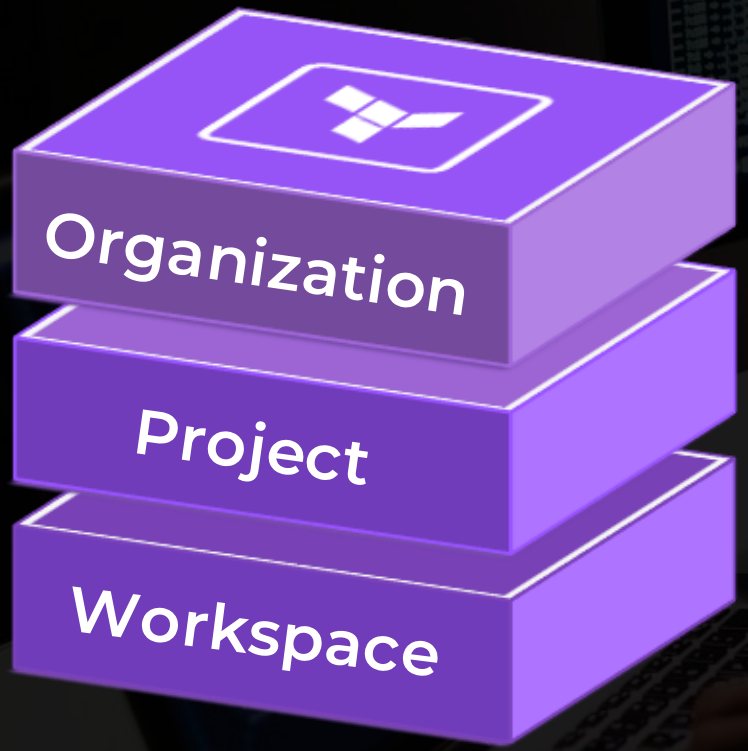
Quality Assurance Project

- monitoring-qa
- call-center-qa

Production Project

- processing-prod
- customer-data-prod
- web-prod
- app21-prod
- app1-prod

Projects – The Organizational Layer



ORGANIZATION

Your top-level account in HCP Terraform. It holds all your teams, projects, and workspaces, and is where you set organization-wide settings.

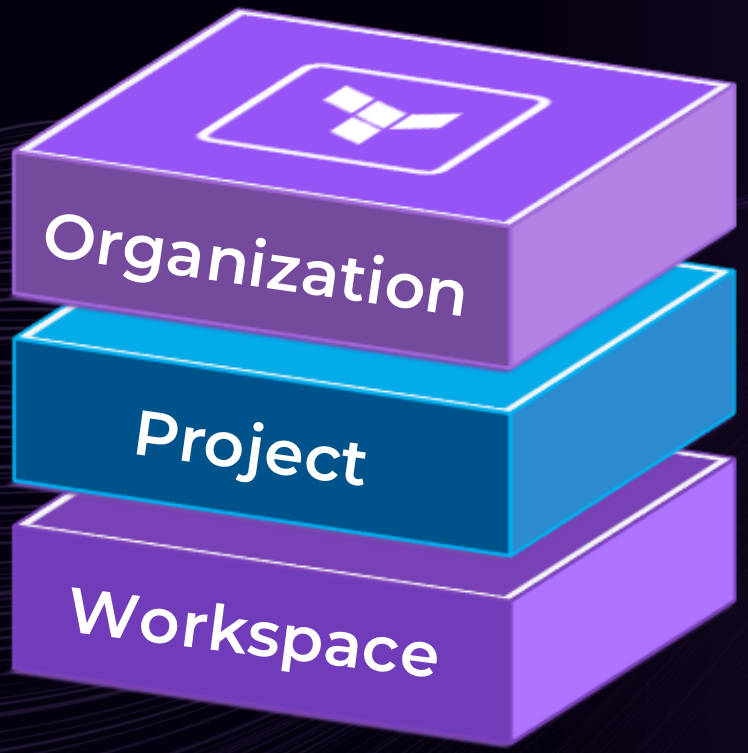
PROJECT

Organizes related workspaces together. Lets you scope team permissions, variable sets, and execution modes to a collection of workspaces. Every workspace must belong to exactly one project.

WORKSPACE

The individual unit that manages a specific set of infrastructure. Each workspace contains its own Terraform configuration, state file, variables, and run history.

Projects



- Every workspace belongs to exactly one project - can't belong to two, can't belong to none
- Default Project - every org gets one automatically; you can rename it but can't delete it
- Scoped team permissions - grant a team access to an entire project instead of workspace-by-workspace (least privilege)
- Project-level variable sets - apply variables (like cloud credentials) to all current and future workspaces in the project automatically

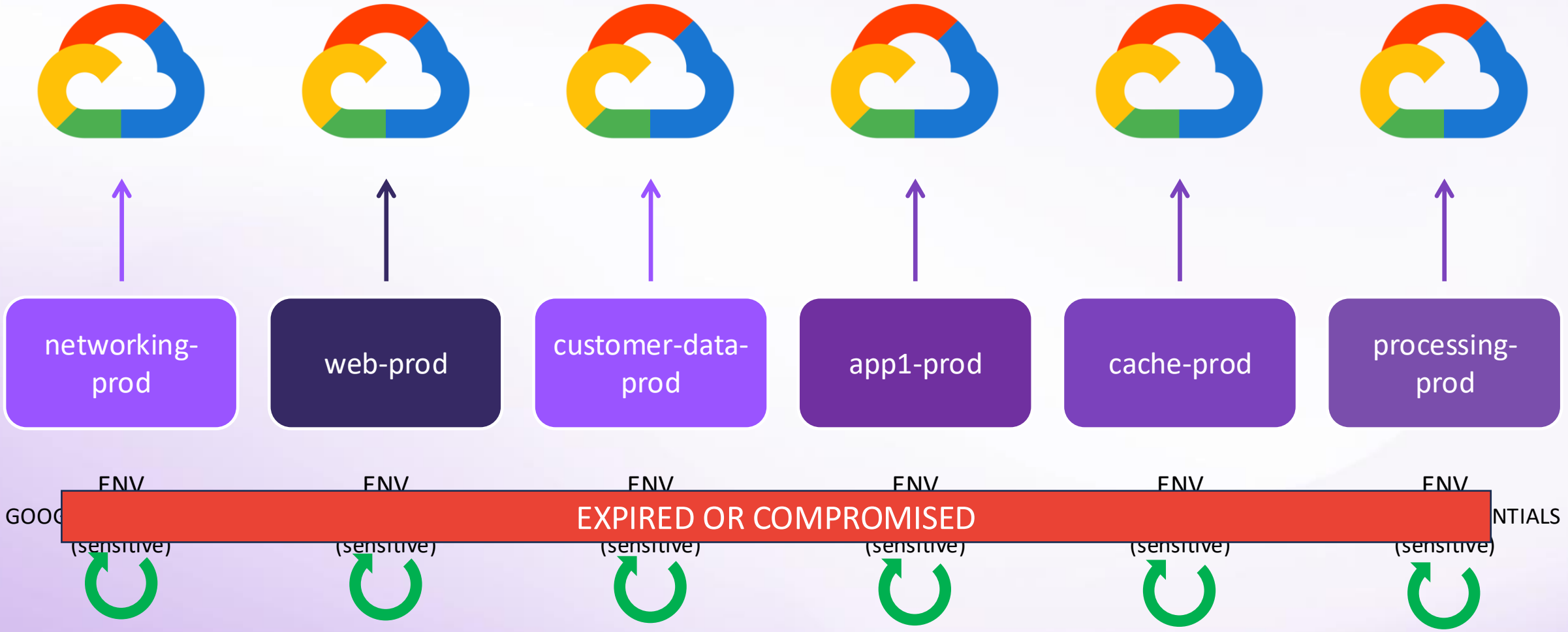
Projects

- Execution mode inheritance - set it at the project level; workspaces inherit from their project, which inherits from the org
- You can move workspaces between projects and specify a project in the cloud block (Terraform 1.6+)
- Only empty projects can be deleted

LECTURE

Sharing Variables with Variable Sets

Variable Repetition



Organize Your Variables

Variable Set

Production Cloud

- AWS_SECRET_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_REGION

Variable Set

Database Settings

- DB_WRITE_CAPACITY=1
- DB_READ_CAPACITY=1

Variable Set

VM-Size

- QA_SIZE=m5.large
- TEST_SIZE=m6.2xlarge
- PROD_SIZE=M6.4xlarge

Variable Set

Audit Configs

- PATH=/var/log/app
- ROTATE=7d

networking -prod

web-prod

app1-prod

cache-prod

qa-database-project

test-db-project

networking -prod

web-prod

app1-prod

cache-prod

customer-data-prod

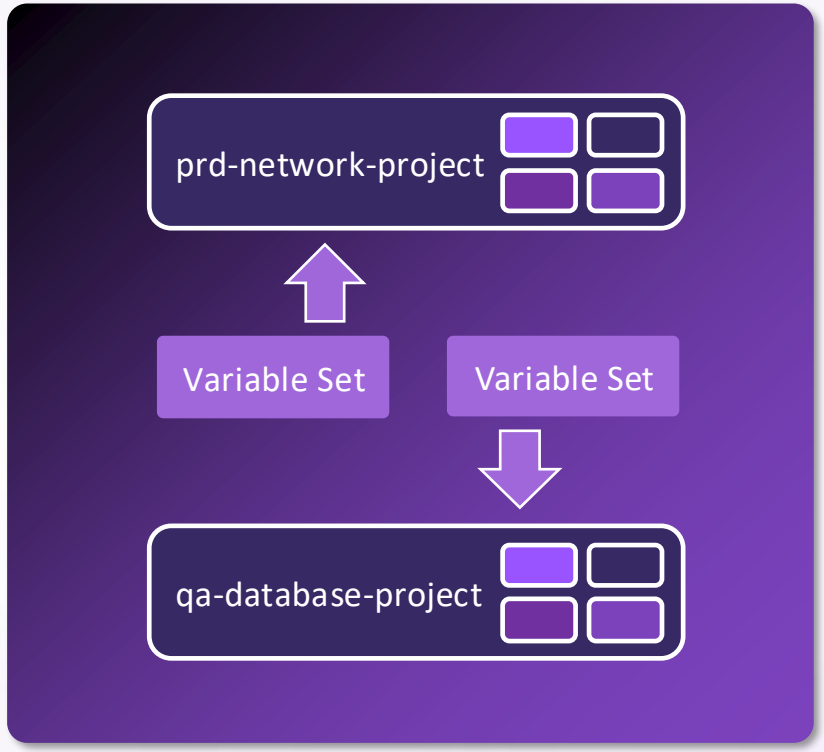
processing-prod

All Projects and Workspaces

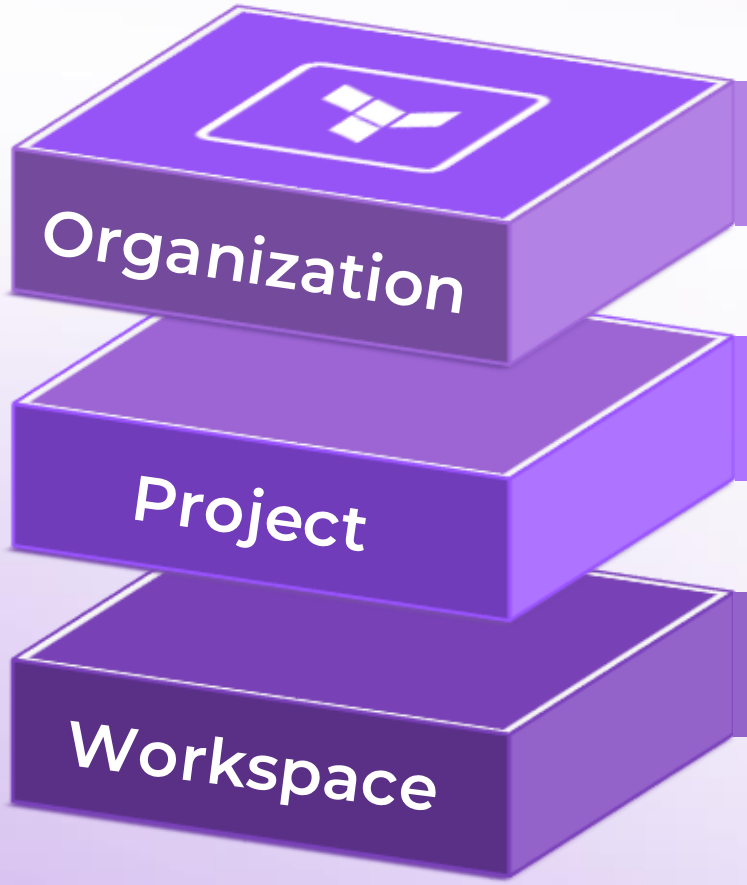
What Are Variable Sets?

Variable sets let you reuse variables in an efficient and centralized way, so you can set variable values once and use them in multiple workspaces.

- Reusable collections of variables you define once and apply to many workspaces
- Can contain both Terraform variables and environment variables
- Update a value once and changes propagate to all associated workspaces automatically
- Workspace-specific variables can override variable set values



Variable Set Scopes



Global

Applies to all current and future workspaces in the organization

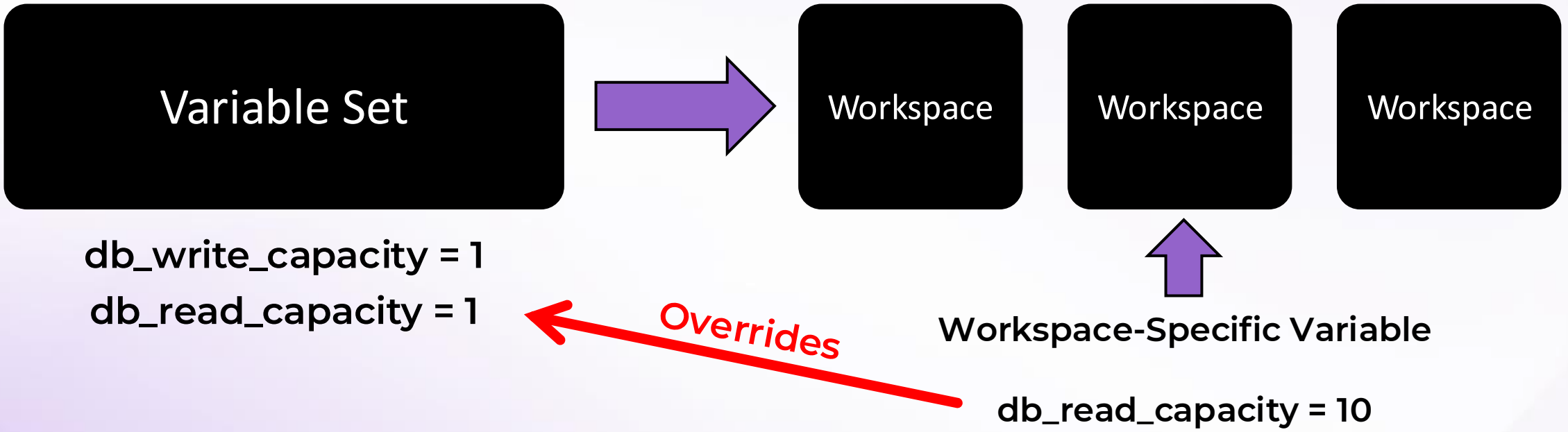
Project-Scoped

Applies to all current and future workspaces within selected projects

Workspace-Scoped

Applies only to specifically selected workspaces

Variable Precedence



HCP Terraform will always use workspace-specific variables over any variables defined in variable sets applied to the workspace.

Variable Precedence

Priority variable sets can be flagged to override more specific scopes

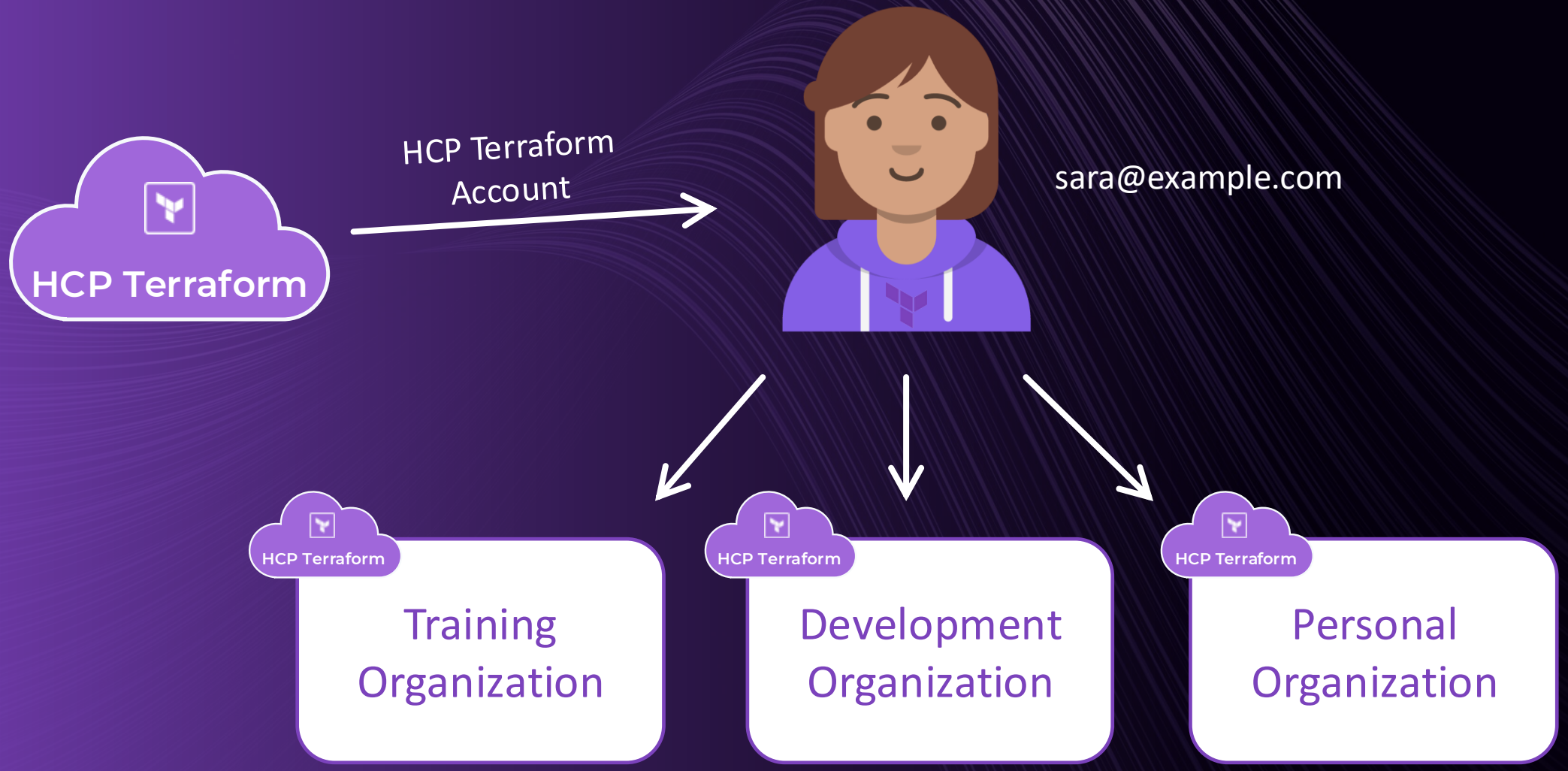
CLI flags and `TF_VAR_` environment variables override workspace and variable set values in CLI-driven workflows

When two variable sets at the same scope conflict, HCP Terraform uses lexical (alphabetical) order of the set names

LECTURE

HCP Terraform
Teams & Permissions

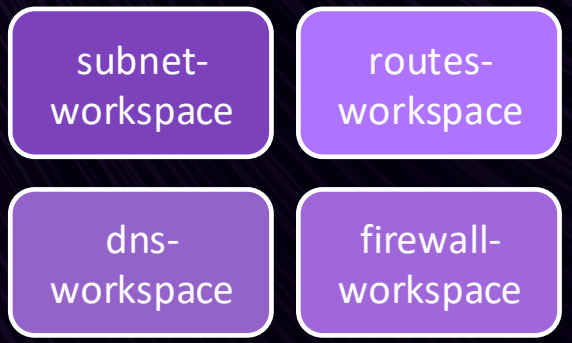
HCP Terraform Users



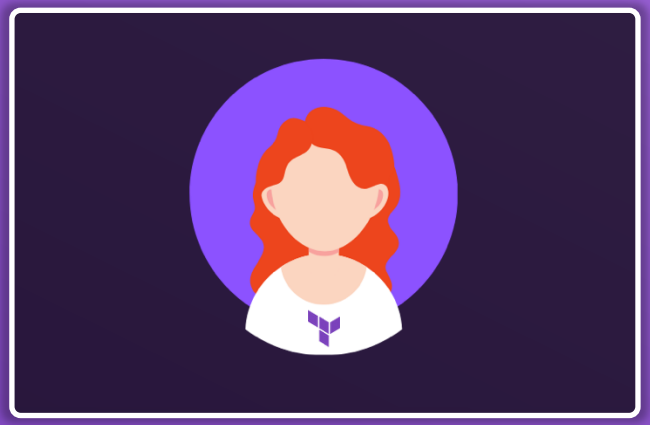
HCP Terraform Teams

Mobile App Team

Network Engineering Team



The Access Control Hierarchy



Users

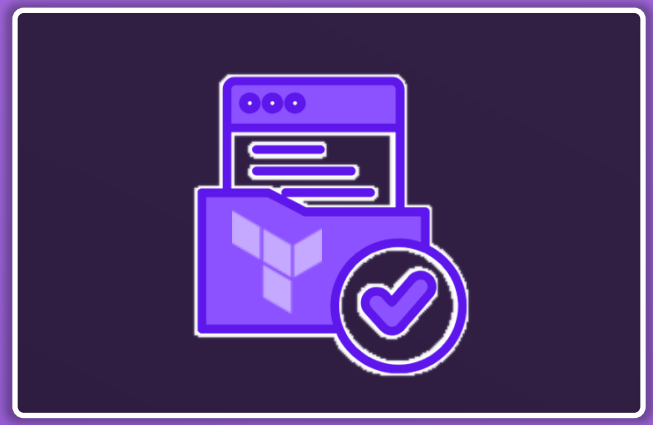
Individuals with HCP Terraform accounts

Can belong to multiple teams and multiple organizations

Teams

Groups of users within an organization

Granted permissions at the organization, project, and workspace levels



Permissions

Additive model - the highest permission always wins

Follow the principle of least privilege

The Owners Team

Every Organization has a default Owners team that has full admin access to everything

- Manage Policies
- Manage Policy Overrides
- Manage Workspaces
- Manage Private Registry
- Manage VCS Settings
- Manage Run Tasks

HCP Terraform Teams

Team Visibility

- Visible: Visible to every member of this organization
- Secret: Only visible to team members and organization owners

Team Tokens

- Team API tokens are designed for performing API operations on workspaces. They have the same access level to the workspaces the team has access to.

Organization, Project, & Workspace Permissions

Broad permissions like Manage all workspaces, Manage policies, Manage VCS settings, View all workspaces

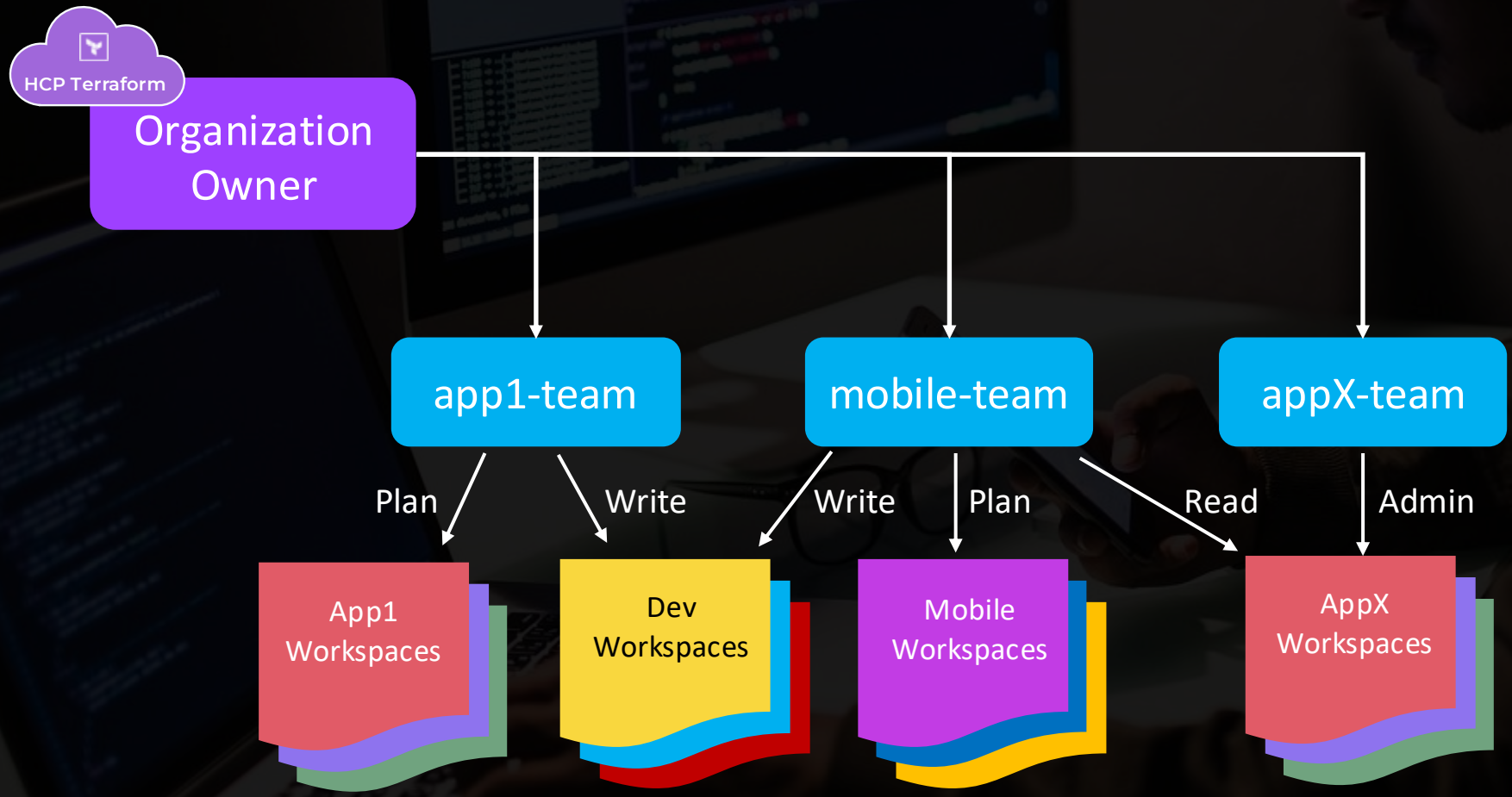
Permissions scoped to a project and all workspaces within it (Read, Write, Maintain, Admin, or Custom)

Permissions on a specific workspace such as Admin, Write, Plan, Read, or Custom

Permissions are additive

HCP Terraform always grants the highest permission level

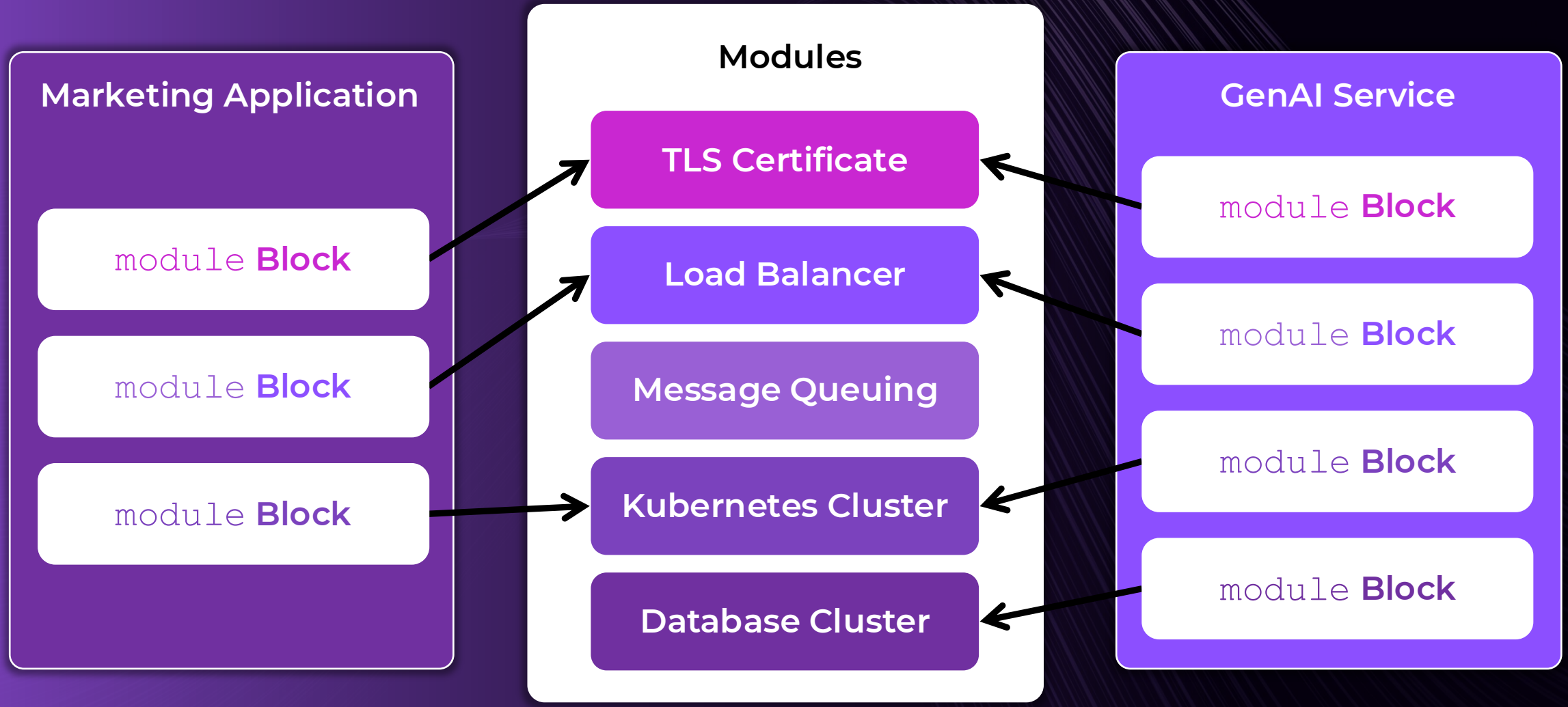
Role Based Access Control (RBAC)



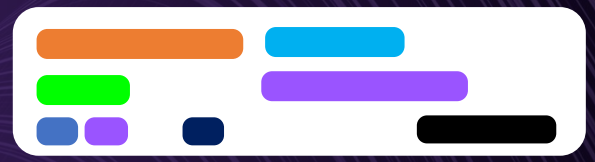
LECTURE

The HCP Terraform Private Registry

Modules for Repeatability

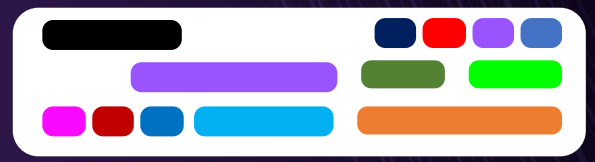


Module Versions



Baseline Configuration

Added New Functionality



Provider API Changed

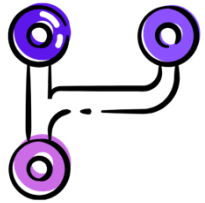
> ...what if we only want our organization to see our modules and not the general public?

Private Registry

Share Modules Privately

Modules and providers are only available to members of your organization

They are not public



Version Controlled

Supports semantic versioning with version constraints, just like the public registry

Enforce Standardization

Develop approved modules and providers so teams build from tested, compliant components

Wait...This Looks Like the Public Registry

	Public Registry	Private Registry
Hosted At:	registry.terraform.io	Inside your HCP Terraform organization
Visibility	Anyone on the internet	Only members of your organization
Who Publishes Content	Anyone	Your team from your connected VCS repos
Content	Community modules & providers	Your private modules + curated public modules
Source Format	terraform-aws-modules/iam/aws	app.terraform.io/krausen-hcp/webapp/gcp

Same experience, different audience

What's Inside the Private Registry?

Private Modules & Providers

Published from your organization's VCS repos, only accessible to organization members

Curated Public Modules

Public registry modules/providers you've added to your private registry as "approved" for your org

Publishing Private Modules

Module code lives in GitHub, GitLab, Bitbucket, etc. Must follow standard module structure.

Publish to your private registry. New versions auto-created from Git tags (v1.0.0, v1.1.0).

Organization members browse, search, and reference modules in their configurations.

Using Private Modules (The Source Format)

Modules from the private registry can be referenced using a registry source address of the form:

app.terraform.io/<ORGANIZATION-NAME>/terraform/<NAME>/<PROVIDER>

```
module "webapp" {
  source  = "app.terraform.io/krausen-hcp/webapp/gcp"
  name    = var.name
  prefix = var.prefix
  version = "1.0.0"
}
```

Key Takeaways

Private Registry

- The private registry lets you share modules and providers privately within your organization
- You can also curate approved public modules/providers alongside your private ones
- Private modules are published from VCS repositories and versioned with Git tags
- The source format includes the hostname: app.terraform.io/<ORG>/<MODULE>/<PROVIDER>
- All org members can view and use the registry; publishing requires Manage Private Registry permissions

LECTURE

Policy Enforcement with Sentinel & OPA

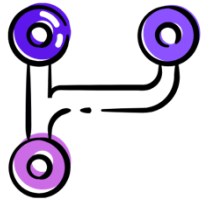
Why Policy Enforcement?

What if we could catch misconfigurations and policy violations before resources ever get deployed?

Policy as Code

Codified Rules

Security, compliance, and operational requirements written as code



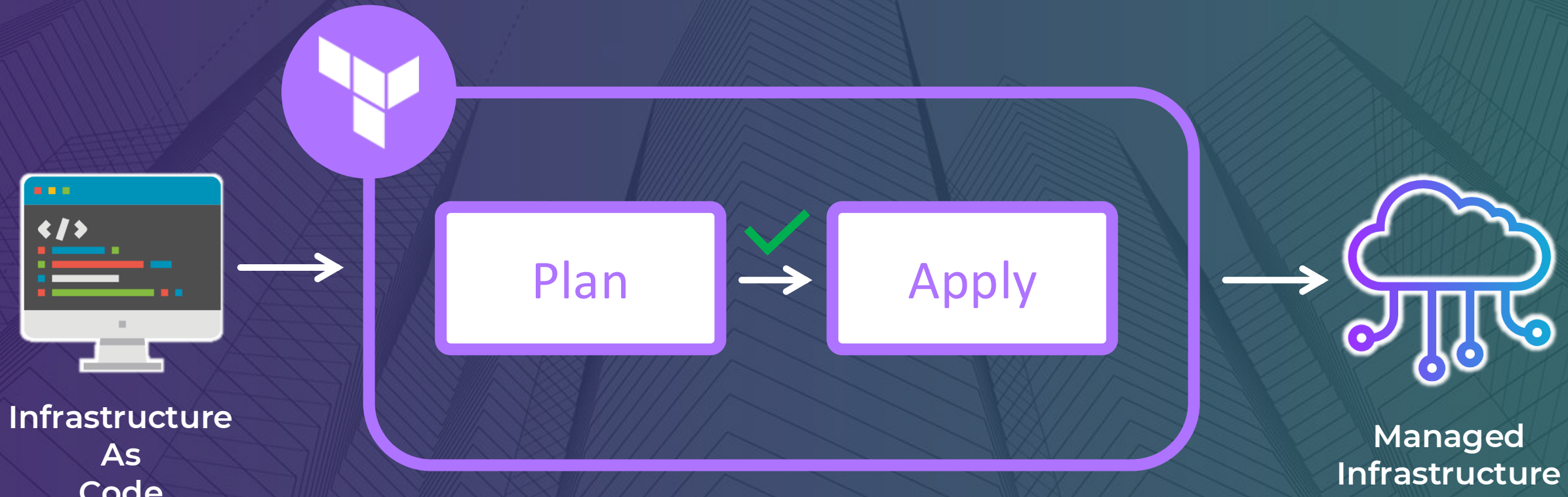
Version Controlled

Stored in VCS, reviewed through pull requests, just like infrastructure code

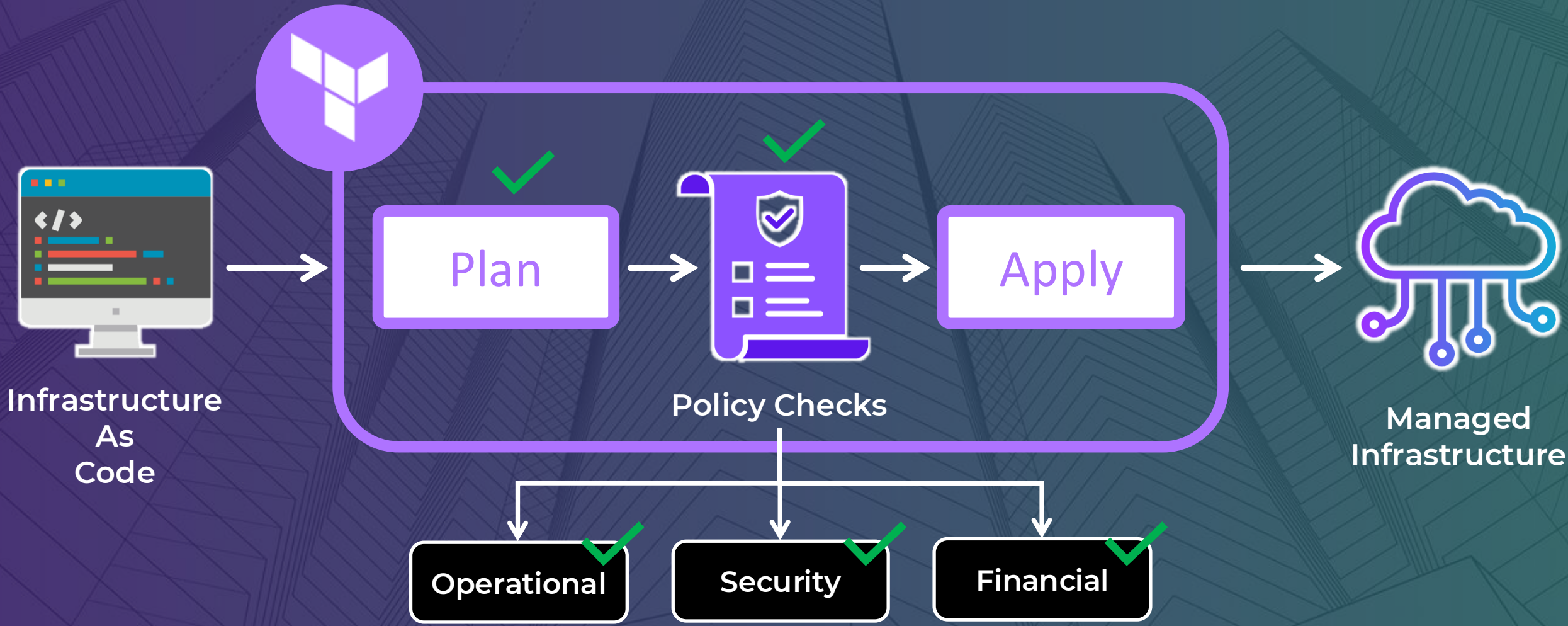
Automatically Enforced

Evaluated on every run, no manual review required

Terraform Run Before Policies



Terraform Run With Policies



Policy Check

- Policies evaluate the plan data before any infrastructure is created
- Failed policies can block the apply entirely
- Sentinel policy checks also have access to cost estimation data
- If the plan itself fails, policies are never evaluated

Two Frameworks: Same Goal

HashiCorp

Sentinel

- Created and Maintained by HashiCorp
- Policy Language: Sentinel
- Ecosystem: HashiCorp Stack
- Pre-Written Policies: Yes, from HashiCorp

VS

Open Policy Agent (OPA)

- Created and Maintained by CNCF
- Policy Language: Rego
- Ecosystem: Cloud-Native
- Pre-Written Polices: Community-Driven

You can use both in the same workspace.

Sentinel

GCP Example

SSH is restricted from the internet

```
import "tfplan/v2" as tfplan

allUnsupportedSourceRanges = [
  "/0",
  "0.0.0.0",
  "0.0.0.0/0",
]

allFirewallResources = filter tfplan.resource_changes as _, resource_changes {
  resource_changes.type is "google_compute_firewall" and
    resource_changes.mode is "managed" and
    (resource_changes.change.actions contains "create" or
      resource_changes.change.actions is ["update"])
}

print("Ensure that SSH access is restricted from the internet")

deny_public_ssh_access = rule {
  all allFirewallResources as _, firewall {
    all filter firewall.change.after.allow as _, allow {
      allow.ports contains "22"
    } as _ {
      all firewall.change.after.source_ranges as _, range {
        range not in allUnsupportedSourceRanges
      }
    }
  }
}

main = rule {
  deny_public_ssh_access
}
```

OPA

AWS Example

Checks whether Security Groups include Ingress from 0.0.0.0/0

```
1  package terraform.policies.public_ingress
2
3  import input.plan as plan
4
5  deny[msg] {
6    r := plan.resource_changes[_]
7    r.type == "aws_security_group"
8    r.change.after.ingress[_].cidr_blocks[_] == "0.0.0.0/0"
9    msg := sprintf("%v has 0.0.0.0/0 as allowed ingress", [r.address])
10 }
```

Sentinel Enforcement Levels

Advisory

- Failed policies produce a warning but the run continues
- Use for new policies you're introducing but not yet enforcing

Soft Mandatory

- Failed policies stop the run, but can be overridden
- Requires the Manage Policy Overrides permission to override
- All overrides are logged for auditing

Hard Mandatory

- Failed policies stop the run and cannot be overridden
- The violation must be fixed in the configuration before the run can proceed
- The default enforcement level for Sentinel policies

OPA Enforcement Levels

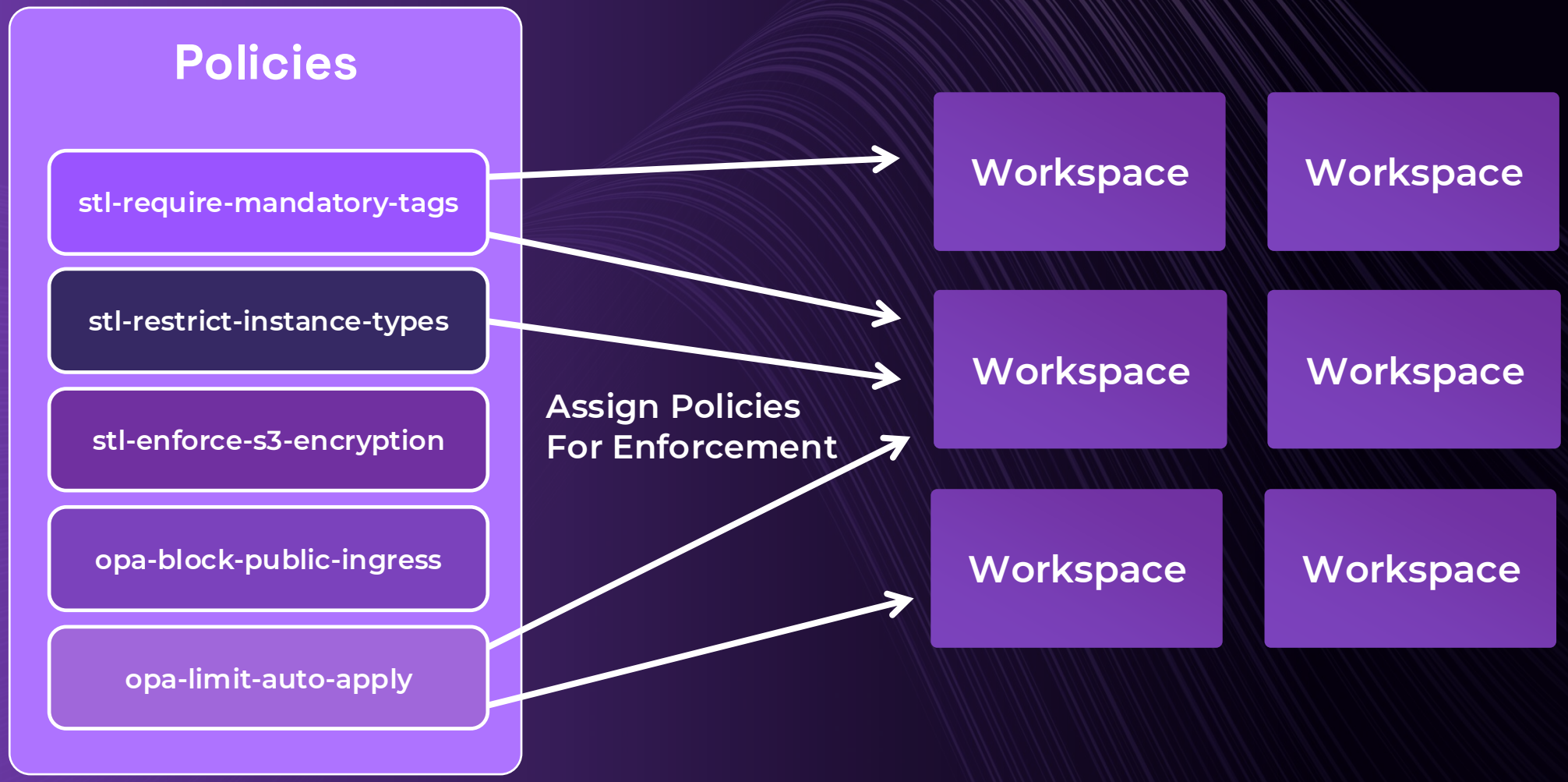
Advisory

- Failed policies produce a warning but the run continues
- Use for new policies you're introducing but not yet enforcing

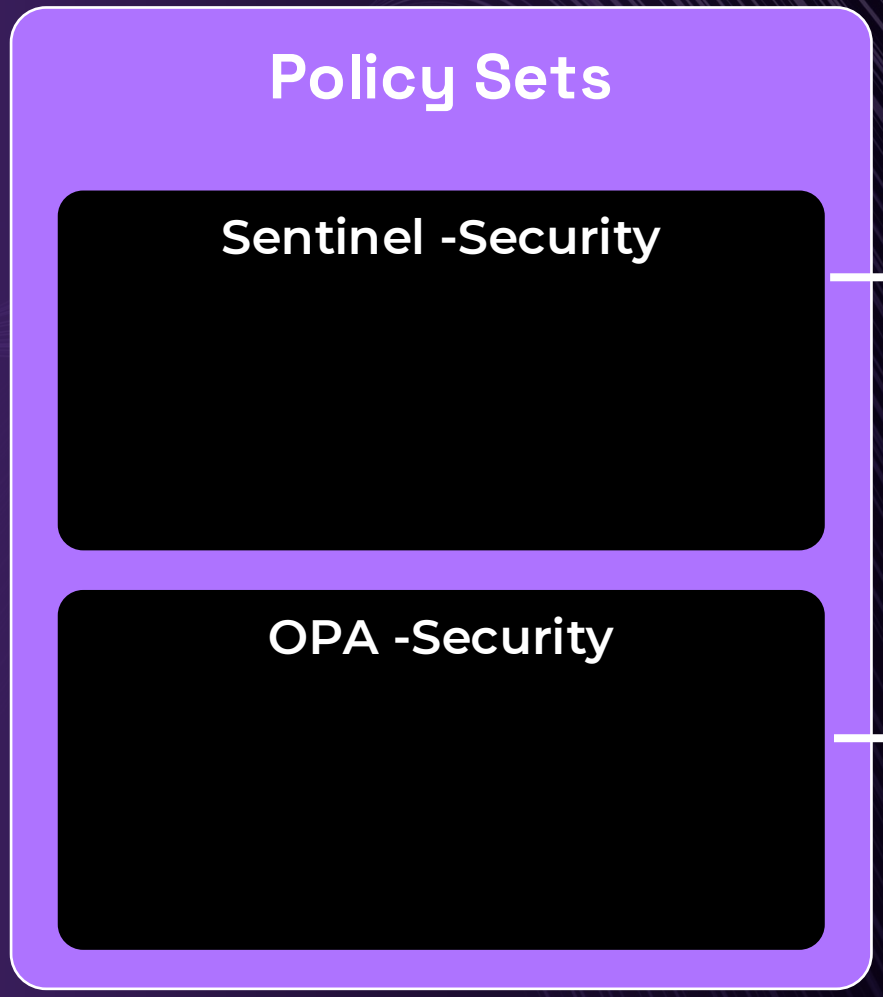
Mandatory

- Failed policies stop the run and can be overridden
- The violation must be fixed in the configuration before the run can proceed

Develop Policies



Policy Sets



Assign Policy Set to Workspace

Workspace

Workspace

Workspace

Policy Sets

Policies Are Grouped Into Policy Sets

Each Set Is One Framework Only (All Sentinel or All OPA)

Sets Can Be Scoped Globally, to Projects, or Specific Workspaces

You Can Exclude Workspaces From A Set

Three Management Methods (UI, VCS, API)

What Policies Can and Can't Do

Examples of What Policies Can Do

- ✓ Block Public S3 Buckets
- ✓ Restrict Instance Types
- ✓ Enforce Tagging
- ✓ Limit Regions
- ✓ Enforce Cost Measures

What Policies Can NOT Do

- ✗ Continuously Monitor After Deployment
- ✗ Remove Existing Violating Resources
- ✗ Validate That Attribute Values Are Valid
- ✗ Analyze Runtime Behavior

Key Takeaways

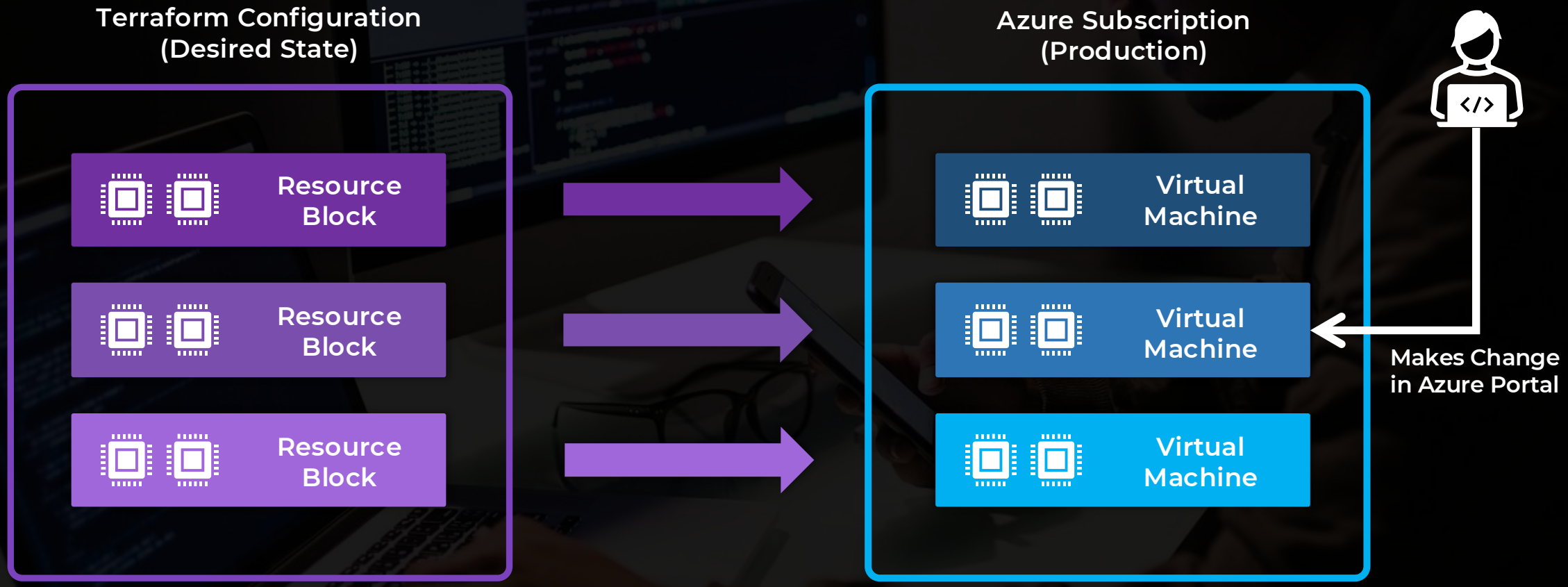
Policy Enforcement

- Policies evaluate after the plan phase but before apply, catching issues before infrastructure is created
- HCP Terraform supports two policy frameworks: Sentinel (HashiCorp) and OPA (open source, uses Rego)
- Sentinel has three enforcement levels (advisory, soft mandatory, hard mandatory) while OPA has two (advisory, mandatory)
- Policy sets can be scoped globally, to specific projects, or to individual workspaces
- Overriding a failed policy requires the Manage Policy Overrides permission

LECTURE

Additional HCP Terraform Workspace Features

Health Assessments



How Do You Know Your Infrastructure Is Still Correct?

Health Assessments

Once enabled, HCP Terraform can automatically run health assessments on your workspaces once every 24 hours or so

Drift Detection

Compares Real Infrastructure Against Your State File

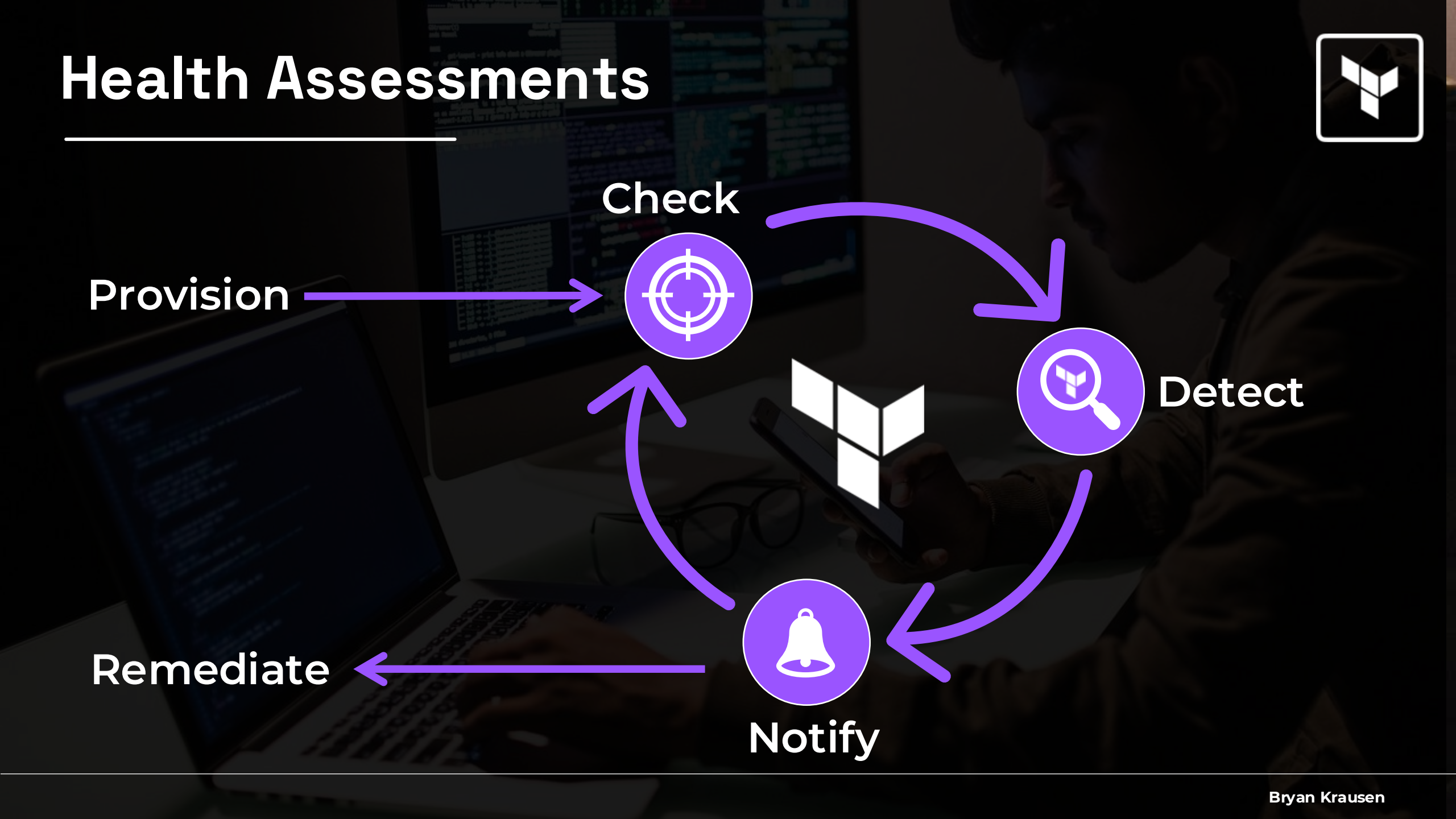
Did Something Change Outside Terraform?

Continuous Validation

Evaluates `check` Blocks, Preconditions, And Postconditions

Are Your Custom Rules Still Passing?

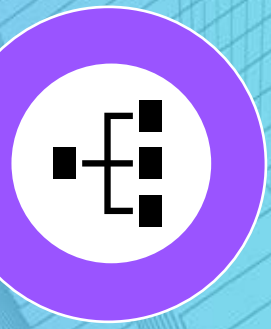
Health Assessments



Health Assessments

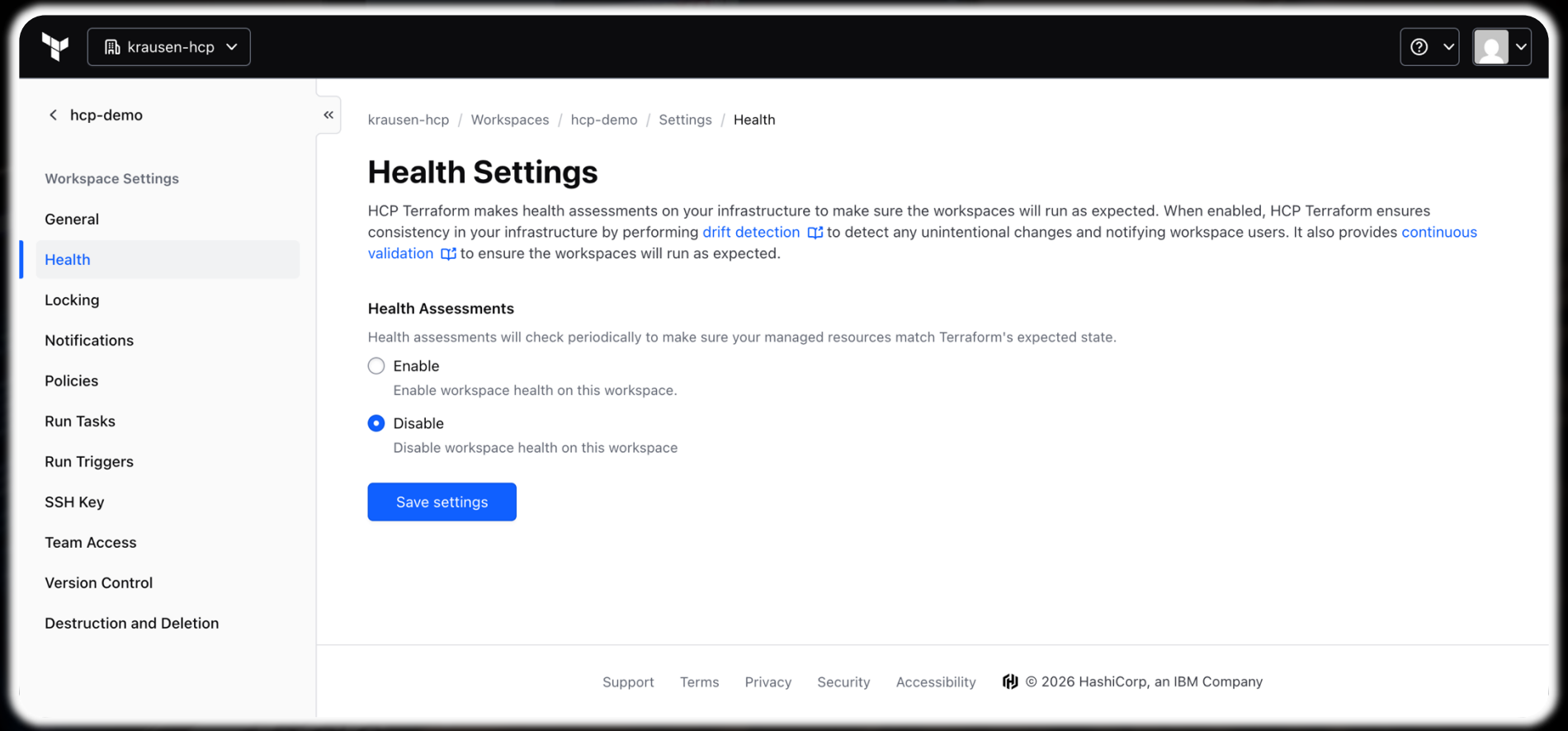
If something's wrong, your workspace shows a "Health warning" status

Health assessments are available in HCP Terraform Standard and Premium editions

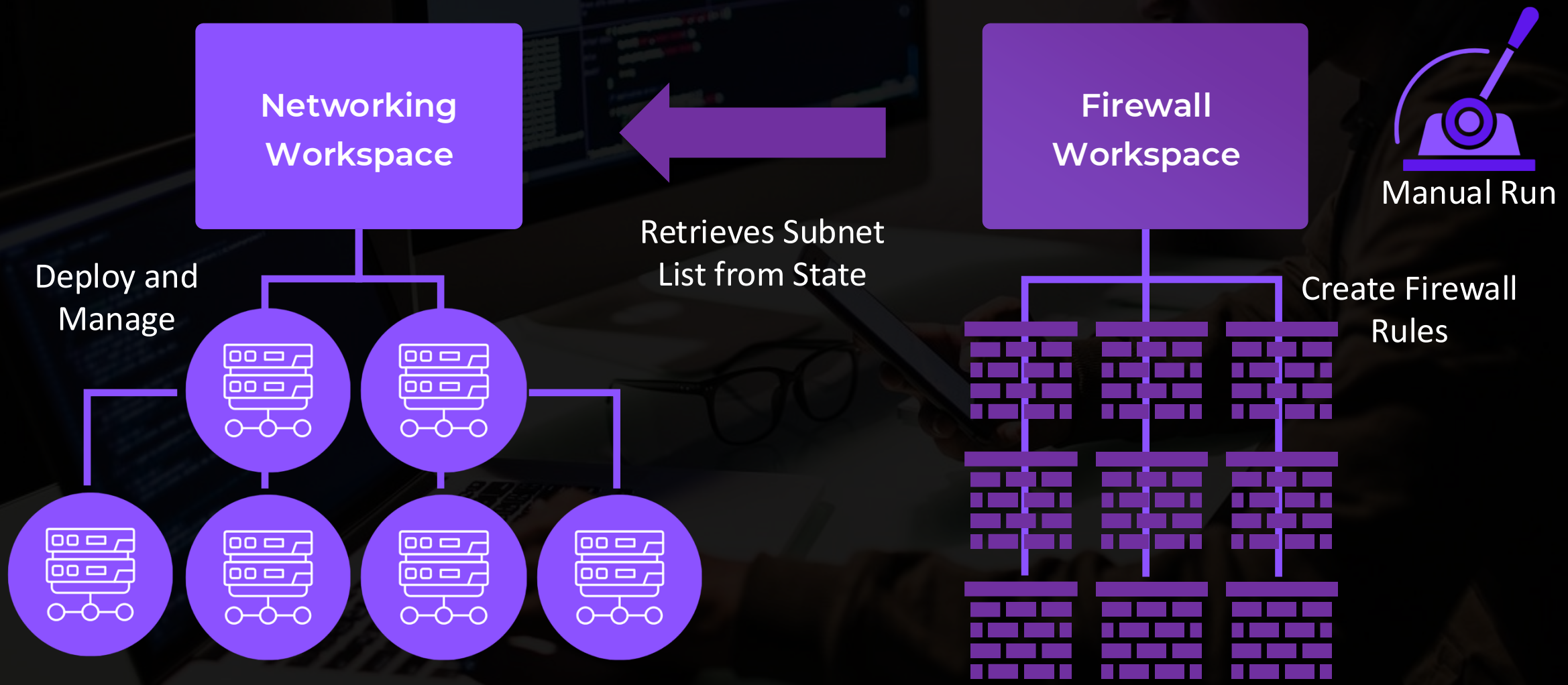


Can be enabled per-workspace or enforced across the entire organization

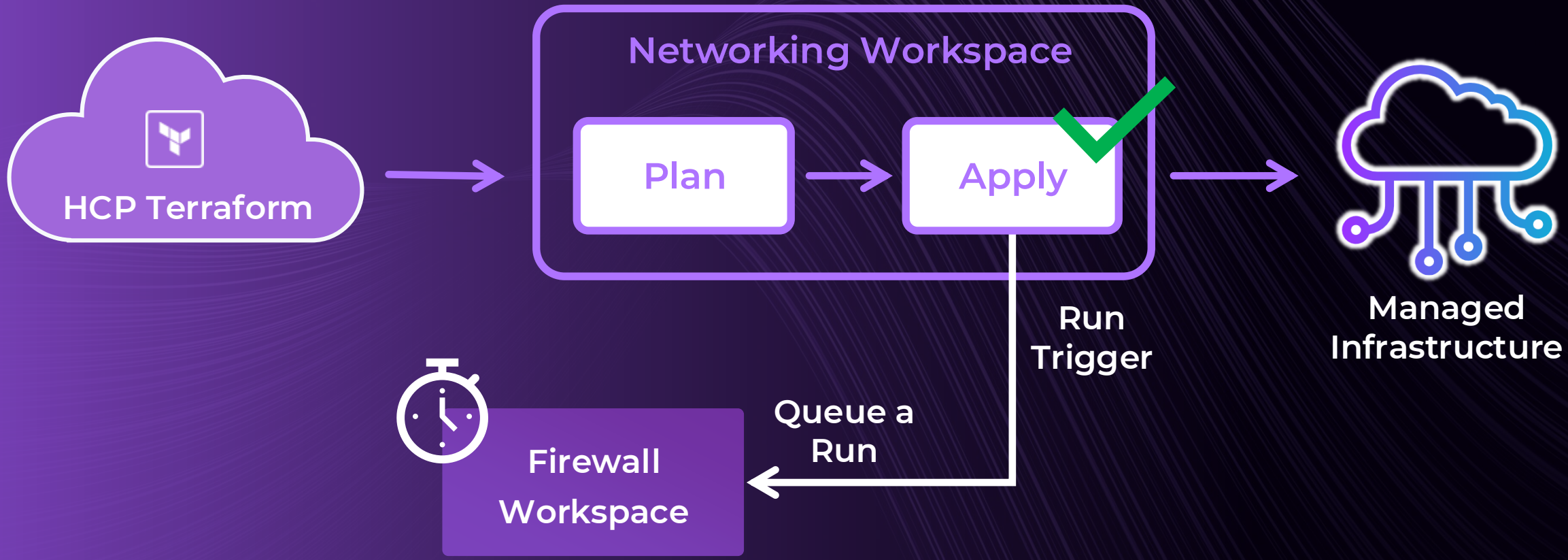
Health Assessments



Workspaces Often Depend On Each Other



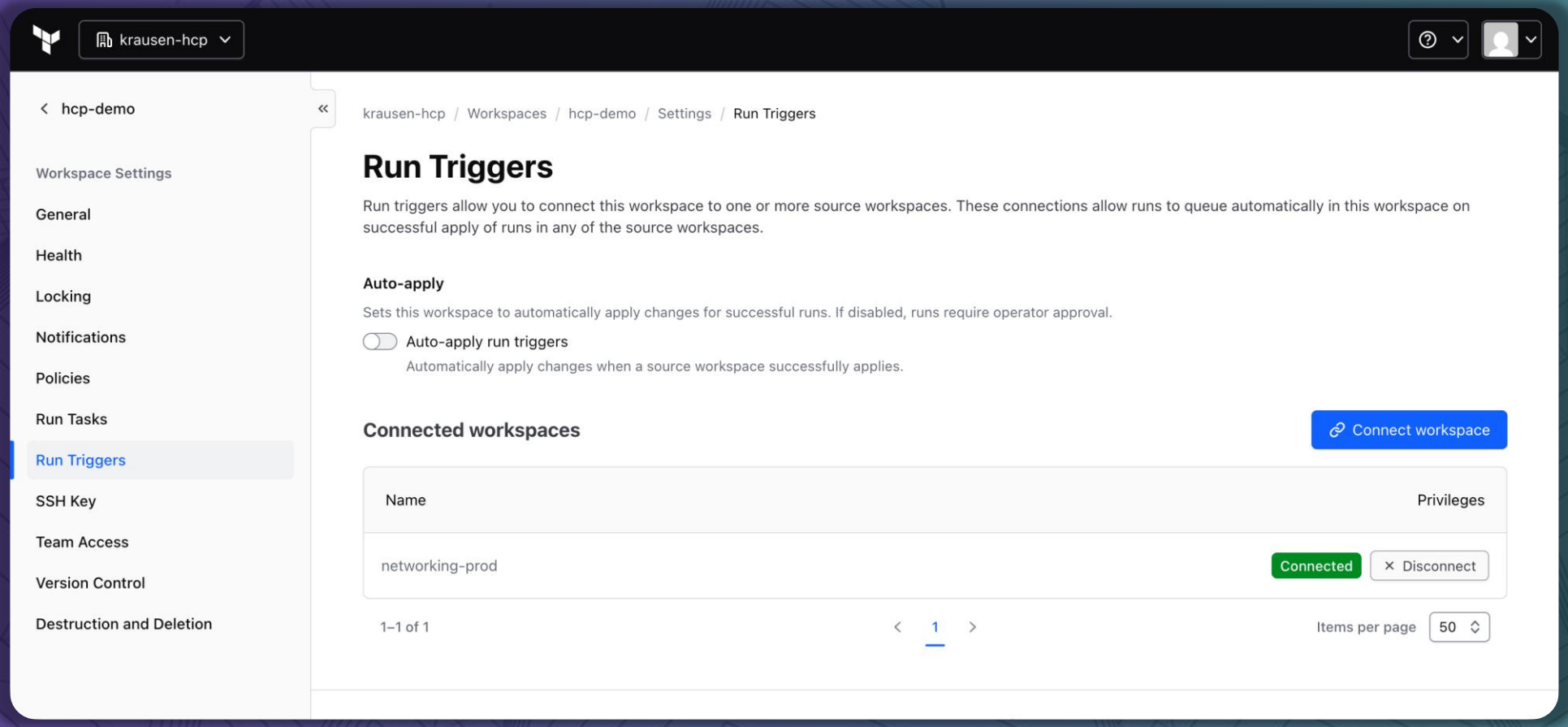
Automating Runs with Run Triggers



Benefits of Run Triggers

- Run Triggers let you connect a workspace to one or more source workspaces
- HCP Terraform automatically queues a run in the connected downstream workspace
- Each workspace can connect to up to 20 source workspaces
- Run triggers fire on successful applies in source workspaces and that they enable workspace-to-workspace automation

Creating a Run Triggers

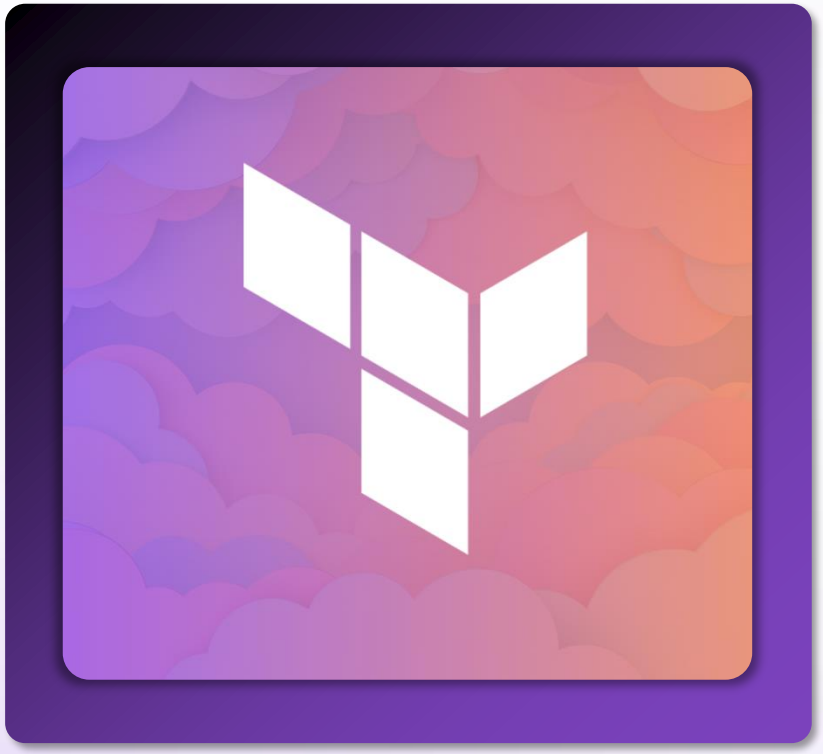


Tracking Work

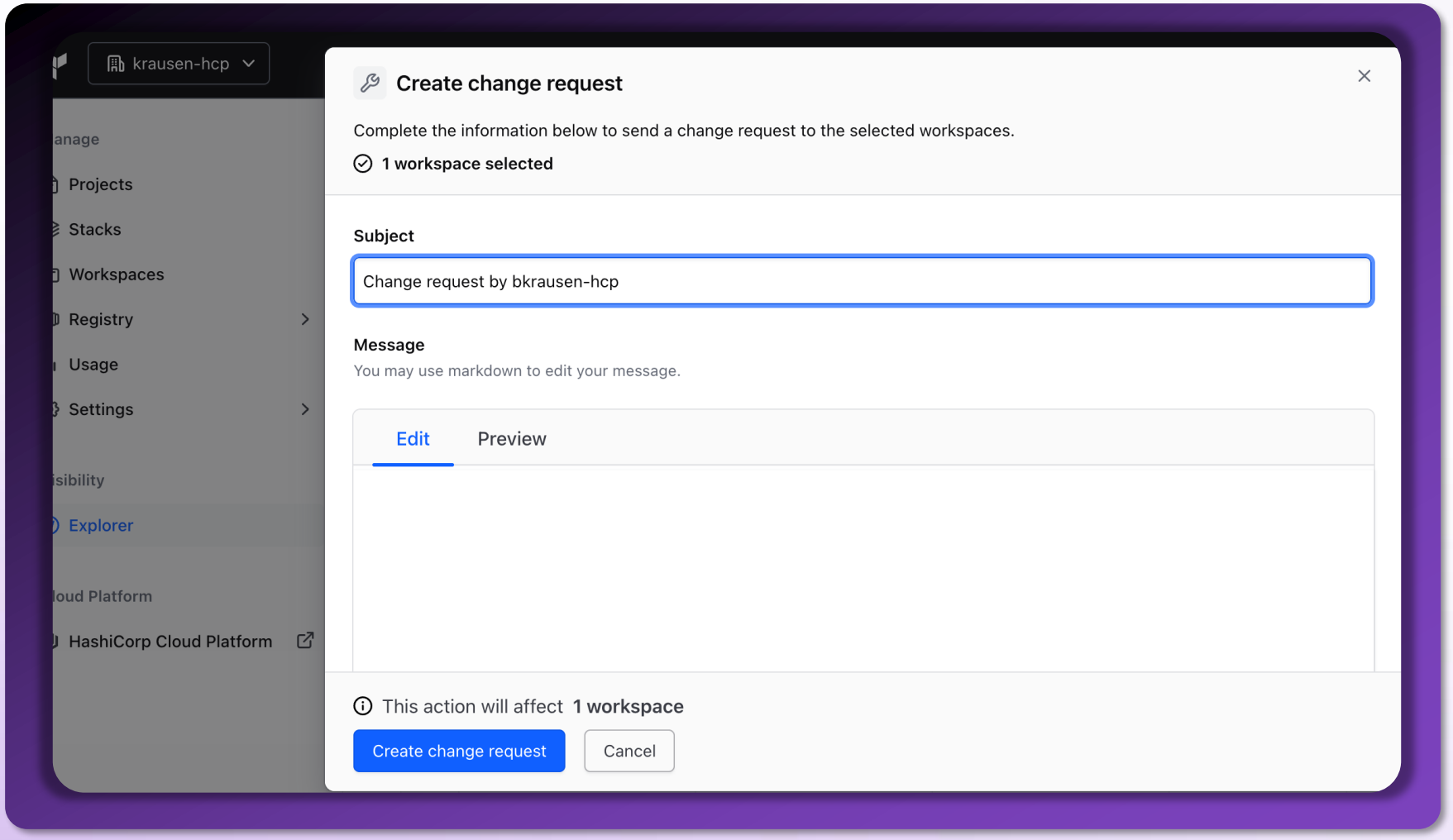
- As your organization scales, you'll have hundreds of workspaces
- Some might need updates, version, upgrades, or fixes
- As an HCP Terraform admin, you need the ability to add to-do items you can attach to workspaces

Tracking Work with Change Requests

- Change Requests are actionable "to-do" items you can attach to workspaces
- You add the messages directly from the Explorer and they get tracked for each workspace
- Teams can be notified when new change requests are created on their workspaces
- Available in Standard and Premium editions



Tracking Work with Change Requests



Gaining Visibility with the Explorer